

実験ログ A: プロシージャルモデリング 基礎

2026-09-14 | 収録 54 件

hython で実際にノードを組み、その結果を実測して記録したもの。数値はすべて実行して得た値で、推測は含まない。

各実験は「何を作ったか」「ノードの接続」「変更したパラメータ」「結果」「わかったこと」「次にやること」の順で並んでいる。用語は別ソースの「用語集: Houdini 初心者向け」にまとめてある。

収録している実験

- Box を山にして細分割し、位置で色を付ける
- subdivide の回数で、形と数はどう変わるか
- mountain と subdivide は、順序を変えると何が変わるか
- mountain のノイズパラメータは、それぞれ何に効くのか
- basis (ノイズの種類) 14通りで、形はどこまで変わるか
- height は純粋な倍率か。gain と bias は何に効くか
- 平面で地形を作り、指標を作り直す
- scatter と copy to points — 地面に物を生やす
- attribute wrangle で VEX を書く — 何が何回転くのか
- for-each ループと VEX — 同じことをやって比べる
- crease で角を残す — どれだけの重みが要するのか
- polyextrude と boolean — 硬い形を作る2つの道具
- 時間軸への対応 — フレームを進めて記録する
- RBD 破壊 — 最初の本物のシミュレーション
- Vellum クロス — 布で保存されるべき量は何か
- Pyro 煙 — dissipation の正体を式で突き止める
- FLIP 液体 — 組み方が分からず保留
- Karma でのレンダリング — ノイズはサンプル数で本当に減るのか
- FLIP 液体 — 配線が解けた。そして粒の数は体積ではなかった
- POP パーティクル — 生まれる数と落ち方を式で確かめる
- 煙はなぜ動かなかったのか — 温度を入れると立ち上る
- 液体に表面を張る — そして019の係数が間違っていた
- 炎を出す — shredding が効かなかった理由
- 動く煙で撮り直す — 018の結論は変わらなかった
- シミュレーションのキャッシュ — 速くなるが、値は変わっていないか
- 炎に色を付ける — 黒体放射の式を数字で追う
- キャッシュを道具に組み込む — 古いファイルを読む事故を防ぐ
- 布は破れなかった — Vellum の breaking が効かない条件を潰す

- ・粒に力かける — 風は「押す」のではなく「追いつかせる」
- ・過去29回を測り直す — 書いたことは、いまも正しいか
- ・Vellum の breaking は動く — ただし布の伸びには効かない
- ・狙った線で破れる布 — 先に切って、あとから貼る
- ・布を物にぶつけて破る — 破れ始めは当たった場所、そのあとは全体へ
- ・HeightField で地形を作る — 高さを面ではなく数の並びで持つ
- ・地形の上に生やし分ける — 浸食が残した情報で木の場所を決める
- ・流れの向きに草を寝かせる — flowdir は x と y が入れ替わっている
- ・浸食のレイヤーで地形を塗り分ける — 色が地形の形と対応しているか
- ・塗り分けた地形を Karma で出す — 見立ては外れ、白飛びが本命だった
- ・Copernicus でテクスチャを作る — 細かさは解像度をどう食うのか
- ・作ったテクスチャを地形に貼る — 貼れたかどうかを画の数字で判定する
- ・凸凹の付け方3通り — 輪郭を見れば、どれが本当に形を変えたか分かる
- ・毛を生やす — density は本数ではなく「面積あたりの本数」だった
- ・毛を束ねる — つなぐ順番が、思っていたのと逆だった
- ・つないだのに、ガイドが1本も使われていなかった — rest が (0,0,0) だった
- ・毛を Karma で出す — 既定の太さは、画の上で 0.1ピクセルしかなかった
- ・毛に色を付ける — 光を消して測ると、色は位置ぴったりに乗った
- ・土台が動いたとき、毛は付いてくるのか — 入力2をつなぐかどうかだけの差
- ・毛を揺らす — 曲がりにくさは「垂れる距離」ではなく「形」に効いていた
- ・毛に風を当てる — 実験029と同じ「二次の抵抗」だった
- ・APEX に入る — 2,220個の部品と、いちばん小さいグラフ
- ・骨を作って動かす — rotate と prerotate を取り違えると、関節ごと飛んでいく
- ・骨に肉を付ける — 曲げると痩せる。90度で 27% 減った
- ・痩せない曲げ方はあった — Dual Quaternion で体積の減りが 154分の1 になる
- ・なぜ Linear が既定なのか — 速さではなかった。差は 4%

Box を山にして細分割し、位置で色を付ける

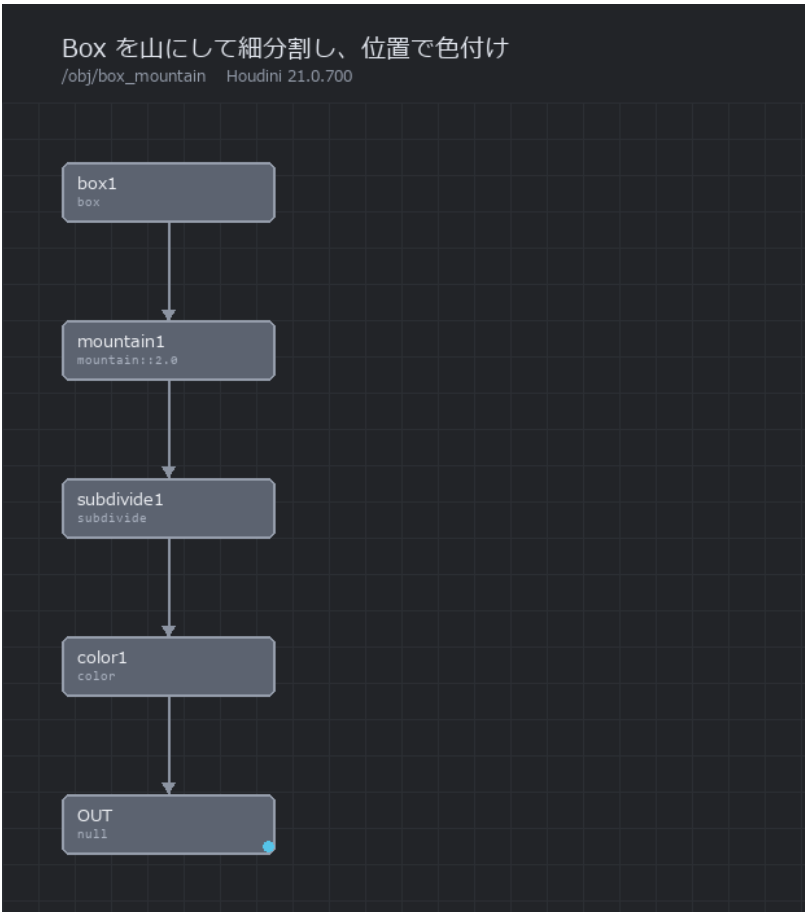
2026-09-14 | Houdini 21.0.700 | /obj/box_mountain | ノード 5 / 接続 4

何を作ったか

プロシージャルモデリングの最小構成を確認するための実験。box SOP で立方体を作り、mountain SOP でノイズによる凹凸を与え、subdivide SOP で細分割し、color SOP で色を乗せた。最後に null ノードを OUT として置き、そこを出力点にしている。

Houdini の基本的な考え方である「ジオメトリを次のノードに流し込んで加工を積み重ねる」形をそのまま辿っている。各ノードは前のノードの出力だけを入力として受け取るため、途中のどのノードでも結果を確認できる。

ノードの接続

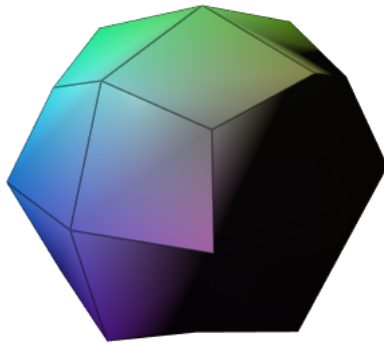


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
mountain1	mountain::2.0	elementsize = 0.8 height = 0.35
subdivide1	subdivide	(変更なし)
color1	color	colortype = 1 ramp2cb = 1.0 ramp2cg = 1.0 ramp2cr = 1.0 ramp2pos = 1.0

ノード	タイプ	デフォルトから変更したパラメータ
OUT	null	(変更なし)

結果



Houdini

OpenGL ROP による Smooth Wire Shaded

表示。ワイヤーフレームが乗っているため、ネットワークが生成したトポロジが確認できる。画像内のウォーターマークは Apprentice ライセンスによるもの。

ジオメトリ

ポイント数	26
プリミティブ数	24
バウンディングボックス	[-0.4794, -0.4768, -0.4767] ~ [0.4932, 0.4959, 0.4959]
ポイント属性	P, Cd
プリミティブ属性	なし
ディテール属性	なし

わかったこと

- ・ box は 8 ポイント / 6 プリミティブ。subdivide を 1 回かけると 26 ポイント / 24 プリミティブになる。Catmull-Clark 細分割なので、回数を増やすと立方体はどんどん球に近づく（2 回で既に球状になり、元が箱だと分からなくなった）。

- ・ mountain SOP

は既存のポイントを法線方向にノイズで押し出すもので、ポイントを増やさない。つまり box のままかけても 8 点しか動かないため、凹凸のディテールを出すには先に subdivide などで分割数を上げる必要がある。

- ・ color SOP の colortype は数値メニュー（0=Constant, 1=Bounding Box, 2=Random, 3=Ramp from Attribute, 4=Random from Attribute）。文字列で指定すると TypeError になる。今回は 1

を使い、バウンディングボックス内の位置をそのまま RGB に割り当てた。

- ・ color SOP を通すとポイント属性に Cd が追加される（P のみ → P, Cd）。属性が増えたかどうかはジオメトリの統計で確認できる。

次にやること

- ・ subdivide の回数を変えて、ポイント数と見た目がどう変わるかを並べて比較する
- ・ mountain を subdivide の後ろに移動し、分割後に凹凸をかけた場合との違いを見る
- ・ grid から地形を作る流れに広げ、プロシージャルモデリングの定番パターンに入る

subdivide の回数で、形と数はどう変わるか

2026-09-14 | Houdini 21.0.700 | /obj/subdiv_compare | ノード 5 / 接続 4

何を作ったか

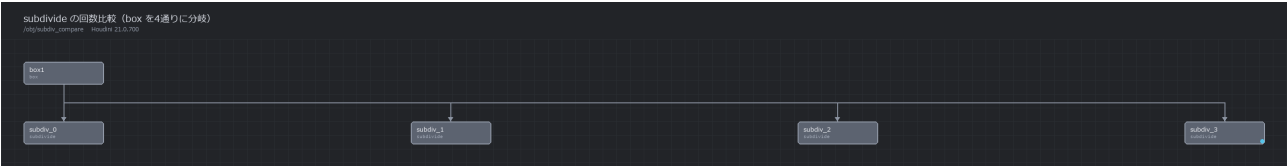
実験001で「subdivideの回数を増やすと立方体が球に近づく」ことに気づいたので、それを数値で確かめる実験。box SOP に subdivide SOP

を4通り（0回・1回・2回・3回）ぶら下げ、すべて同じカメラでレンダして並べた。

同じカメラで撮っているのが重要な点で、各変種を自動でフレームに収めてしまうと、大きさの違いが消えて比較にならない。画を見ると球が立方体よりはっきり小さいことが分かるが、これは目の錯覚ではなく実測でも縮んでいる。

さらに「なぜ縮むのか」「どこまで縮むのか」「1回目だけ縮まないのはなぜか」を追加検証で確定させた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
subdiv_0	subdivide	iterations = 0
subdiv_1	subdivide	(変更なし)
subdiv_2	subdivide	iterations = 2
subdiv_3	subdivide	iterations = 3

回数を変えた結果

すべて同じカメラ・同じ表示モード（Smooth Wire Shaded）。元の立方体の輪郭に対して、分割を重ねるほど内側に縮んでいくのが分かる。



iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
0	8	6	—	1.00000
1	26	24	4.00	1.00000

iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
2	98	96	4.00	0.87847
3	386	384	4.00	0.84918
4	1538	1536	4.00	0.84192
5	6146	6144	4.00	0.84011
6	24578	24576	4.00	0.83966

アルゴリズム別の比較（iterations = 3 で固定）

subdivide SOP の algorithm パラメータには5つの選択肢がある。既定値は 2（OpenSubdiv Catmull-Clark）。同じ3回分割で結果がどう違うかを測った。

algorithm	ポイント数	プリミティブ数	一辺の長さ
Houdini Catmull-Clark	386	384	0.84918
Mantra-Compatible Catmull-Clark	386	384	0.83951
OpenSubdiv Catmull-Clark（既定）	386	384	0.84918
OpenSubdiv Loop	386	768	0.86932
OpenSubdiv Bilinear	386	384	1.00000

回数を変えたときの一辺（2つのアルゴリズムの比較）

OpenSubdiv Catmull-Clark は回数を増やすほど値が動くが、Mantra-Compatible は回数を変えても一定。これが何を意味するかは「わかったこと」の最後に書いた。

iterations	OpenSubdiv CC	Mantra-Compatible
1	1.00000	0.83951
2	0.87847	0.83951
3	0.84918	0.83951
4	0.84192	0.83951
5	0.84011	0.83951

わかったこと

- ・ iterations = 0 は素通し。8点 / 6面のまま何も起きない。比較の基準として使える。
- ・ プリミティブ数は1回ごとに正確に4倍になる（6 → 24 → 96 → 384 → 1536 → 6144 → 24576）。四角形1枚が4枚に割れるため。3回で64倍、6回で4096倍なので、回数を安易に増やすと破綻する。
- ・ ポイント数は常に「プリミティブ数 + 2」になっている。これは偶然ではなくオイラーの多面体定理から導ける。閉じた四角形メッシュでは各辺を2つの面が共有するので $E = 2F$ 、これを $V - E + F = 2$ に代入すると $V = F + 2$ 。実測した7通りすべてで一致した。
- ・ 1回目だけ一辺が 1.00000 のまま縮まない。検証したところ、+X方向の端を保持しているポイントは (0.5, 0, 0) の1点だけだった。これは元の +X 面の中心そのもの。Catmull-Clark は各面の中心に新しい点を作るため、1回目はその新点元が元の面の上に乗り、バウンディングボックスが変わらない。2回目でその点も内側に引かれ 0.87847 に縮む。

- ・ 一辺の縮み方は等比的に収束する。差を取ると $0.02929 \rightarrow 0.00726 \rightarrow 0.00181 \rightarrow 0.00045$ で、毎回およそ $1/4$ （実測 $0.248 / 0.249 / 0.249$ ）。ここから外挿した極限值は約 0.83951 になる。

- ・ 縮みの原因は「分割」ではなく「平滑化」だと確定した。OpenSubdiv Bilinear（平滑化せず分割するだけの方式）で3回分割しても一辺は 1.00000 のまま変わらない。面数は同じ384なので、分割量は同じで平滑化の有無だけが違う。

- ・ OpenSubdiv Loop だけプリミティブ数が 768 になる（他は384）。Loop は三角形メッシュ向けの方式なので、四角形が三角形に割られて枚数が倍になる。四角形を保ちたい場面では選んではいけない。

- ・ Mantra-Compatible Catmull-Clark は回数を $1 \sim 5$ のどれにしても一辺が 0.83951 で一定だった。これは分割を繰り返して極限に近づけるのではなく、極限曲面上の位置を直接求めていることを示している。実際この値は、OpenSubdiv CC の収束列から外挿した極限值 0.83951 と一致した。つまり「無限回分割した先の形」を1回で出しているのがこのモード。

次にやること

- ・ crease（折り目）を使って角を保持できるか。立方体らしさを残したまま分割する方法があるはず

- ・ mountain と subdivide

の順序を入れ替えて、凹凸のディテールがどう変わるかを比較する（実験003）

- ・ polyextrude

など他のSOPでもポイント数・プリミティブ数の増え方を測り、重くなる原因を数で把握する

mountain と subdivide

は、順序を変えると何が変わるか

2026-09-14 | Houdini 21.0.700 | /obj/order_compare | ノード 14 / 接続 13

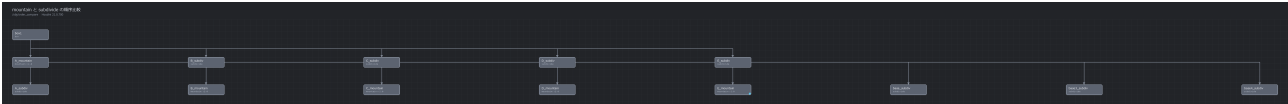
何を作ったか

実験001で「mountain はポイントを増やさないの、細かい凹凸を出すには先に分割数を上げる必要がある」と書いた。これは筋は通っているが、実際に確かめてはいなかった。そこで順序違いを並べて比較し、違いを数字で出す。

比較したのは6通り。ノイズなしの基準形、ノイズを先にかけたもの、分割を先にしたもの、分割をさらに細かくしたもの、ノイズの特徴サイズを小さくしたもの、そのうえで点数を増やしたもの。すべて同じカメラでレンダした。

この実験では測り方を一度間違えている。最初に選んだ指標では違いが正しく出ず、途中で別の指標に切り替えた。その経緯も含めて記録した。

ノードの接続

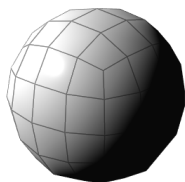


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
A_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
A_subdiv	subdivide	iterations = 2
B_subdiv	subdivide	iterations = 2
B_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
C_subdiv	subdivide	iterations = 3
C_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
D_subdiv	subdivide	iterations = 3
D_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
E_subdiv	subdivide	iterations = 4
E_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
base_subdiv	subdivide	iterations = 2
base3_subdiv	subdivide	iterations = 3
base4_subdiv	subdivide	iterations = 4

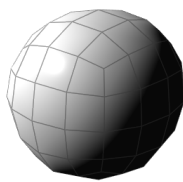
6通りの結果

すべて同じカメラ・同じ表示モード。左から右へ、凹凸がどう変わるかを見てほしい。mountainのパラメータは height = 0.35 で固定し、elementsize（ノイズの特徴サイズ）だけを D と E で 0.8 から 0.25 に変えている。



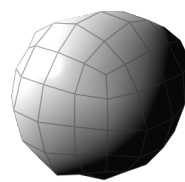
Houdini

基準: 分割のみ、ノイズなし 98点 / 96面



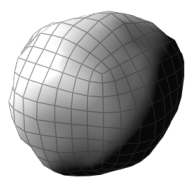
Houdini

A: mountain → subdiv(2) 98点 / 96面



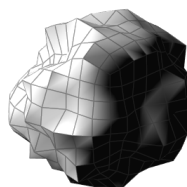
Houdini

B: subdiv(2) → mountain 98点 / 96面



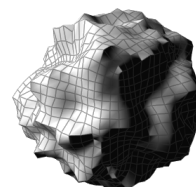
Houdini

C: subdiv(3) → mountain 386点 / 384面



Houdini

D: C と同じだが elementsize 0.25 / 386点



Houdini

E: D と同じノイズで 点数のみ増加 / 1538点

基準形からの点ごとのずれ

同じ分割回数ならポイントの順序も同じなので、ノイズなしの基準形と1点ずつ引き算できる。これが「ノイズがどれだけ形を動かしたか」を直接表す。A・B は2回分割の基準形と、C・D は3回分割の基準形と、E は4回分割の基準形と比べている。

変種	ポイント数	平均のずれ	最大のずれ
A: mountain → subdiv(2)	98	0.02069	0.03671
B: subdiv(2) → mountain	98	0.02479	0.08062
C: subdiv(3) → mountain	386	0.02504	0.08009
D: C + elementsize 0.25	386	0.03236	0.11985
E: D + 点数4倍	1538	0.03239	0.13173

最初を選んで失敗した指標（表面積）

凹凸が増えれば表面積も増えるはずだと考え、まず表面積を測った。ところが mountain を足すと表面積は減った。ノイズは外側にも内側にも押すため全体が縮み（一辺が 0.8785 から 0.8545

へ)、面積は大きさの2乗に比例するのでその影響が凹凸の効果を打ち消していた。一辺の2乗で割って正規化しても、値の大小が凹凸の量と対応しない。大きさと凹凸が混ざった指標だったということ。

変種	表面積	基準比	一辺	面積 / 一辺 ²
基準 (ノイズなし)	2.43164	1.000	0.8785	3.15076
A	2.30052	0.946	0.8545	3.15066
B	2.24947	0.925	0.8535	3.08797
C	2.16988	0.892	0.8314	3.13918
D	2.49922	1.028	0.9123	3.00282
E	2.66892	1.098	0.9278	3.10046

わかったこと

- ・順序を入れ替えてもポイント数とプリミティブ数は同じ (どちらも98点 / 96面)。つまりデータの重さは変わらない。変わるのは形だけ。
- ・最大のずれは A が 0.03671、B が 0.08062 で、B が約2.2倍大きい。同じ mountain の設定でも、分割したあとにかけた方が凹凸が残る。実験001で立てた見込みは正しかった。
- ・なぜそうなるかも測れた。box に mountain をかけた直後は、8点すべてが平均 0.03607 動いている。ところがそのあと subdivide をかけると、基準形からのずれは平均 0.02069・最大 0.03671 まで下がる。Catmull-Clark の平滑化は周囲の点を平均するので、せっかく動かした凹凸をなめしてしまう。
- ・C は B とほぼ同じずれだった (平均 0.02504 対 0.02479、最大 0.08009 対 0.08062)。点を4倍に増やしても凹凸の大きさは変わらない。点数はノイズの振幅を決めていない。
- ・振幅を決めていたのは elementsize だった。C と D は同じ386点だが、elementsiz を 0.8 から 0.25 に下げると平均のずれが 0.02504 から 0.03236 に増え、画も明確に岩ようになった。
- ・E は D と同じノイズ設定で点数だけ4倍にしたもの。平均のずれは 0.03236 から 0.03239 でほぼ変わらない。一方で画は、同じ起伏をより滑らかな面で表現できている。つまり点数は「ノイズをどれだけ忠実に表現できるか (解像度)」を決めていて、「起伏の大きさ (振幅)」は決めていない。
- ・ただし最大のずれだけは点数とともに少し増える (0.11985 → 0.13173)。点が増えるとノイズのより極端な場所まで拾えるようになるため、振幅そのものが増えたわけではない。
- ・実用的な結論: 細かいディテールが欲しいときは、分割数を上げるだけでは足りない。elementsiz も一緒に小さくする。そして mountain は分割のあとに置く。この3つが揃って初めて狙った凹凸になる。
- ・測り方についての反省。表面積のような「全体をひとつの数にまとめた量」は、大きさと形の変化が混ざってしまう。基準形と1点ずつ対応づけて引き算する方が、知りたいことを素直に測れた。比較実験では、まず基準形を作り、それと1対1で比べられる量を選ぶべき。

次にやること

- ・mountain の他のパラメータ (roughness, octaves) が振幅と細かさのどちらに効くのかを、同じやり方で切り分ける
- ・grid から地形を作り、平面の場合も同じ結論になるか確かめる (実験004)
- ・crease を使って、分割しても角を保持できるかを確かめる (実験002からの持ち越し)

mountain

のノイズパラメータは、それぞれ何に効くのか

2026-09-14 | Houdini 21.0.700 | /obj/mountain_params | ノード 15 / 接続 14

何を作ったか

実験003で「点数は解像度を決め、elementsize は振幅を決める」と分かった。では残りのノイズパラメータは何を変えているのか。同じ切り分け方で調べる。

最初に分かったのは名前の問題だった。パラメータ名は roughness や octaves ではなく rough / oct / lac。実際に mountain::2.0 が持っている名前を確認してから使わないと、そもそも動かない。

この実験でも測り方を追加している。003で使った「基準形からのずれ」だけでは、これらのパラメータをほとんど区別できなかった。振幅ではなく形の性質を変えるパラメータだったため、性質を測る指標を新しく用意した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
rough_00	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 0.0
rough_05	mountain::2.0	elementsize = 0.25 height = 0.35
rough_10	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 1.0
oct_1	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 1
oct_3	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 3
oct_8	mountain::2.0	elementsize = 0.25 height = 0.35
lac_15	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 1.5
lac_20	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 2.0
lac_40	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 4.0

ノード	タイプ	デフォルトから変更したパラメータ
fractal_hmFT	mountain::2.0	elementsiz = 0.25 height = 0.35
fractal_fBm	mountain::2.0	elementsiz = 0.25 fractal = fBm height = 0.35
fractal_mfT	mountain::2.0	elementsiz = 0.25 fractal = mfT height = 0.35
fractal_none	mountain::2.0	elementsiz = 0.25 fractal = none height = 0.35

rough（粗さ） — 振幅は変えず、細かさだけを変える

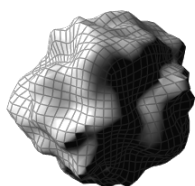
固定条件はすべての変種で共通: subdivide 4回（1538点）、height 0.35、elementsiz 0.25。rough だけを動かしている。平均のずれはほぼ変わらないのに、見た目はなだらかな丘から棘だらけの形まで変わる。



rough	平均のずれ	最大のずれ	隣との食い違い
0.0	0.03377	0.14799	0.01350
0.5（既定）	0.03237	0.13153	0.01733
1.0	0.03235	0.13588	0.03233

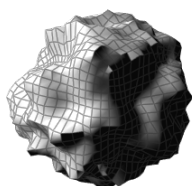
oct（オクタブ数） — 3 以上で効かなくなる

オクタブとは、細かさの違うノイズを重ねる層のこと。oct はその層の数。1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733）。点数が足りず、細かい層を表現できていないため。



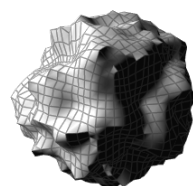
Houdini

oct = 1 食い違い 0.01519



Houdini

oct = 3 食い違い 0.01732



Houdini

oct = 8 (既定) 食い違い 0.01733

oct	平均のずれ	最大のずれ	隣との食い違い
1	0.03231	0.13417	0.01519
3	0.03250	0.13274	0.01732
8 (既定)	0.03237	0.13153	0.01733

予測の検証 — oct が効かないのは点数のせいかな

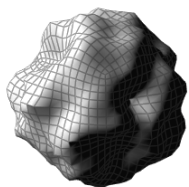
「oct が 3 以上で効かないのは点数が足りないからだ」という説明が正しいなら、点数を増やせば oct=8 が oct=3 を上回るはず。実際にそうだった。386点では oct=8 が oct=3 を下回っているが、6146点では上回っている。

注意: 食い違いの絶対値は点数が増えると小さくなる（隣同士の距離が縮まるので差も縮む）。この表は各行の中で比較だけが有効で、行をまたいで比べてはいけない。

subdivide	点数	oct = 1	oct = 3	oct = 8	8 と 3 の差
3回	386	0.02692	0.02868	0.02856	-0.00012
4回	1538	0.01519	0.01732	0.01733	+0.00001
5回	6146	0.00782	0.00941	0.00963	+0.00022

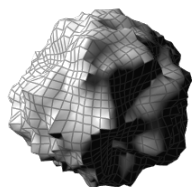
lac（ラキュナリティ） — オクターブ間の周波数の開き

重ねる層ごとに、どれだけ細かさを飛ばすかの倍率。既定は 1.841。大きくすると層の間隔が開いて細かい成分が強くなる。ただし lac = 1.5 では平均のずれも下がっており、振幅にも影響が出ている。



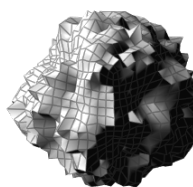
Houdini

lac = 1.5 平均 0.02625 / 食い違い 0.01259



Houdini

lac = 2.0 平均 0.03160 / 食い違い 0.01734



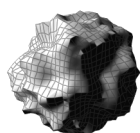
Houdini

lac = 4.0 平均 0.03169 / 食い違い 0.02218

lac	平均のずれ	最大のずれ	隣との食い違い
1.5	0.02625	0.13925	0.01259
2.0	0.03160	0.13528	0.01734
4.0	0.03169	0.13334	0.02218

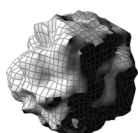
fractal（フラクタルの種類）－ 差は小さい

層の重ね方の方式。4種類あるが、今回の条件では数値の差が小さかった。Terrain だけやや滑らか。既定は Hybrid Terrain。



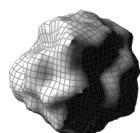
Houdini

Hybrid Terrain（既定）
食い違い 0.01733



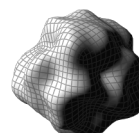
Houdini

Standard (fBm) 食い違い
0.01786



Houdini

Terrain 食い違い 0.01481



Houdini

None 食い違い 0.01350

fractal	平均のずれ	最大のずれ	隣との食い違い
Hybrid Terrain（既定）	0.03237	0.13153	0.01733
Standard (fBm)	0.03183	0.13207	0.01786
Terrain	0.03288	0.13504	0.01481
None	0.03377	0.14799	0.01350

わかったこと

- ・パラメータ名は rough / oct / lac。roughness や octaves という名前は存在せず、そのまま書くと何も設定されない。既定値は rough = 0.5、oct = 8、lac = 1.841、fractal = Hybrid Terrain、basis = Simplex。使う前に実際の名前を確認する必要がある。
- ・003で使った「基準形からのずれ」だけでは、これらのパラメータはほとんど区別できなかった。平均のずれはどれも 0.032～0.034 の範囲に収まる。振幅を決めているのは height と elementsize であり、今回のパラメータは振幅に効いていない。
- ・そこで「隣り合う点どうして、ずれの向きと量がどれだけ食い違うか」を測る指標を足した。なだらかな起伏なら隣の点も同じように動くので値は小さく、細かい凹凸なら大きくなる。基準形は原点中心なので、中心からの距離の差を符号付きのずれとして使っている（外に出たら正、へこんだら負）。これで完全に分離できた。
- ・rough が細かさに最も強く効く。0.0 から 1.0 で食い違いが 0.01350 から 0.03233 へ約2.4倍になる一方、平均のずれは 0.03377 から 0.03235 でほぼ変わらない。つまり rough は、同じ振幅のまま、なだらかな丘から棘だらけの形まで性質だけを変えるパラメータ。
- ・rough = 0.0 の数値は fractal = None と完全に一致した（平均 0.03377、最大 0.14799、食い違い 0.01350）。rough = 0 では追加の層が何も足さないの、層を重ねない単一ノイズに退化している。別の経路で同じ結果に到達したので、この解釈は確かだと言える。
- ・oct は 1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733、差は 0.00001）。点数が足りず細かい層を表現できていないため。この説明を予測に変えて検証したところ、点数を増やす

と oct = 8 が oct = 3 を上回り始めた（386点では -0.00012、1538点で +0.00001、6146点で +0.00022）。予測どおりだったので、この解釈も確かだと言える。

- ・実用的には、oct を上げるだけでは無駄になる。点数が足りていなければ、上げた層は形に現れない。003の「点数は解像度を決める」という結論と同じことが、別の角度から確認できた。

- ・lac も細かさに効く（1.5 → 4.0 で 0.01259 → 0.02218）。ただし lac = 1.5 では平均のずれも 0.02625 に下がっており、振幅にも影響している。rough のようにきれいに性質だけを変えるパラメータではない。

- ・fractal の4種類は、今回の条件では差が小さかった（食い違い 0.01350~0.01786）。地形向け的方式が揃っているので、平面の地形（実験005の予定）ではもっと差が出る可能性がある。

- ・指標の注意点。食い違いの絶対値は点数が増えると小さくなる。隣同士の距離が縮まれば、その間のずれの差も縮むため。したがって同じ点数の中でだけ比較できる。点数をまたいだ比較には、辺の長さで割るなどの正規化が必要になる。

- ・basis（ノイズの基礎となる種類）は14種類あった。既定は Simplex。Worley Cellular のようなセル状のものも含まれており、見た目が大きく変わるはずなので、別の実験にする価値がある。

次にやること

- ・basis を14種類すべて描き出して並べる。セル状ノイズは岩や結晶の見た目に直結するので、実用価値が高い
- ・grid から地形を作り、平面では fractal の種類の差が出るかを確かめる（実験005）
- ・食い違いの指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

basis（ノイズの種類） 14通りで、形はどこまで変わるか

2026-09-14 | Houdini 21.0.700 | /obj/noise_basis | ノード 16 / 接続 15

何を作ったか

004で basis に14種類あることが分かったので、同じ条件で全部描き出した。height 0.35、elementsize 0.25、subdivide 4回（1538点）で固定し、basis だけを変えている。

結果は他のパラメータと性質がまったく違った。rough は振幅を変えずに形の性質だけを変えたが、basis は振幅も性質も両方変える。同じ height を指定しても、種類によって実際のずれが4.5倍違う。

測る指標には004で作った「隣との食い違い」に加えて、外に出た点の割合を足した。ノイズが外向きに押すのか内向きに削るのかで、使いどころが変わるため。

ノードの接続



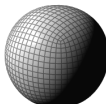
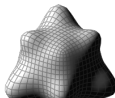
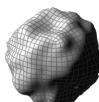
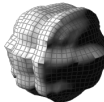
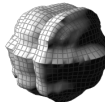
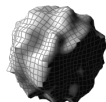
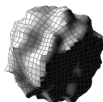
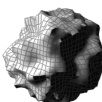
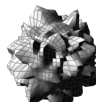
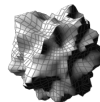
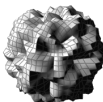
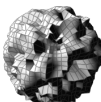
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
basis_sine	mountain::2.0	basis = sine elementsize = 0.25 height = 0.35
basis_perlin	mountain::2.0	basis = perlin elementsize = 0.25 height = 0.35
basis_pperlin	mountain::2.0	basis = pperlin elementsize = 0.25 height = 0.35
basis_simplex	mountain::2.0	elementsize = 0.25 height = 0.35
basis_sparse	mountain::2.0	basis = sparse elementsize = 0.25 height = 0.35
basis_flow	mountain::2.0	basis = flow elementsize = 0.25 height = 0.35
basis_pflow	mountain::2.0	basis = pflow elementsize = 0.25 height = 0.35
basis_worleyFA	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
basis_worleyFB	mountain::2.0	basis = worleyFB elementsize = 0.25 height = 0.35

ノード	タイプ	デフォルトから変更したパラメータ
basis_mworleyFA	mountain::2.0	basis = mworleyFA elementsiz = 0.25 height = 0.35
basis_mworleyFB	mountain::2.0	basis = mworleyFB elementsiz = 0.25 height = 0.35
basis_cworleyFA	mountain::2.0	basis = cworleyFA elementsiz = 0.25 height = 0.35
basis_cworleyFB	mountain::2.0	basis = cworleyFB elementsiz = 0.25 height = 0.35
basis_alligator	mountain::2.0	basis = alligator elementsiz = 0.25 height = 0.35

14種類すべて（細かさの小さい順）

すべて同じカメラ・同じ設定。左上のなめらかなものから、右下の硬く割れたものへ並んでいる。

					
Houdini	Houdini	Houdini	Houdini	Houdini	
基準: ノイズなし 1538点	Sinusoid 平均 0.14454 / 細かさ 0.00444	Sparse Convolution 平均 0.01985 / 細かさ 0.00965	Periodic Perlin Flow 平均 0.01551 / 細かさ 0.01005	Periodic Perlin 平均 0.01662 / 細かさ 0.01062	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Perlin 平均 0.02426 / 細かさ 0.01268	Perlin Flow 平均 0.02639 / 細かさ 0.01441	Simplex (既定) 平均 0.03237 / 細かさ 0.01733	Worley F2-F1 平均 0.14144 / 細かさ 0.01831	Worley F1 平均 0.08017 / 細かさ 0.02065	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Alligator 平均 0.12904 / 細かさ 0.02747	Manhattan F1 平均 0.08103 / 細かさ 0.02996	Chebyshev F2-F1 平均 0.11841 / 細かさ 0.03104	Chebyshev F1 平均 0.09937 / 細かさ 0.03193	Manhattan F2-F1 平均 0.11505 / 細かさ 0.03323	
basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sine	Sinusoid	0.14454	0.15748	0.00444	0.0%

basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sparse	Sparse Convolution	0.01985	0.08378	0.00965	32.9%
pflow	Periodic Perlin Flow	0.01551	0.05039	0.01005	51.2%
pperlin	Periodic Perlin	0.01662	0.05705	0.01062	58.8%
perlin	Perlin	0.02426	0.09921	0.01268	40.9%
flow	Perlin Flow	0.02639	0.10496	0.01441	49.9%
simplex	Simplex (既定)	0.03237	0.13153	0.01733	35.8%
worleyFB	Worley Cellular F2-F1	0.14144	0.17436	0.01831	0.0%
worleyFA	Worley Cellular F1	0.08017	0.17328	0.02065	5.4%
alligator	Alligator	0.12904	0.17456	0.02747	2.3%
mworleyFA	Manhattan Cellular F1	0.08103	0.16473	0.02996	91.1%
cworleyFB	Chebyshev Cellular F2-F1	0.11841	0.17439	0.03104	1.4%
cworleyFA	Chebyshev Cellular F1	0.09937	0.16540	0.03193	94.4%
mworleyFB	Manhattan Cellular F2-F1	0.11505	0.17052	0.03323	2.5%

わかったこと

- basis は振幅と性質の両方を変える。rough

が振幅を保ったまま性質だけを変えたのとは性格が違う。同じ height 0.35 でも、平均のずれは Periodic Perlin Flow の 0.01551 から Sinusoid の 0.14454 まで約9.3倍の開きがある。basis を変えたら height も再調整が必要。

- Sinusoid は例外的な挙動をする。平均のずれが 0.14454 と最大級に大きいのに、隣との食い違いは 0.00444 で全種類中もっとも小さい。外向きの点は 0.0%、つまり全部の点が内側に動いている。画を見ると、丸い4つの膨らみを持つ滑らかな形になっていた。規則的な波なので、細かい凹凸ではなく大きなうねりを作る。地形の大きな起伏には使えるが、ディテールには使えない。

- Perlin 系 (perlin / pperlin / flow / pflow) は外向きの点が 40~59% で、外と内にほぼ均等に動く。平均のずれも 0.0155~0.0264 と控えめ。自然な揺らぎを足したいときの素直な選択肢。

- 既定の Simplex は Perlin 系より振幅も細かさも大きい (平均 0.03237、細かさ 0.01733) が、セル系ほど極端ではない。中間的な位置にある。

- セル系 (Worley / Manhattan / Chebyshev / Alligator) は平均のずれが 0.08~0.14 と大きく、隣との食い違いも上位を占める。岩、割れ、結晶のような硬い見た目はここから作る。

- セル系の中で、外向きの点の割合が極端に分かれた。Manhattan F1 が 91.1%、Chebyshev F1 が 94.4% で、ほぼ全部の点が外に出る。一方 Worley F1 は 5.4%、Alligator は 2.3%、Chebyshev F2-F1 は 1.4% で、ほぼ全部が内側に削られる。

- ・画で確認すると理由が見えた。Worley F1

は、細胞の中心から遠いところが深く削られた侵食岩のような形になる。Manhattan F1 は距離の測り方が軸に沿っているため、軸に平行な面を持つ角ばった塊が外に突き出す形になった。同じ「F1」でも距離の定義が違えば、削る方向と形がまったく変わる。

・ F1 と F2-F1 の違いも外向き率に出る。Manhattan は F1 が 91.1% (外向き) なのに F2-F1 は 2.5% (内向き)。Chebyshev も F1 が 94.4%、F2-F1 が 1.4% で同じ傾向。F2-F1 は細胞の境界線を強調する計算なので、境界以外が下がる = 内向きになる。

・最大のずれはセル系でほぼ頭打ちになっている (0.164~0.175)。height 0.35 に対して 0.5 倍弱で止まっており、何らかの上限が効いている可能性がある。この確認は別の実験にする。

・外向きの点の割合という指標は安く作れて情報量が多かった。ノイズが押すのか削るのかは、実際に作りたいもの (突起か窪みか) に直結する。

次にやること

- ・セル系で最大のずれが 0.175 付近で止まる理由を確かめる。height を上げて止まるのかを測る
- ・grid の平面で同じ14種類を並べる。地形として見たときの向き不向きは球体とは違うはず
- ・外向き率を、突起を作りたいとき／窪みを作りたいときの選択指針としてまとめる

height は純粋な倍率か。gain と bias は何に効くか

2026-09-14 | Houdini 21.0.700 | /obj/height_shape | ノード 10 / 接続 9

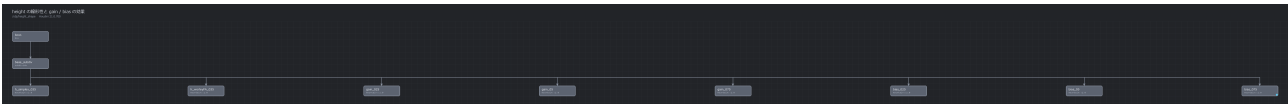
何を作ったか

005でセル系の最大のずれが 0.164~0.175
に揃っていたので、何らかの上限が効いているのではないかと書いた。それを確かめる実験。height を 0.0875 から 1.4 まで16倍の範囲で変え、最大のずれが比例するかを見る。

結論を先に書くと、005の見立ては誤りだった。上限は存在しない。そして高い height で比率が崩れたのは Houdini の制限ではなく、私が使っている指標が壊れていたためだった。

あわせて未調査だった gain / bias / clipmin / clipmax も測った。bias は実用上かなり重要なパラメータだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
h_simplex_035	mountain::2.0	elementsize = 0.25 height = 0.35
h_worleyFA_035	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
gain_025	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.25 height = 0.35
gain_05	mountain::2.0	dogain = 1 elementsize = 0.25 height = 0.35
gain_075	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.75 height = 0.35
bias_025	mountain::2.0	bias = 0.25 dobias = 1 elementsize = 0.25 height = 0.35
bias_05	mountain::2.0	dobias = 1 elementsize = 0.25 height = 0.35
bias_075	mountain::2.0	bias = 0.75 dobias = 1 elementsize = 0.25 height = 0.35

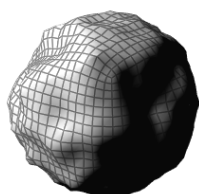
height を変えたときの最大のずれ

固定条件: subdivide 4回 (1538点)、elementsize 0.25。「最大 / height」が一定なら、height は単なる倍率ということになる。

basis	height	平均のずれ	最大のずれ	最大 / height	隣との食い違い
simplex	0.0875	0.00809	0.03289	0.37594	0.00433
simplex	0.1750	0.01619	0.06578	0.37590	0.00867
simplex	0.3500	0.03237	0.13153	0.37579	0.01733
simplex	0.7000	0.06473	0.26273	0.37532	0.03466
simplex	1.4000	0.12822	0.43122	0.30801	0.06868
worleyFA	0.0875	0.02005	0.04332	0.49513	0.00516
worleyFA	0.1750	0.04009	0.08665	0.49512	0.01033
worleyFA	0.3500	0.08017	0.17328	0.49508	0.02065
worleyFA	0.7000	0.16025	0.34636	0.49480	0.04125
worleyFA	1.4000	0.26687	0.42980	0.30700	0.07338

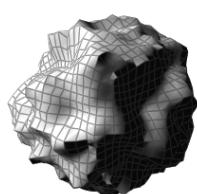
gain — 振幅の倍率。外向きと内向きの比率は変えない

gain は dogain を有効にしないと効かない（既定は無効）。gain = 0.5 のときの数値は、dogain を無効にした既定状態と完全に一致した。つまり 0.5 が中立値。



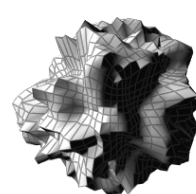
Houdini

gain 0.25 平均 0.01494 / 外向き 35.8%



Houdini

gain 0.50 (中立) 平均 0.03237 / 外向き 35.8%



Houdini

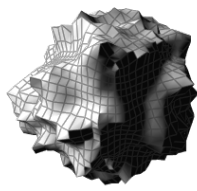
gain 0.75 平均 0.05519 / 外向き 35.8%

gain	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.01494	0.07691	0.00816	35.8%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.05519	0.16402	0.02870	35.8%

bias — 外向きと内向きの比率を直接変える

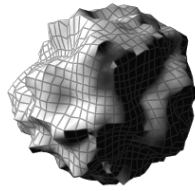
bias も dobias を有効にしないと効かない。0.5 が中立。0.25 にするとほぼ全部が内側に削られ、0.75 にするとほぼ全部が外側に膨らむ。005では basis

を選ぶことでしか得られなかった「突起か窪みか」の制御が、1つのパラメータで連続的にできる。



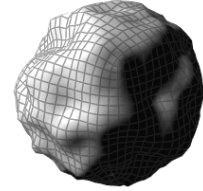
Houdini

bias 0.25 外向き 0.9% =
ほぼ全部が窪み



Houdini

bias 0.50 (中立) 外向き 35.8%



Houdini

bias 0.75 外向き 98.7% =
ほぼ全部が膨らみ

bias	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.09756	0.16935	0.01614	0.9%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.07644	0.14054	0.01156	98.7%

わかったこと

- ・ height は純粋な倍率だった。「最大 / height」は 0.0875 から 0.7

まで8倍の範囲で小数点以下4桁まで一定 (simplex は 0.3759、worleyFA は 0.4951) 。 height を2倍にすれば、ずれもきっちり2倍になる。

- ・ 005で疑った上限は存在しなかった。セル系の最大のずれが揃って見えたのは、セル系ノイズのピーク値の分布が互いに似ているため。実際 005 のセル系7種類の「最大 / height」を計算すると 0.47~0.50 の範囲に収まる。一方 simplex は 0.376

で明確に違う。上限ではなく、ノイズの種類ごとの固有の比率だった。

- ・ ただし height = 1.4 では比率が両方とも 0.307 に落ちた。これは Houdini の制限ではなく、私の指標が壊れているため。基準形の半径は約 0.42 (subdivide 4回のバウンディングボックスの一辺 0.84192 の半分) で、そのとき測れた最大のずれ 0.4312 / 0.4298 はこれとほぼ一致している。中心を貫通するほど強く押し込まれると、「中心からの距離の差」という測り方では値が頭打ちになる。

- ・ つまりこの指標には適用範囲がある。ずれが対象の半径より十分小さいときにしか使えない。今回でいえば height 0.7 まで。005の観察を「上限がある」と解釈しかけたのは、この限界を意識していなかったため。指標を作ったら、それが壊れる条件も押さえておく必要がある。

- ・ clipmin / clipmax の既定は 0.0 / 1.0、dogain / dobias は既定で無効。つまり既定状態ではクリップもゲインもバイアスも効いていない。005までの測定に余計な処理は入っていなかったことが確認できた。

- ・ gain は振幅の倍率として働く。0.25 / 0.5 / 0.75 で平均のずれが 0.01494 / 0.03237 / 0.05519 と増える。一方で外向きの点の割合は 35.8% のまま完全に動かない。形の偏りには触れず、強さだけを変えるパラメータ。

- ・ gain = 0.5 の数値は、dogain を無効にした既定状態と完全に一致した (平均 0.03237、最大 0.13153、細かさ 0.01733) 。0.5 が中立値だと確認できた。

- ・ bias は外向きと内向きの比率を直接変える。0.25 で外向き 0.9% (ほぼ全部が内側に削られる) 、0.5 で 35.8%、0.75 で 98.7% (ほぼ全部が外側に膨らむ) 。画で見ると、0.25 は深くえぐられて鋭い稜線が残る侵食的な形、0.75 は膨らんで浅いへこみが残る塊になる。

- ・ただし bias を 0.5 から動かすと平均のずれも増える (0.03237 → 0.09756 / 0.07644)。分布が片側に寄ると、ゼロから離れた点が増えるため。純粋に形の偏りだけを変えるパラメータではなく、振幅も一緒に動く。

- ・ここまでで mountain のパラメータ地図がほぼ埋まった。振幅を決めるのは height (純粋な倍率) と gain (倍率)。細かさを決めるのは rough (最も強い)、lac、oct (点数が足りている範囲で)。外向きか内向きかを決めるのは bias (連続的) と basis (種類で決まる)。そして何を表現できるかの上限は点数が決める。

次にやること

- ・grid の平面で同じ測定をする。球体では中心からの距離で測れたが、平面では別の基準が必要になるので、指標の作り直しも兼ねる

- ・指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

- ・パラメータ調査はここで区切り、scatter と copy to points に進む

平面で地形を作り、指標を作り直す

2026-09-14 | Houdini 21.0.700 | /obj/grid_terrain | ノード 12 / 接続 8

何を作ったか

これまでの指標は「中心からの距離の差」で、球体でしか使えず、点数をまたいだ比較もできなかった（006でその限界にぶつかった）。平面なら高さの差をそのまま測れる。さらに点の間隔で割れば勾配（傾き）になるので、これなら解像度によらない量になるはずだと考えた。

結果として、その期待は外れた。勾配は解像度を上げるほど増え続ける。ただしこれは失敗ではなく、狙っていたものより役に立つ道具になった。勾配が増えなくなる点が「その形を表現するのに十分な解像度」を教えてくれる。

あわせて、004で「平面ならもっと差が出るかもしれない」と書いた fractal の種類も確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid_20	grid	cols = 20 rows = 20
mt_20	mountain::2.0	elementsize = 2.0
grid_40	grid	cols = 40 rows = 40
mt_40	mountain::2.0	elementsize = 2.0
grid_80	grid	cols = 80 rows = 80
mt_80	mountain::2.0	elementsize = 2.0
grid_160	grid	cols = 160 rows = 160
mt_160	mountain::2.0	elementsize = 2.0
frac_none	mountain::2.0	elementsize = 2.0 fractal = none
frac_fBm	mountain::2.0	elementsize = 2.0 fractal = fBm
frac_mfT	mountain::2.0	elementsize = 2.0 fractal = mfT
frac_hmfT	mountain::2.0	elementsize = 2.0

解像度を変える（elementsize 2.0、height 1.0 固定）

10×10 の grid の分割数を 20 から 160 まで上げた。点の間隔はそれに応じて 0.53 から 0.06 まで細くなる。

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
20	400	0.5263	0.07919	0.29407	0.13595

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
40	1600	0.2564	0.08187	0.32680	0.16634
80	6400	0.1266	0.08183	0.32995	0.18745
160	25600	0.0629	0.08196	0.33513	0.20430

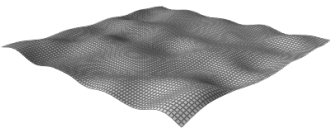
オクターブ数ごとに、勾配がどこで止まるか

各行が1つの解像度。オクターブ数を増やすほど細かい成分が入るので、それを表現しきるのに必要な解像度も上がる。数字が横に並んで止まっていれば、その解像度で足りているということ。

分割数	oct = 1	oct = 2	oct = 4	oct = 8	oct=8 の 80→160 増加率
20	0.12897	0.13673	0.13593	0.13595	—
40	0.14205	0.15845	0.16659	0.16634	—
80	0.14492	0.16561	0.18604	0.18745	—
160	0.14570	0.16752	0.19605	0.20430	+9.0%
収束したか	した (+0.5%)	ほぼした (+1.2%)	まだ (+5.4%)	まだ (+9.0%)	

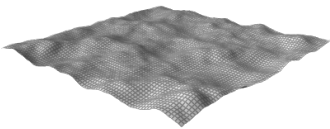
fractal の4種類（分割数80）

004では球体で比べて差が小さかった（細かさ 0.01350~0.01786）。平面では勾配 0.12216 から 0.19064 まで1.6倍の開きが出た。004で立てた「平面ならもっと差が出るかもしれない」という見込みは当たっていた。



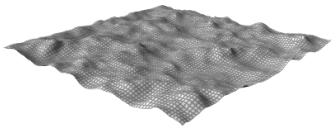
Houdini

None 勾配 0.12216 — 最も滑らか



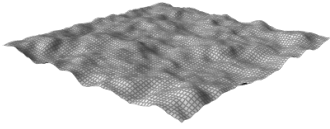
Houdini

Terrain 勾配 0.14766



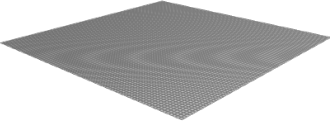
Houdini

Hybrid Terrain（既定） 勾配 0.18745



Houdini

Standard (fBm) 勾配 0.19064 — 最も粗い



Houdini

基準: ノイズなしの grid 6400点 / 6400面

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
None	0.08386	0.39362	0.12216	44.4%
Terrain	0.08257	0.38162	0.14766	43.6%

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
Hybrid Terrain (既定)	0.08183	0.32995	0.18745	42.3%
Standard (fBm)	0.08269	0.37818	0.19064	45.4%

わかったこと

- ・ grid SOP の既定は 10×10 のサイズ、分割は 10×10、向きは ZX 平面（水平）、種類はポリゴン。水平に置かれるので、mountain の押し出しは真上（Y方向）になる。だから高さの差をそのまま測れる。
- ・ 狙いは外れた。勾配は解像度によらない量にはならず、分割数を上げるほど増え続けた（0.13595 → 0.16634 → 0.18745 → 0.20430）。fBm 系のノイズは細かい成分を何段も持っているので、細かく刻むほど急な斜面が新たに見えてくるため。
- ・ ただしこの性質のおかげで、別のことが分かるようになった。オクターブ数ごとに勾配の伸びを見ると、oct = 1 は 80分割で収束し（80→160 で +0.5%）、oct = 2 はほぼ収束（+1.2%）、oct = 4 はまだ伸び（+5.4%）、oct = 8 もまだ伸びている（+9.0%）。つまり勾配が増えなくなったら、その解像度でその形を表現しきれているという判定に使える。
- ・ 低い解像度では余分なオクターブがまったく見えない。20分割では oct = 2 / 4 / 8 がほぼ同じ値（0.13673 / 0.13593 / 0.13595）になり、oct = 2 がわずかに最大という順序の逆転まで起きている。004で球体を使って得た結論が平面でも再現し、しかも解像度との関係まで数字で見えた。
- ・ 高さの平均は40分割以降ほぼ一定（0.0819前後）。振幅は低い解像度でもすぐ正しくなるが、細かさはずっと遅れて収束する。形が正しく見えるのに必要な解像度は、高さが合う解像度よりはるかに高いということ。点数を決めるときは高さではなく細かさを基準にする必要がある。
- ・ 最大の高さも解像度とともに増える（0.29407 → 0.33513）。これも同じ理由で、細かく刻むほどノイズの尖った部分を拾えるようになるため。005・006で使った「最大のずれ」も同じ性質を持っていたことになる。
- ・ fractal の種類は平面で明確に差が出た。滑らかな順に None（0.12216）、Terrain（0.14766）、Hybrid Terrain（0.18745）、Standard fBm（0.19064）。一方で平均の高さはどれも 0.0818~0.0839 でほぼ同じ。fractal は振幅ではなく細かさを変えるパラメータだと確定した。
- ・ None が最も滑らかなのは筋が通っている。層を重ねない単一のノイズなので、細かい成分が入らない。004で「rough = 0 は fractal = None と完全に一致する」と分かったことと整合する。
- ・ カメラのフレーミングを直した。以前はバウンディングボックスを立方体とみなして距離を決めていたため、平たい地形が画面の4分の1ほどにしか写らなかった。8つの隅をカメラ空間に投影して必要な距離を解く方式に変更し、対象が画面いっぱい収まるようになった。地形を扱う以上、これは必須の修正だった。

次にやること

- ・ elementsize・オクターブ数・必要な分割数の関係を式にまとめる。「この設定ならこれだけの点数が必要」と事前に見積もれるようにする
- ・ 地形専用の heightfield ノード群と、この grid + mountain のやり方を比べる
- ・ パラメータの調査はここで区切り、scatter と copy to points に進む

scatter と copy to points — 地面に物を生やす

2026-09-14 | Houdini 21.0.700 | /obj/scatter_copy | ノード 12 / 接続 13

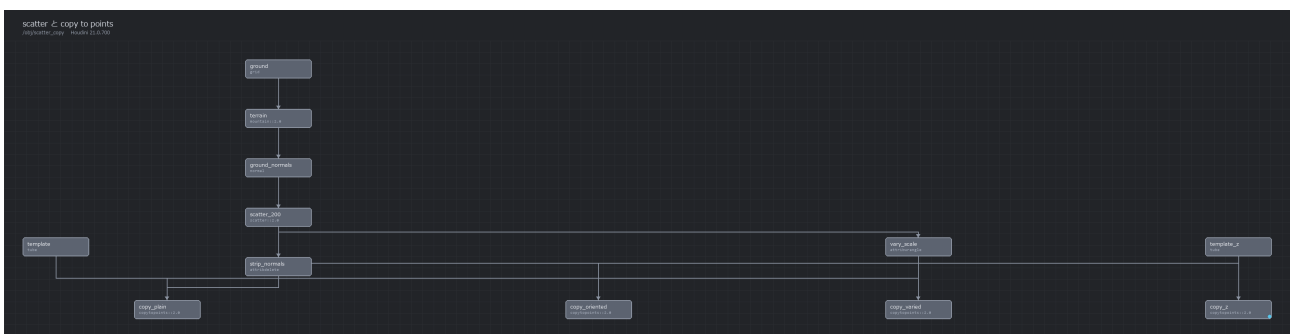
何を作ったか

面の上に点をばらまき、その点ごとにジオメトリを複製する。プロシージャルモデリングの中核パターンで、草や岩や建物を並べるのは全部これで行う。

測ったのは3つ。散布点の数は指定どおりになるか、複製後の点数はどう増えるか、そして複製が地面の傾きに沿って回るかどうか。

3つめで重要な落とし穴が見つかった。複製は確かに回るのだが、回り方が想像と違う。数字で確定させた。

ノードの接続



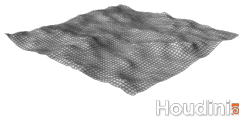
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
ground	grid	cols = 60 rows = 60
terrain	mountain::2.0	elementsizе = 3.0 height = 1.2
ground_normals	normal	(変更なし)
scatter_200	scatter::2.0	npts = 200
template	tube	cap = 1 cols = 10 height = 0.9 rad1 = 0.0 rad2 = 0.18 type = 1
strip_normals	attribdelete	ptdel = N
copy_plain	copytopoints::2.0	(変更なし)
copy_oriented	copytopoints::2.0	(変更なし)
vary_scale	attribwrangle	snippet = @pscale = fit01(rand(@ptnum * 7.3), 0.35, 1.4);
copy_varied	copytopoints::2.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
template_z	tube	cap = 1 cols = 10 height = 0.9 orient = 2 rad1 = 0.0 rad2 = 0.18 type = 1
copy_z	copytopoints::2.0	(変更なし)

複製の向きと大きさ

地面は 10×10 の grid（60分割）に mountain をかけたもの。散布点は200個、テンプレートは20点の円錐。3つの条件で複製した。



地面 grid 60分割 + mountain



法線なしの点に複製
向きのばらつき 0.00度



法線ありの点に複製 + pscale
向きのばらつき 10.84度



テンプレートをZ軸向きにした場合 N と完全に一致する

条件	複製後の点数	複製数	予想点数	向きのばらつき（平均 / 最大）	大きさの幅
法線なし	4000	200	4000	0.00度 / 0.00度	0.45 固定
法線あり	4000	200	4000	10.84度 / 31.69度	0.45 固定
法線あり + pscale	4000	200	4000	10.84度 / 31.69度	0.16 ~ 0.63

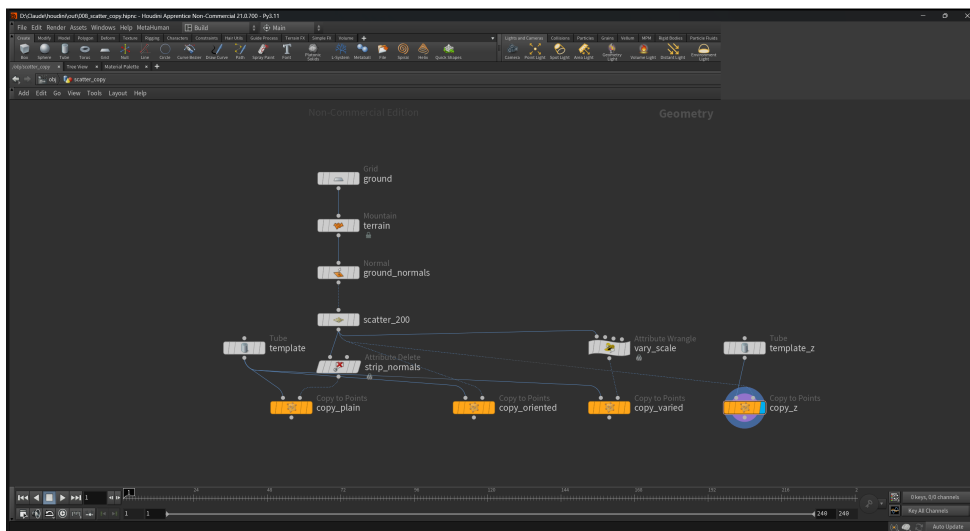
N はテンプレートのどの軸に対応するのか

複製が地面に沿って回ってはいるのに、円錐が横倒しに見えた。そこで、各複製の軸と、その複製が乗っている点の N との角度を全200個について測った。結果はばらつきゼロで、完全に一方の答えを示した。

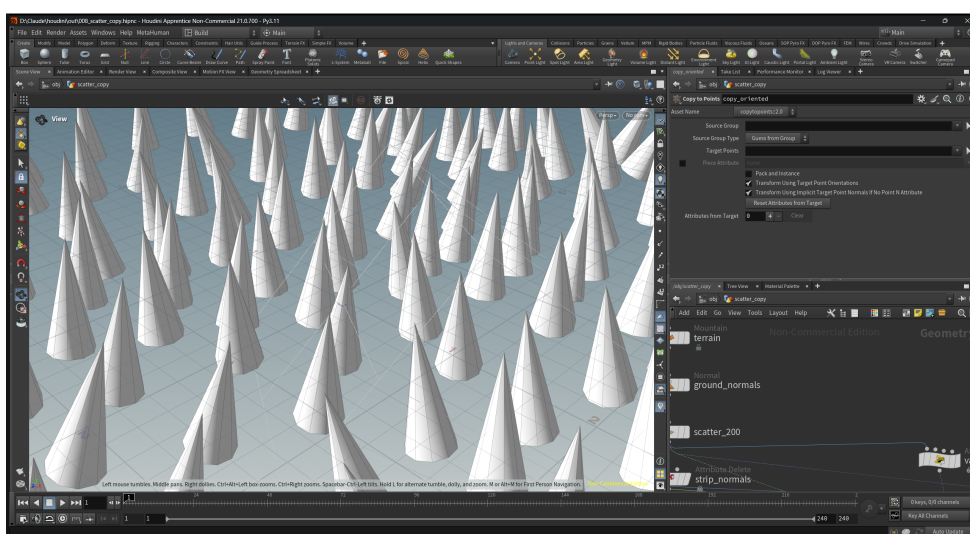
テンプレートの向き	N との角度（平均）	最小	最大
Y軸（tube の既定）	90.00度	90.00度	90.00度
Z軸	0.00度	0.00度	0.00度

実際の Houdini の画面

自動生成の接続図と違い、これは起動中の Houdini を外から操作して撮った本物の画面。自分の Houdini で同じものを開いたときに何が見えるかがそのまま分かる。パラメータ側に写っている「Transform Using Target Point Orientations」が、この実験で測った「N で回転するかどうか」の正体。



ネットワークエディタ。scatter_200 から4つの Copy to Points に分岐している



copy_oriented のパラメータと、その結果のビューポート

scatter の点数は指定どおりか

指定した数	実際の点数	一致
50	50	はい
200	200	はい
800	800	はい

わかったこと

- ・ scatter の npts は正確に守られる。50 / 200 / 800 を指定して、実際の点数も 50 / 200 / 800 になった。およその数ではなく、指定した数ちょうどが出る。
- ・ scatter の出力は P だけでなく N も持っている。元の面の法線が引き継がれるため。この N があるかないかで、次の copy to points の挙動が変わる。
- ・ copy to points の点数は掛け算そのまま。散布点200個 × テンプレート20点 = 4000点。予想と実測が完全に一致した。テンプレートが重いと一気に破綻するので、散布数とテンプレートの点数は掛け算で効くと覚えておく。

- ・ N を持たない点に複製すると、複製は一切回転しない。向きのばらつきは平均も最大も 0.00度。200個すべてが同じ向きに立つ。
- ・ N を持つ点に複製すると、点ごとに回転する。向きのばらつきは平均 10.84度、最大 31.69度で、これは地面の傾きのばらつきそのもの。
- ・ ただし N はテンプレートの Y 軸ではなく Z 軸に対応している。tube の既定 (Y軸向き) のまま複製すると、複製の軸と N の角度が全200個すべて正確に 90.00度、つまり完全に横倒しになる。テンプレートを Z軸向きに作り直すと全200個すべて 0.00度で完全に一致した。ばらつきがゼロなので、これは偶然ではなく仕様。
- ・ tube SOP の orient の既定は Y軸。そのため何も考えずに tube を複製すると必ず横倒しになる。地面に立てたいなら、テンプレートを Z軸向きに作るか、あとで回転させる必要がある。
- ・ pscale は大きさだけを変える。向きのばらつきは 10.84度 / 31.69度 のまま完全に同じで、大きさが 0.45 固定から 0.16~0.63 に広がった。向きと大きさは独立して制御できる。
- ・ tube SOP の type の既定は Primitive で、1点 / 1面の「本物の円柱プリミティブ」になる。ポリゴンとして扱いたいなら Polygon に変える必要がある。これに気づかず、最初は1点のテンプレートを200個複製していた (200点になるだけで何も見えない) 。
- ・ 測り方の失敗も1つ記録しておく。最初は「重心から最も遠い点」で複製の向きを測ろうとしたが、円錐の底面リングの点がすべて等距離のため、選ばれる点が複製ごとにばらついて 21.44度 という無意味な値が出た。固定の点番号を使う方式に変えたら 0.00度 になった。等距離の点が並ぶ形では「最も遠い点」は一意に決まらない。
- ・ 実用の型はこうなる。地面 → normal → scatter → copy to points。テンプレートは Z軸向きに作る。大きさを変えたいなら散布点に pscale を持たせる。この4ノードで「地面に物を生やす」が完成する。

次にやること

- ・ 複製の向きに、傾きだけでなくランダムな回転を混ぜる (up ベクトルの扱い)
- ・ density や面積のアトリビュートで散布密度に偏りをつける
- ・ for-each で複製ごとに違う処理をかける (実験010の予定)

attribute wrangle で VEX を書く ー 何が何回動くのか

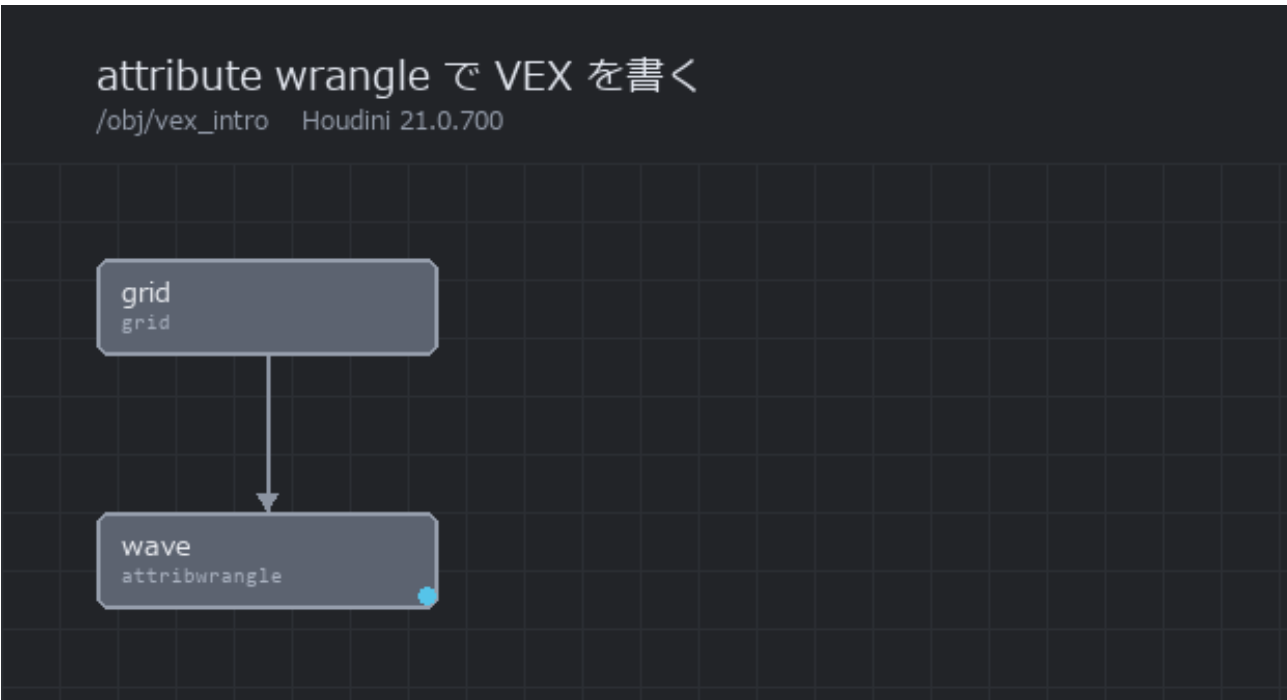
2026-09-14 | Houdini 21.0.700 | /obj/vex_intro | ノード 2 / 接続 1

何を作ったか

VEX は「ノードを並べる代わりに数行書く」ための言語。ただし初心者が最初につまずくのは文法ではなく、そのコードが何に対して何回走るのかという点。それを実測で確かめる。

あわせて「VEXは速い」という話を数字にしようとしたが、ここで測り方を二度も間違えた。最初の方法では計算量を100倍にしても時間が変わらず、公式の性能計測機能を使っても同じだった。三度目でようやく実際の計算を捉えられた。その経緯も残す。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 20 rows = 20
wave	attribwrangle	snippet = @P.y = sin(@P.x * 2.0) * 0.5;

実行対象を変えると、作られるアトリビュートの種類が変わる

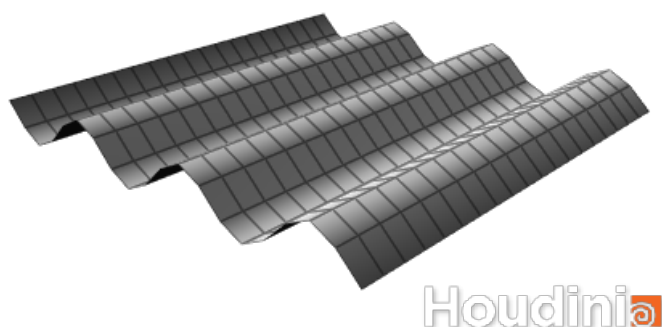
20×20 の grid（ポイント400 / プリミティブ361 / バーテックス1444）に対して、同じコード @marker = 1.0; を実行対象だけ変えて走らせた。

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Detail (only once)	ディテール	1
Primitives	プリミティブ	361
Points (既定)	ポイント	400

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Vertices	バーテックス	1444
Numbers	(作られない)	—

予測してから測る — $@P.y = \sin(@P.x * 2.0) * 0.5$

grid は 10×10 で原点中心なので x は -5 ~ +5。sin の値は -1 ~ +1 の範囲を取るので、y はちょうど -0.5 ~ +0.5 に収まるはず。



VEX 1行で作った波 $@P.y = \sin(@P.x * 2.0) * 0.5;$

	値
予測した範囲	-0.5 ~ +0.5
実測した範囲	-0.499983 ~ +0.499983
差	0.000017

点数と処理時間（毎回新しいノードを作り、5回測った中央値）

1秒あたりの点数が点数とともに増えているのは、ノードを作る固定の費用が大きいため。点が増えるほどその費用が薄まる。

分割数	点数	中央値（ミリ秒）	1秒あたりの点数
200	40,000	1.487	2,690万
400	160,000	2.005	7,979万
800	640,000	2.472	2億5,891万
1600	2,560,000	4.580	5億5,895万

その計測は本物か — 同じ点数で計算量だけ増やす（64万点）

計算量を約100倍にしたときに時間が増えなければ、計測が実際の計算を捉えていないということ。結果の最大yが変わっていることも、確かに別の計算が走った証拠になる。

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
軽い (sin × cos を1回)	2.48	1.0倍	0.49999

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
重い（100回の繰り返し）	43.91	17.7倍	0.32169

わかったこと

- ・ attribute wrangle の「実行対象（class）」は5種類ある。Detail (only once) / Primitives / Points / Vertices / Numbers で、既定は Points。
- ・ 実行対象を変えると、作られるアトリビュートの種類がそれに従って変わる。同じ @marker = 1.0; というコードでも、Points なら点に、Primitives なら面に、Vertices なら頂点に、Detail ならジオメトリ全体に1つだけ作られる。コードは同じでも結果の置き場所が変わるというのが、最初に押さえるべき点。
- ・ Numbers だけはアトリビュートが作られなかった。これはジオメトリの要素ではなく、指定した個数の番号に対して走るモードなので、値を書き込む先がないため。
- ・ 予測を立ててから測る形にした。sin の値域は -1 ~ +1 なので、0.5 倍すれば y は -0.5 ~ +0.5 に収まるはず。実測は -0.499983 ~ +0.499983 だった。
- ・ この 0.000017 のずれも説明できる。grid は20分割なので点の間隔は $10/19 = 0.526316$ 。sin(2x) が最大になるのは $x = \pi/4 \approx 0.785398$ のときだが、最も近い格子点は $x = 0.789474$ で、わずかに外れている。この点での値を計算すると 0.4999834 になり、実測値と小数点以下6桁まで一致する。誤差ではなく、格子がピークをまたいでいるだけだった。
- ・ rand(@ptnum) は完全に再現する。同じコードを別のノードで2回走らせて、全400点が小数点以下9桁まで一致した。乱数といっても点番号から決まる固定の値なので、実行するたびに変わることはない。
- ・ その分布も妥当だった。0~2 の範囲に散らす設定で平均 1.03046（理論値 1.0）、範囲は 0.00103 ~ 1.99050。400点しかないので 3% ほどのずれは想定内。
- ・ 処理時間の計測は二度失敗した。一度目は cook(force=True) の前後を time.perf_counter で挟む方法。これだと計算量を100倍にしても時間が 0.9 倍にしかならなかった。ジオメトリを要求した時点で初めて計算されるため、計測区間の外で処理が走っていた。
- ・ 二度目は Houdini の性能計測機能（hou.perfMon）を使ったが、軽い処理も重い処理も同じ 0.008000 秒を返した。同じノードを焼き直す限り、キャッシュが効いて実際には再計算されないらしい。
- ・ 三度目にようやく成功した。毎回新しいノードを作って初回のクックを測る。キャッシュが存在しないので必ず計算が走る。この方法だと計算量100倍の処理が 17.7倍 の時間になり、結果の最大yも 0.49999 から 0.32169 に変わった。時間と結果の両方が動いたので、今度は本物。
- ・ 得られた数字。64万点に sin × cos を1回かける処理が 2.47ミリ秒、256万点で 4.58ミリ秒。この2点から傾きを出すと、1点あたりの限界コストは約1.1ナノ秒（毎秒約9億点）、固定の費用が約1.8ミリ秒。
- ・ つまり「VEXが速い」効果は点数が多いときにしか出ない。4万点では1.49ミリ秒かかっており、そのほとんどが固定費。少ない点数なら、どんな書き方をしても体感は変わらない。
- ・ 17.7倍という比率が100倍にならないのは、繰り返しの中身が単純で、ノードを作る固定費も含まれているため。それでも計算量に応じて時間が動くことは確認できたので、計測としては成立している。

次にやること

- ・ for-each ループと VEX の使い分け。同じことをどちらでもできる場面で、書きやすさと速さを比べる
- ・ VEX からアトリビュートを読み書きする（point() / setpointattrib()）
- ・ 測り方の失敗が3回続いたので、計測の型を作っておく。「結果が変わること」と「時間が変わること」の両方を確認するのを標準にする

for-each ループと VEX — 同じことをやって比べる

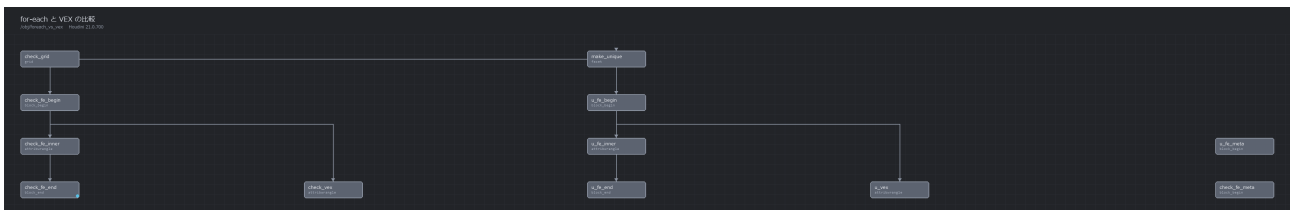
2026-09-14 | Houdini 21.0.700 | /obj/foreach_vs_vex | ノード 12 / 接続 9

何を作ったか

「まとめて処理できないときは for-each」とよく言われるが、VEX でも書ける場面でどれだけ差が出るのかは測らないと分からない。そこで、面ごとに違う高さへ動かすという同じ処理を2通りで作り、結果と速さを比べた。

ところが最初は結果が一致しなかった。その理由を追いかけた結果、この2つは「速い遅い」で選ぶものではなく、そもそも別のことをしているのだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
check_grid	grid	cols = 8 rows = 8
check_fe_begin	block_begin	blockpath = ../check_fe_end method = 1
check_fe_meta	block_begin	blockpath = ../check_fe_end method = 2
check_fe_inner	attribwrangle	snippet = @P.y += detail("op:../check_fe_meta", "iteration", 0) * 0.001;
check_fe_end	block_end	blockpath = ../check_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../check_fe_begin
check_vex	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) { vector p = point(0, "P", pt); p.y += @primnum * ...
make_unique	facet	unique = 1
u_fe_begin	block_begin	blockpath = ../u_fe_end method = 1
u_fe_meta	block_begin	blockpath = ../u_fe_end method = 2
u_fe_inner	attribwrangle	snippet = @P.y += detail("op:../u_fe_meta", "iteration", 0) * 0.001;
u_fe_end	block_end	blockpath = ../u_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../u_fe_begin

分割数	プリミティブ数	for-each (ミリ秒)	VEX (ミリ秒)	何倍
32	961	53.55	1.414	37.9倍
64	3,969	191.87	2.233	85.9倍

結果が一致しなかった理由を追う

同じ処理のはずなのに、49面中48面で高さが違った。しかも差は全部 0.004 で一定。ここから3段階で原因を絞り込んだ。

調べたこと	結果	分かったこと
出力の点数を比べる	元 64点 → for-each 196点 / VEX 64点	for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される
VEX を3回走らせて結果を比べる	3回とも完全に同じ	複数の面が同じ点に書き込む競合ではあるが、順序は決まっていて再現する
点の共有をほどいてから比べる	完全に一致	食い違いの原因は点の共有。ほどけば2つは同じ結果になる

わかったこと

- ・ for-each は Block Begin (method = Extract Piece or Point) と Block End で処理を挟んで作る。Block End の Template Path が空だと「Missing template geometry」で止まる。何を何回繰り返すかはここから決まる。
- ・ 何回目かを知るには、メタデータ用の Block Begin (method = Fetch Metadata) をもう1つ置き、そのディテールアトリビュート iteration を読む。numiterations で総回数も取れる。
- ・ 取り出した中では @primnum が常に 0 になる。1個だけを切り出した状態で処理しているため。「何番目の面か」を使いたいならメタデータを読むしかない。
- ・ detail() のパスには op: が要る。detail("../メタデータノード", "iteration", 0) と書くとファイルパスとして扱われ、エラーも警告も出さずに 0 を返す。ループが動いていないように見えるので原因が分かりにくい。detail("op:../メタデータノード", ...) と書けば動く。実際にこれで1時間相当を溶かした。
- ・ for-each は面の数に比例して遅くなる。49面で11.79ミリ秒、3,969面で191.87ミリ秒。1面あたりおよそ0.05ミリ秒で、面の数がそのまま効く。
- ・ VEX はほとんど変わらない。49面で1.36ミリ秒、3,969面で2.23ミリ秒。実験009で測った「ノード1つあたり約1.8ミリ秒の固定費」がそのまま出ており、処理そのものの時間はまだ見えていない水準。
- ・ その結果、差は面の数とともに広がる。49面で 8.7倍、3,969面で 85.9倍。面が増えるほど for-each を選ぶ代償が大きくなる。
- ・ ただしこの2つは単純に置き換えられるものではない。for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される (64点 → 196点)。これは副作用ではなく、面ごとに独立して動かすために必要なこと。点を共有したままでは、隣り合う面が同じ点を別の場所へ動かそうとして取り合いになる。
- ・ その取り合いは競合ではあるが、結果は毎回同じだった (3回とも一致)。書き込む順序が決まっているため、実行のたびに変わることはない。最初は並列処理による非決定性を疑ったが、それは外れた。
- ・ 点の共有をほどいてから (facet の Unique Points で 64点 → 196点にしてから) 比べると、for-each と VEX の結果は完全に一致した。つまり食い違いの原因は処理内容ではなく、点が共有されているかどうか

かだけだった。

- ・実用的な結論。面ごとに独立した変形をしたいなら、まず点の共有をほどこ。ほどいてしまえば VEX で書ける場面が増え、数十倍速い。for-each が必要なのは、1回分の処理が VEX 1つでは書けないほど複雑なとき（中で複数のノードを通したいときなど）。

- ・測り方についても1つ。最初は面の中心を (x, y, z) でソートして比べようとしたが、比べたい量である y をソートの鍵に含めてしまったため、対応がずれて必ず食い違った。比べたい量は鍵にしない。水平位置 (x, z) を鍵にして y を値にしたら正しく比較できた。

次にやること

- ・facet の Unique Points を挟むことで、for-each が必要だった処理をどこまで VEX に置き換えられるかを試す
- ・for-each の中で複数ノードを通す場合（VEX 1つでは書けない場合）の実例を作る
- ・polyextrude と boolean に進む

crease で角を残す — どれだけの重みが必要なのか

2026-09-14 | Houdini 21.0.700 | /obj/crease_test | ノード 15 / 接続 14

何を作ったか

実験002で「subdivide を重ねると立方体が球になる」と測った。3回分割すると一辺が 1.0 から 0.84918 まで縮む。crease（折り目）を付ければ角が保たれるはずだが、どれだけの重みが必要なのかは分からない。

理屈の上では「重み N なら N 回分の分割まで角が保たれる」はず。3回分割するなら重み3で完全に元の立方体のままになる、という予測を立てて確かめた。

結果は予測どおりだった。ただしこの実験では、2つの指標が違う答えを出すという厄介な状況にも当たった。

ノードの接続

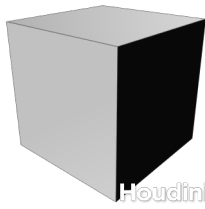


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

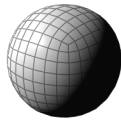
ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
crease_0_0	crease	crease = 0.0 op = 1
sub_0_0	subdivide	iterations = 3
crease_0_5	crease	crease = 0.5 op = 1
sub_0_5	subdivide	iterations = 3
crease_1_0	crease	op = 1
sub_1_0	subdivide	iterations = 3
crease_2_0	crease	crease = 2.0 op = 1
sub_2_0	subdivide	iterations = 3
crease_3_0	crease	crease = 3.0 op = 1
sub_3_0	subdivide	iterations = 3
crease_4_0	crease	crease = 4.0 op = 1
sub_4_0	subdivide	iterations = 3
crease_10_0	crease	crease = 10.0 op = 1
sub_10_0	subdivide	iterations = 3

重みを変えて3回分割する

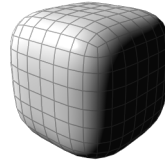
すべて同じカメラ。点数・面数はどの重みでも 386点 / 384面で変わらない。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。



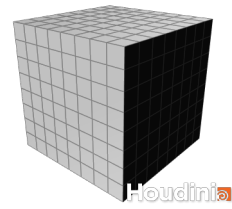
分割前の box 一辺
1.00000 / 折れ角 90.00度



重み 0.0 (crease なし)
一辺 0.84918 / 折れ角
13.93度



重み 1.0 一辺 1.00000 /
折れ角 28.07度

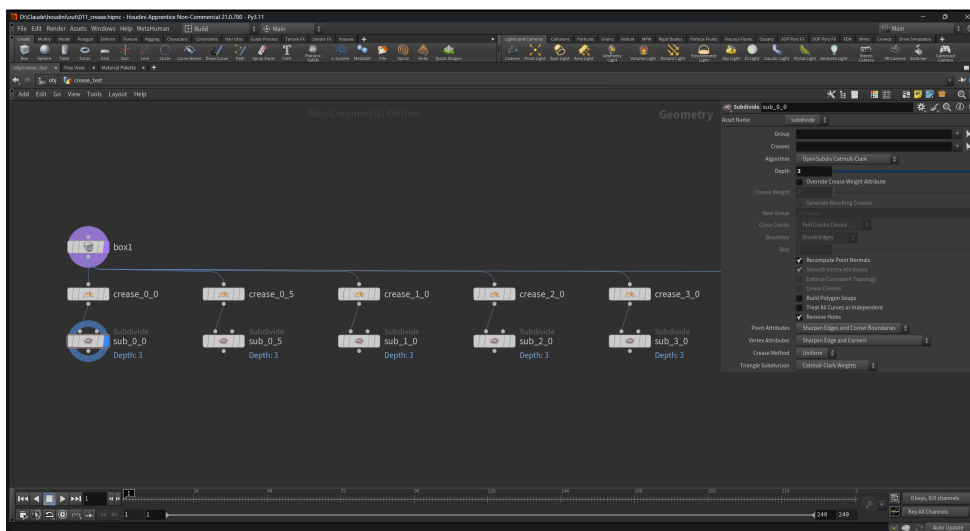


重み 3.0 一辺 1.00000 /
折れ角 90.00度

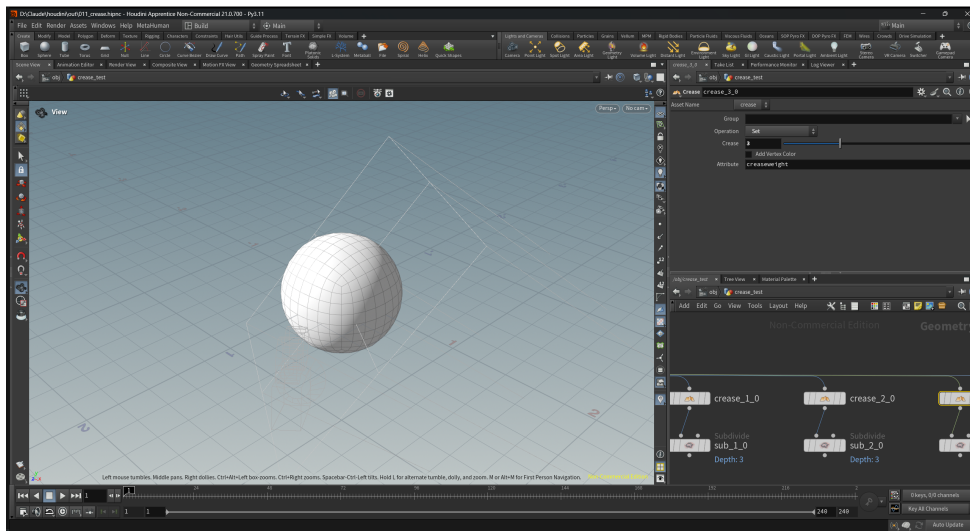
crease の重み	点数	面数	一辺の長さ	最大の折れ角
0.0 (なし)	386	384	0.84918	13.93度
0.5	386	384	0.92459	21.45度
1.0	386	384	1.00000	28.07度
2.0	386	384	1.00000	53.13度
3.0	386	384	1.00000	90.00度
4.0	386	384	1.00000	90.00度
10.0	386	384	1.00000	90.00度
(参考) 分割前の box	8	6	1.00000	90.00度

実際の Houdini の画面

crease SOP で折り目の重みを付け、その下流の subdivide がそれを読む。crease SOP の Crease Attribute の既定は creaseweight で、subdivide はこの名前の属性を自動で見ると。



box から crease と subdivide の組を7通り並べた



crease SOP のパラメータ。Operation を Set にして Crease に重みを入れる

わかったこと

- ・予測は当たった。3回分割する場合、重み 3.0 で折れ角が 90.00度 になり、分割前の box とまったく同じ角の鋭さになる。「重み N なら N 回分の分割まで角が保たれる」という理解でよい。
- ・重みを 3 より上げて何も変わらない (4.0 も 10.0 も 90.00度 のまま)。分割回数を超える重みは無駄。逆に言えば、分割回数を後から増やすつもりなら重みも合わせて上げる必要がある。
- ・点数・面数はどの重みでも 386点 / 384面で変わらなかった。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。角を残すためにポリゴンを足す必要はない。
- ・2つの指標が違う答えを出した。一辺の長さは重み 1.0 で 1.00000 に達し、そこから変わらない。この指標だけを見ていたら「重み1で十分」と判断してしまう。しかし折れ角は 28.07度 で、まだ元の 90度 には程遠い。画を見ると角がはっきり丸い。
- ・つまり一辺の長さは「どこまで縮んだか」しか測っておらず、「角が鋭いかどうか」は測れていない。重み1で外形は元に戻るが、角は丸いまま。何を知りたいかによって、見るべき指標が変わる。
- ・折れ角の指標は、辺を共有する2つの面の法線のなす角の最大値として作った。角が残っていれば大きく、丸まれば小さくなる。分割前の box で 90.00度 と出るので、基準としても正しく働いている。
- ・重み 0.5 のとき一辺は 0.92459 だった。これは crease なし (0.84918) と重み1.0 (1.00000) のちょうど中間で、小数点以下5桁まで一致する。小数の重みは2つの結果を線形に混ぜている。
- ・ただし折れ角のほうは線形ではない。0.0 / 0.5 / 1.0 で 13.93 / 21.45 / 28.07度 となり、中間値の 21.00度 とはわずかにずれる。位置は線形に混ざるが、角度はそうならない。
- ・crease SOP の Operation の既定は「Add To Existing Value」で、既にある値に足し込む。重みを決め打ちしたいなら「Set」にする。既定のまま何度も通すと重みが積み上がる。
- ・crease SOP が書き込む属性名の既定は creaseweight で、subdivide はこの名前を自動で見る。名前を変えたい場合は subdivide 側の Creases にも同じ名前を指定する必要がある。

次にやること

- ・一部の辺だけに crease を付けて、角を残す場所と丸める場所を作り分ける
- ・polyextrude で作った形に crease を付け、建物のような硬い形を作る

- ・折れ角の指標を、面ごとではなく辺ごとに出して分布を見る（どの辺が丸まっているかを特定する）

polyextrude と boolean — 硬い形を作る2つの道具

2026-09-14 | Houdini 21.0.700 | /obj/extrude_boolean | ノード 15 / 接続 20

何を作ったか

ここまではノイズで有機的な形を作ってきた。建物や機械のような硬い形は別の道具で作る。面を押し出す polyextrude と、立体どうしを足し引きする boolean。

押し出しは距離が指定どおりになるのか、点数はどう増えるのか。boolean は演算ごとに何が変わるのか。そして boolean が正しく計算できているかを、体積の関係式で検算した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 5 rows = 5
ext_0_0	polyextrude::2.0	thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_0_5	polyextrude::2.0	dist = 0.5 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_1_0	polyextrude::2.0	dist = 1.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5

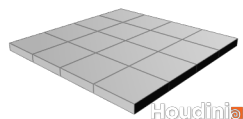
ノード	タイプ	デフォルトから変更したパラメータ
ext_2_0	polyextrude::2.0	dist = 2.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
bool_box	box	scale = 2.0
bool_sphere	sphere	cols = 30 radx = 1.35 radx = 1.35 radz = 1.35 rows = 30 type = 1
bool_union	boolean::2.0	(変更なし)
bool_intersect	boolean::2.0	booleanop = 1
bool_subtract	boolean::2.0	booleanop = 2
bool_shatter	boolean::2.0	booleanop = 3
bool_custom	boolean::2.0	booleanop = 4
bool_seam	boolean::2.0	booleanop = 6
bool_detect	boolean::2.0	booleanop = 8
bool_resolve	boolean::2.0	booleanop = 9

polyextrude の距離

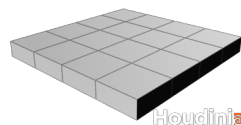
10×10 の grid (5分割 = 25点 / 16面) を押し出した。距離を 0 にすると何も起きず、0 より大きければ点数・面数は距離によらず一定になる。



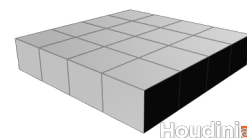
距離 0.0 25点 / 16面



距離 0.5 41点 / 32面



距離 1.0 41点 / 32面



距離 2.0 41点 / 32面

距離	点数	面数	できた高さ	指定との差
0.0	25	16	0.00000	0.00000
0.5	41	32	0.50000	0.00000
1.0	41	32	1.00000	0.00000
2.0	41	32	2.00000	0.00000

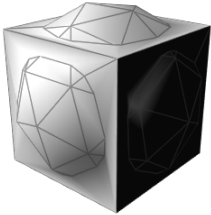
inset (内側への縮み)

距離 1.0 で固定して inset だけ変えた。上面がどれだけ小さくなるかを測っている。元の一辺は 10.0。

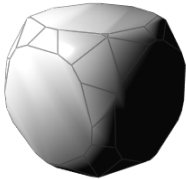
inset	点数	面数	上面の一辺	元との差
0.0	41	32	10.00000	0.0
0.3	41	32	9.40000	0.6 = 0.3 × 2
0.8	41	32	8.40000	1.6 = 0.8 × 2

boolean の演算ごとの結果

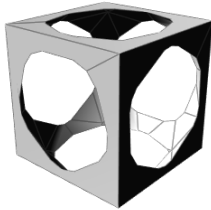
2×2×2 の box（体積 8.00000）と、半径 1.35 のポリゴン球（体積 9.00180）を組み合わせた。



Union（和集合） 体積 9.91613



Intersect（積集合） 体積 7.08567



Subtract（差集合） 体積 0.91433

演算	点数	面数	体積
Union（和集合）	130	120	9.91613
Intersect（積集合）	96	60	7.08567
Subtract（差集合）	104	60	0.91433
Shatter（砕く）	200	120	8.00000
Custom	200	120	8.00000
Seam（境界線）	88	6	0.00000
Detect（検出）	8	6	8.00000
Resolve（重なるの解消）	152	496	2.91612

boolean が正しく計算できているかの検算

集合の大きさには必ず成り立つ関係がある。これを体積で確かめれば、boolean が正しく動いているかを数値で検証できる。

確かめた式	左辺	右辺	差
和集合 + 積集合 = box + 球	17.00180	17.00180	0.0000049
差集合 = box - 積集合	0.91433	0.91433	0.0000000

わかったこと

・ polyextrude の距離は指定どおりに出る。0.5 / 1.0 / 2.0 を指定して、できた高さは 0.50000 / 1.00000 / 2.00000 で差はゼロ。これは当たり前に見えて重要で、ノイズ系のパラメータ（height など）が種類によって実際のずれが9倍も違ったのとは対照的。押し出しは寸法をそのまま指定できる。

- ・点数・面数は距離によらず一定（41点 / 32面）。距離 0 のときだけ何も起きない（25点 / 16面のまま）。形を変えても重さは変わらない。
- ・元の 25点 / 16面 が 41点 / 32面 になる内訳も説明できる。5×5 の grid は境界に 16点あり、押し出すとその 16点が複製されて $25 + 16 = 41$ 点。面は上面 16 + 側面 16 = 32面。
- ・inset は各辺から内側に引っ込む距離。0.3 にすると一辺が両側から 0.3 ずつで 0.6 縮み、10.0 → 9.40000 になった。0.8 なら 1.6 縮んで 8.40000。指定した値がそのまま距離として効く。
- ・パラメータ名は dist と inset。分割数は divisions ではなく divs。ここでも一般的な名前ではなく実際の名前を確認する必要があった。
- ・boolean には8つの演算がある。よく使う Union / Intersect / Subtract のほかに、Shatter（Aを砕いて全部の破片を残す）、Seam（境界線だけを出す）、Detect（検出だけして形は変えない）、Resolve（自分自身の重なりを解消する）がある。
- ・Shatter の体積は 8.000000 で box と完全に一致した。砕いても全部の破片が残るので体積は変わらない。破壊エフェクトの下準備がこれ。
- ・Seam は面数 6 のまま体積 0 になる。境界線だけを取り出す演算なので立体にならない。
- ・体積の関係式で検算できた。集合には「和集合 + 積集合 = 元の2つの合計」という関係が必ず成り立つ。実測すると 17.00180 対 17.00180 で差は 0.0000049。また「差集合 = box - 積集合」も 0.91433 対 0.91433 で差は完全にゼロ。boolean が正しく計算できていることを数値で確認できた。
- ・この検算は、形が正しいかどうかを目で見ずに確かめられる点で価値がある。複雑な boolean の結果が正しいか不安なときは、体積の関係を見れば判断できる。
- ・sphere SOP で rows / cols を 30 に設定したが、面数は 80 のままだった。ポリゴン球は rows / cols ではなく別の分割方法（多面体の細分）を使っている。分割数を上げたいなら別のパラメータを探す必要がある。
- ・その粗い球（80面）の体積は 9.00180 だった。半径 1.35 の真球なら 10.30575 なので、約 13% 小さい。ポリゴンで近似した球は必ず内側に寄るので、体積を当てにするなら分割数を上げる必要がある。

次にやること

- ・polyextrude を繰り返して階層のある建物を作る。for-each との組み合わせ
- ・sphere の分割数を上げる方法を調べ、体積が真球にどう近づくかを測る
- ・エフェクトに入るための下準備（複数フレームの書き出し、シミュレーションのキャッシュ）

時間軸への対応 — フレームを進めて記録する

2026-09-14 | Houdini 21.0.700 | /obj/time_axis | ノード 4 / 接続 2

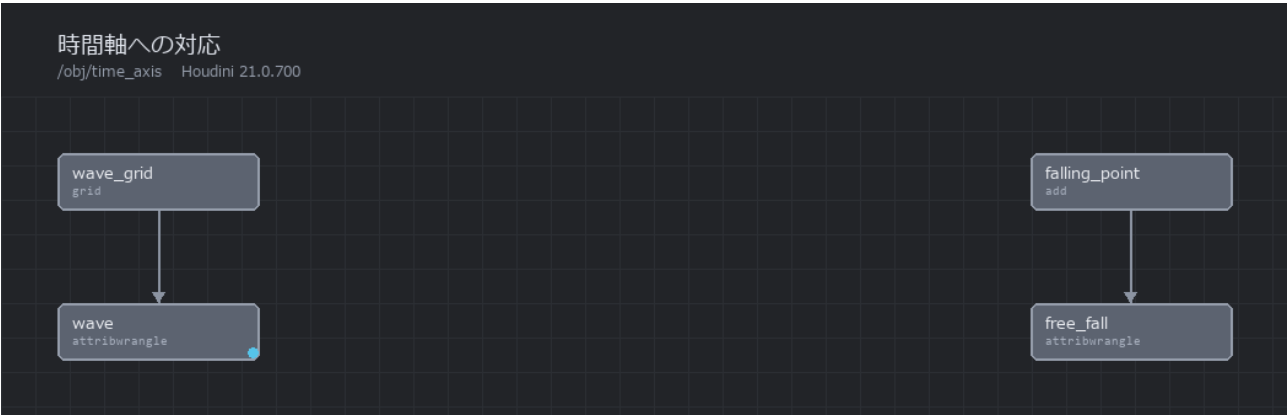
何を作ったか

ここまでの12実験はすべて1フレームだけを見ていた。エフェクトは時間変化そのものが結果なので、フレームを進めて連番で書き出し、1枚のコンタクトシートと動くGIFにまとめる仕組みが要る。

仕組みが正しいかは、答えが分かっている動き（自由落下の式）で確かめた。

あわせて、この先のシミュレーションで使うノードがどこにあるのかを数えた。その結果、当初の予定を変えるべきだと分かった。

ノードの接続

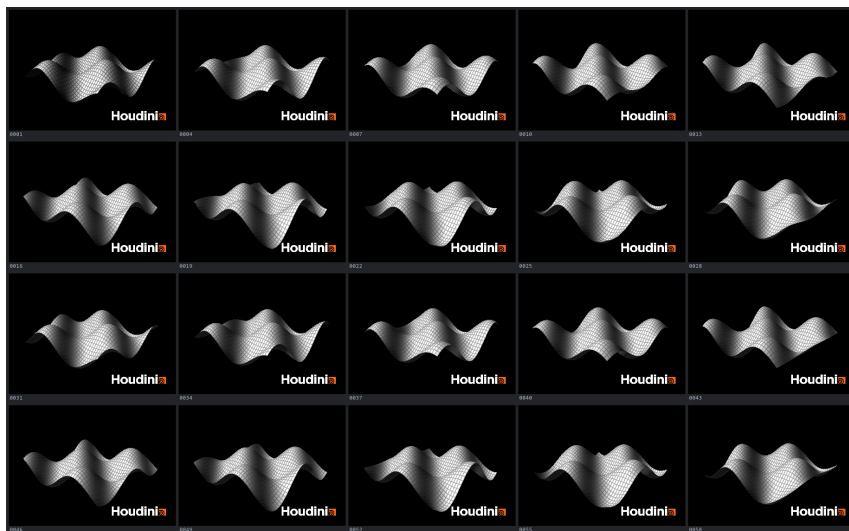


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
wave_grid	grid	cols = 40 rows = 40
wave	attribwrangle	snippet = @P.y = sin(@P.x * 0.8 + @Frame * 0.2) * cos(@P.z * 0.8) * 1.5;
falling_point	add	points = 1
free_fall	attribwrangle	snippet = float t = (@Frame - 1) / 24.0; @P.y = 20.0 - 0.5 * 9.8 * t * t;

連番の書き出し（20コマ）

フレーム1から58まで3コマおき。カメラは全フレームを通して固定してある。動いているのか大きさが変わっているのかを画から判断できるようにするため。



20コマを1枚に並べたコンタクトシート。左下の数字がフレーム番号

検証: 自由落下の式と一致するか

高さ20から重力9.8で落ちる点を作り、理論値 $h = \frac{1}{2}gt^2$ と比べた。24コマで1秒として計算している。

フレーム	経過秒	測った高さ	理論値	差
1	0.0000	20.00000	20.00000	0.0000000
10	0.3750	19.31094	19.31094	0.0000004
20	0.7917	16.92899	16.92899	0.0000002
30	1.2083	12.84566	12.84566	-0.0000005
40	1.6250	7.06094	7.06094	-0.0000006

シミュレーション関連ノードがどこにあるか

SOP は形を作る階層、DOP はシミュレーションの階層。どちらに何種類あるかを数えると、どこから手を付けるべきかが見えてくる。

対象	キーワード	SOP にある数	DOP にある数
パーティクル	pop	0 種	67 種
剛体（破壊）	rbd	47 種	21 種
布	vellum	22 種	7 種
煙・炎	pyro	14 種	3 種
液体	flip	8 種	4 種

わかったこと

- ・連番の書き出しは正しく動いている。自由落下の式との差は最大でも 0.0000006 で、これは小数の計算精度そのもの。フレームを進めて記録する仕組みに間違いはない。
- ・20枚の書き出しに 1.18 秒（1枚あたり 59 ミリ秒）。60フレーム全部でも3.5秒程度で、実用的な速さ。レンダリングの重さは問題にならない。
- ・カメラは全フレームを覆う範囲で一度だけ決める。フレームごとに枠を合わせ直すと、対象が動いているのか大きさが変わっているのかを画から判断できなくなる。そのために、全フレームを走査して範囲を

求める関数を用意した。

- ・出力は2つの形にした。PDFには動きを載せられないのでコンタクトシート（全コマを1枚に並べてフレーム番号を振る）、HTMLにはGIF。同じ連番から両方を作る。

- ・パーティクルは SOP に1つもない（0種）。すべて DOP

側にある（67種）。つまりパーティクルを扱うには DOP

ネットワークを組む必要があり、他の実験のようにSOPをつなぐだけでは始められない。

- ・一方で剛体（RBD）は SOP に 47種もある。布（Vellum）も

22種。SOPレベルのラッパーが充実している分野の方が、入口としては易しい。

当初はパーティクルから始める予定だったが、RBD か Vellum から入る方が理にかなっている。

- ・煙・炎（Pyro）は SOP 14種 / DOP 3種、液体（FLIP）は SOP 8種 / DOP 4種。どちらも SOP 側にラッパーはあるが数は少ない。

- ・ solver SOP

を使って自分で重力を書こうとしたが、これはロックされたアセットで、中にノードを作るには `allowEditingOfContents()` が必要。しかも中身は `dopnet` と4つの `null` で、想像していた「前フレームの結果を受け取って加工する」という単純な構造ではなかった。プログラムから組むには追加の調査が必要

。

- ・そのため今回は、自由落下を数式で書いて仕組みの検証にとどめた。これはシミュレーションではない（前のフレームの結果を使っていない）。本物のシミュレーションは次の実験から。

次にやること

- ・ RBD（剛体）から始める。SOPレベルのノードが47種あり、boolean の Shatter で作った破片をそのまま落とせる

- ・ Vellum（布）も SOPレベルのノードが揃っているので入りやすい

- ・シミュレーションの結果をファイルに保存して使い回す（キャッシュ）。毎回計算し直すと実験が遅くなる

RBD 破壊 — 最初の本物のシミュレーション

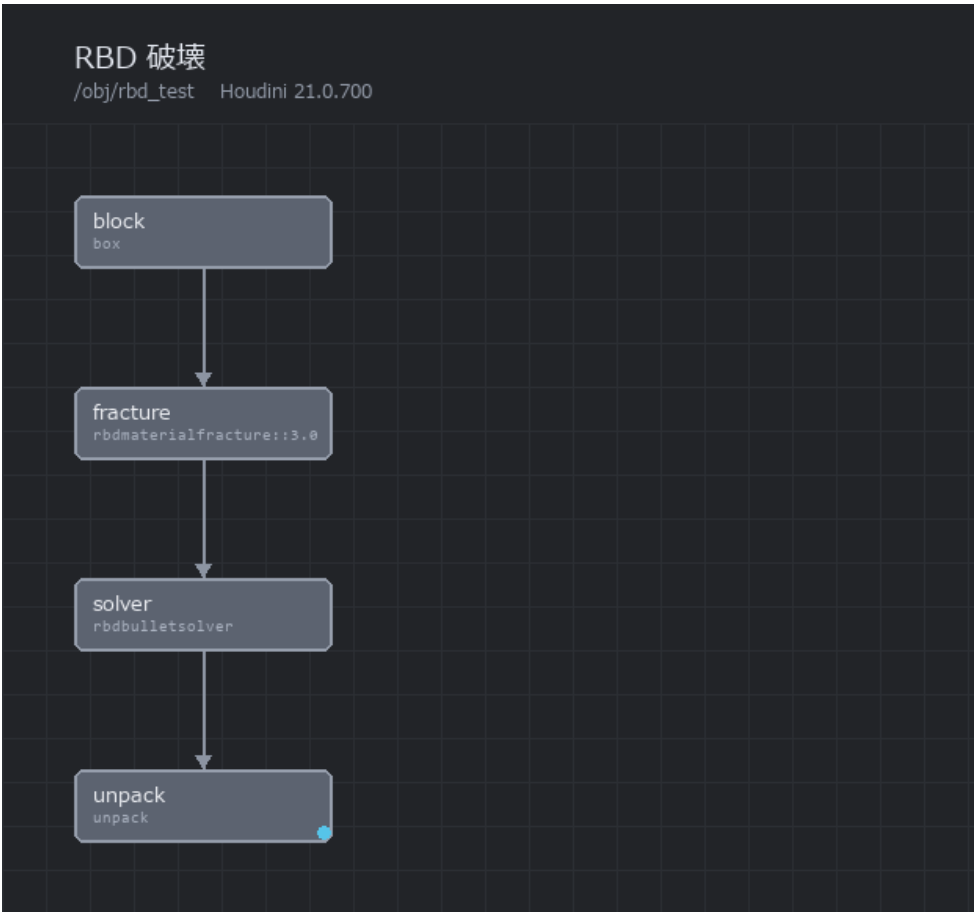
2026-09-14 | Houdini 21.0.700 | /obj/rbd_test | ノード 4 / 接続 3

何を作ったか

実験013で、剛体（RBD）は SOP レベルのノードが47種あり、パーティクルより入口が易しいと分かった。ここまでの実験はすべて「1フレームの計算」だったが、今回は前のフレームの結果を次に渡していく本物のシミュレーション。

シミュレーションは結果が複雑で、目で見ても正しいかどうか判断しづらい。そこで検証の軸を1つ決めた。剛体は変形しないので、砕けても落ちてもぶつかっても、全部の破片の体積の合計は変わらないはず。これが崩れたら計算のどこかがおかしい。

ノードの接続



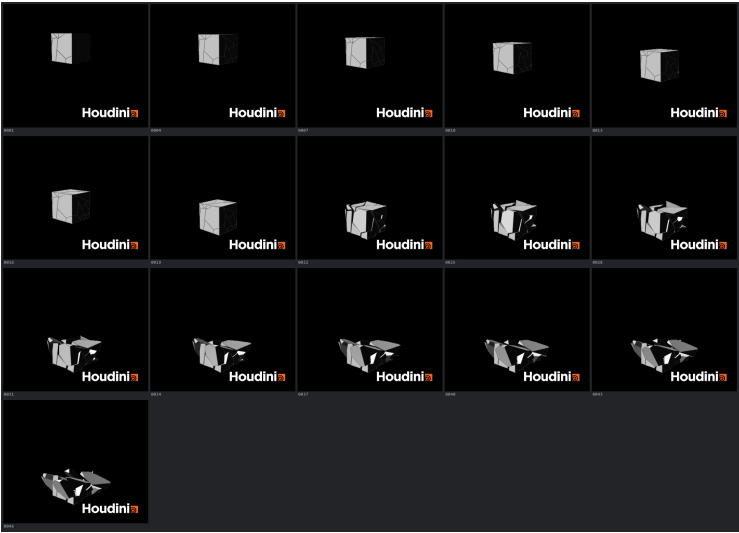
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
block	box	scale = 2.0 ty = 4.0
fracture	rbdmaterialfracture::3.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solver	rbdbulletsolver	folder10 = 1 folder13 = 1 folder21 = 1 folder24 = 1 folder26 = 1 folder26_1 = 1 folder26_2 = 1 folder27 = 1 folder3 = 1 folder34 = 1 folder35 = 1 folder38 = 1 …ほか 11 件
unpack	unpack	(変更なし)

高さ4から落として砕く（16コマ）

2×2×2 の箱を rbdmaterialfracture（コンクリート）で25個の破片に砕き、rbdbulletsolverで落とした。地面はソルバの機能で用意している。



16コマ。落下中は塊のまま、着地して初めて砕ける

フレームごとの状態

「横の広がり」はバウンディングボックスの水平方向の大きさ。元の箱は 2.0
なので、これが増えていけば破片が散らばったということ。

フレーム	体積の合計	最下点	横の広がり	高さ	計算にかかった秒
1	8.00000	3.0000	2.0000	2.0000	0.38
4	8.00000	2.9208	2.0000	2.0000	0.02
10	8.00000	2.3028	2.0000	2.0000	0.03
20	8.00000	−0.0492	2.0589	2.0331	0.05
30	8.00000	−0.0225	3.7483	2.0181	0.03
46	8.00000	−0.0219	5.6408	1.9498	0.05

検証: 体積は保たれているか

調べたこと	結果
元の箱 (2×2×2) の体積	8.00000
砕いた直後の合計	8.00000 (差 0.00000)
全フレームでの振れ幅	0.0000000 (0.0000%)

わかったこと

- ・体積は完全に保たれた。全6フレームで 8.00000 ちょうど、振れ幅は 0.0000000。元の箱の体積 8.0 とともに完全に一致する。砕いても落としてもぶつかっても体積が変わらないので、剛体シミュレーションとして正しく動いている。

- ・これは目で見ても分からない種類の検証。破片が散らばった画を見ても「正しく計算できているか」は判断できないが、体積が保たれていれば少なくとも物質は増減していないと言える。実験012で boolean に使った検算と同じ考え方。

- ・落下中は砕けない。フレーム1から10まで、横の広がり 2.0000 のまま完全に変化しない。砕いてはあるが、破片はくっついたまま1つの塊として落ちる。

- ・着地の瞬間に広がり始める。フレーム20で最下点が -0.0492 になり（地面を少し突き抜ける）、同時に広がりが 2.0589 に増える。その後 30で 3.7483、46で 5.6408。元の 2.8倍まで散らばった。

- ・同時に全体の高さは 2.0000 から 1.9498 に下がる。積み上がっていたものが平たく崩れたということ。

- ・最下点は着地時に -0.0492 まで沈み、その後 -0.0219 で落ち着く。破片は地面を少し突き抜けたまま止まる。ソルバが衝突判定に持たせている余裕（マージン）によるもので、厳密に地面の上に乗るわけではない。

- ・rbdmaterialfracture のコンクリートモードで、2×2×2 の箱が 25個の破片になった（282点 / 191面）。破片の区別は name というプリミティブ属性で管理されている。

- ・計算は速かった。最初のフレームが 0.38秒、以降は 0.02～0.05秒。全部で1秒程度。シミュレーションといっても、この規模なら実験を回すのに支障はない。

- ・rdbbulletsolver の既定は substeps = 10、cacheenabled = 1。1フレームを10回に分けて計算しており、結果は自動でキャッシュされる。2回目以降のフレーム送りが速いのはこのため。

- ・地面はソルバの useground で用意できる。別途ジオメトリを作る必要はない。

次にやること

- ・substeps を減らすと結果がどう変わるか。突き抜けや不安定さが出るはず
- ・破片の数を増やしたときの計算時間の伸び方を測る
- ・Vellum（布）でも同じように、保存されるべき量を見つけて検証する

Vellum クロス — 布で保存されるべき量は何か

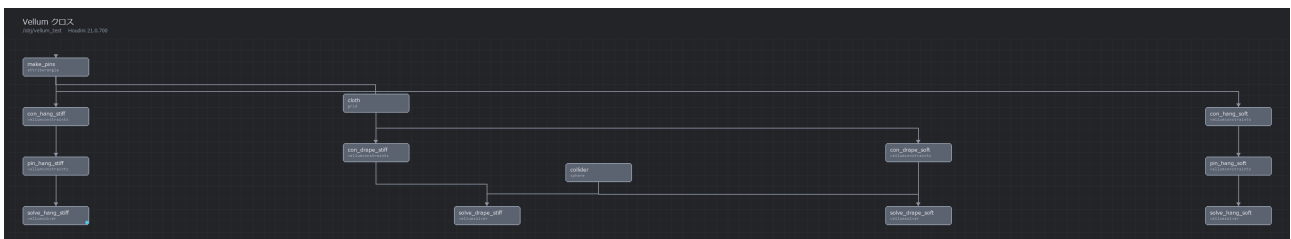
2026-09-14 | Houdini 21.0.700 | /obj/vellum_test | ノード 13 / 接続 19

何を作ったか

剛体では体積が保たれた（実験014）。布は面なので体積では測れない。代わりに「辺の長さの合計」を見る。布は伸びにくい素材なので、たわんでも吊るしても、辺の長さの合計はほぼ変わらないはず。

さらに、伸びにくさ (stretchstiffness) を下げれば保存が崩れるはず。そう考えて試したところ、まったく変化しなかった。原因を追ったら、このパラメータの本当の仕組みが分かった。

ノードの接続



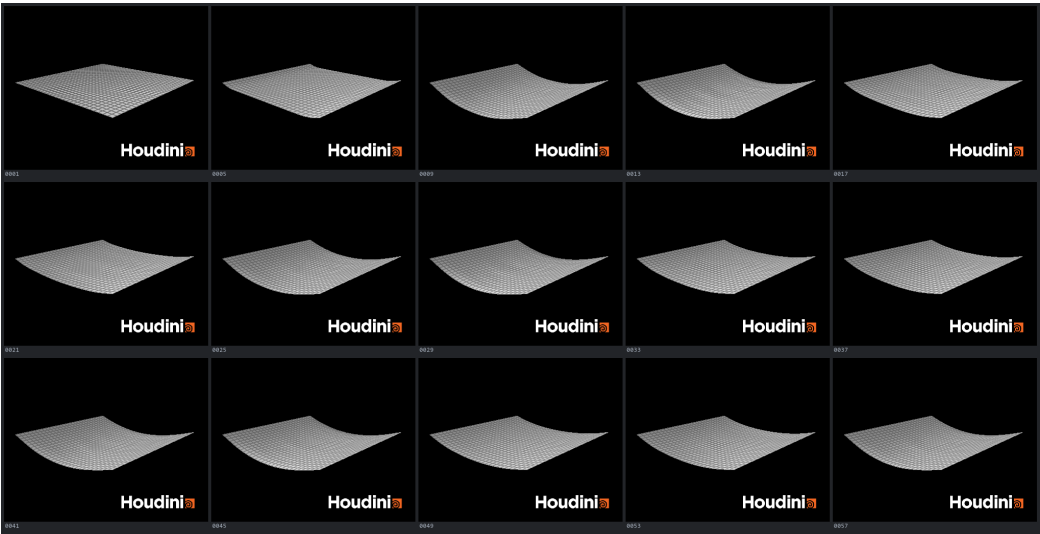
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 30 rows = 30 sizeX = 4.0 sizeY = 4.0 ty = 3.0
collider	sphere	radX = 1.2 radY = 1.2 radZ = 1.2 ty = 1.0 type = 1
make_pins	attribwrangle	snippet = if (abs(@P.x) > 1.9) { @group_pins = 1; }
con_drape_stiff	vellumconstraints	constrainttype = 3
solve_drape_stiff	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_drape_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
solve_drape_soft	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_hang_stiff	vellumconstraints	constrainttype = 3
pin_hang_stiff	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_stiff	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

ノード	タイプ	デフォルトから変更したパラメータ
con_hang_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
pin_hang_soft	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_soft	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

両端をピンで留めて吊るす（15コマ）

4×4 の布（900点 / 841面）の両端をピンで固定し、自重でたわませた。ピンで留めると布に張力がかかるので、伸びにくさの違いが出やすくなる。



15コマ。両端が固定されたまま中央が垂れ下がる

条件ごとの伸び率

「伸び率」は辺の長さの合計を、たわむ前の状態（464.00000）で割った値。1.0 なら一切伸びていない。

条件	最大の伸び率	伸びた割合
たわむ・かたい（stiffness 1.0）	1.00272	0.272%
たわむ・やわらかい（stiffness 0.02）	1.00257	0.257%
吊るす・かたい（stiffness 1.0）	1.00842	0.842%
吊るす・やわらかい（stiffness 0.02）	1.00842	0.842%

かたさを大きく振る（吊るした布、最終フレーム）

stiffness を 50倍変えても結果が同じだったので、指数のパラメータを含めて広く振った。値と指数を入れ替えたペアの結果が完全に一致したので、実効的なかたさは「値 × 10の指数乗」だと確定した。

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1.0	6	10 ⁶	1.00843	2.6042

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1000000.0	0	10^6	1.00843	2.6042
1.0	0	10^0	2.74029	-3.9926
1.0	-6	10^{-6}	3.53114	-5.7265
0.000001	0	10^{-6}	3.53114	-5.7265

わかったこと

- ・点数は全フレームで 900 のまま変わらない。布は形が変わるだけで、点が増減することはない。剛体の体積保存にあたるのが、布ではこの「構造が変わらないこと」。

- ・たわませただけでは、辺の長さの合計はほとんど変わらなかった（最大 0.272% の伸び）。布が自分の重さで垂れるだけでは、張力がほとんどかからないため。

- ・両端をピンで留めて吊ると 0.842% まで増えた。たわむ場合の3.1倍。布を引っ張る力がかかっている分だけ伸びている。指標が張力を捉えられていることが確認できた。

- ・ところが、かたさを 1.0 から 0.02 に下げても結果がまったく変わらなかった。50分の1にしたのに伸び率は 1.00842 で小数点以下5桁まで同じ。

- ・原因は stretchstiffnessexp というパラメータで、既定値が 10 だった。実効的なかたさは「値 × 10の指数乗」なので、既定では 1.0×10^{10} という極端に硬い状態。ここから 0.02 にしても 2×10^8 で、やはり伸びない。

- ・これを確かめるために値と指数を広く振った。(値 1.0, 指数 6) と (値 1000000, 指数 0) の結果が完全に一致し、(値 1.0, 指数 -6) と (値 0.000001, 指数 0) も完全に一致した。掛け算であることが実測で確定した。

- ・指数を 0 まで下げると伸び率 2.74029 (174%の伸び) 、-6 まで下げると 3.53114 (253%の伸び) になり、布というよりゴムのような挙動になる。最下点も 2.60 から -5.73 まで下がる。

- ・実用上の意味は大きい。画面に見えている数字 (1.0) だけを触っても、実際にはほとんど変わらない。効くのは指数のほう。布が硬すぎる・柔らかすぎると感じたなら、値ではなく指数を疑う。

- ・ピンの指定でも1つつまずいた。group type の既定はプリミティブなので、点のグループを渡すなら先に points へ切り替える必要がある。さらに group を空のままにすると「全部が対象」と解釈され、布全体がピン留めされて一切動かなくなる。最下点が全フレーム 3.0000 のまま動かないという分かりやすい症状が出た。

- ・計算は速く、1フレームあたり 0.3~0.8秒。900点の布ならシミュレーションでも実験のテンポは保てる。

次にやること

- ・ bendstiffness にも同じ指数があるか確かめる。あるなら曲がりにくさも同じ罫がある

- ・布と球の衝突で、厚み (thickness) がどう効くかを測る

- ・Pyro (煙) へ進む。ボリウムなので、保存されるべき量がまた変わる

Pyro 煙 — dissipation の正体を式で突き止める

2026-09-14 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

剛体は体積、布は辺の長さが保たれた。煙はボリウム（空間を格子で区切って各マスに値を持たせたもの）なので、測るのは密度の合計になる。

当初は「煙は消えていくので密度は減るはず」と考えたが、実際には増えた。発生源が毎フレーム供給し続けるためで、減衰と供給が釣り合う。そこで「供給と減衰の釣り合い」を式に立てて実測と突き合わせたところ、小数点以下4桁まで一致した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

密度の合計の変化（最初のフレームを1とした比）

半径0.5の球から煙を出し続けた。dissipation は煙が消える速さのパラメータで、既定は 0.1。

フレーム	散逸なし (0.0)	既定 (0.1)	強め (0.3)
1	1.00000	1.00000	1.00000
5	5.00000	4.09489	2.77298
10	10.00000	6.51284	3.23902
20	20.00000	8.78371	3.33052
30	30.00000	9.57551	3.33310
46	46.00000	9.92084	3.33318

立てた式と実測の突き合わせ

供給が毎フレーム一定で、毎フレーム $(1 - d)$ 倍に減るとすれば、 t フレーム後の合計は $(1 - (1-d)^t) / d$ 倍になるはず。この式と実測を比べた。

条件	フレーム	実測の比	式の予測	差
散逸なし	10	10.00000	10.00000	0.000000
散逸なし	46	46.00000	46.00000	0.000000
既定 0.1	5	4.09489	4.09510	-0.000210
既定 0.1	20	8.78371	8.78423	-0.000523
既定 0.1	46	9.92084	9.92145	-0.000608

条件	フレーム	実測の比	式の予測	差
強め 0.3	5	2.77298	2.77310	-0.000120
強め 0.3	46	3.33318	3.33333	-0.000153

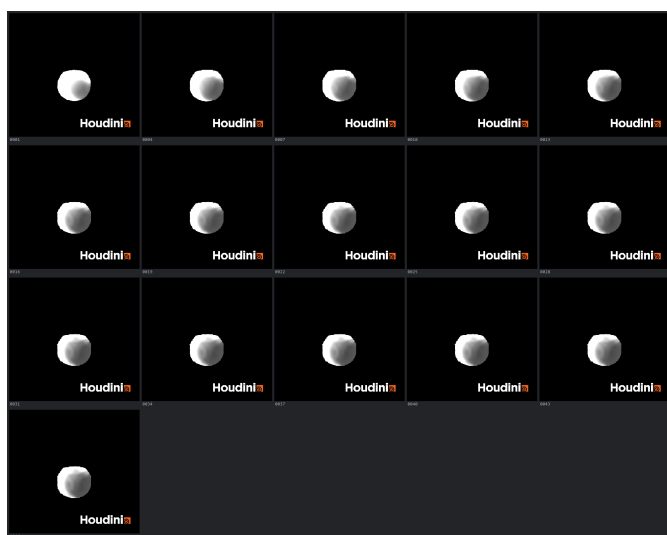
行き着く先（釣り合いの値）

供給と減衰が釣り合うと、密度はある値で止まる。式の上では $1 / \text{dissipation}$ になるはず。

条件	最後の実測	$1 / \text{dissipation}$	差
既定 0.1	9.92084	10.00000	0.079
強め 0.3	3.33318	3.33333	0.00015

見た目（16コマ）

数値ははっきりしているが、画はあまり情報を持っていない。理由は下の「わかったこと」に書いた。



16コマ。ハードウェアレンダラーはポリウムを塊としてしか描けない

わかったこと

- ・当初の予想は外れた。「煙は消えていくので密度は減る」と考えていたが、実際には増えた。発生源が毎フレーム煙を供給し続けるため、減る量より足される量が多い間は増え続ける。
- ・散逸なし（ $\text{dissipation} = 0$ ）のとき、密度の合計はフレーム番号とぴったり同じ比になった。5フレーム目で5.000000倍、46フレーム目で46.000000倍。差は完全にゼロ。毎フレーム同じ量が足され、一切減らないので当然だが、これがきれいにすることでシミュレーションが余計なことをしていないと確認できる。
- ・ dissipation は「毎フレーム $(1 - d)$ 倍に減らす」という意味だと確定した。この仮定から立てた式 $(1 - (1-d)^t) / d$ と実測を比べると、全フレームで差が 0.0006 以下（最大でも 0.006%）。
- ・行き着く先も式のとおり。供給と減衰が釣り合う値は $1 / \text{dissipation}$ で、0.3 のとき理論値 3.33333 に対し実測 3.33318。0.1 のときは 10 に対し 9.92084 で、まだ完全には収束していないが確実に近づいている。
- ・この形の検証は応用が効く。パラメータの意味が分からないときは、仮定を式にして実測と突き合わせればよい。説明を読むより確実に、桁が合っているかまで分かる。実験015で「かたさ = 値 × 10の指数乗」を突き止めたのと同じやり方。

- ・ボリウムの構成でつまづいた点。pyrosource は既定で P と pscale しか作らず、density 属性を明示的に追加しないと次のラスライズが空を返す。そして紛らわしいことに、pyrosource の attributes は「作る属性の個数」（数値）、volumerasterizeattributes の attributes は「属性名を並べた文字列」で、同じ名前なのに意味が違う。

- ・ハードウェアレンダラー（OpenGL ROP）はボリウムの描画に向いていない。煙が塊のようにしか描かれず、濃淡も立ち上りも分からない。これまでの実験ではこのレンダラーで十分だったが、ボリウムを扱うならKarmaのような本式のレンダラーが要る。

- ・計算は速く、1フレームあたり 0.04~0.2秒。ボリウムでもこの規模なら実験のテンポは保てる。

次にやること

- ・Karma でボリウムをレンダリングし、煙らしい見た目を出す。レンダリング時間も測る
- ・buoyancy（浮力）を変えて、立ち上る速さがどう変わるかを測る
- ・FLIP（液体）へ。液体なら体積が保存されるはずなので、また別の検証軸になる

FLIP 液体 — 組み方が分からず保留

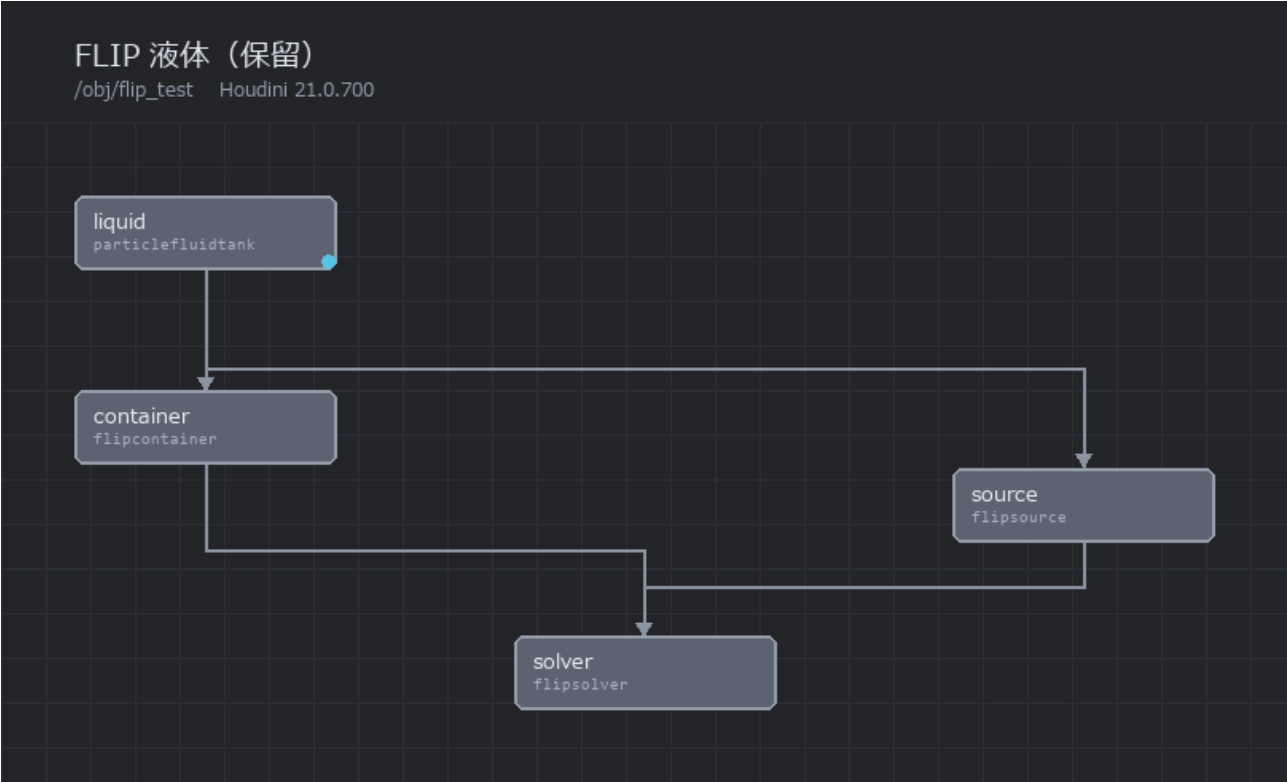
2026-09-14 | Houdini 21.0.700 | /obj/flip_test | ノード 4 / 接続 4

何を作ったか

液体は押し縮められないので体積が保たれるはず。FLIP は液体を粒で表す方式なので、粒1つが一定の体積を担うなら「粒の数」がそのまま体積にあたる。さらに FLIP には reseeding という粒を足したり間引いたりする仕組みがあり、これを切れば粒の数が完全に保たれるはずーという筋書きで実験を組んだ。

ところがソルバが動かなかった。4通りの配線を試したがすべて同じエラーで止まり、原因を特定できていない。ここまでの調査内容を残して保留とする。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
liquid	particlefluidtank	particlesep = 0.15 sizex = 2.0 sizey = 2.0 sizez = 2.0
container	flipcontainer	particlesep = 0.15
source	flipsource	particlesep = 0.15

ノード	タイプ	デフォルトから変更したパラメータ
solver	flipsolver	folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 particlesep = 0.15 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 vis_ramp2cg = 0.5 vis_ramp2pos = 0.4993646740913391 …ほか 5 件

試した配線と結果

すべて flipsolver が「Not enough sources specified.」というエラーを返した。出力2には入力の粒がそのまま出てくるが、出力1（本来の結果）は空のまま。

配線	出力1	出力2	出力3	エラー
タンクの粒 → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1、粒 → 入力2	なし	1460点（素通し）	0点	Not enough sources specified.
コンテナ → 入力1、flipsource → 入力2	なし	1点（素通し）	0点	Not enough sources specified.

分かっていること

調べたこと	結果
particlefluidtank で粒は作れる	2×2×2、間隔0.15 で 1460粒
flipcontainer は単体で動く	出力1が7点、出力2が82点、出力3が2点。エラーなし
flipsource は単体で動く	1点1プリミティブ（ポリウム）。エラーなし
flipsolver の入力数	4つ（input1～input4）
flipsolver の出力数	3つ
source という名前のパラメータ	sourceband（既定 2.0）の1つだけ

わかったこと

- ・エラーメッセージは一貫して「Not enough sources specified.」。ソルバは入力のどこかに「液体の供給源」を期待しているが、粒を渡しても、コンテナを渡しても、flipsource を渡しても満たされなかった。
- ・他のソルバとの違いが大きい。rdbbulletsolver（実験014）も vellumsolver（実験015）も pyrosolver（実験016）も、入力1にジオメトリをつなぐだけで動いた。FLIP だけが追加の作法を要求している。

- ・パラメータ側にも手がかりが少ない。source を含む名前は sourceband ひとつしかなく、「どの入力が供給源か」を指定する仕組みは見当たらない。つまり入力の順番や種類が決まっていて、そこに正しいものを渡す必要があるということ。

- ・途中で1つ罌を踏んだので記録しておく。particlefluidtank を上に移動させたら粒が0になった。このノードは「水位」を基準に中身を満たす仕組みで、タンクを水位より上に動かすと空になる。落下させたいときはタンクを動かすのではなく、地面のほうを下げる。

- ・この保留は失敗ではあるが、記録しておく価値がある。「他と同じやり方が通じないノードがある」と分かったこと自体が情報で、次に取り組むときに同じ4通りを試さずに済む。

- ・なお粒から見積もった体積は 4.92750 で、箱の体積 8.0 の 62%

だった。粒は格子状に並ぶので隙間があり、粒の数 ×

間隔の3乗では体積を正確には表せない。実際に検証に使うなら、この係数を先に求めておく必要がある。

次にやること

- ・Houdini 付属のヘルプ (nodes.zip) で flpsolver

の入力の説明を読む。ネットではなく手元の一次情報から

- ・シェルフツールが作る FLIP のネットワークを GUI で作り、その構造を .hipnc から読み取る

- ・先に Karma でのレンダリングを片付ける。実験016でボリウムが描けないと分かっているため

Karma でのレンダリングー ノイズはサンプル数で本当に減るのか

2026-09-14 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

実験016で煙を描いたとき、ハードウェアレンダラー（OpenGL ROP）では濃淡が分らないと書いた。そこで本式のレンダラーである Karma を使う。ただ「きれいになった」で終わらせず、測れるものを測る。

Karma は1ピクセルの色を、乱数で飛ばした何本もの光線の平均で決める。平均のばらつきは本数の平方根に反比例するはずなので、サンプル数を4倍にすればノイズは半分になるという式が立つ。これを実測で確かめた。

結果、式が当たるのは狭い範囲だけだった。そして016で書いた「ハードウェアレンダラーはボリウムを描けない」も、言い過ぎだったと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

サンプル数を変えたときのノイズ

ノイズの量は、基準画との差ではなく、3枚を連立させて1枚ずつ求めた。基準画にもノイズが残っている
ので、基準画に頼ると結果がその出来に左右されるため。640×360、煙とみなした画素は 21,525 個。

サンプル数	ノイズ	前段との比	式の予測	ずれ
2	18.316	—	—	—
4	11.321	0.6181	0.7071	−12.6%
8	6.516	0.5756	0.7071	−18.6%
16	4.559	0.6997	0.7071	−1.0%
32	3.413	0.7486	0.7071	+5.9%
64	2.640	0.7734	0.7071	+9.4%
128	2.345	0.8883	0.7071	+25.6%

サンプル数	ノイズ	前段との比	式の予測	ずれ
256	2.088	0.8904	0.7071	+25.9%

サンプル数を4倍にしたとき（式なら 0.5000）

8→32 の区間だけが式どおり。その下では式より速く、その上では遅く減る。

区間	実測の比	式との差
2 → 8	0.3558	式より速く減る
4 → 16	0.4028	式より速く減る
8 → 32	0.5238	ほぼ式どおり
16 → 64	0.5789	やや遅い
32 → 128	0.6870	明らかに遅い
64 → 256	0.7909	ほとんど減らない

指定したサンプル数と、実際に使われたサンプル数

2～256 の範囲で時間を直線に当てはめると「固定費 2.26秒 + 1サンプルあたり 7.44ミリ秒」になる。この直線から逆算した実効サンプル数を右端に置いた。指定を64倍にしても、実際には6倍にもなっていない。

指定	実測の時間	直線の予測	実測/予測	実効サンプル
64	3.00 秒	2.74 秒	1.10	99
128	3.09 秒	3.21 秒	0.96	112
256	3.43 秒	4.16 秒	0.82	157
512	4.23 秒	6.07 秒	0.70	264
1024	5.52 秒	9.88 秒	0.56	439
2048	5.94 秒	17.50 秒	0.34	495
4096	6.48 秒	32.73 秒	0.20	567

頭打ちの犯人 — varianceaa_thresh

samplesperpixel を 4096 に固定したまま、打ち切りのしきい値だけを下げた。しきい値を50分の1にすると、実際の仕事量は21.9倍になった。頭打ちはハードウェアの限界ではなく、この設定によるもの。

varianceaa_thresh	時間	実効サンプル	既定との比
0.01（既定）	5.39 秒	420	1.00
0.005	6.29 秒	542	1.29
0.001	8.01 秒	773	1.84
0.0002	70.64 秒	9,191	21.87

同じ煙を2つのレンダラーで — 同じカメラ・同じ露出

「煙の画素」は真っ白でない画素の数。「最も暗い」は、その画素がどこまで濃く描かれたか。

レンダラー	時間	煙の画素	階調	平均の明るさ	ばらつき	最も暗い
OpenGL ROP	0.71 秒	29,242	235 段	101.0	86.16	15
Karma 256サンプル	3.64 秒	21,545	200 段	109.0	52.83	50

煙の中心を横切る線の明るさ

左から右へ等間隔に見たときの値。255 が背景。数字が小さいほど濃い。

レンダラー	縁の手前	縁	中へ	中心	中心	中へ	縁	縁の外
OpenGL ROP	254	224	121	58	42	50	112	219
Karma	255	255	184	109	104	104	189	255

わかったこと

- ・式が当たったのは 8〜32 の区間だけだった。サンプル数を4倍にしたときの比は、8→32 で 0.5238（式は 0.5000）とよく一致する。ところが 64→256 では 0.7909 で、ほとんど減らない。

- ・減らなくなる理由を突き止めた。samplesperpixel

は「ここまで使ってよい」という上限であって、「必ずこれだけ使う」という指定ではない。指定を 64 から 4096 へ64倍にしても、時間から逆算した実効サンプル数は 99 から 567 へ5.7倍にしかならなかった。

- ・犯人は varianceaa_thresh（既定

0.01）だった。これは「このくらい収束したら、そのピクセルはもう打ち切る」というしきい値。4096 を指定したまましきい値を 0.0002 まで下げると、実効サンプル数は 420 から 9,191 へ21.9倍になり、時間も 5.39秒から70.64秒に伸びた。上限ではなく打ち切りの設定が効いていた。

- ・実用的な結論。この場面では 32 サンプルあたりで費用対効果が折り返す。それ以上は、数字を上げるだけでは絵は良くなる。本当に詰めたいときは varianceaa_thresh も一緒に下げる。ただし時間は一気に13倍になる。

- ・時間の内訳も分かった。640×360 で固定費が 2.26 秒、1サンプルあたり 7.44 ミリ秒。2サンプルでは時間のほぼ全部が固定費で、256サンプルでも半分強が固定費だった。軽い絵を何枚も撮るときは、サンプル数より枚数のほうが効く。

- ・実験016に書いた「ハードウェアレンダラーはボリュームを描けない」は言い過ぎだった。同じカメラ・同じ露出で撮り直すと、OpenGL ROP も煙の輪郭と、厚みによる濃淡（中心ほど濃い）を描けている。中心を横切る線を見ると、縁の 254 から中心の 42 までなだらかに変化している。

- ・2つのレンダラーの本当の違いは、光が中を通るかどうかが違った。OpenGL は濃さの分だけ暗くするだけで、どちらから光が当たっているかが絵に出ない。Karma は光源側が明るく、反対側が暗い。Karma のほうが最も暗い画素が 50 と薄いのも、光が内部を通過して抜けてくるため。

- ・速さの差は大きい。OpenGL が 0.71 秒に対し、Karma は 256サンプルで 3.64 秒。5倍。確認のための描画は今後も OpenGL でよく、Karma は見せる絵に使う。

- ・測り方で3回つまずいた。(1) 出力画像には透明度が付いていて、透明部分の色は黒だった。白い背景に見えるのは表示側が白で合成しているため。これに気づかず「白から離れた画素＝煙」というマスクを作り、画面の88%を拾って、ほとんど何も測っていなかった。

- ・ (2) 煙の芯が真っ白 (255) に張り付いていた。張り付いた画素は元の値が分からないので、ノイズを測っても意味がない。ライトを下げ、中間調の画素だけを対象にした。

- ・ (3) OpenGL と Karma でカメラの寄り方が違っていた (margin 1.12 対 0.5)。煙の写る画素数が違って当たり前で、最初の比較は無効だった。実験003以降ずっと守ってきた「全変種を同じカメラで撮る」を、自分で破っていた。

- ・ Apprentice でも Karma

は問題なく動く。出力には右下にウォーターマークが入るので、測る範囲からは外している。

次にやること

- ・ 構造のある煙を作って撮り直す。いまの煙はほぼ一様な球で、レンダラーの差を見せる題材として弱い

- ・ 018で決めた Karma の設定 (denoiser off、varianceaa_thresh、露出) を hou_tools に関数としてまとめる

- ・ 019 として FLIP に再挑戦する。付属ヘルプで flipsolver の入力の作法を読む

FLIP 液体 —

配線が解けた。そして粒の数は体積ではなかった

2026-09-14 | Houdini 21.0.700 | /obj/flip | ノード 5 / 接続 6

何を作ったか

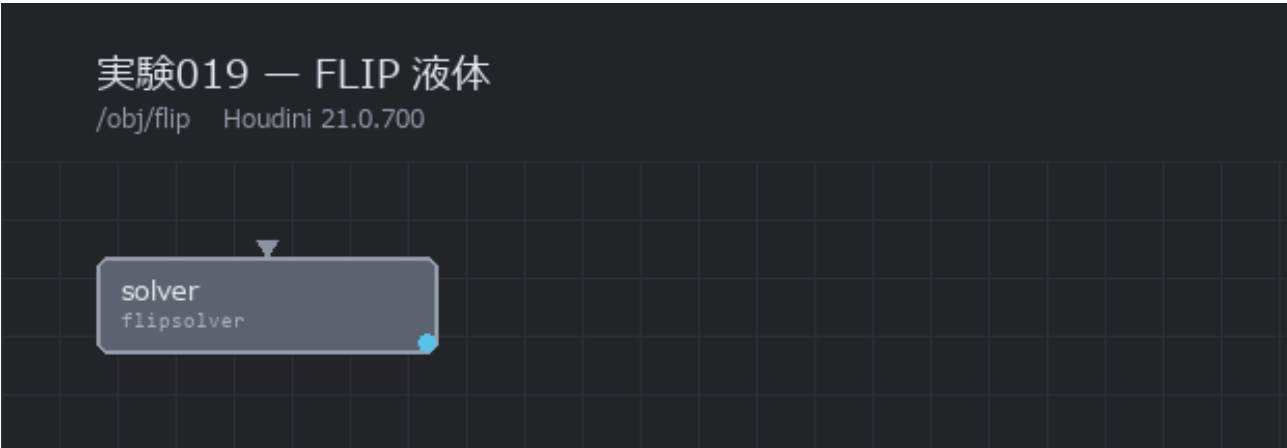
実験017で4通りの配線を試して全部同じエラーで止まり、保留にしていた。付属ヘルプ（nodes.zip）を読んだら原因がはっきりした。

コンテナは単なる箱ではなく、3つのチャンネル（液体の粒・容器・衝突）を束ねる中継点だった。粒はソルバに直接つながるのではなく、いったんコンテナに入れて、コンテナの3出力をソルバの3入力へ渡す。017で試した4通りは、どれもこの形になっていなかった。

さらに調べると、そのチャンネルを流れているのは粒ではなく名前付きのVDBで、source という名前のものだけが液体を湧かせると分かった。

動くようになってから本題に入った。液体は押し縮められないので体積が保たれる。粒1つが一定の体積を担うなら「粒の数」が体積の代わりに使えるはずーという筋書きは、半分外れた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
container	flipcontainer	particlesep = 0.07 sizex = 3.0 sizey = 3.0 sizez = 3.0
blob	box	sizex = 0.9 sizey = 0.9 sizez = 0.9 ty = 1.0
src	flipsource	particlesep = 0.07 volumename = source
merge_sources	merge	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solver	flipsolver	donarrowband = 0 folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 ground_posy = -1.5 particlesep = 0.07 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 …ほか 7 件

017で試した配線と、正解

flipsolver の入力 は 1=Sources（液体の粒）、2=Container、3=Collisions、4=Boundary Flow。ヘルプにそう書いてある。ただしそれだけでは足りず、コンテナ自身が同じ3チャンネルの入出力を持つ中継点であることが鍵だった。

配線	結果
017(a) 粒 → ソルバ入力1	Not enough sources specified.
017(b) コンテナ → ソルバ入力1	Not enough sources specified.
017(c) コンテナ → 入力1、粒 → 入力2	Not enough sources specified.
017(d) コンテナ → 入力1、flipsource → 入力2	Not enough sources specified.
019 粒 → ソルバ入力1、コンテナ → 入力2	エラーは消えたが粒が流れない
019 粒 → コンテナ入力1、コンテナの3出力 → ソルバの3入力	動いた

Sources チャンネルを流れているもの

コンテナの出力1を調べると、点は7つでプリミティブは7つ、すべてVDB（ボリューム）だった。ヘルプには「液体の粒を期待する」と書いてあるが、実際に流れているのは粒ではない。だからタンクの3,145点を入れても何も起きなかった。

VDBの名前	役割の見当
source	液体が湧く場所
sink	液体が消える場所
velsurface	速度を与える面
guidingsurface	流れを誘導する面
pressuresurface	圧力を与える面
vel	速度の場
pressure	圧力の場

VDBの名前を変えると液体は湧くか

同じ箱から flipsource でVDBを作り、名前だけを変えてコンテナの Sources チャンネルに合流させた。flipsource の既定の名前は surface

なので、そのままつないでも何も起きない。

VDBの名前	F1	F5	F10	F20	F40	判定
source	3588	3667	4913	5725	5517	液体が湧く
surface (flipsource の既定)	5	5	5	5	5	何も起きない
notasource	5	5	5	5	5	何も起きない

水位を変えたときの粒の数 — narrow band の有無

コンテナは 3×3×3 で原点中心。水位 −1.0 のとき深さ0.5、+1.0 のとき深さ2.5になる。水の体積は5倍になる。

水位	水の体積	narrow band あり	narrow band なし
−1.00	4.5000	19,973	33,252
−0.50	9.0000	19,081	48,926
0.00	13.5000	19,537	64,517
0.50	18.0000	19,885	79,896
1.00	22.5000	19,484	94,621

narrow band を切ったときの当てはまり

粒の数 = 3415.7 × 体積 + 18130 という直線に当てはめた結果。

水の体積	実測	直線の予測	ずれ
4.500	33,252	33,500.8	−0.74%
9.000	48,926	48,871.6	+0.11%
13.500	64,517	64,242.4	+0.43%
18.000	79,896	79,613.2	+0.36%
22.500	94,621	94,984.0	−0.38%

40フレーム進めたときの粒の数（水位 0.0、narrow band なし）

017では「reseeding を切れば粒の数が完全に保たれるはず」と書いていた。実測は逆だった。

reseeding	F1	F5	F10	F20	F30	F40	最大-最小
あり（既定）	64,517	64,500	64,439	64,421	64,408	64,385	132 (0.20%)
なし	64,517	66,906	69,929	75,904	81,898	87,921	23,404 (36.28%)

わかったこと

・017を止めていた原因は、コンテナの役割の誤解だった。flipcontainer は箱を作るだけのノードではなく、液体の粒・容器・衝突の3チャンネルを束ねて流す中継点になっている。粒はソルバに直接つながるのではなく、コンテナに入れてから、コンテナの3出力をソルバの3入力へ渡す。

- ・そのチャンネルを流れているのは粒ではなく名前付きのVDBだった。コンテナの出力1は7つのVDBで、名前は source / sink / velsurface / guidingsurface / pressuresurface / vel / pressure。タンクの3,145点をいくら入れても無視されたのは当然だった。

- ・名前が役割を決めている。同じ箱から作ったVDBを、名前だけ変えて合流させた。source なら3,588粒が湧き、surface (flipsurface の既定) や notasource では5粒のまま何も起きない。つまり flipsurface をそのままつないでも動かず、名前を source に変える必要がある。

- ・粒の数は体積を表していなかった。水位を動かして水の体積を5倍にしても、粒の数は約19,500でほとんど変わらない (一番多い19,973と一番少ない19,081の差はわずか4.7%)。体積が5倍になっているのに、である。

- ・原因は donarrowband (narrow band、既定で有効) だった。ヘルプに「水位より下の水はボリュームと場で表される」と書いてある。粒は水面付近の帯にしか置かれていないので、深さを増やしても粒は増えない。

- ・narrow band を切ると、粒の数は体積にきれいに比例した。粒の数 = $3415.7 \times \text{体積} + 18130$ で、5点すべてずれが1%未満 (最大 0.74%)。

- ・切片の 18,130 は、narrow band ありのときの粒の数 (平均 19,592) に近い。体積に比例しない分が、ちょうど帯の分に相当する。2つの測定が別々の道から同じ量を指している。

- ・体積が増えた分の粒1つが担う体積は 0.000293 で、これは粒の間隔の3乗 ($\text{sep}^3 = 0.000512$) の 0.572倍。粒は間隔の立方格子より密に詰まっている。017で「粒の数 × 間隔の3乗」では体積を表せないと書いたが、係数はこの 0.572 だった。

- ・017で立てた予想は逆だった。「reseeding を切れば粒の数が完全に保たれるはず」と書いていたが、実測では reseeding ありのほうが保たれる (40フレームで 0.20% しか動かない)。切ると36.28%も増える。

- ・理由はヘルプに書いてある。境界では水位を保つために粒が絶えず作られ、また消されている。reseeding はその「消す」側を担っているので、切ると作られた分が溜まり続ける。reseeding は粒を増やす仕組みではなく、数を一定に保つ仕組みだった。

- ・床を置かないと粒は容器の下から抜けて消える。落下する水を撮ったとき、粒の数は湧く量と消える量が釣り合って約3,400で落ち着いた。床 (useground) を入れると水が溜まり、約4,450で落ち着いた。

- ・ヘルプを読むのが一番速かった。017ではネットも配線の総当たりも試して進まなかった。付属の nodes.zip に入力の意味が書いてあり、それを読んだ時点でほぼ解けた。手元の一次情報を先に当たる。

次にやること

- ・particlefluidsurface で粒に表面を張る。粒のままだと液体に見えない
- ・衝突するもの (Collisions チャンネル) を入れて、物にぶつかる水を作る
- ・020 として POP パーティクルに進む

POP パーティクル — 生まれる数と落ち方を式で確かめる

2026-09-14 | Houdini 21.0.700 | /obj/pop_test/popnet | ノード 5 / 接続 3

何を作ったか

実験013で「パーティクルはSOPに0種、DOPに67種」と分かってから保留にしていた題材。SOPだけでは組めず、DOPネットワークを作る必要がある。

配線は019と同じやり方で、総当たりではなく役割から組んだ。popobject（粒の入れ物）をソルバの入力1へ、popsource（粒を生む部品）を入力2へ、popforce（力を加える部品）を入力3へ。結果はdopimport でSOP側に戻す。

測ったのは2つ。毎秒200個と指定したら本当に毎秒200個生まれるのか。そして初速0で落ちる粒は $y = y_0 - \frac{1}{2}gt^2$ に従うのか。

1つ目はぴたりと合った。2つ目は合わなかったので、合わない分の正体を突き止めた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
output	output	(変更なし)
particles	popobject	(変更なし)
solver	popsolver::2.0	(変更なし)
source	popsource::2.0	constantrate = 200.0 impulseactiveate = 0.0 soppath = /obj/pop_test/emitter
gravity	popforce	forcey = -9.80665

生まれる数 — 毎秒200個と指定したとき

24fps なので1フレームあたり $200/24 = 8.333$

個のはず。フレーム1に9個の初期分があるので、そのぶん全体が上に平行移動する。

フレーム	経過秒	粒の数	式の予測	差
1	0.0000	9	0.00	+9.00

フレーム	経過秒	粒の数	式の予測	差
5	0.1667	44	33.33	+10.67
10	0.3750	87	75.00	+12.00
20	0.7917	169	158.33	+10.67
40	1.6250	336	325.00	+11.00
60	2.4583	501	491.67	+9.33

落ち方 — 最も古い粒の高さ

$y_0 = 3.0$ 、 $g = 9.80665$ 。粒の age アトリビュートをそのまま t に入れて計算した。

フレーム	齢(秒)	実測の高さ	式の予測	差
10	0.4509	2.23386	2.00295	+0.23091
20	0.8676	-0.23483	-0.69089	+0.45606
30	1.2843	-4.40606	-5.08727	+0.68121
40	1.7009	-10.27984	-11.18619	+0.90635
50	2.1176	-17.85616	-18.98766	+1.13150
60	2.5343	-27.13502	-28.49169	+1.35667

ずれを「時間の原点のずれ」と置き直す

$y = y_0 - \frac{1}{2} g (t - \delta)^2$ と置いて δ を逆算した。実際に落ちていた時間は $\sqrt{(2 \times \text{落下距離}/g)}$ で求まる。

フレーム	齢	実際に落ちた時間	δ
10	0.4509	0.39528	0.05565
20	0.8676	0.81223	0.05537
30	1.2843	1.22899	0.05528
40	1.7009	1.64570	0.05523
50	2.1176	2.06239	0.05521
60	2.5343	2.47908	0.05519

δ は計算の刻みで変わるか

DOPネットワークの substeps を変えて、1刻みの長さと δ を比べた。substeps を2倍にすると1刻みは半分になる。

substeps	δ の平均	1刻みの長さ(秒)	$\delta / 1\text{刻み}$
1	0.05532	0.041667	1.328
2	0.02761	0.020833	1.325
4	0.01379	0.010417	1.324
8	0.00669	0.005208	1.284

わかったこと

・配線は役割から組んだ。popobject は入力なし・出力1つ、popsolver は入力3つ、popsurface と popforce も入力なし・出力1つ。ヘルプに「ソルバの緑の入力に POP の部品を挿す」と書いてあるので、入れ物を入力1、部品を入力2以降に挿した。一度で動いた。

・止まったのは結果の取り出し側だった。シミュレーション自体は最初から動いていたのに、SOP側で0点と出ていた。原因は dopimport の objpattern (読み込む対象の名前) が空だったこと。* を入れたら読めた。エラーも警告も出ないので気づきにくい。

・生まれる数は指定どおりだった。直線に当てはめると毎秒 199.96 個。指定は 200 個なのでずれは 0.02%。切片の +10.18 は、フレーム1で一度に出る初期分。

・落ち方は式と合わなかった。実測はいつも「まだ落ちていない」側にずれ、差は齢が進むほど大きくなった (0.23 → 1.36)。ただし比率で見ると逆に小さくなっていく (-23% → -4.3%) ので、単純な倍率のずれではない。

・ずれ方の形から、速度の違いではなく時間の原点のずれを疑った。 $y = y_0 - \frac{1}{2}g(t - \delta)^2$ と置いて δ を逆算すると、6点すべてで 0.0552~0.0557 (幅 0.83%) と一定だった。ばらばらなら仮定が間違い、一定なら正しい。

・ δ の正体は計算の刻みだった。substeps を 1→2→4→8 と増やすと δ は 0.05532 → 0.02761 → 0.01379 → 0.00669 と半分ずつになる。1刻みの長さで割ると 1.28~1.33 でほぼ一定。つまり粒は、齢より約1.3刻みぶん遅れて落ち始めている。

・これは物理が間違っているのではなく、時間を刻んで計算していることから来るずれ。刻みを細かくすれば小さくなる。substeps 8 ならずれは 0.0067秒 (0.16フレーム相当) まで下がる。

・実用面では、粒の位置を式で予測したいときは substeps を上げる。見た目だけなら既定で問題ない。1.3刻みという値は覚えておくと、ずれの大きさを見積もれる。

・ツマミが効いていることの確認もした。popforce の力を0にすると、最も古い粒の高さは60フレーム経っても 3.00000 のまま動かない。重力ありでは -27.13502 まで落ちている。

次にやること

・021 として、構造のある煙を作る。018でレンダラーの差を見せる題材として弱いと分かったため

・022 として、液体に表面を張る (particlefluidsurface)

・粒に衝突を入れる。床で跳ねる・転がるところまで

煙はなぜ動かなかったのか — 温度を入れると立ち上る

2026-09-14 | Houdini 21.0.700 | /obj/smoke | ノード 4 / 接続 3

何を作ったか

実験018で「いまの煙は一様な球で、レンダラーの差を見せる題材として弱い」と書いた。形のある煙を作ろうとして測ったら、もっと手前に問題があった。

煙がまったく動いていなかった。中身のあるボクセルの数が全フレームで504のまま、ばらつきも1.4927のまま。合計だけが増えていく。これは「同じ形が濃くなっていくだけ」を意味する。昇りも広がりもしていない。実験016の煙も、実は同じだった。

浮力は温度で決まる。density だけを供給して temperature を供給していないので、昇る理由がなかった。temperature を足したら、格子が [16,32,16] から [48,144,48] に伸び、中身のあるボクセルが28倍になった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rad y = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance = 2.0 disturbance_controlramp2pos = 1.0 …ほか 101 件

density だけを供給したとき

「中身」は密度が0より大きいボクセルの数。「ばらつき」は変動係数（標準偏差÷平均）で、全体を何倍しても変わらない量。形が変われば動くが、濃さが変わるだけなら動かない。

フレーム	密度の合計	中身のあるボクセル	重心の高さ	ばらつき	格子
5	17,379.98	504	0.4677	1.4927	16×32×16
15	33,702.64	504	0.4677	1.4927	16×32×16
25	39,394.00	504	0.4677	1.4927	16×32×16
40	41,813.50	504	0.4677	1.4927	16×32×16

temperature も供給したとき

同じ組み方で、供給する属性を1つ足しただけ。

フレーム	密度の合計	中身のあるボクセル	重心の高さ	ばらつき	格子
5	18,356.35	1,072	0.5718	1.6673	48×32×48
15	33,901.98	5,851	0.5109	1.9247	48×64×48
25	41,407.58	14,006	0.5148	2.7519	48×96×48
40	48,289.94	30,105	0.5217	3.7187	48×144×48

かき乱す設定は効いたか（40フレーム目）

disturbance と shredding を、温度ありの状態に足して比べた。

条件	密度の合計	中身	ばらつき	温度ありとの差
温度あり（基準）	48,289.94	30,105	3.7187	—
disturbance 2.0	52,247.42	30,350	3.5663	合計 +8.2%
shredding 1.0	48,289.94	30,105	3.7187	完全に同じ
shredding 20.0	48,289.94	30,105	3.7187	完全に同じ

わかったこと

- ・煙は動いていなかった。density だけを供給した場合、中身のあるボクセルは全フレームで 504個、ばらつきは 1.4927、重心の高さは 0.4677。どれも1桁も動かない。合計だけが増えていく。つまり同じ形が濃くなっていくだけで、昇りも広がりもしていない。

- ・格子の大きさも 16×32×16 のまま変わらなかった。煙が広がれば計算領域も広がるはずなので、これも「動いていない」ことの裏付けになる。

- ・原因は温度を供給していなかったこと。浮力は温度差で生まれる。密度だけ渡しても、昇る理由がない。pyrosource で属性を2つにして temperature を足し、volumerasterizeattributes にも density temperature と書いたら動き出した。

- ・temperature を足した結果、格子は [48,32,48] から [48,144,48] へ縦に4.5倍伸び、中身のあるボクセルは 1,072 から 30,105 へ28倍になった。ばらつきも 1.6673 から 3.7187 へ 2.2倍で、形ができています。

- ・実験016と018の煙も同じ状態だった。016で測った散逸の式（毎フレーム (1-d) 倍）は密度の増減だけの話なので、結論は変わらない。018のノイズの式もレンダー側の話なので変わらない。ただしどちらも「動かない煙」を撮っていたので、絵が地味だったのはこれが理由。

- ・かき乱す設定も試した。disturbance は効いた。合計が 8.2% 増え、ばらつきは少し下がる (3.7187 → 3.5663)。細かくばらけさせる設定なので、ばらつきが下がるのは筋が通る。ただし合計が動いたので「動かすだけ」ではない。私の最初の仮説は外れた。

- ・shredding はまったく効かなかった。1.0 でも 20.0 でも、数値が小数点以下まで基準と完全に一致する。付属ヘルプには「温度の勾配にもとづいて速度場を押し引きし、炎に特有の筋や裂け目を作る」とある。炎（燃焼）を伴わない煙だけの設定では出番がない、というのが今のところの読み。ここは確かめきれていないので、炎を出す設定で改めて試す。

- ・ヘルプには「温度の拡散が小さいほど効果が出やすい」とも書いてあったが、tempdiffusion は既定で 0.0、つまりすでに最小だった。この線では説明がつかない。

次にやること

- ・炎（燃焼）を出す設定にして、shredding が効くかどうかを確かめる
- ・この煙で Karma を撮り直す。実験018は動かない煙で比べていた
- ・022 として、液体に表面を張る (particlefluidsurface)

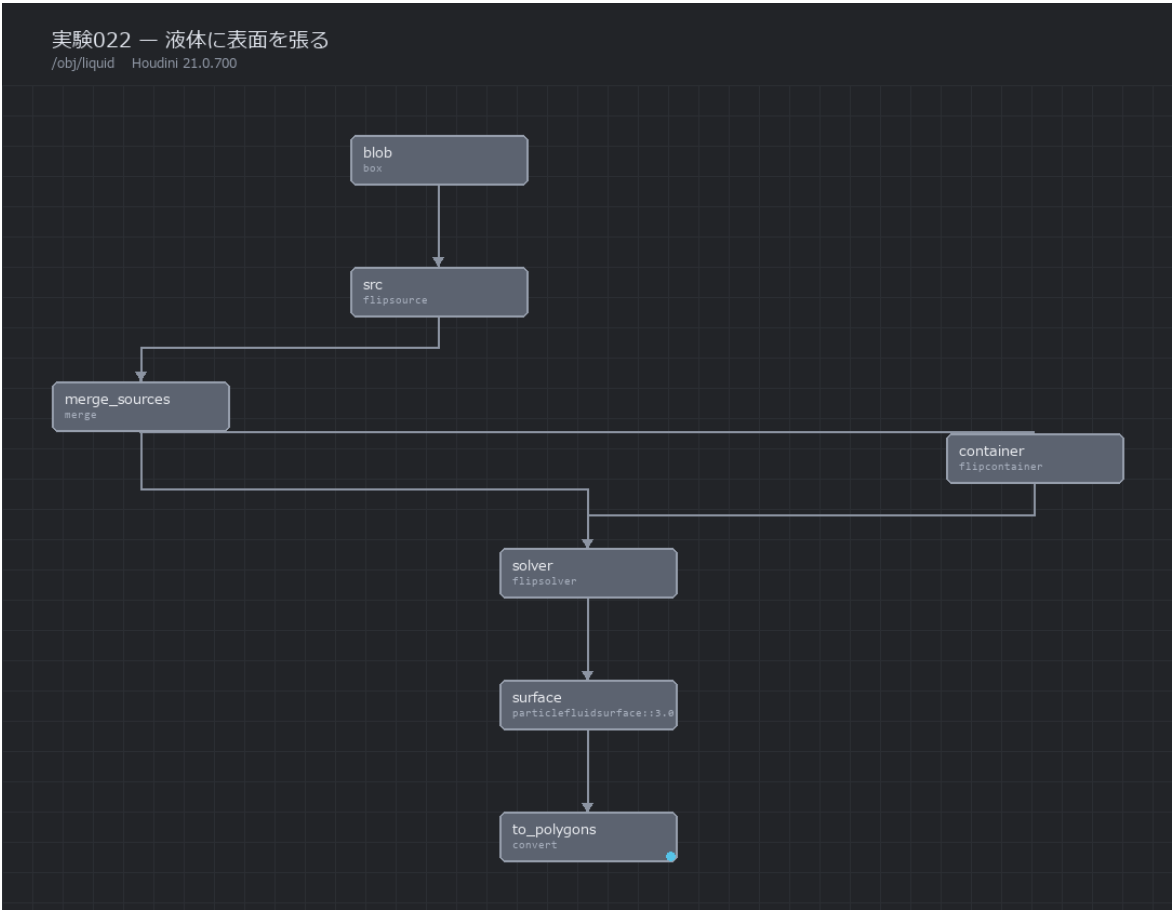
液体に表面を張る — そして019の係数が間違っていた

2026-09-14 | Houdini 21.0.700 | /obj/liquid | ノード 7 / 接続 8

何を作ったか

実験019では粒のままだったので、液体に見えなかった。particlefluidsurface で表面を張る。
張るだけなら簡単だったが、体積を測ったら019から予測した値の2.75倍になった。原因を追ったところ、2つの別々の問題が重なっていた。ひとつは境界の外側に作られる粒まで面に含めていたこと。もうひとつは019の係数そのものが間違っていたこと。
最後は、粒1つが担う体積はちょうど粒の間隔の3乗である、というきれいな形に落ち着いた。比は1.0006 だった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
container	flipcontainer	particlesep = 0.07 size _x = 3.0 size _y = 3.0 size _z = 3.0

ノード	タイプ	デフォルトから変更したパラメータ
blob	box	size _x = 0.9 size _y = 0.9 size _z = 0.9 ty = 1.0
src	flipsource	particlesep = 0.07 volumename = source
merge_sources	merge	(変更なし)
solver	flipsolver	donarrowband = 0 folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 ground_posy = -1.5 particlesep = 0.07 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 …ほか 7 件
surface	particlefluidsurface:: 3.0	closedends = 1 dobbox = 1 folder4 = 1 folder5 = 1 folder6 = 1 folder7 = 1 maskvisramp1cb = 1.0 maskvisramp1cr = 0.2 maskvisramp2cb = 1.0 maskvisramp2cg = 0.85 maskvisramp2pos = 0.25 maskvisramp3cb = 0.10000000149011612 …ほか 27 件
to_polygons	convert	(変更なし)

体積が分かっている水で確かめる

器を水位まで満たせば体積は計算で出せる。3×3 の器を深さ1.5まで満たすので 13.5。ここで面を張って 13.5 が出れば、面の張り方は正しい。

切り取り	影響半径	体積	本当の体積との比	ずれ
なし	2.0	60.0394	4.4474	+344.7%
なし	3.0	59.7756	4.4278	+342.8%
あり	2.0	13.3606	0.9897	−1.0%
あり	3.0	13.2493	0.9814	−1.9%

器の中と外で、粒はどう分かれているか

水位 0.0、粒の間隔 0.07 のとき。器は 3×3×3 で原点中心、水は y が −1.5 から 0 まで。

場所	粒の数	割合
器の中（水のある範囲）	40,871	50.4%

場所	粒の数	割合
器の外	40,235	49.6%
合計	81,106	100%

水位を変えたときの、粒の増え方

水位 -0.5（体積9.0）と

+0.5（体積18.0）で数えた。体積が9.0増えたときに、どこの粒が何個増えたか。

水位	水の体積	全部の粒	器の中	器の外
-0.50	9.0000	60,152	27,688	32,464
0.50	18.0000	101,884	53,912	47,972
増えた分	9.0000	41,732	26,224	15,508

粒1つが担う体積

体積1あたり何個増えるかの逆数。粒の間隔は 0.07 なので、その3乗は 0.000343。

数え方	体積1あたりの粒	粒1つぶんの体積	間隔の3乗との比
全部の粒	4,636.9	0.000216	0.63
器の中だけ	2,913.8	0.000343	1.0006
器の外だけ	1,723.1	—	—

わかったこと

- ・まず面が正しく張れているかを確認めた。落ちて広がった水でいきなり測ると、正解が分からないので合っているか判断できない。器を水位まで満たせば体積は計算で出せるので、そちらで先に確かめた。

- ・切り取らないと体積が4.4倍になる。器を深さ1.5まで満たした水（体積 13.5）に面を張ると、60.04 になった。付属ヘルプ (flipcontainer) に理由が書いてある。「境界の外側にも粒が作られる。面を張るときこの粒も数えられるので、メッシュが大きくなる。Bounding Box で外側を切り取れる」。

- ・切り取ると 13.36 になり、ずれは 1.0% まで下がった。面の張り方そのものは正しい。

- ・切り取るときは「切り口を閉じる」も入れる。これを入れないと、切った面が開いたままになる。開いた面には「囲まれた体積」という考え方が成り立たず、測ると 0.016 というでたらめな値が出る。最初これで一度つまずいた。

- ・境界の粒は想像よりずっと多い。水位0.0のとき、粒の 49.6% が器の外にあった。半分である。面を張るときだけでなく、粒の数を何かの目安に使うときも、この分を除かないと話が合わない。

- ・そして実験019の係数が間違っていた。019では水位を変えたときの傾きから「増えた粒1つが担う体積」を 0.000293 と出した。切片を分けたので境界の分は除けているつもりだったが、除けていなかった。

- ・理由は、境界の粒も水位と一緒に増えるから。器の側面に沿って作られるので、水が深くなるほど側面の濡れる面積が増え、粒も増える。実測では、体積が9.0増えたときに増えた 41,732個のうち 15,508個（37.2%）が器の外だった。傾きにこの分が混ざっていた。

- ・器の中の粒だけで数え直すと、体積1あたり 2,913.8個。粒1つぶんは 0.000343 になる。粒の間隔の3乗 ($0.07^3 = 0.000343$) との比は 1.0006。

- ・つまり「水の体積 = 器の中の粒の数 × 粒の間隔の3乗」が、そのまま成り立つ。019で「0.572倍」と書いた半端な係数は、境界の粒が混ざったせいで出ていた見かけの数字だった。

- ・面の太さを決めるのは influenceradius（粒の間隔の何倍まで届くか、既定3.0）。1.0まで下げると「Droplet Scale must not exceed Influence Scale」というエラーで面が作れない。2.5を超えると体積が変わらなくなる（2.5で3.49395、3.0で3.49347）。

- ・面をなめらかにすると体積は減る。0回で 3.4935、8回で 3.1243 と 10.6% 減った。実験002で「分割ではなく平滑化が形を縮ませる」と分かったが、同じことがここでも起きている。

- ・出力は

PolySoup（多数の面を1つにまとめた形）なので、そのままでは体積を測る仕組みが使えない。convertで普通のポリゴンに戻すと測れる。なお面ごとの measuredvolume を足しても体積は出ない（平らな面には体積が無い）。三角形ごとに原点との四面体の符号付き体積を足す方法で求めた。

次にやること

- ・019の記事に訂正の注記を出す。読んだ人が気づけないため
- ・023 として炎を出す。021で shredding が効かなかったのを確かめ直す
- ・024 として、動く煙で Karma を撮り直す

炎を出す — shredding が効かなかった理由

2026-09-14 | Houdini 21.0.700 | /obj/burn | ノード 4 / 接続 3

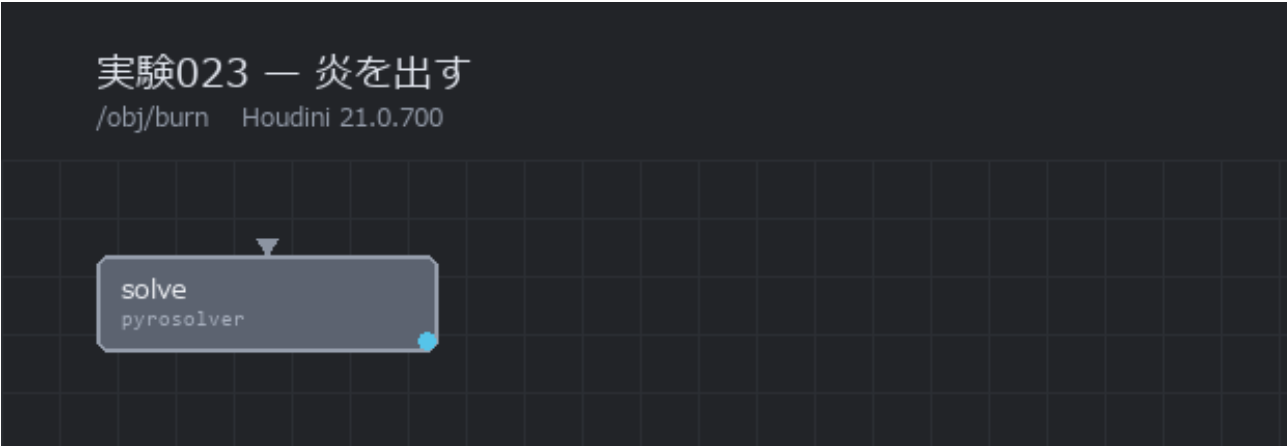
何を作ったか

実験021で shredding を 1.0 でも 20.0 でも試したが、数値が小数点以下まで完全に一致した。まったく効いていない。

付属ヘルプにはこうある。「温度の勾配にもとづいて速度場を押し引きし、炎に特有の筋・裂け目・舌のような形を作る」。021では「炎を伴わない煙だけの設定では出番がないのだろう」と書いたが、そこで止めていた。確かめていない見立てを残したままにしたくないので、今回それを確かめた。

燃烧を入れると shredding は効いた。炎の合計が40%減り、形のばらつきが9%増え、立ち上る高さも縮んだ。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 2 attribute2 = 1 attributes = 2 name1 = burn name2 = temperature
rasterize	volumerasterizeattributes	attributes = burn temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 103 件

煙だけのとき（021の再確認）

供給するのは density と temperature。shredding を 0 と 5.0 で比べた。

条件	F40の合計	中身のマス	ばらつき	格子
shredding なし	48,289.94	30,105	3.7187	48×144×48
shredding 5.0	48,289.94	30,105	3.7187	48×144×48
差	0	0	0	なし

燃焼を入れたとき

pyrosource の初期化プリセット sourceburn を使うと、作られる属性が burn と temperature になる。density は作られないので、どの条件にもある temperature を主な指標にした。

条件	F40の温度の合計	中身のマス	ばらつき	格子
shredding なし	4,724.59	38,637	1.8278	48×144×48
shredding 5.0	3,673.59	41,569	1.9914	48×128×48
差	−22.2%	+7.6%	+9.0%	縦が16マス短い

炎そのものの量

flame の場（燃えている部分）だけを見たもの。

条件	F25の合計	F25のマス	F40の合計	F40のマス
shredding なし	3,001.8	7,374	3,727.4	10,621
shredding 5.0	2,424.4	7,314	2,231.5	7,906
差	−19.2%	−0.8%	−40.1%	−25.6%

フレームごとの推移（燃焼あり）

温度の合計と、形のばらつき。

フレーム	なしの合計	ありの合計	なしのばらつき	ありのばらつき
5	765.31	760.67	0.8442	0.8427
15	2,106.41	1,994.95	1.0784	1.1381

フレーム	なしの合計	ありの合計	なしのばらつき	ありのばらつき
25	3,281.97	2,853.95	1.4196	1.5271
40	4,724.59	3,673.59	1.8278	1.9914

わかったこと

- ・021の見立ては当たっていた。shredding は炎（燃焼）があるときだけ効く。煙だけの設定では、値を 0 から 5.0 に変えても数値が1桁も動かない。燃焼を入れると、同じ変更ですべての数字が動く。

- ・効き方にも筋が通っている。炎の合計が40.1%減り、燃えているマスが25.6%減る一方で、温度の広がり（中身のマス）は7.6%増え、ばらつきは9.0%増えた。つまり炎がちぎれて薄く広がり、そのぶん早く燃え尽きる。ヘルプの「筋・裂け目・舌のような形」という説明と合う。

- ・立ち上る高さも縮む。計算領域が 48×144×48 から 48×128×48 になった。縦に16マス短い。ちぎれた炎は上昇の勢いを失いやすい。

- ・違いはフレームが進むほど大きくなる。F5では合計の差が0.6%しかないが、F40では22.2%になる。短いフレームだけ見て「効かない」と判断すると間違える。021で気づけなかった一因でもある。

- ・燃焼の組み方。pyrosource の初期化に sourceburn というプリセットがある。これを押すと、作られる属性が density と temperature ではなく burn と temperature に変わる。burn が「燃える元」で、ソルバがこれを炎に変える。

- ・ソルバ側では addflamefield を有効にすると flame の場ができる。煙も出したい場合は doflamedensity を有効にするが、量の既定が 0.0001 でほとんど出ない。1.0 にして初めて見えるようになった。

- ・この実験で density の合計が 0 のままだったのは、燃焼のプリセットが density を作らないため。指標を temperature に変えて測り直した。「測ろうとしたものが存在しない」と気づけたのは、フィールドの一覧を毎回出していたから。

- ・021で「確かめきれしていない」と書いて残した見立てを、2実験あとで潰せた。見立てを断定せずに残しておく、あとで回収できる。

次にやること

- ・024 として、動く煙で Karma を撮り直す。実験018は動かない煙で比べていた

- ・炎に色を付ける。いまは白飛びしていて炎に見えない

- ・025 として、シミュレーションのキャッシュ（file cache）に対応する

動く煙で撮り直す — 018の結論は変わらなかった

2026-09-14 | Houdini 21.0.700 | /obj/smoke | ノード 4 / 接続 3

何を作ったか

実験018で2つのレンダラーを比べたとき、題材にしていた煙は動いていなかった。それが分かったのは実験021。一様な球が濃くなるだけの絵で比べていたことになる。

題材が悪いと結論も疑わしい。021で立ち上るようにした煙に入れ替えて、同じ指標で測り直した。

結果、数字はほとんど変わらなかった。階調は 235→233 と 200→200、最も暗い画素は 15→17 と 50→49。018の結論はそのまま成り立つ。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rad y = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 99 件

動く煙を2つのレンダラーで（同じカメラ・同じ露出）

実験021で温度を供給して立ち上るようにした煙。フレーム40、460×760、Karma は256サンプル。

レンダラー	時間	煙の画素	階調	平均の明るさ	ばらつき	最も暗い
OpenGL ROP	0.73 秒	33,003	233 段	129.9	72.99	17
Karma 256サンプル	4.30 秒	25,726	200 段	141.5	49.63	49

実験018（動かない煙）との比較

題材を入れ替えても、どの指標もほとんど動かない。

指標	018 OpenGL	018 Karma	024 OpenGL	024 Karma
階調	235	200	233	200
ばらつき	86.16	52.83	72.99	49.63
最も暗い画素	15	50	17	49

煙の中心を横切る線の明るさ

255 が背景。数字が小さいほど濃い。

レンダラー	縁の外	中心	縁の外
OpenGL ROP	254	42	253
Karma	255	93	255

わかったこと

- ・018の結論は変わらなかった。題材を「動かない球」から「立ち上る煙」に入れ替えても、階調は 235→233 と 200→200、最も暗い画素は 15→17 と 50→49。どれも誤差の範囲でしか動かない。
- ・これは2つのレンダラーの違いが、題材の複雑さに左右されないことを意味する。光が中を通るかどうかという違いは、形が単純でも複雑でも同じように出る。
- ・疑って測り直したこと自体には意味がある。018の結論は「悪い題材で出した結論かもしれない」という弱点を抱えていた。測り直したことでその弱点が消えた。結果が変わらなくても、確かめる価値はある。

- ・見た目の差ははっきりしている。OpenGL

は煙の頭も柱も一様に暗く、どちらから光が当たっているか分からない。Karma

は頭の内側に陰影が出て、丸みが分かる。数字では「最も暗い画素」がこの違いを捉えている（17 対 49）。OpenGL は濃いところをほぼ黒まで落とすが、Karma は光が抜けてくるぶん薄く留まる。

- ・速さの差も018と同じ傾向。0.73秒 対 4.30秒で約5.9倍（018では5.1倍）。確認用は OpenGL、見せる絵は Karma、という使い分けはそのままよい。

- ・今回の煙は実験021の組み方をそのまま使った。temperature を供給しないと煙は動かない。この1行がないと、また「動かない球」を撮ることになる。

次にやること

- ・炎に色を付ける。いまは白黒でしか描けていない
- ・025 として、シミュレーションのキャッシュ（file cache）に対応する
- ・実験030に達したら、001～030の振り返り点検を行う

シミュレーションのキャッシュ ー 速くなるが、値は変わっていないか

2026-09-14 | Houdini 21.0.700 | /obj/smoke | ノード 5 / 接続 4

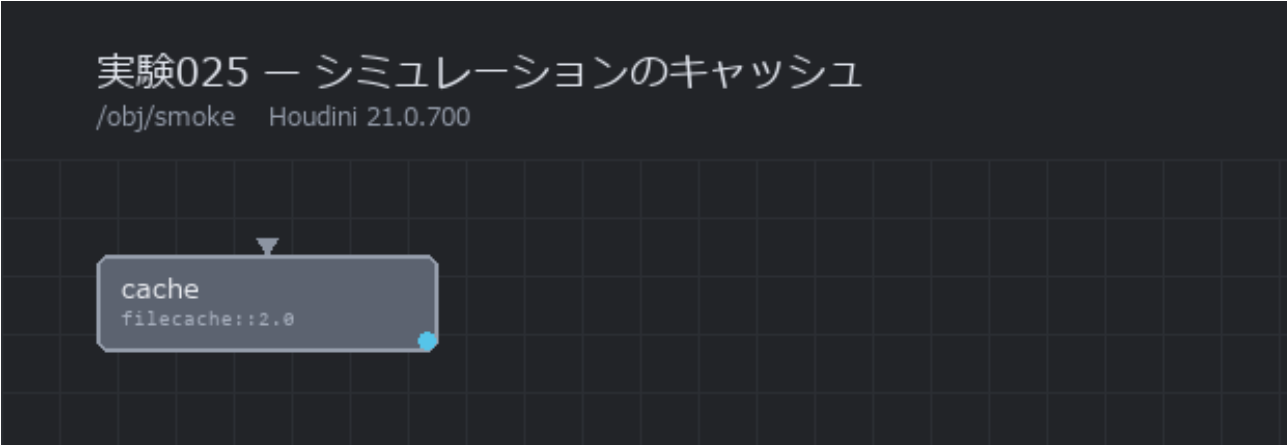
何を作ったか

ここまでの実験で、同じ計算を何度も回してきた。実験018では同じ煙を何十回も描き直している。シミュレーションは前のフレームの結果から次を作るので途中から見ることができず、毎回1フレーム目から計算し直している。

filecache は結果をファイルに書き出し、次からはそれを読む。当然速くなるはずだが、速くなっても値が変わっていたら意味がない。以降の測定がすべて信用できなくなる。

読み込みは4.6倍速く、値は40フレームすべてで完全に一致した。ただしフレーム範囲の指定で一度つまずき、意図の6倍のファイルを書いて3.5GBを消費した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 99 件
cache	filecache::2.0	f1 = 1.0 f2 = 40.0 file = D:/Claude/houdini/out/025_cache/smoke.0040.bgeo.sc filemethod = 1 folder2 = 1 folder3 = 1 loadfromdisk = 1

時間とファイルの量（40フレーム）

実験021と同じ煙。0.05刻みのボクセル。書き出しが速いのは、直前に全フレームを計算済みで、その結果をそのまま書き出しているため。

やったこと	時間	備考
計算しながら40フレーム回す	1.17 秒	毎回1フレーム目から積み上げる
ファイルに書き出す	0.15 秒	40ファイル / 63.3 MB
ファイルから読んで40フレーム回す	0.26 秒	計算より 4.6倍速い

値は変わっていないか

密度の合計。「ほぼ同じ」ではなく「完全に同じ」を求めた。

フレーム	計算した値	読んだ値	差
1	4,244.138830	4,244.138830	0.0000000000
10	29,938.465463	29,938.465463	0.0000000000
20	38,283.067808	38,283.067808	0.0000000000
30	44,966.939905	44,966.939905	0.0000000000
40	48,289.944718	48,289.944718	0.0000000000

フレーム範囲の指定でつまずいた

f1 と f2 に数値を入れたつもりが、式が残っていた。

やり方	書かれたフレーム	ファイル数	容量	書き出し時間
数値を set しただけ	1～240	240	3,578.9 MB	28.81 秒
式を消してから set	1～40	40	63.3 MB	0.15 秒

わかったこと

- ・値は完全に一致した。40フレームすべてで、密度の合計の差が 0.0000000000。書き出しと読み込みで数値が変わらないことを確かめられたので、キャッシュを挟んだあとの測定も信用してよい。
- ・読み込みは 4.6倍速い。1.17秒が 0.26秒になる。今回の煙は軽いのでこの程度だが、重いシミュレーションほど差は開く。
- ・フレーム範囲の指定で一度つまずいた。f1 と f2 の既定は \$FSTART / \$FEND という式で、シーンの範囲（1〜240）を指している。数値を入れるだけでは式が残り、40フレームのつもりが240フレーム書かれた。
- ・その結果、3,578.9 MB を書き、28.81秒かかった。意図の6倍である。式を消してから数値を入れると 63.3 MB、0.15秒になった。56倍の無駄だった。
- ・この手のずれは、書き出したファイルの数を数えれば一発で分かる。40フレームのつもりなら40ファイルのはず。思ったとおりに動いたかを、別の量で確かめる。
- ・容量は40フレームで 63.3 MB。1フレームあたり約 1.6 MB。フレーム1が 7.7 KB、フレーム40が 3.2 MB で、煙が広がるにつれて増えていく。長いシミュレーションでは容量が効いてくる。
- ・キャッシュはリポジトリに入れない。中身は再生成できるうえに大きいので、.gitignore に out/*_cache/ を足した。
- ・書き出し時間の 0.15秒は、直前に全フレームを計算済みだったため。まっさらな状態から書き出せば、計算の時間（1.17秒）がそのまま乗る。キャッシュが得になるのは2回目から。

次にやること

- ・hou_tools にキャッシュを組み込む。重い実験で自動的に使えるようにする
- ・026 として、炎に色を付ける
- ・実験030に達したら、001〜030の振り返り点検を行う

炎に色を付ける — 黒体放射の式を数字で追う

2026-09-14 | Houdini 21.0.700 | /obj/fire | ノード 7 / 接続 7

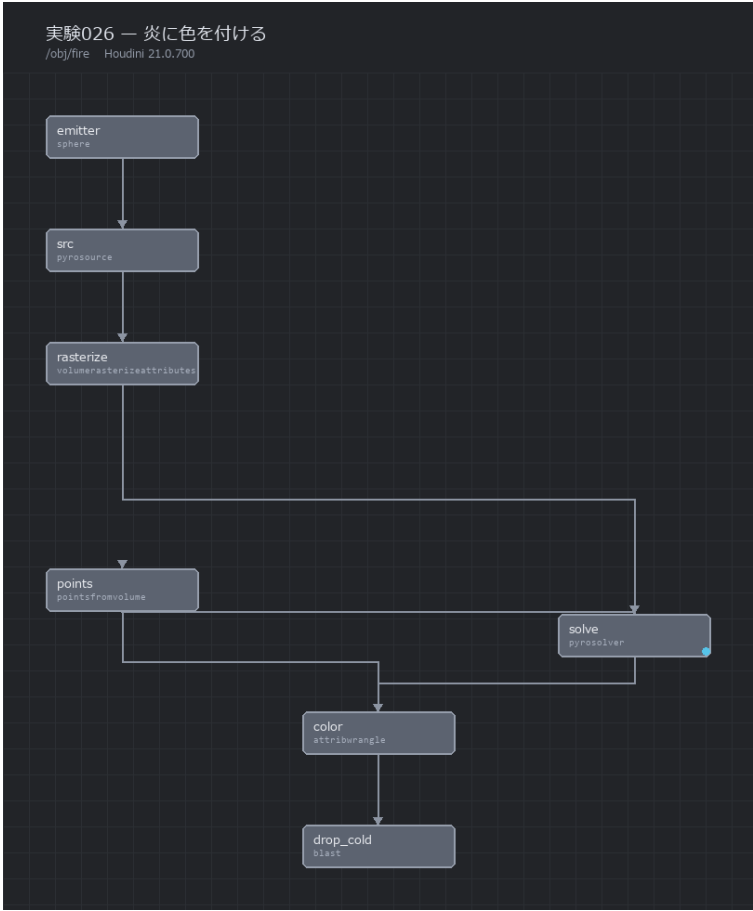
何を作ったか

実験023で炎を出したが、白黒でしか描けていなかった。炎は温度で色が変わる。低いと赤、高くなるにつれて橙・黄・白へ。これは黒体放射という物理の話で、Houdini の VEX には `blackbody()` という関数がある。

「色が付いた」で終わらず、温度ごとに返る色を数字で取り出した。赤成分と青成分の比は 1000K で 17.39、8000K で 0.7381。23.6分の1まで下がり、温度とともにきれいに単調に減る。

見た目にするまでに2回つまずいた。返る値が 6500倍ほどの大きさだったこと、そして 1 で切ると赤が潰れて全部黄色になることである。

ノードの接続



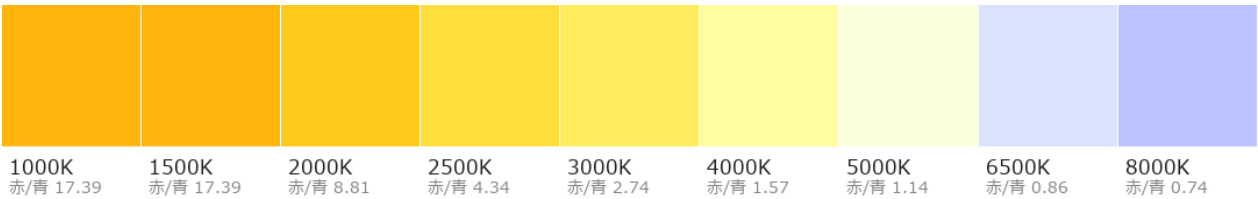
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rad y = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2

ノード	タイプ	デフォルトから変更したパラメータ
src	pyrosource	attribute1 = 2 attribute2 = 1 attributes = 2 name1 = burn name2 = temperature
rasterize	volumerasterizeattributes	attributes = burn temperature folder1 = 1 folder3 = 1 voxelsize = 0.05
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 101 件
points	pointsfromvolume	particlesep = 0.080000000000000002
color	attribwrangle	snippet = // 2本目の入力（ボリウム）から、この点の位置の温度を読む float t = volumesample(1, "temperature", @P); f@temp = t; // 温度の単位をケルビンに直してから色にする vec...
drop_cold	blast	group = @temp<0.02 grouptype = 3

blackbody() が温度ごとに返す色

2つ目の引数は「白として扱う温度」で 6500 を指定した。値は 6500 で割って 0～1 に収めてある。赤と青の比を見ると、色の変化が数字で追える。



左が 1000K、右が 8000K。各マスの下は 赤/青 の比。

温度(K)	赤	緑	青	赤/青
1000	1.4015	1.0000	0.0806	17.3872
1500	1.4015	1.0000	0.0806	17.3872
2000	1.2750	1.0000	0.1448	8.8063
2500	1.1517	1.0000	0.2655	4.3383
3000	1.0802	1.0000	0.3940	2.7415
4000	1.0101	1.0000	0.6443	1.5676
5000	0.9812	1.0000	0.8630	1.1370

温度(K)	赤	緑	青	赤/青
6500	0.9686	1.0000	1.1211	0.8640
8000	0.9686	1.0000	1.3122	0.7381

炎の温度を色にする

ボリウムの中に点をばらまき、点の位置の温度を読んで色にした。点なら確認用の描画（OpenGL）でも色が出る。



実験023の炎。温度は持っているが色になっていない。 温度を色にしたもの。根元が橙、上が黄。

調べたこと	結果
ばらまいた点	4,779 個
温度の範囲	0.3227 ~ 0.9929
ケルビンに直すと (×3000)	968K ~ 2,979K
その範囲の赤/青	約 17.4 ~ 2.8

わかったこと

- ・ 赤と青の比は、温度とともにきれいに単調に下がった。1000K で 17.39、8000K で 0.7381。23.6分の1になる。9点すべてで下がる向きが崩れない。式は思ったとおりに効いている。
- ・ 1000K と 1500K が完全に同じ値だった。どちらも (1.4015, 1.0000, 0.0806)。低いほうで頭打ちになっている。炎の一番冷たいところを描き分けたいなら、この下限より上に収まるよう温度の換算を調整する必要がある。
- ・ 引数は2つ。blackbody(温度) と書くとエラーになる。正しくは blackbody(温度, 白として扱う温度)。エラーメッセージが候補を出してくれるので気づける。
- ・ 返る値は 0~1 ではない。6500 を指定すると、緑成分がちょうど 6500 になる大きさに返ってくる。そのまま色に入れると全部真っ白になる。6500 で割って初めて見える値になった。
- ・ 1 で切ると色味が消える。1000K の (1.4015, 1.0000, 0.0806) を 0~1 に丸めると (1.0, 1.0, 0.08) になり、赤と緑が同じ値 = 黄色になってしまう。実際、最初の画は全部黄色だった。

- ・直し方は「一番大きい成分で割る」。(1.4015, 1.0, 0.0806) ÷ 1.4015 = (1.0, 0.713, 0.058)

で、橙赤のまま明るさだけが揃う。丸めるか割るかで、色が別物になる。

- ・ボリュームそのものには色を付けず、pointsfromvolume で中に点をばらまいて点に色を付けた。こうすると確認用の速い描画でも色が見える。見せる絵にするときは Karma 側でボリュームに色を付ける。

- ・点から温度を読むのは volumesample(1, "temperature", @P)。2本目の入力につないだボリュームを、その点の位置で読む。

次にやること

- ・ Karma でボリュームに直接色を付ける。いまは点に色を付けて代用している
- ・ 027 として、hou_tools にキャッシュを組み込む
- ・ 実験030に達したら、001～030の振り返り点検を行う

キャッシュを道具に組み込む ー 古いファイルを読む事故を防ぐ

2026-09-14 | Houdini 21.0.700 | /obj/smoke | ノード 5 / 接続 4

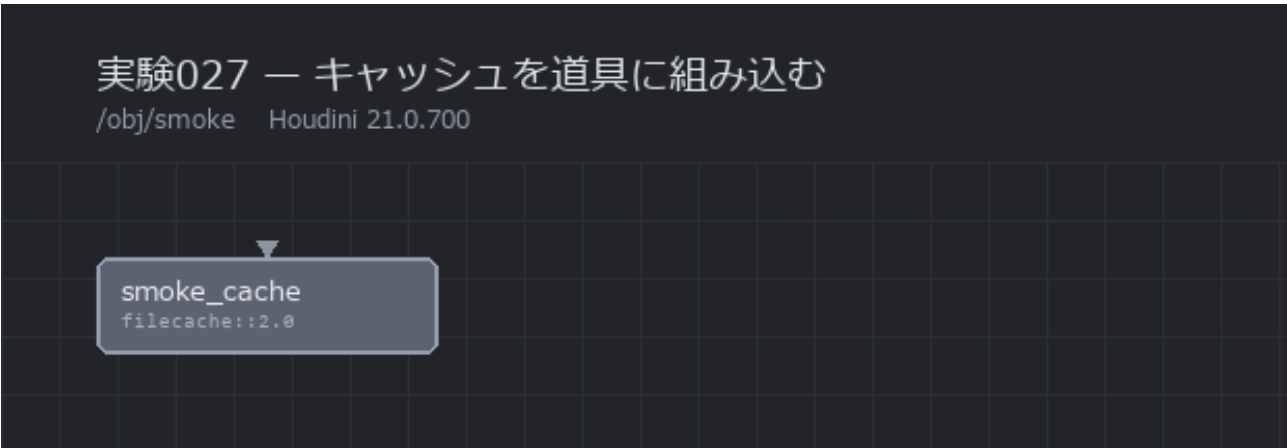
何を作ったか

実験025で filecache の挙動は測った。読み込みは速く、値は完全に一致する。ただし手で組むと毎回同じ手間がかかるうえ、もっと危ない落とし穴がある。

上流を変えたのに、古いキャッシュを読んでしまうことである。このとき画も数字も出てくる。エラーも出ない。ただ、それは前の設定の結果である。測った値が静かに嘘になる。このプロジェクトで一番避けたい事故がこれになる。

そこで hou_tools.cache_sim()
に指紋を持たせた。上流のノードの種類と「既定から動かしたパラメータ」をまとめて SHA-1 を取り、キャッシュの隣に置く。一致したときだけ読み、変わっていれば黙って計算し直す。仕組みが効いているかを、3つの形で確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.06

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.05 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 100 件
smoke_cache	filecache::2.0	f1 = 1.0 f2 = 30.0 file = D:/Claude/houdini/out/027_cache/smoke.0001.bgeo.sc filemethod = 1 folder2 = 1 folder3 = 1 loadfromdisk = 1

1. キャッシュを通して値は変わらないか

「ほぼ同じ」ではなく「完全に同じ」を求める。ここが崩れると、以降の測定がすべて信用できなくなる。

フレーム	計算した値	読んだ値	差
1	4,230.662863	4,230.662863	0.0000000000
10	30,202.378274	30,202.378274	0.0000000000
20	37,789.693870	37,789.693870	0.0000000000
30	43,739.861589	43,739.861589	0.0000000000

2. 速さと、2回目の振る舞い

2回目は同じ設定で呼び直した。指紋が一致するので、書き出しは起きないはずである。

やったこと	書き出し	読み出し	結果
計算しながら30フレーム回す	—	0.83 秒	これが基準
1回目の cache_sim()	0.12 秒	0.15 秒	計算より 5.5倍速い
2回目の cache_sim() (同じ設定)	起きない	0.20 秒	1回目と完全に一致

3. 上流を1つ変えたら、気づくか

pyrosolver の dissipation を、既定の 0.1 から 0.05 に下げた。それ以外は何も触っていない。



Houdini

dissipation 0.1 (既定) 。 F30。



Houdini

dissipation
0.05。同じカメラ・同じフレーム。頭が濃く大きい。

調べたこと	結果
指紋	8f35eab0ca26… → 366303d3368d… (変わった)
cache_sim() の判断	fingerprint changed → 計算し直した
値が変わったフレーム	29 / 30
F30 の密度の合計	43,739.86 → 66,890.32 (+52.9%)
指紋のファイルを壊した場合	no fingerprint → 計算し直した

わかったこと

- ・ キャッシュを通しても値は1ビットも変わらなかった。30フレームすべてで差が 0.0000000000。025の結論が、道具に組み込んだあとでも保たれている。
- ・ 2回目は書き出しが起きなかった。指紋が一致したので読むだけになり、値も1回目と完全に一致した。使う側は毎回 cache_sim() を呼ぶだけでよい。
- ・ 上流を1つ動かしたら、指紋が変わって計算し直した。これが無ければ、30フレーム中29フレームで違う値を、正しい値として読んでいたことになる。F30 で +52.9% のずれ。
- ・ 指紋は上流のノードをたどって、種類と「既定から動かしたパラメータ」だけを集めて作る。既定値を含めないで、Houdiniの版が上がって既定が増えても指紋は暴れない。
- ・ 指紋のファイルが壊れていたら、読まずに計算し直す。壊れたJSONを置いて確かめた。「読めないときは安全側に倒す」ー迷ったら計算し直すほうが、静かに間違えるよりよい。
- ・ 同じ計算を2回書き出すと、ファイルの合計サイズが 138バイト違った (38,927,958 → 38,928,096) 。中身の値は同じでも、ファイルはバイト単位では一致しない。つまりキャッシュの正しさをファイルの比較で確かめることはできない。指紋を「ファイル」ではなく「ネットワーク」から作っている理由がこれ。
- ・ 書き出したファイルの数がフレーム数と違えば、cache_sim() は例外を投げて止まる。実験025で40フレームのつもりが 240フレーム書かれて 3.5GB を無駄にした件を、道具の側で塞いだ。
- ・ pyrosolver の dissipation は既定で有効・値は 0.1。何も触らなくても煙は薄くなり続けている。半分にしかたけで F30 の密度が +52.9% になる。「既定=何もしていない」ではない。

- ・ hython は、レンダしたあとにシーンを作り直すと固まる。今回この順で10分待っても進まなかった。レンダは1プロセスの最後に1枚だけにして、比較の2枚目は別プロセスで撮る。カメラをそろえるため、枠を JSON に残して次のプロセスが読む。

次にやること

- ・ 重い実験 (Karma、FLIP、大きな Pyro) を `cache_sim()` に載せ替える
- ・ 028 として、まだ触っていない分野に進む
- ・ 実験030に達したら、001~030の振り返り点検を行う

布は破れなかった — Vellum の breaking が効かない条件を潰す

2026-09-14 | Houdini 21.0.700 | /obj/tear | ノード 6 / 接続 7

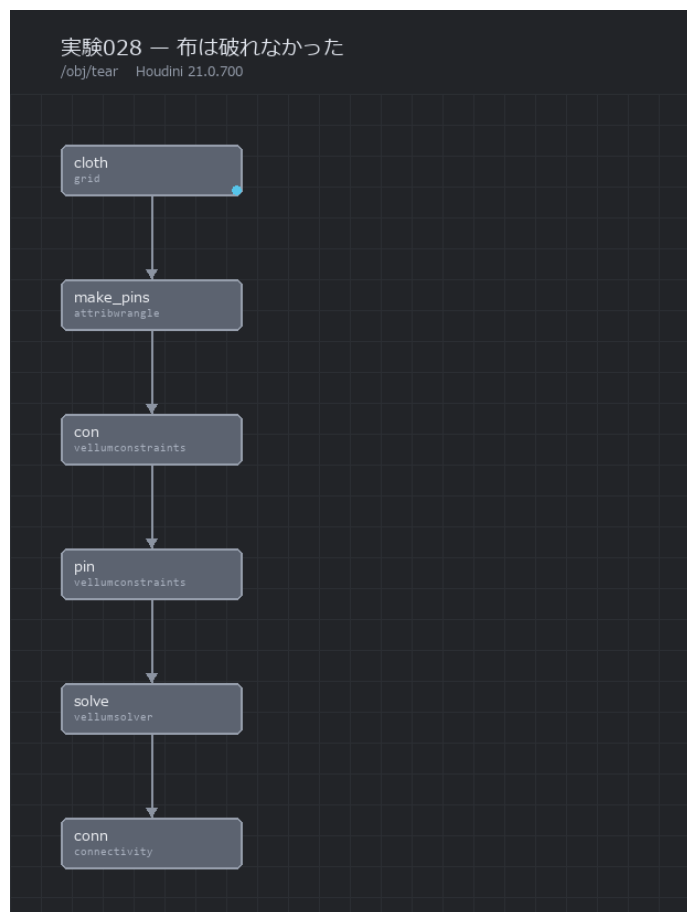
何を作ったか

実験015で布を垂らした。今回はそれを破るつもりだった。vellumconstraints には dobreaking (既定でオフ) と breakthreshold (既定 0.1) がある。しきい値を下げれば破れやすくなるはずである。

結果を先に書く。1つも破れなかった。しきい値を 1.0 から 0.000001 まで下げても、破れ方の種類を3つとも試しても、布を100倍重くして応力を26,552まで上げて、切れた拘束はゼロのままだった。しきい値の 2.7×10 の10乗倍の応力がかかっても、である。

「効かない」と書くには、効くはずの条件を潰しきる必要がある。5つ潰した。しきい値・破れ方の種類・力の強さ・拘束の型・ソルバの設定。それでも破れない。原因はまだ特定できていないので、断定はしない。確かめたことと、まだ確かめていないことを分けて残す。

ノードの接続

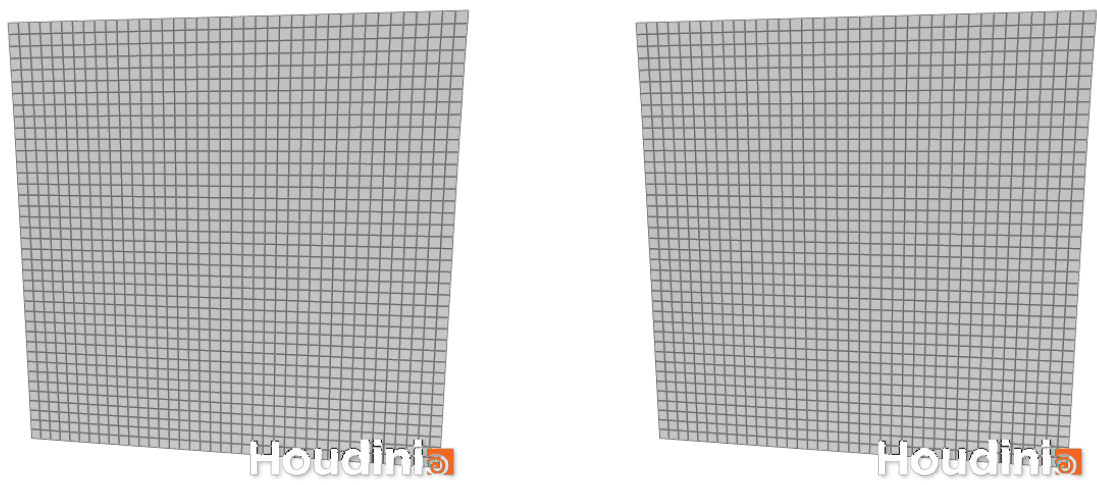


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 40 orient = 0 rows = 40 sizex = 4.0 sizey = 4.0 ty = 2.2
make_pins	attribwrangle	snippet = if (@P.y > 4.1) { @group_pins = 1; }
con	vellumconstraints	constrainttype = 3 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
conn	connectivity	connecttype = 1

組んだもの

4×4の布を40×40に分割し、上の辺1列だけをピンで留めて吊るした。布の全重量が上端の拘束にかかる。48フレーム回す。



breaking を切った状態。F48。

しきい値
0.02。同じカメラ・同じフレーム。見分けがつかない。

項目	値
布	4×4 / 40×40分割 / 1,600点 / 1,521面
留めたところ	上の辺1列だけ
作られた拘束	9,126本 (distance 4,641 + bend 4,485)
拘束の型 / 破れ方	distance / stretchstress
ソルバのモード	full (Minimal ではない)
中の DOP の判定頻度	perframe (Never ではない)

1. しきい値を下げていく

破れ方の種類は既定の stretchstress。48フレーム目の状態を測った。

しきい値	切れた拘束	拘束	点	塊	一番長い辺	最下点
なし（基準）	0	9,126	1,600	1	0.10506	0.1594
1.0	0	9,126	1,600	1	0.10506	0.1594
0.5	0	9,126	1,600	1	0.10506	0.1594
0.2	0	9,126	1,600	1	0.10506	0.1594
0.1（既定値）	0	9,126	1,600	1	0.10506	0.1594
0.05	0	9,126	1,600	1	0.10506	0.1594
0.02	0	9,126	1,600	1	0.10506	0.1594

2. 破れ方の種類を変える（しきい値 0.000001）

breakypestretch の3種類すべて。拘束の属性が正しく書き換わっていることも確認済み。

破れ方	切れた拘束	応力の最大
stretchstress（応力）	0 / 9,126	22.603
stretchdistance（伸びた距離）	0 / 9,126	22.603
stretchratio（伸び率）	0 / 9,126	22.603

3. 力を強くする（しきい値 0.000001）

布を柔らかくし、さらに重さを100倍にした。応力はしきい値の 2.7×10 の10乗倍に達している。

stretchstiffness	重さ	切れた拘束	応力の最大	一番長い辺
1.0	1	0	22.603	0.10506
0.01	1	0	22.605	0.10506
0.001	1	0	22.616	0.10506
0.001	100	0	26,551.953	0.10672

わかったこと

- ・ どの条件でも1つも切れなかった。しきい値7段階・破れ方3種類・力4段階、合わせて14通り。切れた拘束はすべて 0。
- ・ 結果は完全に同一だった。breaking を切った状態と、しきい値 0.02 の状態で、一番長い辺（0.10506）も最下点（0.1594）も小数点以下まで一致する。実験021の shredding と同じ形の「効いていない」である。
- ・ 応力はしきい値をはるかに超えていた。重さ100倍のときの応力の最大は 26,551.953。しきい値は 0.000001。その差は 2.7×10 の10乗倍。「力が足りない」では説明できない。
- ・ 拘束の型は対象だった。cloth が作る拘束のうち 4,641本は型が distance で、breaktype に stretchstress が書かれている。付属ヘルプは breakypestretch を「distance 拘束の破れ方を決める」と説明しており、条件を満たしている。
- ・ ソルバは判定を切っていなかった。SOP の solvermode は full（Minimal モードなら拘束の増減が非対応だが、そうではない）。SOP 中の DOP ソルバの breakfrequency は perframe で、毎フレーム判定している設定だった。

- ・破れた拘束は消えるのではなく broken グループに入る。ソルバの2番目の出力に、その名前のプリミティブグループが用意されている。今回はここが最後まで空のままだった。つまり「消えたかどうか」ではなく「グループに入ったかどうか」で数えるのが正しい。

- ・付属ヘルプに breakfrequency は載っているが、SOP のパラメータには無い。DOP 版の vellumsolver にはあり、SOP 版には出ていない。SOP の中に埋まっている DOP ノードを開いて読むまで、値が分からなかった。

- ・ここまでが確かめたこと。原因は特定できていない。残っている可能性は次の3つ。(1) SOP レベルの vellumsolver では拘束の破れが扱われない、(2) 別の未知の設定が要る、(3) このHoudiniの版に固有の挙動。断定はしない。

次にやること

- ・DOP レベルで vellum を組み、breakfrequency を自分で指定して同じ測定をする
- ・weld (溶接) や glue (接着) の拘束で breaking を試す。付属ヘルプは breaktypeall を「weld と hair の破れ方」と説明しており、そちらが本来の対象かもしれない
- ・029 として POP で粒に力をかける
- ・実験030に達したら、001~030の振り返り点検を行う

粒に力をかける — 風は「押す」のではなく「追いつかせる」

2026-09-14 | Houdini 21.0.700 | /obj/forces | ノード 3 / 接続 0

何を作ったか

実験020では粒を落としたけだった。今回は横から力をかける。POPの力のノードは67種あるが、入口になるのは2つ。popforce は決まった向きに押し続け、popwind は指定した速さに近づけていく。名前は似ているが中身は違うはずである。

最初の見立ては外れた。空気抵抗といえば速さに比例する抵抗（線形抵抗）だと思い、 $v(t) = \text{風速} \times (1 - e^{(-a t)})$ を予測した。この式なら $t = 1/a$ のとき必ず 63.2% になる。測ったら 75.00% で、しかも airresist を変えても 75.00% のまま動かない。

風速を $1 \cdot 3 \cdot 6$ と変えて測り直し、正しい式を決めた。popwind は二乗抵抗である。 $dv/dt = \text{airresist} \times (\text{風速} - v)^2$ 。15点すべてで、ずれは 0.0000003 未満だった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	grid	cols = 12 rows = 12 sizex = 1.0 sizey = 1.0 ty = 3.0
popnet	dopnet	objname1 = *
import	dopimport::2.0	doppath = /obj/forces/popnet

1. popforce と popwind。同じ「3」を与えても別物

どちらも x 方向に

3。粒は1フレーム目だけ出し、初速はゼロ。72フレーム（2.96秒）回した。カメラは2枚とも同じ。



Houdini



Houdini

popforce。x = 13.30 まで進んだ。

popwind。同じカメラで x = 6.62。半分しか進まない。

経過秒	popforce の速さ	速さ÷時間	popwind の速さ	風速に対する割合
0.0417	0.1250	3.0000	0.3333	11.11%
0.1667	0.5000	3.0000	1.0000	33.33%
0.3750	1.1250	3.0000	1.5882	52.94%
0.6667	2.0000	3.0000	2.0000	66.67%
1.0000	3.0000	3.0000	2.2500	75.00%
2.0000	6.0000	3.0000	2.5714	85.71%
2.9583	8.8750	3.0000	2.6962	89.87%

2. popwind の式を決める。風速を変えて2つの式と比べる

線形抵抗 $v = W(1 - e^{(-at)})$ と、二乗抵抗 $v = W \cdot (Wat) / (Wat + 1)$ 。airresist は 1。

風速	経過秒	実測	線形抵抗	その差	二乗抵抗	その差
1.0	0.3750	0.2727	0.3127	-0.0400	0.2727	0.0000
1.0	1.0000	0.5000	0.6321	-0.1321	0.5000	0.0000
1.0	2.0000	0.6667	0.8647	-0.1980	0.6667	0.0000
3.0	0.1667	1.0000	0.4606	+0.5394	1.0000	0.0000
3.0	1.0000	2.2500	1.8964	+0.3536	2.2500	0.0000
6.0	0.3750	4.1538	1.8763	+2.2776	4.1538	0.0000
6.0	2.0000	5.5385	5.1880	+0.3505	5.5385	0.0000

3. 二乗抵抗なら、 $t = 1/(\text{airresist} \times \text{風速})$ でちょうど 50%

式が正しければ、airresist と風速をどう組み合わせても、この一点は 50% に揃うはずである。

airresist	風速	測ったフレーム	実測	式の予測
1.0	1.0	F25	50.00%	50.00%
1.0	6.0	F5	50.00%	50.00%
2.0	3.0	F5	50.00%	50.00%
4.0	3.0	F3	50.00%	50.00%

4. popaxisforce (渦) 。既定では1つも動かない

効く範囲が「原点を中心とした半径1の球」に決まっている。粒は $y=3$ にいたので範囲の外だった。中心と大きさを粒に合わせると回り始める。



渦を効かせたあと。中心から外へ広がっている。

経過秒	接線速度	中心からの平均距離
0.0417	0.0248	0.3669
0.3750	0.1729	0.3684
1.0000	0.3206	0.4033
2.0000	0.5296	0.6149
2.9583	1.0126	1.0140

わかったこと

- ・ popforce の加速度は、入れた数字そのものだった。速さ÷時間が9点すべてで 3.0000。force に 3 と入れたら加速度が 3。速さは伸び続け、2.96秒で 8.875 に達した。頭打ちにならない。
- ・ popwind は風速で頭打ちになった。同じ「3」でも 2.96秒で 2.6962（風速の 89.87%）。決して 3 を超えない。進んだ距離は 13.30 と 6.62 で、ちょうど半分ほど。
- ・ 最初の見立て（線形抵抗）は外れた。 $t = 1/\text{airresist}$ で 63.2% になるはずが、測ると 75.00%。しかも airresist を $0.5 \cdot 1 \cdot 2 \cdot 4$ と変えても 75.00% のまま。1つの条件で外れただけなら誤差を疑うが、4条件すべてで同じ値が出るのは式が違う証拠である。
- ・ 正しい式は二乗抵抗だった。 $dv/dt = \text{airresist} \times (\text{風速} - v)^2$ 。解くと $v = W \cdot (W \cdot a \cdot t) / (W \cdot a \cdot t + 1)$ 。風速3種類×時刻5点の15点すべてで、ずれは 0.0000003 未満。線形抵抗との最大のずれが 2.28 だったのと比べて桁が違う。
- ・ 式を決められたのは、風速を変えたから。風速3だけで測っていたら、係数が「3」という定数なのか「風速そのもの」なのか区別がつかなかった。風速を 1 と 6 でも測って初めて、係数が風速であると分かった。1つの設定で合う式は、いくらでも作れてしまう。
- ・ 二乗抵抗なら $t = 1/(\text{airresist} \times \text{風速})$ でちょうど 50%。airresist と風速の4通りの組み合わせで、すべて 50.00%。ずれは 0.000006 ポイント。式の検算として使える一点である。

- ・ popaxisforce は既定では何も起きない。効く範囲が

type (球かトーラス) ・ t (中心) ・ r (半径1) ・ height

で決まっていて、既定は原点を中心とした半径1の球。粒が $y=3$ にいると範囲の外なので、接線速度は 0.0000 のまま、位置も1ミリも動かない。エラーは出ない。

- ・ 中心と大きさを粒に合わせると回り始めた。接線速度は 0.0248 → 1.0126、中心からの距離は 0.3669 → 1.0140。回りながら外へ広がる。向きは、軸を +Y にしたとき上から見て反時計回りだった。

- ・ 粒は点のままではレンダに写らないほど小さい。小さな球を copytopoints で被せて撮った。また比較の2枚は、出発点を含めた同じ枠で撮っている。枠を別々に取ると「どれだけ進んだか」が画から消える。

次にやること

- ・ 030 として、001~030の振り返り点検を行う

- ・ popwind の amp / swirlscale (乱れ) を測る。今回は 0 のまま触っていない

- ・ 実験028で残した「Vellum の breaking が効かない」を、DOP レベルで確かめ直す

過去29回を測り直す ― 書いたことは、いまも正しいか

2026-09-14

何を作ったか

実験が30回たまったので、001～029を1つずつ見直した。読み返すだけでは「そう書いてあった」以上のことは分からないので、確かめ直せる主張はもう一度動かして測った。

7つ測り直して、5つは確認、2つは書き方が不正確だった。訂正が要ったのは実験008（scatter は N を作らない）と実験012（rows / cols が効かないのは Polygon だけ）。どちらも「自分の組み方では正しいが、一般化すると間違い」という形の誤りだった。

もう1つ、指標の取り方で数字が変わる例も見つかった。実験011の折れ角を数え直したとき、最初は違う値が出た。原因は Houdini ではなく、面の隣り合わせを「辺を共有」ではなく「点を共有」で数えていたこと。数え方を揃えたら4桁まで一致した。

測り直した7項目

「外れていたら過去の記事を直さなければならない」主張だけを選んだ。

元の実験	確かめたこと	結果
002	点数はいつも「面数 + 2」か	○ 7通りすべて成立
002	一辺の縮みは外挿値 0.83951 に収束するか	○ 8回で差 0.000006
004	パラメータ名は rough / oct / lac か	○ roughness・octaves・lacunarity は存在しない
009	rand(@ptnum) は何度走らせても同じか	○ 400点で差 0.000000000000
011	重み N は分割 N 回まで角を保つか	○ 分割4回・5回でも成立（重み4・5が必要）
008	scatter の出力はいつも N を持つか	× 訂正が必要
012	sphere の rows / cols は効かないのか	× 訂正が必要

訂正1 ― 実験008「scatter の出力は N も持つ」

scatter は N を作らない。入力が N を持っていたときだけ、それを受け継いで点に持たせる。008 は上流に normal SOP を置いた状態だけを見て書いていた。

上流に normal SOP	入力の N の置き場所	scatter の出力の属性
なし	どこにも無い	P だけ
あり	バーテックス	P と N

訂正2 ― 実験012「ポリゴン球は rows / cols を使わない」

効かないのは type が Polygon のときだけ。Polygon Mesh なら効く。type を7種類すべて測った。

type	表示名	6分割の面数	30分割の面数	rows / cols が効くか
prim	Primitive	1	1	効かない

type	表示名	6分割の面数	30分割の面数	rows / cols が効くか
poly	Polygon	80	80	効かない
polymesh	Polygon Mesh	30	870	効く
mesh	Mesh	1	1	効かない
nurbs	NURBS	1	1	効かない
bezier	Bezier	1	1	効かない
polysoup	Polygon Soup	1	1	効かない

補足 — 実験011 を分割4回・5回まで伸ばす

011 は分割3回だけで測っていた。「重み N なら分割 N 回まで」が本当なら、分割4回では重み4が必要になるはずである。折れ角は辺を共有する面どうしで測った。

分割回数	重み3	重み4	重み5	元の鋭さ(90度)になる重み
3回	90.00度	90.00度	90.00度	3
4回	53.13度	90.00度	90.00度	4
5回	28.07度	53.13度	90.00度	5

数え方の違いで数字が変わった例

同じ形の「折れ角の最大」を、面の隣り合わせの数え方を変えて測った。分割3回の場合。

crease の重み	辺を共有する面どうし	点を共有する面どうし	011 の記載
0.0	13.93度	17.85度	13.93度
1.0	28.07度	28.77度	28.07度
2.0	53.13度	53.13度	53.13度
3.0	90.00度	90.00度	90.00度

わかったこと

- ・訂正1（実験008）。scatter は N を作らない。上流に normal SOP が無ければ、出力の属性は P だけ。008 は「scatter の出力は P だけでなく N も持つ。元の面の法線が引き継がれる」と書いたが、前半だけ読むと「scatter が N を作る」と読めてしまう。自分の組み方では正しいが、一般化すると間違いになる書き方だった。

- ・ついでに分かったこと。normal SOP は既定で バーテックスに N を書く。それを scatter が受け取ると、出力では点の N になっている。置き場所が途中で移る。

- ・訂正2（実験012）。rows / cols が効かないのは Polygon のときだけ。Polygon Mesh にすると6分割で30面、30分割で870面と、指定どおりに増える。012 は「ポリゴン球は rows / cols ではなく多面体の細分を使っている」と書いたが、これは Polygon だけの話だった。実験026で球の rows / cols を24にして効いていたのは、type が Polygon Mesh だったため。

- ・type は7種類。Primitive / Mesh / NURBS / Bezier / Polygon Soup
はいずれも面が1枚で返る（面ではなく1個のプリミティブとして持つ）。ポリゴンとして扱いたいなら Polygon か Polygon Mesh。

- ・実験011 の規則は、分割4回・5回でも成立した。分割4回では重み3では53.13度どまりで、重み4にして初めて 90.00度。分割5回では重み5が必要。「重み N なら分割 N 回まで角を保つ」でよい。011 は分割3回だけで確かめていたので、今回2ケース足した。
- ・数え方を変えたら数字が変わった。011 の折れ角を測り直したとき、最初は 13.93度 が 17.85度 に、28.07度 が 28.77度 になった。原因は Houdini ではなく、面の隣り合わせを「辺を共有（点を2つ共有）」ではなく「点を1つでも共有」で数えていたこと。角が鋭いケース（53.13度・90.00度）では両者が一致するので気づきにくい。数え方を揃えたら011の数字と4桁まで一致した。
- ・実験002 の外挿は当たっていた。「一辺は 0.83951 に収束する」と外挿していた。8回まで回すと 0.839516 で、差は 0.000006。差が前回の約1/4ずつ減るという読みも、2回目から7回目まで 0.243 / 0.248 / 0.249 / 0.249 / 0.245 / 0.231 と保たれた。
- ・実験004 のパラメータ名も、実験009 の乱数の再現性も、そのまま成立した。roughness / octaves / lacunarity はいまも存在せず、rand(@ptnum) は400点すべてで差 0.000000000000。
- ・2つの訂正は、どちらも同じ形をしていた。「自分が組んだ1通りで観察したことを、一般の性質として書いた」。次からは、条件を1つ外した場合も測ってから書く。scatter なら normal SOP を外す、sphere なら type を変える。それだけで防げた。

次にやること

- ・031～060 の実験を進め、060 で次の振り返り点検を行う
- ・実験028で残した「Vellum の breaking が効かない」を、DOP レベルで確かめ直す
- ・手順ページを全実験分そろえる

Vellum の breaking は動く — ただし布の伸びには効かない

2026-09-14 | Houdini 21.0.700 | /obj/glue | ノード 8 / 接続 10

何を作ったか

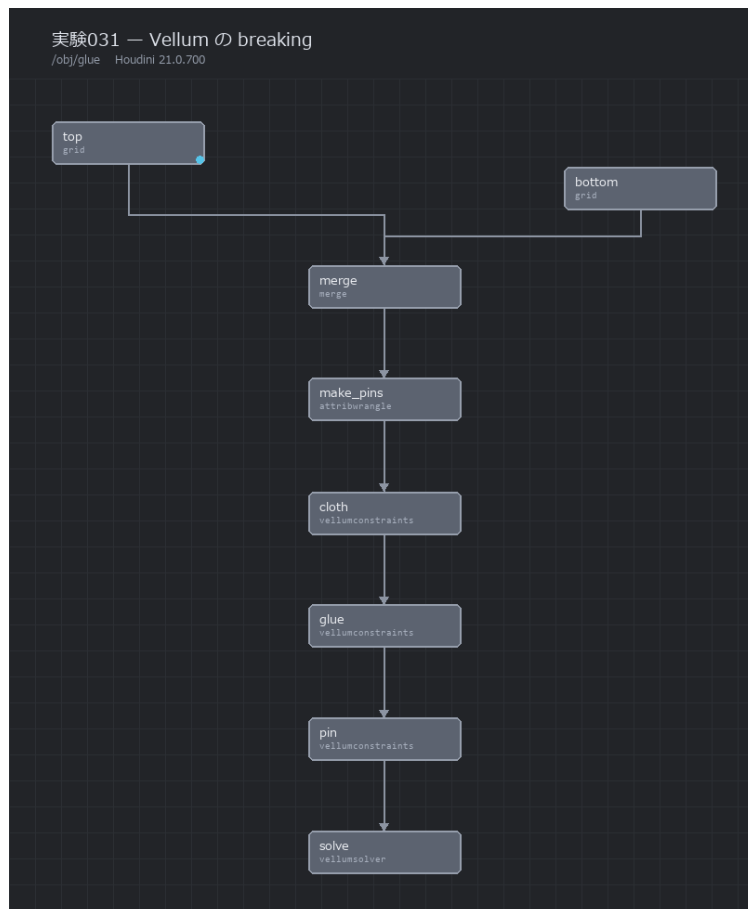
実験028で布を破ろうとして、14通り試して1本も破れなかった。原因は特定できず、3つの可能性を残した。今回それを潰した。

答えが出た。breaking は動く。ただし効く拘束の種類が決まっている。glue が作る stitch 拘束はしきい値どおりに破れ、cloth が作る distance / bend 拘束は同じ場面・同じしきい値でも1本も破れない。

決め手は、1つのシーンの中で両方に breaking を入れたこと。消えたのは stitch の12本だけで、distance 770本と bend 682本は1本も消えていない。solver も、しきい値も、フレーム範囲も、かかる力もすべて同じである。「設定が足りない」のではなく「この種類には効かない」だった。

あわせて、028に書いた説明が逆だったことも分かった。028では「破れた拘束は消えるのではなく broken グループに入る」と書いたが、実際は破れた拘束は取り除かれ、broken グループは空のままである。12本が消えても broken は 0 だった。

ノードの接続

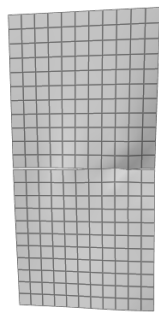


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
top	grid	cols = 12 orient = 0 rows = 12 sizeX = 2.0 sizeY = 2.0 ty = 4.0
bottom	grid	cols = 12 orient = 0 rows = 12 sizeX = 2.0 sizeY = 2.0 ty = 2.02
merge	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 4.9) { @group_pins = 1; }
cloth	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 1e-06 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

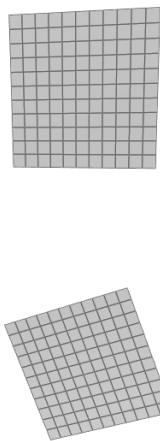
1. 同じ場で、どの種類の拘束が破れるか

上の布を吊るし、その下にもう1枚を glue でくっつけた。下の布の重さは12本の stitch 拘束だけが支える。しきい値はどちらも 0.000001。



Houdini

cloth に breaking。1本も破れず、2枚がつながったまま垂れている。



Houdini

glue に breaking。同じカメラ・同じフレーム。下の布が外れて落ちていく。

設定	distance 770本	bend 682本	stitch 12本	broken グループ	最下点
どちらも breaking なし	0本 消えた	0本	0本	—	0.9608

設定	distance 770本	bend 682本	stitch 12本	broken グループ	最下点
glue だけ breaking	0本	0本	12本 消えた	0	-8.6439
cloth だけ breaking	0本	0本	0本	0	0.9608
両方に breaking	0本	0本	12本 消えた	0	-8.6439

2. しきい値は、ちゃんと効いている

glue 側の breaking のしきい値だけを変えた。拘束の本数がどのフレームで減るかを見る。入力はどれも 1,464本。

しきい値	F3	F10	F40	最下点	起きたこと
なし	1,464	1,464	1,464	0.9608	破れない
0.000001	1,452	1,452	1,452	-8.6439	F3までに12本すべて
1.0	1,452	1,452	1,452	-8.6439	F3までに12本すべて
10.0	1,464	1,456	1,452	-5.6540	少しずつ破れる
100.0	1,464	1,464	1,464	0.9608	破れない
1000.0	1,464	1,464	1,464	0.9608	破れない

3. 028のカーテンで、cloth と distance を比べる

上端だけで吊るした4×4の布。実験028と同じ場面。cloth プリセットを通さない素の distance 拘束でも試した。

拘束の種類	しきい値	拘束の本数（入力 → F40）	最下点
cloth	なし	2,166 → 2,166	0.1929
cloth	0.000001	2,166 → 2,166	0.1929
distance	なし	1,121 → 1,121	0.1931
distance	0.000001	1,121 → 1,121	0.1931
distance	10.0	1,121 → 1,121	0.1931

4. DOP レベルで判定の頻度を指定しても同じ

SOP 版の中の配線をまねて DOP を自分で組むと、SOP には出ていない breakfrequency（破れ判定の頻度）を指定できる。しきい値 0.000001。

breakfrequency	拘束の本数（入力 → F40）	最下点
never（判定しない）	2,166 → 2,166	0.1928
perframe（毎フレーム）	2,166 → 2,166	0.1928
persubstep（毎サブステップ）	2,166 → 2,166	0.1928

わかったこと

- ・ breaking は動いていた。glue が作る stitch

拘束は、しきい値どおりに破れる。12本が消え、下の布が落ちて最下点が 0.9608 から -8.6439 になった。実験028の「1つも破れない」は、拘束の種類によるものだった。

・ 決め手は、1つのシーンの中で両方に breaking を入れたこと。消えたのは stitch の12本だけ。distance 770本と bend 682本は1本も消えていない。solver も、しきい値も、フレーム範囲も、かかる力もすべて同じ場面である。条件が同じなら、残る違いは種類しかない。

・ しきい値はちゃんと効いている。0.000001 と 1.0 では F3 までに12本すべてが破れ、10.0 では少しずつ破れ（F10で8本、F40で12本）、100.0 以上では1本も破れない。境目は 10 と 100 の間にある。「効かない」ではなく「効く範囲が違う」。

・ 028に書いた説明は逆だった。「破れた拘束は消えるのではなく broken グループに入る」と書いたが、実際は取り除かれ、broken グループは空のままである。12本が消えても broken は 0 だった。028では本数も数えていたので結論は変わらないが、理由の説明を訂正する。

・ cloth プリセットを通さない素の distance 拘束でも破れない。「cloth プリセットが何か余計なことをしている」という筋は消えた。distance という種類そのものが、この構成では breaking の対象になっていない。

・ DOP レベルでも同じだった。SOP 版には出ていない breakfrequency を自分で persubstep（毎サブステップ判定）にしても、拘束は1本も減らない。never との差もゼロ。判定の頻度の問題ではない。

・ 実用の結論。布を破りたいなら、cloth の拘束を破ろうとしない。破りたい線に沿ってあらかじめ布を切り分け、glue でくっつけておく。あとはしきい値で、どれだけの力で外れるかを決められる。

・ 道具として覚えておくこと2つ。DOP の拘束ジオメトリは dopimport の geodatapath に ConstraintGeometry と書けば取り出せる。もう1つ、normalizestress を切ると stress の値が 7.925 から 0.014 に変わる（566分の1）。同じ「stress」という名前でも、設定によって桁が変わる。

・ 028で「断定はしない」と書いて3つの可能性を残したのが効いた。そのうち「SOPレベルでは扱われない」は今回否定され（DOPでも同じ）、残ったのは「別の条件が要る」だった。その条件が拘束の種類だと分かった。見立てを断定せずに残しておく、あとで潰せる。

次にやること

- ・ 切れ目を入れた布を glue でつなぎ、狙った線で破れる布を作る
- ・ 032 以降、まだ触っていない分野に進む
- ・ 実験060で 031~060 の振り返り点検を行う

狙った線で破れる布 — 先に切って、あとから貼る

2026-09-14 | Houdini 21.0.700 | /obj/tear_line | ノード 11 / 接続 14

何を作ったか

実験031で分かったのは、Vellum の breaking は glue が作る stitch 拘束にだけ効き、cloth が作る distance 拘束には効かないということだった。つまり布の伸びを破ることはできないが、貼り合わせを外すことはできる。

ならば順序を逆にすればよい。先に切っておいて、あとから貼る。破れてほしい線で布を2つに切り分け、glue

で貼り直し、しきい値を決めて吊るす。重さが貼り目にかかり、しきい値を超えたところから外れる。

狙いどおりになった。波打つ線で切った布は、その線のかたちのまま裂けた。貼り目の位置は狙った線から平均 0.044、最大でも 0.101

しか離れておらず、これはマス目1つぶん (0.103) より小さい。しきい値 100

以上なら1枚の布として垂れたまま、10 以下なら下半分が外れて落ちる。

ノードの接続

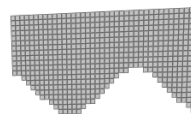
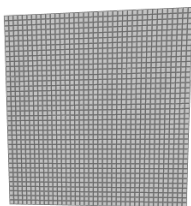


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

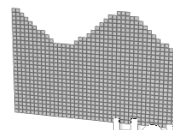
ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 40 orient = 0 rows = 40 sizeX = 4.0 sizeY = 4.0 ty = 2.2
mark_cut	attribwrangle	class = 1 snippet = float line = 2.6 + 0.45 * sin(@P.x * 2.2); i@lower = (@P.y < line) ? 1 : 0;
upper	blast	group = @lower==1 grouptype = 4
lower	blast	group = @lower==0 grouptype = 4
two_pieces	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 4.1) { @group_pins = 1; }
cloth1	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 1.0 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
conn	connectivity	(変更なし)

できたもの

4×4の布を40×40に分割し、上端だけをピンで留めて吊るした。切った線は $y = 2.6 + 0.45 \cdot \sin(2.2x)$ の波。カメラは2枚とも同じ。



Houdini



Houdini

しきい値 1000。1枚の布として垂れる。貼り目は見えない。

しきい値 1.0。切った波の線のまま裂けて、下半分が落ちる。

段取り	やること
1	布を1枚作る (grid)
2	破れてほしい線で面を2組に分ける (attribwrangle でプリミティブに印)
3	2つに切り分けて合流させる (blast ×2 → merge)
4	cloth の拘束を付ける。ピースごとに独立して付く
5	glue で貼り直す。ここに breaking としきい値を入れる
6	上端をピンで留めて解く

1. 貼り目は、狙った線の上にあるか

stitch 拘束は2点を結ぶプリミティブなので、その重心を「その拘束のある場所」とみなして測った。

調べたこと	結果
元の布	1,600点 / 1,521面
切ったあと	1,666点 (+66)
作られた拘束	distance 4,706 / bend 4,420 / stitch 66
stitch の高さの範囲	2.1487 ~ 3.0718
狙った線の範囲	2.15 ~ 3.05 ($y = 2.6 + 0.45 \cdot \sin(2.2x)$)
線からのずれ	平均 0.04405 / 最大 0.10071
マス目の間隔	0.1026 (4 ÷ 39)
検算: 増えた点 = stitch の数	66 = 66

2. しきい値で、外れる／外れないを決められるか

拘束の本数は cloth 9,126本 + stitch 66本 = 9,192本から始まる。外れると stitch が取り除かれて 9,126本になる。

しきい値	F5	F15	F40	吊るした側 (683点)	外れた側 (983点)
0.01	9,126	9,126	9,126	2.140 ~ 4.200	-11.335 ~ -8.448
1.0	9,126	9,126	9,126	2.140 ~ 4.200	-11.055 ~ -8.187
10.0	9,161	9,126	9,126	2.141 ~ 4.200	-8.953 ~ -5.728
100.0	9,192	9,192	9,192	2.116 ~ 4.200	0.155 ~ 3.053
1000.0	9,192	9,192	9,192	2.116 ~ 4.200	0.155 ~ 3.053

3. 切らない布は、やはり破れないまま (031の再確認)

同じ吊るし方で、切らずに cloth の拘束へ直接 breaking を入れた場合。

しきい値	拘束の本数 (入力 → F40)	最下点
0.000001	9,126 → 9,126	0.1667

しきい値	拘束の本数（入力 → F40）	最下点
1.0	9,126 → 9,126	0.1667

わかったこと

- ・狙った線のまま裂けた。貼り目の位置は、狙った線から平均 0.044 ・最大 0.101 しか離れていない。マス目1つぶん (0.1026) より小さい。線をどう引いても、その形のまま破れる。
- ・数が合っている。切ると切り口の点が二重になる。増えた点は 66個で、glue が作った stitch もちょうど 66本。重なった1組につき1本という理屈どおり。ここがずれていたら、貼り忘れか貼りすぎがある。
- ・しきい値で「どれだけの力で外れるか」を決められる。10以下なら外れ、100以上なら外れない。10のときは F5 で 9,161本、F15 で 9,126本と少しずつ外れる。境目は 10 と 100 の間で、実験031の glue のときと同じ。
- ・外れないときは、貼り目がまったく見えない。しきい値 1000 の画は1枚の布そのもので、どこで切ったか分からない。貼っておけば、破れるまでは普通の布として振る舞う。
- ・切らない布は、やはり破れない。同じ吊るし方で cloth の拘束に breaking を入れても、しきい値 0.000001 でも 9,126本のまま。実験031の結論がそのまま再現した。
- ・blast の grouptype でつまずいた。選択肢は 0=推測 / 1=ブレイクポイント / 2=辺 / 3=点 / 4=面。面のつもりで 1 を入れると「ブレイクポイント」になり、何も消えずに素通りする。エラーは出ない。
- ・気づけたのは点の数を数えていたから。切ったはずなのに合流後の点数が元のちょうど2倍 (3,200) になり、stitch が 1,600本 (=元の点数そのもの) できた。切れていない2枚が丸ごと重なっていたと分かった。「思ったとおりに動いたか」を別の量で確かめる。
- ・面の印は、点ではなくプリミティブに付ける。attribwrangle の実行対象を Primitives にして、面の重心 @P が線より下かどうかで振り分ける。点に付けると、1枚の面の4隅で答えが割れて、切り口がぎざぎざに崩れる。

次にやること

- ・衝突する物にぶつけて破る。今回は重さだけで外している
- ・切る線を画像やカーブから作り、狙った形に破る
- ・033 以降、まだ触っていない分野に進む

布を物にぶつけて破る — 破れ始めは当たった場所、そのあとは全体へ

2026-09-14 | Houdini 21.0.700 | /obj/impact | ノード 25 / 接続 42

何を作ったか

実験032では布の重さだけで貼り目を外した。破れる線は自分で決めた波で、どこが破れるかは最初から決まっていた。今回はぶつける。格子状に切れ目を入れた布を吊るし、球を投げ込む。

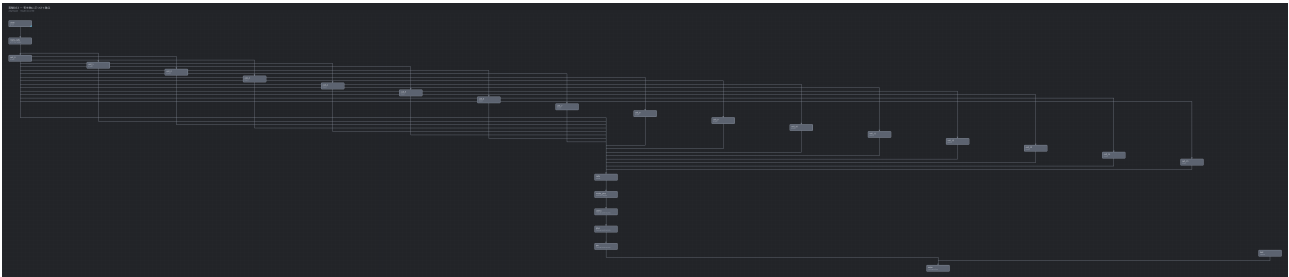
最初の測り方は失敗した。しきい値を 1.0 にしたところ、球のあり／なしで外れた数がまったく同じ 146 本になった。重さだけで全部外れてしまい、球の影響が見えなくなっていたためである。「重さでは外れない」しきい値まで上げて測り直した。

そうすると、しきい値 100~800 では重さだけでは1本も外れず、球を通すと 80／55／55／43／4本 が外れた。球が原因であることは、これで疑いようがない。速さを上げれば外れる数も増える（0.08で0本、0.64で71本）。

ただし「当たった場所に集中する」とは言えなかった。最初に外れる1本は球の通ridorの内側だが、最後には内側 20.0%・外側 21.8%

とほぼ同じ割合になる。破れ始めは局所でも、そのあとは布全体へ広がっていく。

ノードの接続



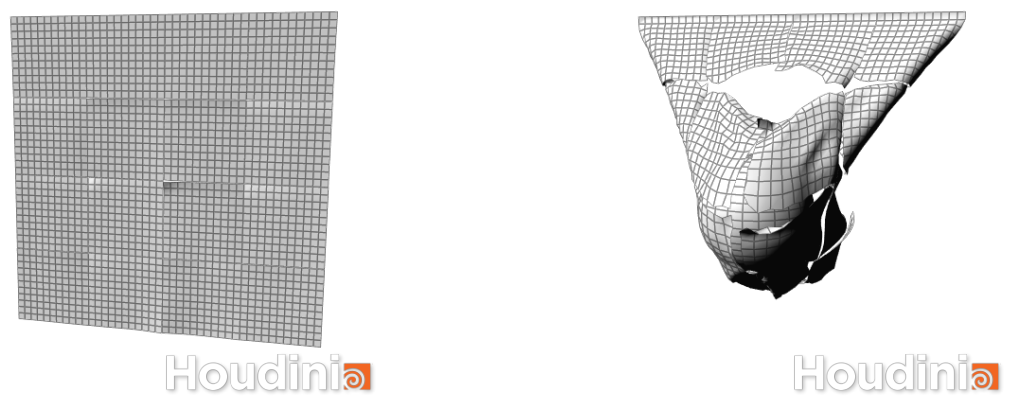
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 45 orient = 0 rows = 45 sizex = 3.0 sizey = 3.0 ty = 2.2
mark_cells	attribwrangle	class = 1 snippet = int cx = int(floor((@P.x + 1.5) / 3.0 * 4)); int cy = int(floor((@P.y - 0.7) / 3.0 * 4)); i@cell = clamp(cx, 0, 3) * ...
cell_0	blast	group = @cell!=0 grouptype = 4
cell_1	blast	group = @cell!=1 grouptype = 4
cell_2	blast	group = @cell!=2 grouptype = 4
cell_3	blast	group = @cell!=3 grouptype = 4

ノード	タイプ	デフォルトから変更したパラメータ
cell_4	blast	group = @cell!=4 grouptype = 4
cell_5	blast	group = @cell!=5 grouptype = 4
cell_6	blast	group = @cell!=6 grouptype = 4
cell_7	blast	group = @cell!=7 grouptype = 4
cell_8	blast	group = @cell!=8 grouptype = 4
cell_9	blast	group = @cell!=9 grouptype = 4
cell_10	blast	group = @cell!=10 grouptype = 4
cell_11	blast	group = @cell!=11 grouptype = 4
cell_12	blast	group = @cell!=12 grouptype = 4
cell_13	blast	group = @cell!=13 grouptype = 4
cell_14	blast	group = @cell!=14 grouptype = 4
cell_15	blast	group = @cell!=15 grouptype = 4
cells	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 3.63) { @group_pins = 1; }
cloth1	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 200.0 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
ball	sphere	radx = 0.42 rady = 0.42 radz = 0.42 rows = 24 tx = 0.0 ty = 2.2 type = 2 tz = -2.6

できたもの

3×3の布を45×45に分割し、4×4のマスキに切り分けて glue で貼り直した。上端だけをピンで留めて吊るし、奥から球（半径0.42）を投げ込む。カメラは2枚とも同じ。



球なし。しきい値200では重さだけでは1本も外れず、そのまま垂れる。球あり。同じカメラ・同じフレーム。穴が開いて破片が落ちていく。

調べたこと	結果
元の布	2,025点 / 1,936面
4×4に切ったあと	2,304点 (+279)
作られた拘束	distance 6,160 / bend 5,456 / stitch 261
球	半径 0.42、1フレームあたり z へ 0.16 進む
球が布を通るフレーム	F17 あたり

1. 球を通すと、外れる貼り目は増えるか

しきい値 1.0 では重さだけで全部外れてしまい、球のあり／なしで差が出なかった。上げていくと、重さでは外れず球でだけ外れる範囲が現れる。

しきい値	球なし	球あり	差	最下点
1.0	146	146	0	-7.979
100.0	0	80	+80	1.044
200.0	0	55	+55	1.031
300.0	0	55	+55	1.127
500.0	0	43	+43	1.190
800.0	0	4	+4	1.201
1600.0	0	0	0	1.201

2. 外れた貼り目は、球の通りの近くに偏っているか（しきい値 200）

球の通り道（x=0, y=2.2）から半径 0.9 の内と外に分ける。内側に 105本、外側に 156本の貼り目がある。位置はシミュレーション前の位置で数えた。

フレーム	外れた計	内で外れた	外で外れた	内の率	外の率
14	0	0	0	0.0%	0.0%

フレーム	外れた計	内で外れた	外で外れた	内の率	外の率
18	0	0	0	0.0%	0.0%
22	1	1	0	1.0%	0.0%
28	37	16	21	15.2%	13.5%
36	55	21	34	20.0%	21.8%

3. 球の速さを変えると、外れる数は増えるか（しきい値 500）

1フレームあたりに進む距離を変えた。同じしきい値のまま、速さだけを変えている。

1フレームあたりの距離	外れた貼り目	最下点
0.08	0本	0.712
0.16	43本	1.190
0.32	56本	0.185
0.64	71本	1.434

わかったこと

- ・最初の測り方は失敗した。しきい値 1.0 で測ったところ、球のあり／なしで外れた数がまったく同じ 14 6本だった。重さだけで全部外れていて、球の影響が埋もれていた。差が出ないときは「効かない」ではなく「他の原因で飽和している」ことを先に疑う。
- ・しきい値を上げると、球の効果がはっきり出た。100～800 では重さだけで 0本、球を通すと 80／55／55／43／4本。球なしが 0 なので、外れた分はすべて球のせいである。
- ・速さを上げると、外れる数も増えた。1フレームあたり 0.08 で 0本、0.16 で 43本、0.32 で 56本、0.64 で 71本。単調に増える。ぶつける強さで破れ具合を決められる。
- ・「当たった場所に集中する」とは言えなかった。最後の時点で、内側 20.0%・外側 21.8%。ほぼ同じ割合である。当初の見立ては外れた。
- ・ただし破れ始めは局所だった。最初に外れた1本（F22）は球の通りの道の内側にある。F28 では内 15.2% 対 外 13.5% でわずかに内側が先行し、F36 で追いつかれる。「当たったところから破れ始め、そのあと全体へ広がる」が正しい言い方になる。
- ・広がる理由は測っていないが、布がぶつけられて大きく揺れるので、その動き自体が残りの貼り目に力をかけていると考えられる。ここは確かめていないので断定しない。局所に留めたいなら、球を小さく速くするか、揺れが収まる前にフレームを切る必要がある。
- ・貼り目の見分け方でつまずいた。最初は位置で照合したが、布は動くので、シミュレーション前の位置と後の位置が一致しない。ほぼ全部が「外れた」と数えられ、内 100%・外 96.2% という無意味な結果が出た。つないでいる点の番号で見分けるように直した。番号は動かない。
- ・球は sphere の type を Polygon Mesh にしている。実験030で測ったとおり、Polygon では rows / cols が効かない。

次にやること

- ・球を小さく速くして、破れを当たった場所に留められるか試す
- ・破れた縁を見せる絵にする（いまは確認用の速い描画）
- ・034 以降、まだ触っていない分野に進む

HeightField で地形を作る — 高さを面ではなく数の並びで持つ

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 4 / 接続 3

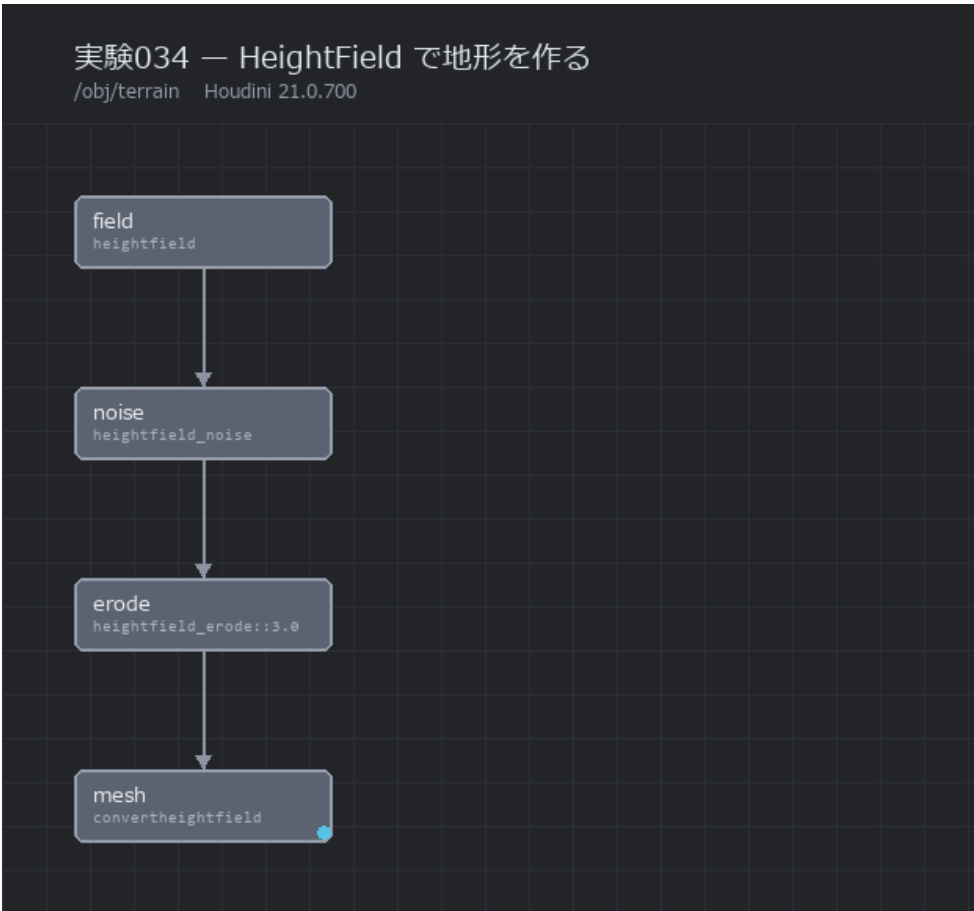
何を作ったか

実験007では grid に mountain をかけて地形を作った。点を並べて高さをずらす、ポリゴンのやり方である。Houdini にはもう1つ HeightField という仕組みがあり、こちらは専用のノードが45種もある。

違いは持ち方にある。HeightField は高さを2次元のボリュームとして持つ。面も点も持たず、プリミティブはたった2つ（height と mask）。そのぶん、ポリゴンでは難しい浸食（erode）のような計算ができる。

測って分かったこと。解像度は $\text{size} \div \text{gridspacing}$ で決まり、Grid Samples という名前のパラメータは 128 にしても 1024 にしても解像度を1ミリも動かさない。浸食はレイヤーを2枚から7枚に増やし、勾配を32.7%下げる。amp は純粋な倍率で、4通り試して $\text{幅} \div \text{amp}$ が小数点以下6桁まで同じだった。

ノードの接続



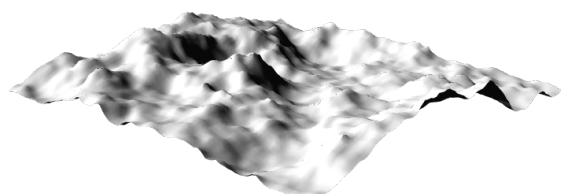
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size _x = 200.0 size _y = 200.0

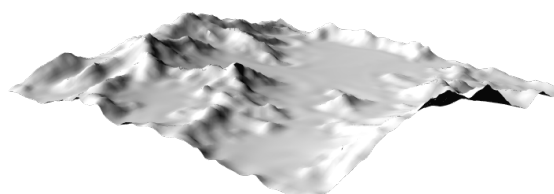
ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 30.0 elementsiz = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 2 seed = 0.5160613059997559
mesh	convertheightfield	(変更なし)

浸食のあるなし

同じノイズから作った地形。画は起伏を見せるため amp を 120 にしている（amp は純粋な倍率なので形は変わらない）。カメラは2枚とも同じ。



ノイズをかけただけ。丸い起伏が並ぶ。



浸食を4回。尾根と谷ができ、山肌に筋が出る。

調べたこと	結果
HeightField の中身	2点 / 2プリミティブ (height と mask のボリューム)
高さの解像度	100×100×1
メッシュに直すと	10,000点 / 9,801面
点の数と解像度	100×100 = 10,000 でぴったり一致

1. 解像度は何が決めるか

size と gridspacing を変えて、できた高さボリュームの解像度を読んだ。

size	gridspacing	解像度	size ÷ gridspacing
200	2.0	100×100	100
200	1.0	200×200	200
200	4.0	50×50	50
400	2.0	200×200	200
100	2.0	50×50	50

2. Grid Samples は効くのか

名前からすると解像度を決めそうだが、size 200 / gridspacing 2 のまま値だけ変えた。

gridsamples	解像度
128	100×100
512（既定）	100×100
1024	100×100

3. 浸食は何を変えるか

heightfield_erode の Iterations per Frame を変えた。解像度は 100×100。

回数	かかった秒	レイヤー数	最低	最高	平均	勾配
0（かけない）	0.04	2	-3.747	4.993	0.330	0.07893
1	0.68	7	-2.626	4.926	0.288	0.06710
2	0.80	7	-2.619	4.728	0.255	0.06124
4	2.35	7	-2.609	4.615	0.196	0.05315

4. amp は高さの倍率か（実験006と同じ問い）

heightfield_noise の Amplitude だけを変えて、高さの幅を測った。

amp	高さの幅	幅 ÷ amp
7.5	2.1850	0.291329
15.0	4.3699	0.291329
30.0	8.7399	0.291329
60.0	17.4797	0.291329

わかったこと

- ・ HeightField はプリミティブが2つしかない。height と mask という2枚のボリウムで、点は2個。同じ地形をメッシュに直すと 10,000点 / 9,801面になる。同じ情報を、まったく違う入れ物で持っている。
- ・ 点の数は解像度そのものだった。100×100 の高さボリウムをメッシュにすると、ちょうど 10,000点。マス目1つが点1つになる。
- ・ 解像度は $\text{size} \div \text{gridspacing}$ で決まる。5通り試してすべて一致した。細かくしたいなら gridspacing を下げる。
- ・ Grid Samples は効かなかった。128 にしても 1024 にしても 100×100 のまま。名前からは解像度を決めそうに見えるが、動かない。この2つが並んでいるのが、最初につまずくところになる。
- ・ 浸食はレイヤーを2枚から7枚に増やす。増えるのは debris（がれき）・sediment（堆積）・flow（流れ）・flowdir.x・flowdir.y（流れの向き）。削った結果だけでなく、その過程も残る。あとで色を塗り分けるのに使える。
- ・ 浸食は地形をなだらかにした。勾配は 0.07893 から 0.05315 へ、32.7%下がった。見立てどおり。
- ・ 削って、積もっている。最高は 4.993 → 4.615 と下がり、最低は -3.747 → -2.609 と上がった。山が削れて谷が埋まる。平均も 0.330 → 0.196 と下がっており、全体として土が低いところへ移っている。

- ・時間は回数に比例しない。1回で 0.68秒、2回で 0.80秒、4回で 2.35秒。1回目に下ごしらえの費用がかかり、そのあとは増えていく。100×100 という小さな地形でこれなので、実用の解像度では相応に重い。

- ・amp は純粋な倍率だった。7.5 / 15 / 30 / 60 の4通りで、幅÷amp が 0.291329 と小数点以下6桁まで同じ。実験006で mountain の height について確かめたことが、HeightField 側でも同じように成り立つ。

- ・ただしamp の数字がそのまま高さになるわけではない。amp 30 で実際の幅は 8.74。0.29倍ほどに収まる。狙った高さにしたいなら、この比率を測ってから逆算する。

次にやること

- ・浸食のレイヤー (debris / sediment / flow) で色を塗り分ける
- ・heightfield_scatter で地形の上に物を生やす (実験008の応用)
- ・035 以降、まだ触っていない分野に進む

地形の上に生やし分ける ― 浸食が残した情報で木の場所を決める

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 10 / 接続 11

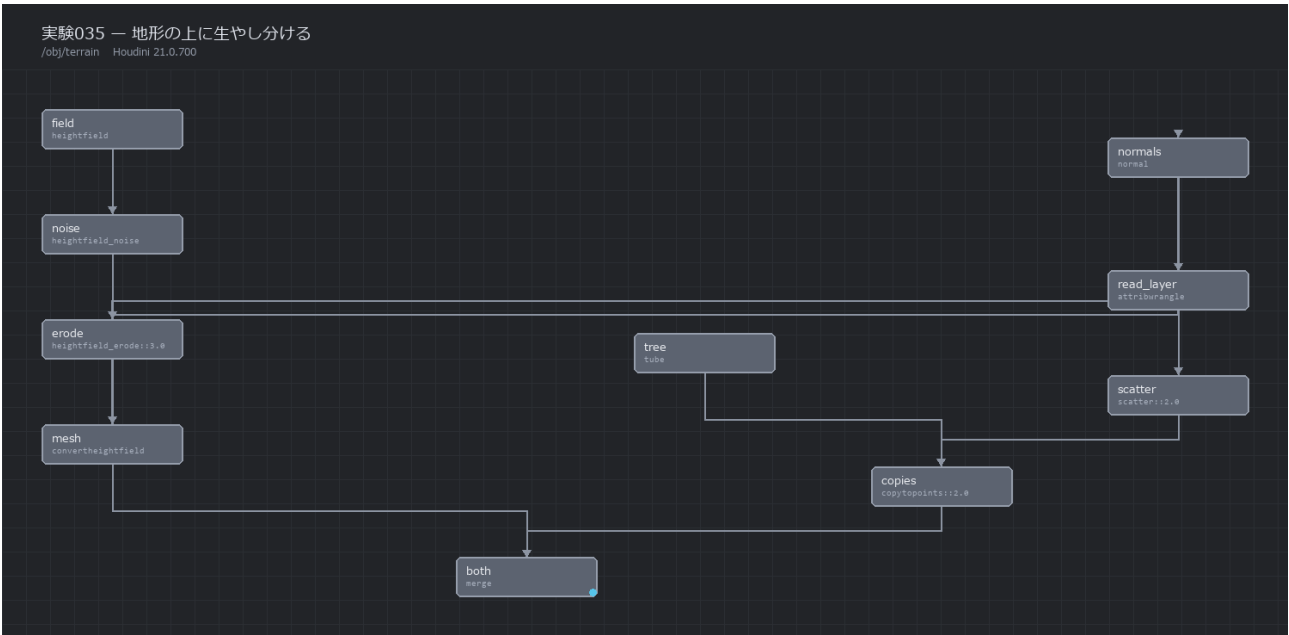
何を作ったか

実験008では平らな地面に木を200本立てた。どこに立てるかは完全に運任せで、地形の形とは関係がなかった。

実験034で、浸食が過程をレイヤーとして残すことが分かった。がれき・堆積・流れ。このうち sediment（堆積）は「土が溜まったところ」を表しているはずなので、これを散布の重みに使えば、崖には生えず平らなところに生える地形が自動で作れる。

狙いどおりになった。まず sediment 自体が、緩いところに急なところの 170.64倍溜まっている。これを重みに使うと、生えた場所の勾配の平均が 0.06474 から 0.00332 へ、94.9%下がった。木は崖を避け、土の溜まった谷筋に集まる。

ノードの接続



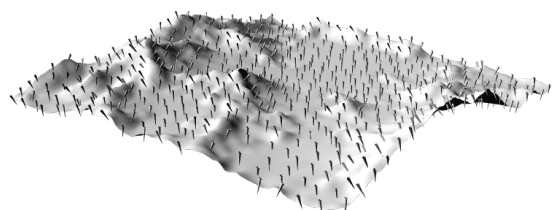
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	sizeX = 200.0 sizeY = 200.0
noise	heightfield_noise	amp = 120.0 elementsSize = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.2596234083175659
mesh	convertheightfield	(変更なし)
normals	normal	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
read_layer	attribwrangle	snippet = f@layer = volumesample(1, "sediment", @P); // 勾配も点ごとに持たせる。あとで「どこに生えたか」を測るのに使う vector up = set(0, 1, 0); f@slope = 1.0...
scatter	scatter::2.0	densityattrib = layer npts = 600 usedensityattrib = 1
tree	tube	cols = 8 height = 6.0 rad1 = 0.6 rad2 = 0.05 rx = 90.0 type = 1
copies	copytopoints::2.0	(変更なし)
both	merge	(変更なし)

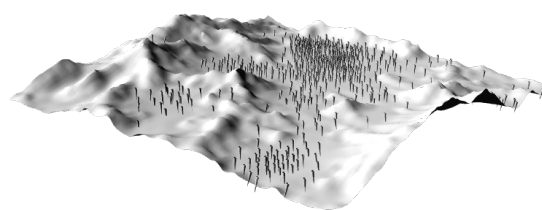
重みのあるなし

同じ地形・同じ本数（600本）。違いは「どこに置くかの重み」だけ。カメラは2枚とも同じ。



Houdini

重み無し。崖も尾根も関係なく、一様にばらまかれる。



Houdini

sediment を重みに。崖が裸になり、谷筋に集まる。

段取り	使うノード
地形を作って浸食する	heightfield → heightfield_noise → heightfield_erode
メッシュに直す	convertheightfield
面の向きを作る	normal（これが無いと複製が回らない）
レイヤーを点に読み込む	attribwrangle で volumesample
重みを付けて散布	scatter の密度アトリビュートに指定
木を複製する	copy to points

1. 浸食が残したレイヤーの中身

100×100 のマス目それぞれが持つ値。「値のあるマス」は 0 でないマスの割合。

レイヤー	最小	最大	平均	値のあるマス
height（高さ）	-10.4156	18.1462	0.6104	49.1%
mask	0.0000	0.0000	0.0000	0.0%
debris（がれき）	0.0000	2.5569	0.0452	32.5%

レイヤー	最小	最大	平均	値のあるマス
sediment（堆積）	0.0000	11.0788	0.8346	49.5%
flow（流れ）	0.0000	4.1504	0.8009	100.0%
flowdir.x	-4.1177	2.9576	-0.1818	39.4%
flowdir.y	-2.2426	4.0191	0.0780	50.6%

2. sediment は本当に「緩いところ」に溜まっているか

メッシュの点 10,000個を、勾配の中央値で2つに分けて比べた。

どちら	sediment の平均
急なほう（勾配が中央値より上）	0.00973
緩いほう（勾配が中央値より下）	1.65957
緩いほうは急なほうの	170.64倍

3. 重みを付けると、生える場所はどう変わるか

どちらも600本。生えた場所の勾配と sediment の値を測った。

	本数	勾配の平均	勾配の中央値	sediment の平均
重み無し	600	0.06474	0.03674	0.75115
重みあり	600	0.00332	0.00063	4.23520
変化	—	-94.9%	-98.3%	5.64倍

わかったこと

- ・ sediment は緩いところに 170.64倍 溜まっていた。勾配の中央値で2つに分けると、急なほうの平均が 0.00973、緩いほうが 1.65957。浸食が「どこに土が溜まったか」をそのまま記録している。

- ・ 重みを付けると、生える場所の勾配が 94.9% 下がった。平均 0.06474 → 0.00332、中央値は 0.03674 → 0.00063 で 98.3% 下がる。木は崖を避ける。

- ・ 生えた場所の sediment の値は 5.64倍になった。0.75115 → 4.23520。狙ったレイヤーの値が高いところへ、はっきり寄っている。

- ・ mask は最初から最後まで全部ゼロだった。最小も最大も 0.0000。heightfield が作る2枚のうち1枚は、何か塗らないかぎり空のままである。「レイヤーがある」と「値が入っている」は別の話。

- ・ flow だけは 100% のマスに値がある。他のレイヤーは 32~51% なのに対し、流れはどこにでもある。地形全体をなでる量なので、散布の重みには向かない。使うレイヤーは、値の散らばり方を見てから選ぶ。

- ・ レイヤーを点に持ってくるのは volumesample 1行。f@layer = volumesample(1, "sediment", @P); と書き、2本目の入力に浸食後の HeightField をつなぐ。実験026で炎の温度を読んだのと同じやり方が、そのまま地形にも効く。

- ・ normal は scatter の前に置く。実験008・030で確かめたとおり、scatter は N を作らない。引き継ぐだけなので、上流で作っておかないと木が全部同じ向きに立つ。

- ・ 木のテンプレートは tube を X に 90度回して Z 軸向きにしている。copy to points はテンプレートの Z 軸を N に合わせる（実験008）。回さないと全部横倒しになる。

次にやること

- flow の向き (flowdir) を使って、草の流れに沿って寝かせる
- debris と sediment で色を塗り分ける
- 036 以降、まだ触っていない分野に進む

流れの向きに草を寝かせる — flowdir は x と y が入れ替わっている

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 11 / 接続 12

何を作ったか

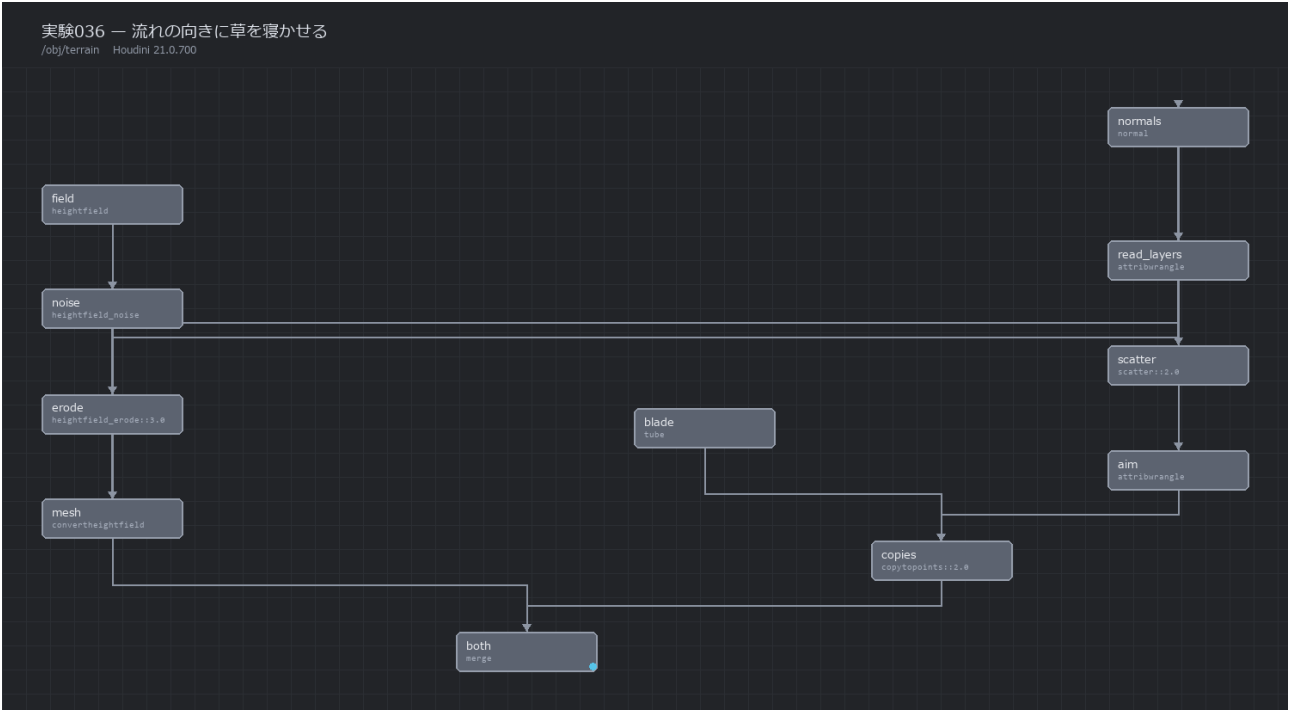
実験035では浸食のレイヤーのうち sediment（堆積）だけを使った。残っているのが flowdir.x と flowdir.y、水が流れた向きである。名前からすると下り坂の向きのはずだが、確かめていない。

素直に書いたら外れた。flowdir.x を3Dのx、flowdir.y を3Dのz に入れると、高さから作った下り坂の向きとの角度が平均96.48度。でたらめ（90度）とほとんど変わらない。

そこで x と y の割り当てを8通り総当たりで測った。1つだけ明らかに違う値が出た。入れ替えた (fy, 0, fx) が平均39.95度・中央29.94度で、30度未満が50.1%を占める。x と y は入れ替えて使う。

向きが決まれば、あとは N に入れるだけ。複製の軸と狙った向きのずれは900本すべてで0.000度、真上からの傾きは0度から90度になった。草はびたりと寝る。

ノードの接続



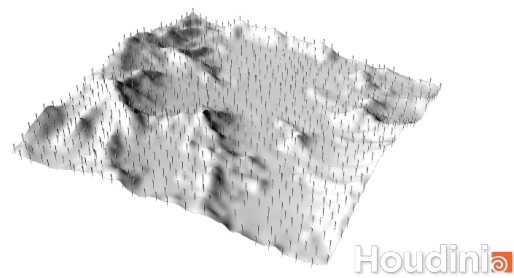
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size_x = 200.0 size_y = 200.0
noise	heightfield_noise	amp = 120.0 elements_size = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.35988283157348633
mesh	convertheightfield	(変更なし)

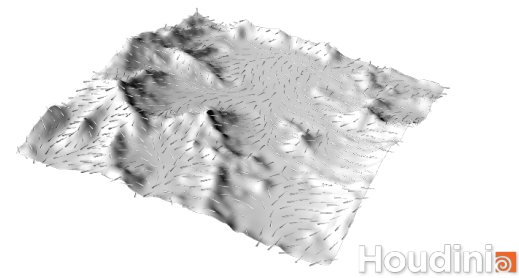
ノード	タイプ	デフォルトから変更したパラメータ
normals	normal	(変更なし)
read_layers	attribwrangle	snippet = f@layer = volumesample(1, "sediment", @P); float fx = volumesample(1, "flowdir.x", @P); float fy = volumesample(1, "f...
scatter	scatter::2.0	npts = 900
aim	attribwrangle	snippet = if (length(v@flowvec) > 1e-6) { @N = normalize(v@flowvec); } else { @N = set(0, 1, 0); }
blade	tube	cols = 6 height = 7.0 rad1 = 0.35 rad2 = 0.02 rx = 90.0 type = 1
copies	copytopoints::2.0	(変更なし)
both	merge	(変更なし)

向きのあるなし

同じ地形・同じ900本。違いは N に何を入れるかだけ。カメラは2枚とも同じ。



N は真上。草はまっすぐ立ち、向きの情報が無い。



N に流れの向き。草が寝て、水の通り道が筋になって見える。

調べたこと	結果
メッシュの点	10,000個
流れのある点	9,996個 (99.96%)
草	900本
convertheightfield が持ち込む点アトリビュート	debris, flow, flowdir_x, flowdir_y, height, mask, sediment, uv

1. x / y の割り当てを8通り総当たり

flowdir.x と flowdir.y を、3D の x と z にどう割り当てるか。高さから作った下り坂の向きとの角度を測った。でたらめなら、どれも平均90度になる。

割り当て	平均	中央値	判定
(fx, 0, fy) 素直な並び	96.48度	99.55度	でたらめと同じ
(-fx, 0, -fy)	83.52度	80.45度	でたらめと同じ

割り当て	平均	中央値	判定
(fy, 0, fx) 入れ替え	39.95度	29.94度	これが正しい
(-fy, 0, -fx)	140.05度	150.06度	逆向き
(fx, 0, -fy)	93.42度	94.01度	でたらめと同じ
(-fx, 0, fy)	86.58度	85.99度	でたらめと同じ
(fy, 0, -fx)	90.56度	88.58度	でたらめと同じ
(-fy, 0, fx)	89.44度	91.42度	でたらめと同じ

2. 正しい割り当てでの、下り坂との一致具合

(fy, 0, fx) で測った角度の分布。0度なら完全に下り坂を向いている。

角度	その範囲に入る割合
30度未満	50.1%
60度未満	78.5%
90度未満	90.5%
120度未満	95.3%
平均 / 中央値	39.95度 / 29.94度

3. 複製は、入れた向きを向くか

草1本ごとに、その軸と N の角度を測った。900本すべて。

N に入れたもの	狙いとのずれ（平均）	狙いとのずれ（最大）	真上からの傾き	ばらつき
真上 (0,1,0)	0.000度	0.000度	0.00度	0.00
流れの向き	0.000度	0.000度	90.00度	0.00

わかったこと

- ・素直に書くと外れる。flowdir.x を3Dのx、flowdir.y を3Dのz

に入れると、下り坂との角度が平均96.48度。でたらめ（90度）とほとんど変わらない。

- ・x と y は入れ替えて使う。8通り試して、set(fy, 0, fx)

だけが平均39.95度・中央29.94度まで下がる。他の7通りはすべて

83~140度で、でたらめか逆向きだった。1つだけ明らかに外れた値が出るので、迷わず決められる。

- ・総当たりが効いた。「合わないから諦める」のでも「たぶんこうだろうと決め打つ」のでもなく、ありうる並びを全部試して数字で選んだ。8通りしかないので、総当たりが一番速い。

- ・正しい割り当てなら、半分が30度未満に入る。50.1%が30度未満、90.5%が90度未満。完全に一致はしないが、これは浸食の流れが局所の傾きだけで決まらない（上流から流れてきた勢いが残る）ためと考えられる。ここは確かめていないので断定しない。

- ・複製は入れた向きをぴたりと向いた。900本すべてで、狙いとのずれが 0.000度。真上を入れれば 0度（立つ）、流れの向きを入れれば 90度（寝る）。ばらつきも 0.00。

- ・草の軸の測り方でつまづいた。最初は複製の最初の点と最後の点を結んで軸にしたが、ぴったり180度ずれた。tube は2つの輪でできていて、輪の並びが想定と逆だった。ずれが「ちょうど180.000度」なので、誤差ではなく向きの取り違いだと分かる。

- ・ 下り坂の向きは、法線からは作れなかった。normal SOP は N を点ではなくバーテックスに書く（実験030で確認済み）。点のラングルから @N を読むと空になる。高さのボリュームから直接、隣との差で作直した。

- ・ convertheightfield はレイヤーを点アトリビュートとして持ち込む。メッシュの時点で flow や sediment が点に付いている。同じ名前で書こうとすると衝突する。最初 v@flow と書いて、既にある float の flow とぶつかり、ベクトルのつもりが数値1つになっていた。

次にやること

- ・ debris と sediment で地形を塗り分ける
- ・ 草の長さを流れの強さで変える
- ・ 037 以降、まだ触っていない分野に進む

浸食のレイヤーで地形を塗り分ける ー 色が地形の形と対応しているか

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 5 / 接続 5

何を作ったか

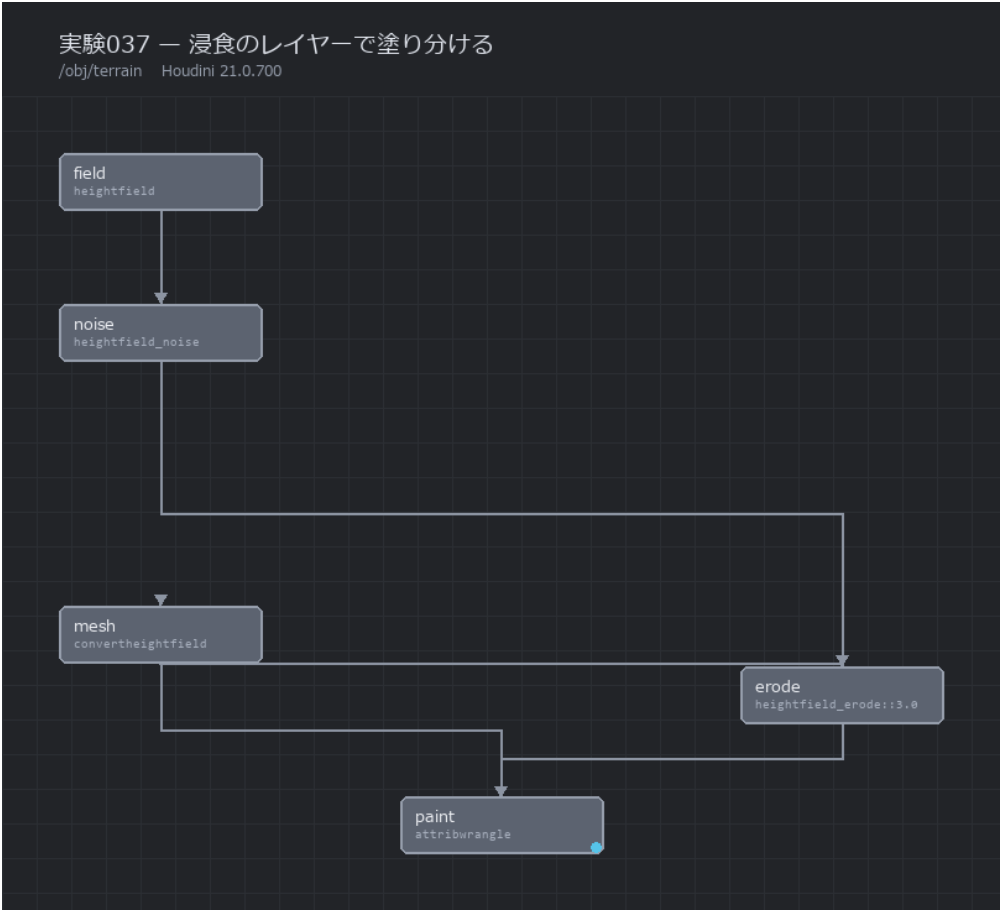
実験034で、浸食が過程をレイヤーとして残すことが分かった。035では sediment を散布の重みに、036では flowdir を向きに使った。今回は色にする。

見た目を作るだけなら適当に塗っても絵にはなる。そうではなく、塗り分けが地形の形と対応しているかを数字で確かめた。

対応していた。岩肌に塗った場所の勾配の平均は 0.44733、土に塗った場所は 0.07076 で、岩肌は土の 6.32倍 急である。水筋はその中間の 0.21210 で、これも筋が通る（水は斜面を下って谷に集まる）。

しきい値は勘で決めず、レイヤーの値の分布を見てから決めた。sediment は半分以上のマスがゼロで、下から75%でようやく 0.8157。この形を見てから 0.30 を境にした。

ノードの接続



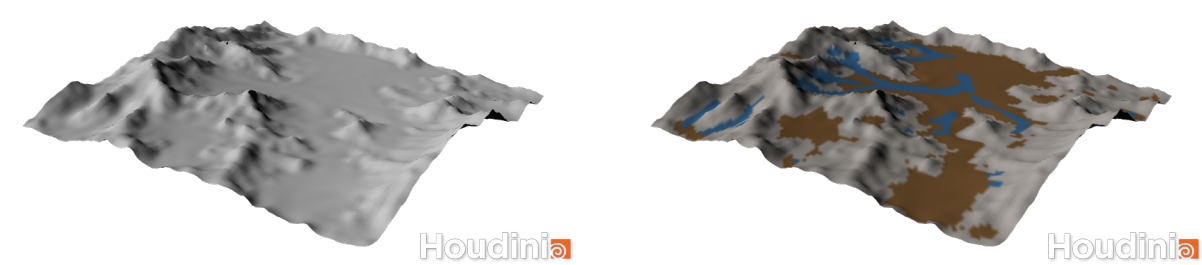
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size _x = 200.0 size _y = 200.0
noise	heightfield_noise	amp = 120.0 elementsize = 80.0

ノード	タイプ	デフォルトから変更したパラメータ
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.2140711545944214
mesh	convertheightfield	(変更なし)
paint	attribwrangle	snippet = f@sediment2 = volumesample(1, "sediment", @P); f@flow2 = volumesample(1, "flow", @P); f@debris2 = volumesample(1, "de...

塗り分けのあるなし

同じ地形。違いは @Cd に何を入れるかだけ。カメラは2枚とも同じ。



塗り分け無し。形は見えるが、どこが土でどこが岩か分からない。浸食のレイヤーで塗り分け。灰 = 岩肌、茶 = 土、青 = 水筋。

色	意味	条件
灰 (0.62, 0.60, 0.58)	岩肌。土が流れ去って何も溜まっていない	どちらにも当てはまらない
茶 (0.45, 0.32, 0.18)	土。堆積が積もった	sediment ≥ 0.30
青 (0.20, 0.38, 0.55)	水筋。水が集まって流れた	flow ≥ 1.60 (こちらが優先)

1. しきい値を決めるために、値の分布を見る

100×100 のマス目 10,000個ぶんの値を、下から何%の位置かで見た。

レイヤー	最小	下から10%	25%	50%	75%	90%	最大
sediment	0.0000	0.0000	0.0000	0.0000	0.8157	2.9817	11.0788
flow	0.0000	0.0969	0.3697	0.6956	1.1336	1.5871	4.1504
debris	0.0000	0.0000	0.0000	0.0000	0.0014	0.0854	2.5569

2. 分けた3つは、勾配で見て別のものか

塗り分けたあとで、それぞれの勾配・堆積・流れの平均を出した。勾配は高さから直接作っている。

種類	点の数	割合	勾配の平均	勾配の中央値	堆積の平均	流れの平均
岩肌	6,370	63.7%	0.44733	0.40695	0.01660	0.59386
土	2,653	26.5%	0.07076	0.05448	2.39543	0.83297
水筋	977	9.8%	0.21210	0.13315	1.93003	2.06320

わかったこと

- ・塗り分けは地形の形と対応していた。岩肌の勾配の平均が 0.44733、土が 0.07076。岩肌は土の 6.32倍 急である。中央値で見ても 0.40695 対 0.05448 で 7.5倍。色は模様ではなく、形の情報を持っている。
- ・水筋は中間に入った。勾配の平均 0.21210 で、岩肌より緩く土より急。水は斜面を下って谷に集まるので、その通り道は「急でも緩でもない途中」になる。狙って作った並びではないが、筋が通っている。
- ・しきい値は勘で決めない。まずレイヤーの値の分布を出した。sediment は半分以上のマスがゼロで、下から75%の位置でようやく 0.8157。この形を見てから 0.30 を境にした。分布を見ずに「0.5くらい」と決めていたら、土がほとんど出ないところだった。
- ・debris は使えなかった。下から90%の位置でも 0.0854 しかなく、ほとんどのマスが実質ゼロ。レイヤーがあることと、塗り分けに使えることは別の話。
- ・flow は逆に、どこにでも値がある。下から10%でも 0.0969。実験035で「100%のマスに値がある」と測ったとおりで、ゼロとの比較では分けられない。上位の値（1.60 以上、全体の約1割）で切って初めて筋になる。
- ・優先順位を決める必要があった。水筋のところは堆積も多い（堆積の平均 1.93）ので、条件が重なる。水筋を先に判定して、残りを土と岩肌に分けた。順番を逆にすると水筋が土に飲まれて消える。
- ・使われた色はちょうど3種で、割合は 63.7% / 26.5% / 9.8%。塗り残しも、意図しない中間色も無い。
- ・勾配はnormal SOP の N からではなく、高さのボリュームから隣との差で作った。normal SOP は N を点ではなくバーテックスに書くため（実験030・036）、点のラングルからは読めない。

次にやること

- ・岩肌と土で、表面の細かさ（ノイズ）を変える
- ・この塗り分けを Karma で見せる絵にする
- ・038 以降、まだ触っていない分野に進む

塗り分けた地形を Karma で出す — 見立ては外れ、白飛びが本命だった

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 6 / 接続 6

何を作ったか

実験037で地形を3色に塗り分けたが、あれは確認用の速い描画（OpenGL）で、点に付けた @Cd がそのまま画になっていた。見せる絵にするときは Karma を使う。

見立ては「材質を割り当てないと点の色は出ない」だった。外れた。材質なしでも色は出る。principled shader の「点の色を使う」も既定でオンで、割り当てても色の出方は変わらない。

代わりに別の問題が出た。白飛びである。材質を割り当てないと、地形の 43.4% の画素が白く飛ぶ。割り当てると 3.1% に落ちる。14倍の差で、これは色ではなく明るさの話だった。

さらに、白飛びの原因はライトではなかった。材質を割り当てたあとはキーライトを 2.4 から 0.3 へ8分の1にしても、白飛びは 3.0% から 2.9% にしか変わらない。全体の明るさは 167.66 から 110.22 へ下がるのに、飛んでいるところは飛んだままである。

ノードの接続



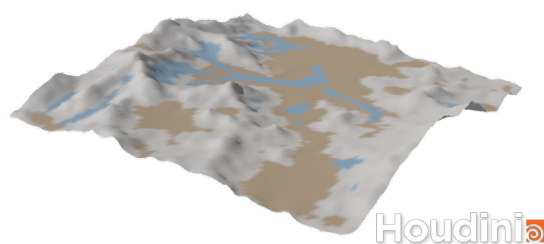
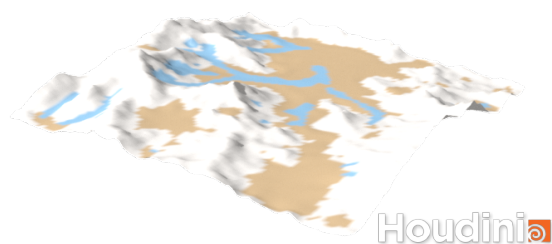
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size_x = 200.0 size_y = 200.0

ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 120.0 elementsiz = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.543621301651001
mesh	convertheightfield	(変更なし)
paint	attribwrangle	snippet = float sed = volumesample(1, "sediment", @P); float flw = volumesample(1, "flow", @P); int water = (flw >= 1.6) ? 1 : ...
assign	material	shop_materialpath1 = /mat/terrain_mat

材質のあるなし（ライトは同じ）

実験024で煙に合わせたライト設定のまま。カメラも解像度も同じ。違いは材質を割り当てるかどうかだけ。



材質なし。色は出るが、岩肌が真っ白に飛ぶ（43.4%）。

材質あり。同じライト。白飛びは 3.1% まで落ちる。

	時間	白飛び	茶	青	灰
材質なし	4.13秒	43.4%	27.4%	7.2%	65.5%
材質あり	3.55秒	3.1%	24.7%	6.4%	68.9%

1. ライトを弱くすると、白飛びは減るか

材質を割り当てた状態で、キーライトだけを8分の1まで落とした。ドームライトは 1.0 で固定。

キーライト	時間	白飛び	明るさ	ばらつき	茶	青
2.4	3.40秒	3.0%	167.66	26.30	23.8%	6.2%
1.2	3.35秒	3.0%	138.95	25.89	22.9%	5.5%
0.6	3.28秒	3.0%	120.93	26.91	21.0%	5.0%
0.3	3.20秒	2.9%	110.22	28.04	19.7%	4.3%

2. OpenGL と Karma

同じカメラ・同じ解像度（720×460）。Karma はキーライト 0.3、サンプル 24。

	時間	白飛び	明るさ	ばらつき	茶	青
OpenGL	1.29秒	2.9%	103.50	44.51	27.4%	6.8%

	時間	白飛び	明るさ	ばらつき	茶	青
Karma	3.20秒	2.9%	110.22	28.04	19.7%	4.3%
比	2.5倍	同じ	1.06倍	0.63倍	—	—

わかったこと

- ・見立ては外れた。点の色は材質なしでも出る。「材質が無いと Karma は色を出しようがない」と考えていたが、既定の材質がちゃんと @Cd を読む。principled shader の「点の色を使う」も既定でオンなので、割り当てても色の出方は変わらない。
- ・代わりに白飛びが出た。材質を割り当てないと、地形の 43.4% の画素が白く飛ぶ。割り当てると 3.1%。14倍の差で、これは色ではなく明るさの話だった。
- ・白飛びの原因はライトではなかった。材質を割り当てたあととは、キーライトを 2.4 から 0.3 へ8分の1にしても、白飛びは 3.0% から 2.9% にしか変わらない。全体の明るさは 167.66 → 110.22 と下がるのに、飛んでいるところは飛んだまま。ライトを絞っても直らない飛び方がある。
- ・測り方で一度失敗した。最初は「明るい画素は背景」として地形と背景を分けたが、白飛びした地形まで背景として捨ててしまい、白飛びが 0.0% と出た。書き出したPNGは透明度を持っているので、そちらで分け直したら 43.4% になった。測りたいものが「真っ白」なのに、真っ白を捨てる仕分けをしていた。
- ・Karma は OpenGL の 2.5倍の時間。720×460・サンプル24 で 3.20秒 対 1.29秒。実験018・024で煙を撮ったときは5倍前後だったので、面だけの地形はボリュームより差が小さい。
- ・ばらつき（明暗の差）は Karma のほうが小さい。44.51 対 28.04 で 0.63倍。ドームライトが影を埋めるためで、「本番のレンダのほうがコントラストが強い」とは限らない。
- ・ライトは題材ごとに決め直す。実験024で煙に合わせた設定（キー 2.4）をそのまま地形に使うと飛ぶ。同じ設定を使い回せるのは、同じ種類のものを撮るときだけ。

次にやること

- ・ドームライトの強さも振って、影の出方を測る
- ・地形に空（環境マップ）を入れて、水面に映り込ませる
- ・039 以降、まだ触っていない分野に進む

Copernicus でテクスチャを作る ー 細かさは解像度をどう食うのか

2026-09-14 | Houdini 21.0.700 | /obj/cops | ノード 2 / 接続 1

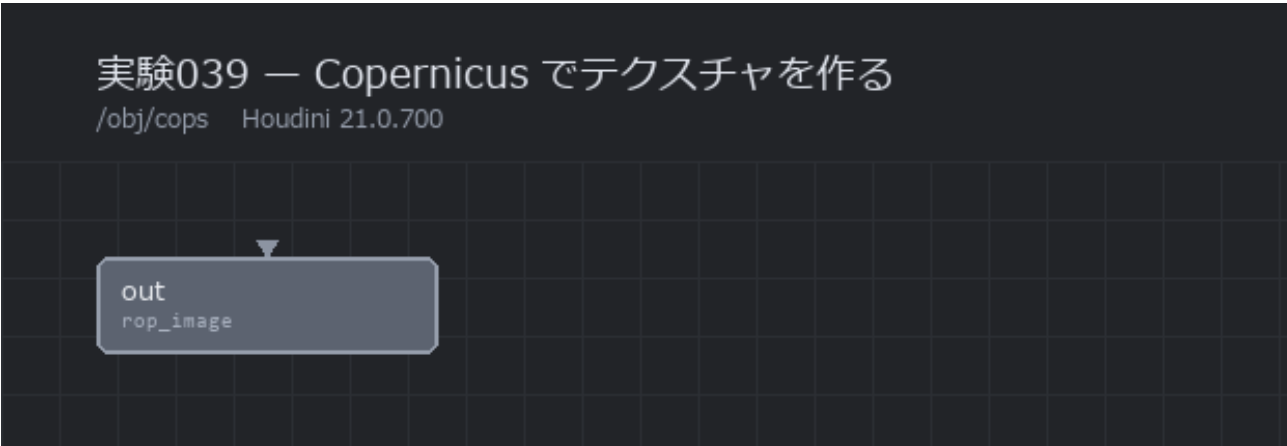
何を作ったか

ここまではすべて形（ジオメトリ）の話だった。Houdini には画そのものを作る仕組みもある。Copernicus（COP）で、専用のノードが262種ある。地形に貼る模様や汚し、マスクを、外で作って読み込むのではなく Houdini の中で手続きとして作れる。

解像度はノイズのノードではなく、ネットワーク（copnet）の側で決まる。しかも「解像度を指定する」のスイッチは既定でオフで、そのときは欄に書いてある 1024×1024 がそのまま使われる。オフなのに値は効いている。

細かさの指標は、実験007と同じ問いを投げた。画素どうしの差をそのまま見ると、解像度を2倍にするたびに約半分になる（6.4940 → 1.0266）。ただしこれは画素が細かくなったぶん隣が近づいただけで、解像度を掛けて「画全体を1とした長さあたりの差」に直すと 831.2 → 945.0 → 1038.4 → 1051.3 と増えて頭打ちになる。増えなくなったところが、その模様に足りている解像度である。実験007で地形について言ったことが、画でもそのまま成り立つ。

ノードの接続



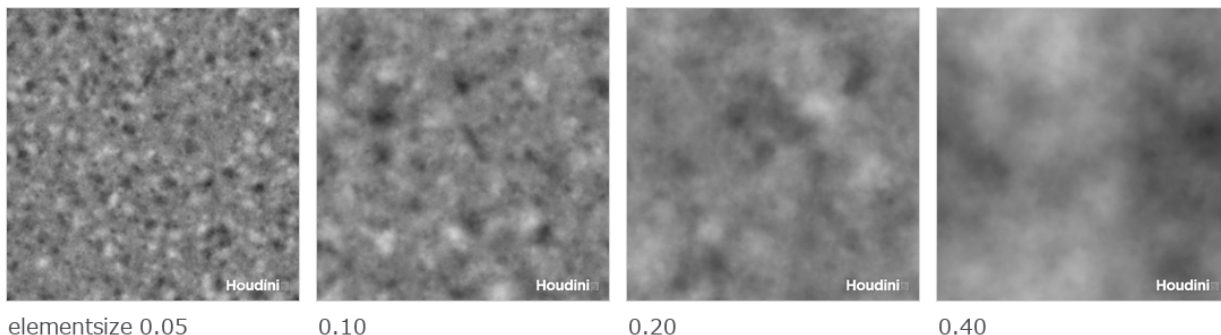
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
noise	fractalnoise	folder0 = 1 folder1 = 1 folder2 = 1
out	rop_image	aov1 = C outputaovs = 1

パラメータを変えたときの見え方

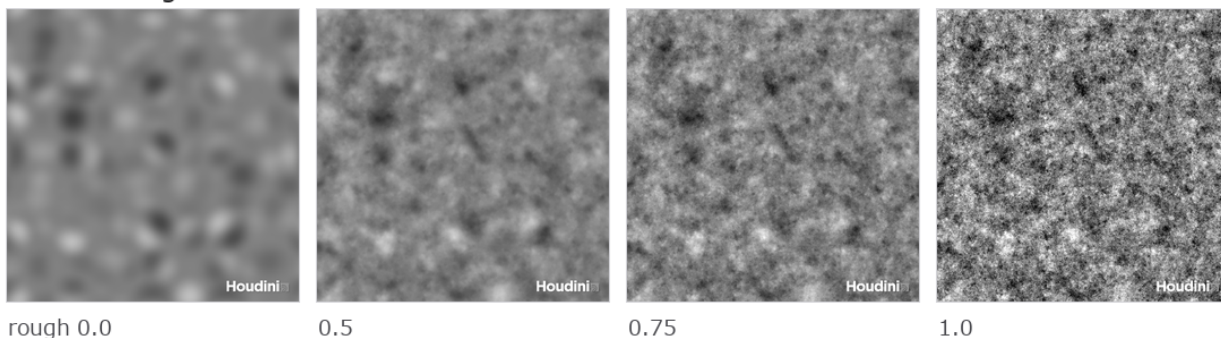
どちらも 1024×1024。上段はうねりの大きさ、下段は細かさを変えている。

うねりの大きさ (elementsize) を変える



elementsizes 0.05 / 0.10 / 0.20 / 0.40。数字が大きいくほど、うねりが大きくなる。

細かさ (rough) を変える



rough 0.0 / 0.5 / 0.75 / 1.0。1.0 では砂嵐に近くなる。

項目	値
使ったノード	copnet → fractalnoise → rop_image
Copernicus のノード	262種
解像度を決める場所	copnet の res1 / res2 (ノイズ側には無い)
書き出した階調	206～215 (8ビットPNG の上限は256)

1. 解像度はどこで決まるか

copnet の「解像度を指定する (setres)」と res1/res2 を変えて、書き出した画の大きさを見た。

設定	書き出した大きさ	階調の数
setres オフ (既定)	1024×1024	210
setres オン / 256	256×256	206
setres オン / 512	512×512	215
setres オン / 1024	1024×1024	210

2. うねりの大きさ (elementsize)

1024×1024 で elementsizes だけを変えた。細かさ = 隣り合う画素の差の平均 (0～255の目盛)。

elementsizes	明るさ	ばらつき	細かさ
0.05	128.00	20.69	1.7913
0.10	129.22	19.79	1.0266

elementsize	明るさ	ばらつき	細かさ
0.20	128.43	16.94	0.5752
0.40	127.50	24.88	0.3657

3. 細かさ (rough)

1024×1024 で rough だけを変えた。

rough	明るさ	ばらつき	細かさ
0.0	129.28	16.92	0.4020
0.25	129.23	17.58	0.4841
0.50	129.22	19.79	1.0266
0.75	129.25	25.72	5.6503
1.0	129.29	49.17	32.9725

4. 細さは解像度によらない量になるか（実験007と同じ問い）

同じノイズ (elementsize 0.10) を解像度だけ変えて書き出した。「細かさ×解像度」は、画全体を1とした長さあたりの差にあたる。

解像度	細かさ	前の行との比	細かさ×解像度	前の行との比	ばらつき
128	6.4940	—	831.2	—	22.77
256	3.6914	0.568	945.0	1.137	22.76
512	2.0280	0.549	1038.4	1.099	22.76
1024	1.0266	0.506	1051.3	1.012	19.79

わかったこと

- ・ 解像度はノイズではなくネットワークの側で決まる。fractalnoise に解像度の欄は無い。copnet の res1 / res2 が決めている。形 (SOP) の感覚で「そのノードに設定がある」と探すと見つからない。
- ・ 「解像度を指定する」は既定でオフなのに、値は効いている。オフのままでも 1024×1024 で出てくる。欄に書いてある既定値がそのまま使われるため、オフ=無効ではない。
- ・ 画素どうしの差は、解像度を2倍にするたび約半分になる。6.4940 → 3.6914 → 2.0280 → 1.0266 で、比は 0.568 / 0.549 / 0.506。これは模様が変わったのではなく、画素が細かくなって隣が近づいただけである。
- ・ 解像度を掛けて直すと、実験007と同じ形になった。831.2 → 945.0 → 1038.4 → 1051.3 で、比は 1.137 / 1.099 / 1.012。増えて頭打ちになる。頭打ちになったところが、その模様に足りている解像度で、elementsize 0.10 なら 1024 で足りている。
- ・ ばらつきは解像度によらずほぼ一定だった。22.77 / 22.76 / 22.76 / 19.79。「どれだけ濃淡があるか」は解像度で変わらないが、「どれだけ細かいか」は変わる。実験007で「高さの平均は早く収束するのに勾配は伸び続ける」と測ったのと同じ関係になる。
- ・ elementsize は素直に効く。0.05 から 0.40 へ8倍にすると、細さは 1.7913 から 0.3657 へ約5分の1。大きくするほどうねりが大きく、なだらかになる。

- ・ rough の効き方は急である。0.0 で 0.4020、1.0 で 32.9725。82倍になる。0.5 までは穏やかで、0.75 から一気に立ち上がる。

- ・ ただし rough は明るさのばらつきも変えてしまう。16.92 → 49.17 で約2.9倍。実験004で mountain の rough を測ったときは「振幅を変えずに形の性質だけを変える」だったが、COP のノイズでは同じ名前でも振る舞いが違う。名前が同じでも、別のノードなら測り直す。

- ・ 書き出した画の階調は 206~215 だった。8ビットPNG の上限は256なので、ほぼ使い切っている。ただし値は 0~255 に丸められるので、マスクとして精度が要るなら EXR など小数のまま持てる形式にする。

次にやること

- ・ 作ったテクスチャを地形の材質に貼って、Karma で確かめる
- ・ COP のノイズを HeightField の高さに流し込む
- ・ 040 以降、まだ触っていない分野に進む

作ったテクスチャを地形に貼る — 貼れたかどうかを画の数字で判定する

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 5 / 接続 4

何を作ったか

実験039で Copernicus のノイズを画像として書き出した。実験038で地形を Karma で出した。今回はその2つをつなぐ。

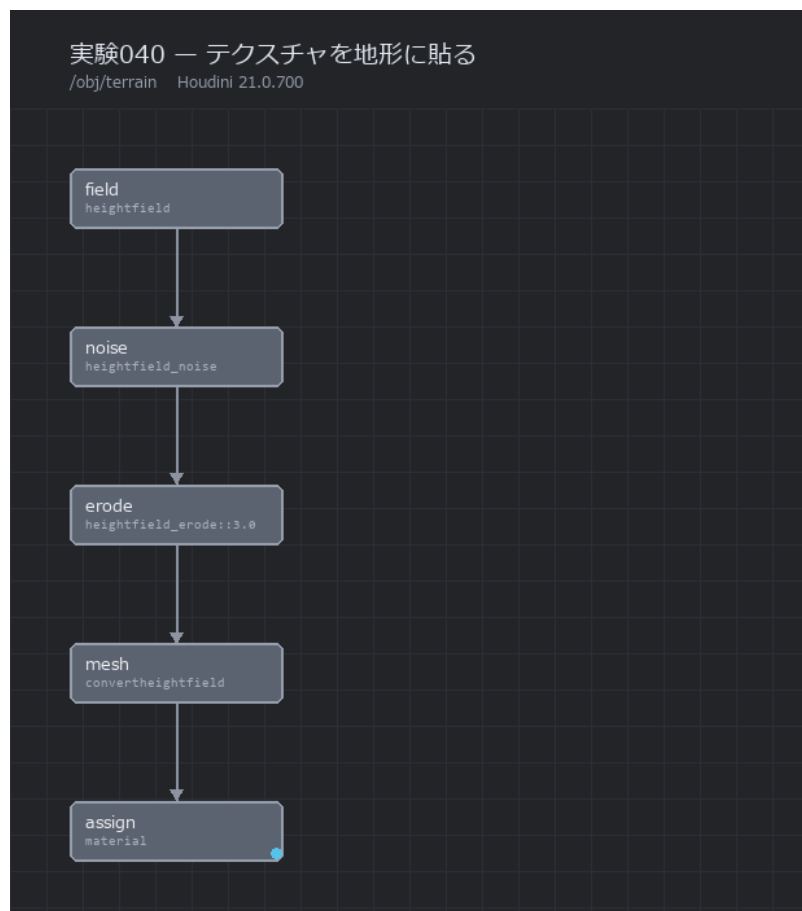
「貼ったつもりで貼れていない」はよくある。そこで貼れたかどうかを画の数字で判定した。テクスチャを貼れば面の中に細かい濃淡が出るはずなので、貼る前と貼ったあとで画の細かさが増えるはずである。増えなければ貼れていない。

増えた。細かさは 2.2143 から 3.0095 へ 1.36倍、濃淡のばらつきは 14.83 から 27.54 へ 1.86倍。貼れている。

ただしテクスチャの解像度を 256 から 1024 へ4倍にしても、画に出る細かさは 2.9028 から 3.0095 で 1.04倍にしかない。この見え方の大きさなら 256 でほぼ足りている。実験039で「頭打ちになったところが足りている解像度」と測ったことが、貼る側でも成り立つ。

もう1つ、Apprentice では COP から書き出した画像にウォーターマークが入る。それを貼ると地形の表面にも写る。

ノードの接続



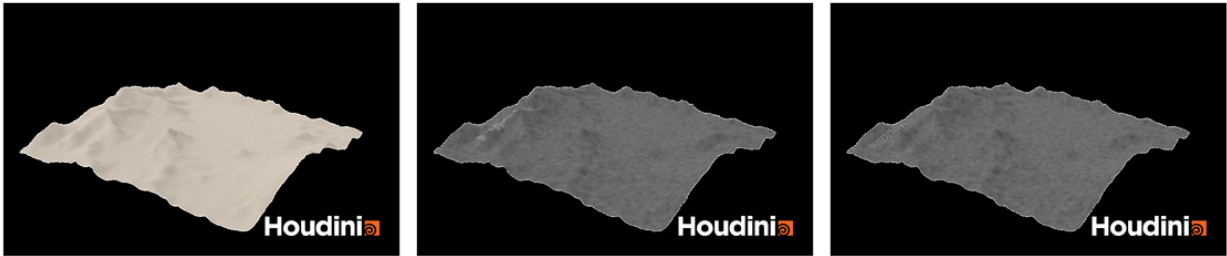
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	sizeX = 200.0 sizeY = 200.0
noise	heightfield_noise	amp = 120.0 elementsSize = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.32275283336639404
mesh	convertHeightfield	(変更なし)
assign	material	shop_materialPath1 = /mat/terrain_mat

貼る前と貼ったあと

同じ地形・同じライト・同じカメラ。違いは貼る画だけ。

同じ地形・同じライト・同じカメラ。違いは貼る画だけ



テクスチャなし 細かさ 2.2143 256×256 2.9028 1024×1024 3.0095

左から、テクスチャなし / 256×256 / 1024×1024。

テクスチャ	時間	明るさ	ばらつき	細かさ
なし（一色）	5.35秒	191.73	14.83	2.2143
256×256	3.44秒	110.31	27.56	2.9028
1024×1024	3.47秒	110.26	27.54	3.0095

地形は UV を持っているか

テクスチャを貼るには UV が要る。convertHeightfield の出力を調べた。

調べたこと	結果
点アトリビュート	P, debris, flow, flowdir_x, flowdir_y, height, mask, sediment, uv
uv はあるか	ある（自分で作らなくてよい）
テクスチャを作ったノード	copnet → fractalnoise → rop_image（実験039と同じ）
貼り付け先	principledshader の basecolor_useTexture / basecolor_texture

わかったこと

・貼れたことを、見た目ではなく数字で確かめた。細かさは 2.2143 → 3.0095 で 1.36倍、ばらつきは 14.83 → 27.54 で 1.86倍。「貼ったつもり」で終わらせないために、増えるはずの量を先に決めておく。

- ・テクスチャの解像度を4倍にしても、画はほとんど変わらなかった。256 で 2.9028、1024 で 3.0095。1.04倍にしかない。この見え方の大きさなら 256 でほぼ足りている。

- ・実験039の結論が、貼る側でも成り立つ。039では「細かさ×解像度が頭打ちになったところが、その模様に足りている解像度」と測った。今回は画面に出る大きさが上限を決めている。地形が画面の一部しか占めていないので、それ以上細かいテクスチャは画に出ない。

- ・UV は自分で作らなくてよい。convertheightfield の出力に uv が最初から付いている。実験036で「レイヤーが点アトリビュートとして持ち込まれる」と測ったときと同じ一覧に入っている。

- ・Apprentice では、COP から書き出した画像にウォーターマークが入る。それを貼ると地形の表面に文字が写る。レンダの隅に出るウォーターマークと違い、テクスチャに焼き込まれているので、地形のどこにでも現れる。学習用に作る画としては問題ないが、本番で使う画をこの経路で作ることはできない。

- ・一色のときのほうが遅かった。5.35秒 対

3.44秒。テクスチャを貼ったほうが速い。理由は測っていないが、一色のほうが明るく（191.73 対 110.26）、明るい面ほど光が何度も跳ね返って計算が増えるためと考えられる。ここは確かめていないので断定しない。

- ・貼ると暗くなる。明るさ 191.73 → 110.26。作ったノイズの平均が中間の灰色で、一色に指定した (0.55, 0.48, 0.40)

より暗いため。テクスチャに差し替えると明るさが変わるので、ライトは貼ったあとに決める。

次にやること

- ・ノイズを高さ (displacement) として使い、面そのものを凸凹させる
- ・岩肌と土で別のテクスチャを使い分ける (037の塗り分けをマスクにする)
- ・041 以降、まだ触っていない分野に進む

凸凹の付け方3通り —

輪郭を見れば、どれが本当に形を変えたか分かる

2026-09-14 | Houdini 21.0.700 | /obj/terrain | ノード 6 / 接続 6

何を作ったか

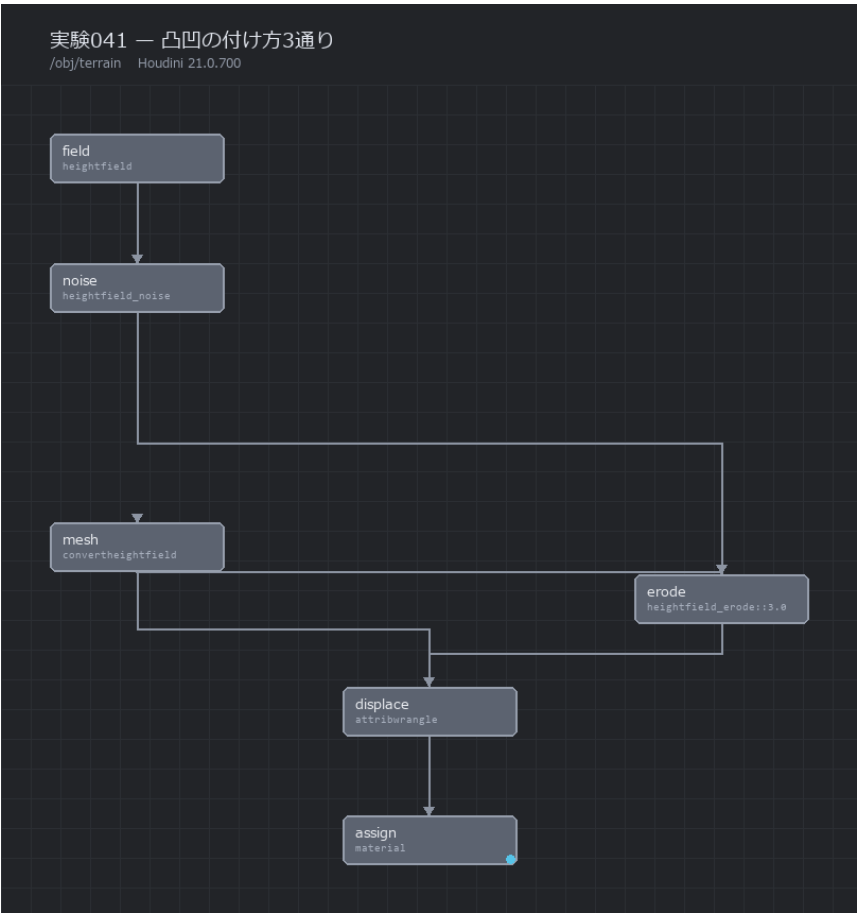
面を凸凹させる方法は3つある。ジオメトリを動かす（点そのものを動かす）、displacement（レンダるときだけ面を動かす）、bump（面は動かさず光の当たり方だけ凸凹に見せる）。

見た目はどれも似る。決定的な違いは輪郭（シルエット）に出るはずである。本当に面が動いていればフチがぎざぎざになり、光のごまかしならフチは滑らかなままになる。そこで輪郭の長さを測った。書き出した画の透明度から、地形と背景の境目になっている画素を数える。

狙いどおりに分かれた。bump は画の細かさを一番大きく増やす（2.2945 → 4.0524 で 1.77倍）のに、輪郭は 0.999倍でまったく変わらない。displacement は点を1ミリも動かさないのに輪郭を 1.020倍にする。ジオメトリを動かした場合だけ、点の位置そのものが変わった（最大 2.22）。

「見た目が凸凹しているか」と「形が凸凹しているか」は別のことで、輪郭を測れば見分けられる。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size _x = 200.0 size _y = 200.0

ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 120.0 elementsiz = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.8801623582839966
mesh	convertheightfield	(変更なし)
displace	attribwrangle	snippet = float e = 2.0; float h0 = volumesample(1, "height", @P); float hx = volumesample(1, "height", @P + set(e, 0, 0)); flo...
assign	material	shop_materialpath1 = /mat/terrain_mat

4通りを同じカメラで撮る

同じ地形・同じライト・同じカメラ。displacement と bump には、まったく同じ画（Copernicus で作ったノイズ）を渡している。

同じ地形・同じライト・同じカメラ。凸凹の付け方だけが違う



左から、何もしない / ジオメトリを動かす / displacement / bump。

やり方	点が動いた量	輪郭の長さ	何もしないとの比	画の細かさ	時間
何もしない	0.000000	3,064	1.000倍	2.2945	4.32秒
ジオメトリを動かす	2.219831	3,118	1.018倍	2.7508	5.02秒
displacement	0.000000	3,126	1.020倍	3.5173	6.32秒
bump	0.000000	3,062	0.999倍	4.0524	8.32秒

わかったこと

- ・ bump は輪郭をまったく変えなかった。3,062 対 3,064 で 0.999倍。それでいて画の細かさは一番大きい（4.0524、何もしない場合の1.77倍）。凸凹して見えるのに、形は1ミリも変わっていない。
- ・ displacement は点を動かさないのに輪郭を変えた。点の移動量は 0.000000 なのに、輪郭は 1.020倍。レンダのときだけ面が動いているので、データを見ても分からない。
- ・ ジオメトリを動かした場合だけ、データが変わる。点が最大 2.22 動き、高さの幅も 28.519 から 29.562 に広がった。あとの工程（散布や当たり判定）に効くのはこれだけ。
- ・ 使い分けの目安。フチに出したい凸凹はジオメトリか displacement、面の中だけでよい細かさは bump。遠景の岩肌なら bump で足り、シルエットに出したい岩の割れ目なら displacement が要る。
- ・ bump が一番遅かった。8.32秒で、何もしない場合の 1.9倍。displacement（6.32秒）より遅い。「面を動かさないから軽い」とは限らない。理由は測っていないので断定しない。

- ・最初、bump がまったく効かなかった。有効化のスイッチだけを入れてテクスチャのパスを指定していなかったため、輪郭も細かさも「何もしない」と完全に同じ数字（3056 と 2.2916）になった。完全に同じ数字が出たら、効いていないことを疑う。

- ・輪郭の測り方。書き出した画の透明度で地形と背景を分け、隣り合う画素で「地形かどうか」が食い違う場所の数を数えた。フチがぎざぎざになるほど、この数が増える。実験038で「明るさで分けると白飛びを取りこぼす」と学んだので、透明度で分けている。

- ・差は 2% 前後と小さい。目で見比べても分からない大きさだが、数えれば順番がはっきり出る。bump だけが 1.000倍から動かず、他の2つは 1.018 / 1.020 と揃って上がる。

次にやること

- ・ displacement の量を振って、輪郭の変化が量に比例するか確かめる
- ・ 地形を寄りで撮り、bump と displacement の差が見える距離を探す
- ・ 042 以降、まだ触っていない分野に進む

毛を生やす — density

は本数ではなく「面積あたりの本数」だった

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 3 / 接続 2

何を作ったか

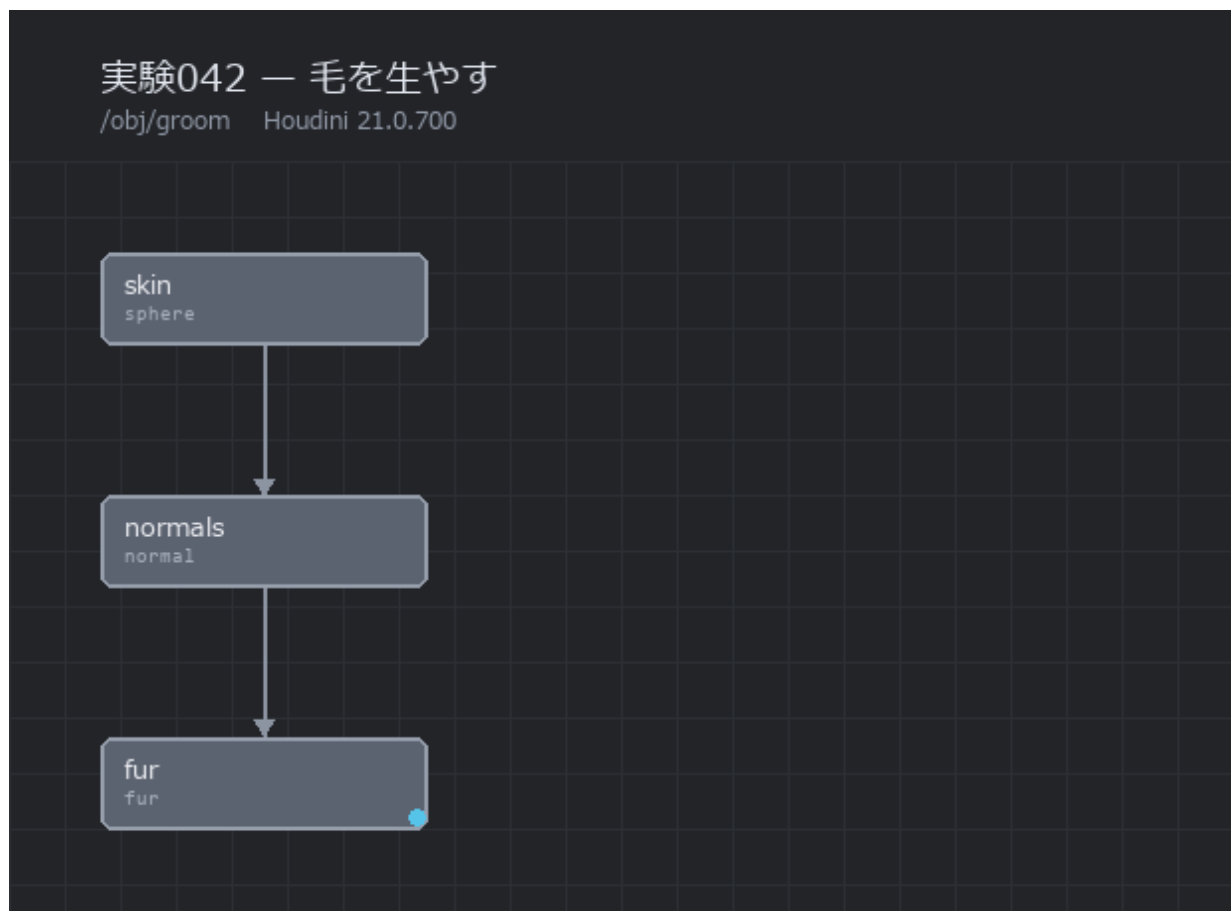
新しい分野に入る。Houdini には毛（グルーム）の仕組みがあり、guide で始まるノードだけで26種ある。ただし最初の1本は fur で足りる。面を渡すと毛が生えてくる。

最初につまずいたのは、どのノードが毛を作るのかだった。名前からすると guideinit が作りそうに見えるが、これは既にあるガイドに風や持ち上がりを付ける整え役で、面だけ渡すと「Not enough sources specified.」で止まる。

数の関係もそのままでは合わない。density に 100 を入れても、生えるのは 1,249本。指定した数そのまま本数になるわけではない。ただし本数を土台の面積で割ると 99.678 で、density の 100 とほぼ一致する。density は「面積1あたりに何本生やすか」だった。

length は純粋な倍率だった。0.125 / 0.25 / 0.5 / 1.0 の4通りで、毛の長さの平均を length で割った値が 0.99808 と小数点以下5桁まで完全に同じ。実験006の height、実験034の amp と同じ性質になる。

ノードの接続

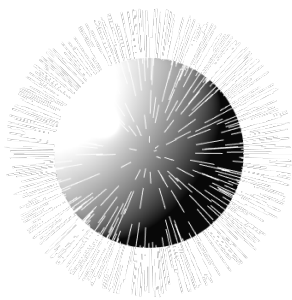


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
fur	fur	(変更なし)

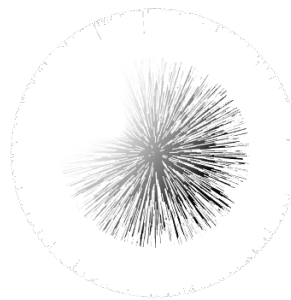
density を変える

土台は 40×40 の球（1,522点 / 1,560面 / 面積 12.53039）。カメラは2枚とも同じ。



Houdini

density 60。毛がまばらで、土台の球が見える。



Houdini

density 600。同じ球とは思えない密度になる。

density	生えた本数	本数 ÷ density	面積あたりの本数	時間
50	612	12.240	48.841	0.01秒
100	1,249	12.490	99.678	0.01秒
200	2,498	12.490	199.355	0.01秒
400	5,050	12.625	403.020	0.01秒

length は純粋な倍率か（実験006・034と同じ問い）

density は 100 のまま、length だけを変えた。毛1本の長さは、区間の長さを足し合わせて測っている。

length	本数	長さの平均	平均 ÷ length	長さのばらつき	毛1本の点数
0.125	1,249	0.12476	0.99808	0.00012	5.00
0.25	1,249	0.24952	0.99808	0.00023	5.00
0.50	1,249	0.49904	0.99808	0.00047	5.00
1.00	1,249	0.99808	0.99808	0.00094	5.00

わかったこと

・ density は本数ではなく「面積あたりの本数」だった。100 を入れて 1,249本。土台の球の面積 12.53039 で割ると 99.678 で、指定した 100 とほぼ一致する。土台を大きくすれば、同じ density でも本数は増える。

・比は4通りでほぼ一定だった。本数÷density が 12.240 / 12.490 / 12.490 / 12.625。ばらつきが残るのは、ばらまきに乱数が入るため。density を2倍にすれば本数も2倍になる。

・length は純粋な倍率だった。4通りで「平均 ÷ length」が 0.99808 と小数点以下5桁まで同じ。実験006の height、実験034の amp と同じ性質になる。Houdini のこの手のパラメータは、倍率として信用してよいことが多い。

・ただし 1.0 ではなく 0.99808 である。指定より 0.19% 短い。4通りすべてで同じ値なので、誤差ではなく決まったずれである。毛が少し反っていて、直線の区間をつないで測ると回り道になるため……とは考えられるが、確かめていないので断定しない（反っているなら逆に長く出るはずで、説明としては弱い）。

・毛1本は5点だった。どの length でも 5点=4区間。長くしても分割は増えない。長い毛をなめらかに曲げたいなら、点数を別に増やす必要がある。

・どのノードが毛を作るのかで、まずつまずいた。名前からすると guideinit が作りそうに見えるが、これは既にあるガイドに風や持ち上げりを付ける整え役で、パラメータも Wind Amount / Lift / Tangential to Skin の11個しかない。面だけ渡すと「Not enough sources specified.」で止まる。

・毛は2段構えで作る。ガイド（案内役、数百本）で形を決め、そこから毛（数万本）を増やす。fur はこの2段を1つにまとめたノードで、最初の1本にはこれでよい。26種のノードは、その2段のそれぞれを細かく調整するためにある。

・面の向き（N）は必要だった。normal を挟んでいる。実験008・030で確かめたとおり、N が無いと向きが決まらない。

次にやること

- ・ガイドを自分で作り、fur ではなく2段構えで組む
- ・hairclump（毛を束ねる）で見え方がどう変わるか測る
- ・043 以降、まだ触っていない分野に進む

毛を束ねる — つなぐ順番が、思っていたのと逆だった

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 7 / 接続 9

何を作ったか

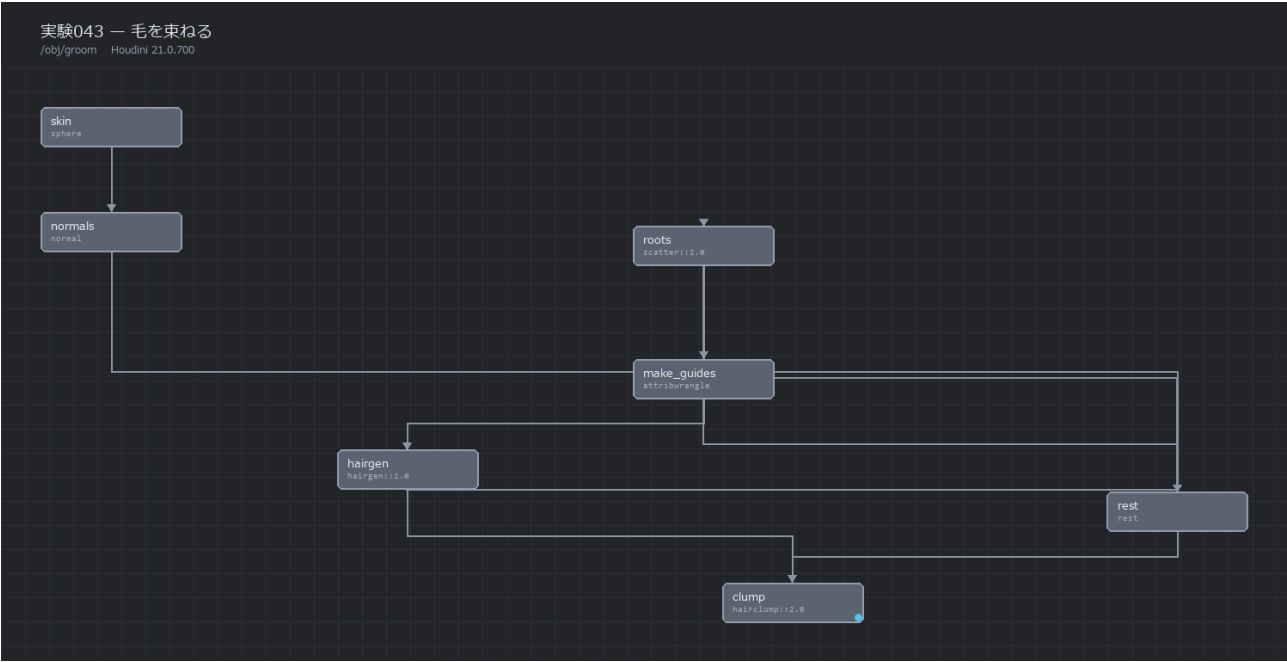
実験042では fur で手早く毛を生やした。今回はその毛を hairclump で束ねる。見た目は、細かい毛が無数に生えた状態から、房にまとまった状態へ変わる。

ところが fur の出力は hairclump につながらない。「Couldn't find skinprim attribute.」で止まる。グルームの道具は「この毛は土台のどの面から生えたか」を番号で知りたがり、fur の出力は N と P しか持っていない。そこでガイドを自分で組み、xyzdist で一番近い面の番号を調べて skinprim に入れた。

それでも動かなかった。原因はつなぐ順番だった。総当たりで確かめたところ、hairgen は「土台 → ガイド」、hairclump は「毛 → 土台」で、2つのノードで順番が逆になる。片方を覚えて、もう片方に当てはめると必ず失敗する。

束ねた結果も測った。clumpsize を 0.05 から 0.8 まで上げると束の数は 9,823 → 27 に減り、毛先どうしの距離は縮む。ただし縮み方は一本調子ではなく、0.2 がいちばん密で、0.8 では逆に広がった。束を大きくすると「太い束」になるためで、これも数字で確かめた。

ノードの接続



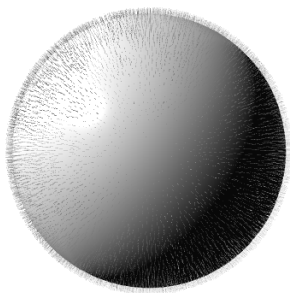
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0
clump	hairclump::2.0	blendremap2pos = 1.0 blendremap2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件

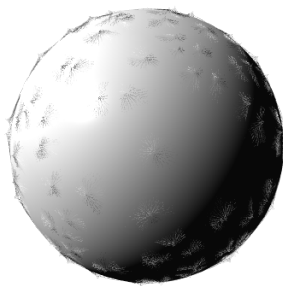
束ねる前と後

土台は 40×40 の球（面積 12.53039）。ガイド300本、density 1000 で 12,516本の毛。カメラは2枚とも同じ。clumpsize 0.4。



Houdini

束ねない。毛が一様に散っていて、全体がうぶ毛のように見える。



Houdini

clumpsize 0.4
で束ねた。144個の房になり、1房あたり約87本。

つないだ順番	ノード	出てきた本数	エラー
土台 → ガイド	hairgen::2.0	12,516	—
ガイド → 土台	hairgen::2.0	出ない	Invalid source .../hairgencore.

つないだ順番	ノード	出てきた本数	エラー
毛 → 土台	hairclump::2.0	12,516	—
土台 → 毛	hairclump::2.0	出ない	Invalid source .../hairclumpcore1.

clumpsize を変えると束の数はどう変わるか

毛は 12,516本で固定。clumpid の種類を数えて束の数とした。

clumpsize	束の数	1束あたりの本数	いちばん大きい束	いちばん小さい束
0.05	9,823	1.27	7	1
0.10	2,607	4.80	22	1
0.20	648	19.31	60	2
0.40	144	86.92	218	14
0.80	27	463.56	966	106

毛先は本当に寄るのか

毛先の点だけを取り出し、いちばん近い毛先までの距離を測った。12,516本から等間隔に1,391本を抜いて比較している（どの条件も同じ本数）。

条件	測った毛先	いちばん近い毛先まで	中央値	束ねない場合との差
束ねない	1,391	0.05885	0.05839	—
clumpsize 0.05	1,391	0.05821	0.05820	-1.1%
clumpsize 0.20	1,391	0.02781	0.02093	-52.7%
clumpsize 0.80	1,391	0.04015	0.03692	-31.8%

なぜ 0.8 では逆に広がったのか

束ごとに、根元の点と毛先の点それぞれについて「束の中心からの平均距離」を測った。これが束の太さにあたる。

clumpsize	束の数	根元の広がり	毛先の広がり	毛先 ÷ 根元
0.05	9,823	0.01381	0.00386	0.279
0.10	2,607	0.02588	0.00627	0.242
0.20	648	0.05408	0.01603	0.296
0.40	144	0.11562	0.05144	0.445
0.80	27	0.26199	0.16565	0.632

わかったこと

- ・つなぐ順番が、2つのノードで逆だった。hairgen::2.0 は「土台 → ガイド」、hairclump::2.0 は「毛 → 土台」。4通りすべてを総当たりで試し、正しい2通りだけが 12,516本を出し、残る2通りは Invalid source で何も出さなかった。片方を覚えて、もう片方に当てはめると必ず失敗する。
- ・fur の出力は、束ねるノードにつながらない。fur が出すのは N と P だけで、skinprim が無い。hairclump は「Couldn't find skinprim

attribute.」で止まる。実験042で使った手軽な道具は、次の工程には渡せない。

- ・土台には rest が要る。球に normal を付けただけでは「Skin input has no rest attribute.」で止まる。rest SOP を1つ挟むと通る。

- ・ガイドは自分で作れた。散らした点から polyline を伸ばし、xyzdist で一番近い面を調べて skinprim と skinprimuv をプリミティブに付ける。これだけで hairgen が受け取る。専用ノードを使わなくても、必要なのは属性2つだった。

- ・束ねて増えるアトリビュートは clumpid ひとつだけだった。束ねる前が id / skinprim / skinprimuv、後が clumpid / id / skinprim / skinprimuv。どの毛がどの束に属するかを示す番号で、これを使えば束ごとに色を変えろといった加工ができる。

- ・clumpsize を上げると束の数は減る。0.05 で 9,823束、0.8 で 27束。0.05 のときは1束あたり 1.27本で、ほとんど束になっていない（毛先の距離も 1.1% しか縮まない）。小さすぎる clumpsize は「効かない」ではなく「1本ずつが1束」になる。

- ・毛先は確かに寄った。ただし一本調子ではない。いちばん近い毛先までの距離は、0.2 で 52.7% 縮む一方、0.8 では 31.8% しか縮まない。clumpsize を上げ続ければ密になる、ではなかった。

- ・その理由は「束が太くなるから」だった。束の毛先の広がり は 0.00386 → 0.16565 と、clumpsize とともに増える。根元に対する比も 0.279 → 0.632 と増えていく。大きい束ほど、根元の広がりに対して毛先が寄りきらない。だから束の中は密でも、毛先どうしの距離の平均は 0.2 を底に反転する。

- ・実験042の density の話が、別の形で裏づけられた。density 1000、土台の面積 12.53039 なので計算上は 12,530本。実際に出たのは 12,516本で、0.1% の差。042 の「density は面積あたりの本数」という結論が、ガイドを自作した今回の組み方でもそのまま成り立つ。

- ・ガイド300本から 12,516本、41.7倍。ガイドの本数は毛の本数に直接は効かない（効くのは density と面積）。ガイドは形を決め、density が数を決める。

次にやること

- ・clumpid を使って束ごとに色を変え、束の境目を目で確かめる
- ・毛にカールやノイズを足す（hairskin 系のノード）
- ・グルームをレンダリングに乗せる（Karma で毛をどう扱うか）

つないだのに、ガイドが1本も使われていなかった — rest が (0,0,0) だった

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 8 / 接続 10

何を作ったか

実験043で自作のガイドを hairgen::2.0 につなぎ、12,516本の毛を出した。今回その続きで「ガイドの向きをばらしたら毛はどう変わるか」を測ろうとしたところ、毛先の平均が小数点以下5桁まで1ミリも動かなかった。ガイドを100本にしても900本にしても、毛の本数も束の数もまったく同じ。

つまりガイドは1本も使われていなかった。エラーも警告も出ない。

原因は rest アトリビュートだった。ガイドを作った wrangle では addpoint

で新しい点を作っている。土台から流れてきたジオメトリには rest が付いているので、新しい点にも rest という欄はできるが、中身は (0,0,0) のまま。hairgen は rest

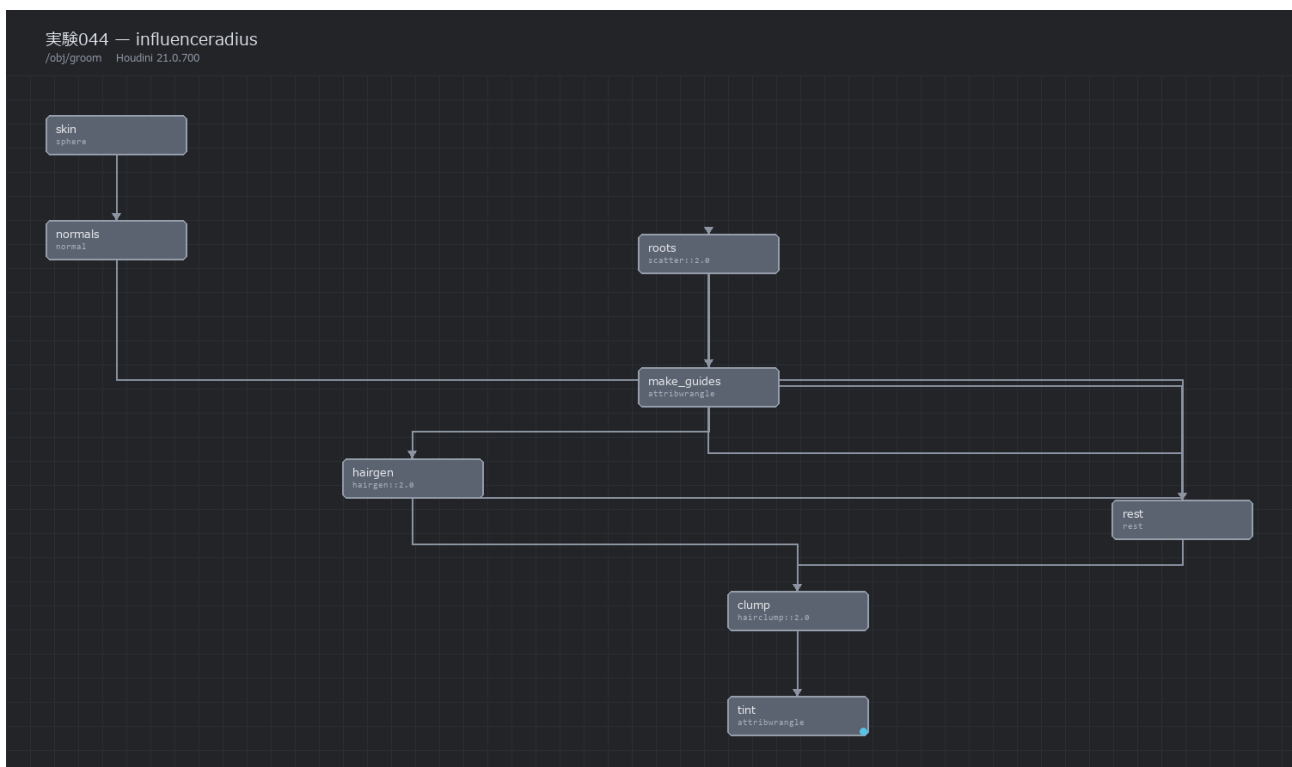
空間で毛とガイドの距離を測るので、ガイドは全部が原点にあることになり、どの毛にも届かなかった。

setpointattrib で rest に実際の位置を入れた瞬間、毛の長さの最大が 0.05000 から 0.49989 に変わった。0.05 は unguidedlength

の既定値で、それまでの毛はすべて「ガイドに拾われなかった毛」の既定の長さだった。

直したうえで influenceradius を上げると、拾われる毛は 18.9% → 73.1% → 100% と増える。既定の 0.05 では2割も拾えない。

ノードの接続



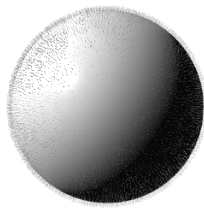
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; float jitter = 0.0; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.4
clump	hairclump::2.0	blendremapramp2pos = 1.0 blendremapramp2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件
tint	attribwrangle	class = 1 snippet = int cid = @clumpid; vector c = set(rand(cid * 3 + 0.1), rand(cid * 3 + 1.7), rand(cid * 3 + 2.3)); c = 0.25 + 0.75 *...

rest を直す前と後

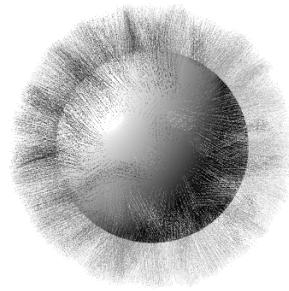
土台は 40×40 の球。ガイド300本（長さ 0.5）、density

1000。カメラは2枚とも同じ。influenceradius 0.2、ガイドの向きはばらしてある。



Houdini

rest が (0,0,0) のまま。ガイドは1本も使われず、長さ 0.05 の短い毛だけが生える。



Houdini

rest を直した。同じつなぎ方・同じパラメータで、ガイドどおりの長さとなり向きになる。

rest	ガイドのばらつき	毛	長さの平均	長さの最大	毛先の広がり
(0,0,0)	0.0	12,516	0.04990	0.05000	0.60504
(0,0,0)	1.2	12,516	0.04990	0.05000	0.60504
直した	0.0	12,516	0.13481	0.49989	0.66185
直した	1.2	12,516	0.13499	0.50000	0.65678

influenceradius を上げると、拾われる毛はどう増えるか

growunguided

を切ると、ガイドに拾われなかった毛は生えない。残った本数が「拾われた毛」になる。長さは growunguided を戻して測った。

influenceradius	全部の毛	拾われた毛	割合	長さの平均	長さの最大
0.05 (既定)	12,516	2,366	18.9%	0.13481	0.49989
0.10	12,516	9,153	73.1%	0.37835	0.49989
0.20	12,516	12,514	100.0%	0.49765	0.49989
0.40	12,516	12,516	100.0%	0.49294	0.49610
0.80	12,516	12,516	100.0%	0.48541	0.48960

clumpid は何を指す番号か

influenceradius 0.20、clumpsize 0.4 で束ねた 12,516本の毛。束は144個。

調べたこと	結果
番号の範囲	204 ~ 12492
0 からの連番か	いいえ (144個が飛び飛び)
番号が毛の id の中にあるか	144 / 144
その id の毛自身が同じ束にいるか	144 / 144
隣の毛が同じ束になる率 (実測)	1,072 / 1,391 = 77.07%

調べたこと	結果
ばらばらに振られていた場合の予想	$1 \div 144 = 0.69\%$

わかったこと

・ガイドは1本も使われていなかった。しかもエラーも警告も出ない。ガイドを100本から900本に増やしても、向きをばらばらに傾けても、毛先の平均が小数点以下5桁まで同一だった。「つないだから効いているはず」は確かめないと言えない。

・原因は rest が (0,0,0) だったこと。wrangle の addpoint で作った点には、既にある属性の欄はできるが中身は既定値のまま入る。土台から rest が流れてきていたので、ガイドの全点の rest が原点になっていた。hairgen は rest 空間で距離を測るので、ガイドは全部が原点にあることになり、どの毛にも届かない。

・気づけたのは、毛の長さが unguidedlength の既定値と一致していたから。長さの最大が 0.05000 ちょうど。ガイドの長さは 0.5 なので、桁がひとつ違う。0.05 は「ガイドに拾われなかった毛」に与えられる既定の長さだった。

・直したら、長さの最大が 0.05000 → 0.49989 に変わった。同時に、ガイドの向きをばらしたときの毛先の差が 0.000000 → 0.005074 になった。ガイドが効いているかどうかは、この2つで判定できる。

・influenceradius の既定 0.05 では、2割弱しか拾えない。ガイド300本を半径1の球に撒くと間隔は約0.2。既定の半径ではほとんどの毛がどのガイドからも遠い。0.20 でようやく 100% (12,514 / 12,516) になる。

・半径を上げすぎると毛が短くなる。長さの平均は 0.49765 (半径0.2) → 0.49294 (0.4) → 0.48541 (0.8)。遠くのガイドまで混ぜると、向きの違うガイドが平均されて縮む。2.5% の差だが、4通りで単調に減っているので誤差ではない。

・clumpid は「束の代表になった毛の番号」だった。0からの連番ではなく 204~12492 の飛び飛びの値で、144個すべてが実在する毛の id と一致し (144/144)、しかもその毛自身が同じ束に属していた (144/144)。番号は通し番号ではなく、束の中心になった毛を指している。

・束はちゃんと空間的にまとまっている。隣の毛が同じ束になる率は 77.07%。ばらばらに振られていたなら $1 \div 144 = 0.69\%$ のはずで、112倍。残りの 23% は束の境目にいる毛になる。

・この実験の測り方そのものが道具になる。「効いているはずのものが効いていない」を見つけるには、入力を大きく変えて出力が動くかを見ればよい。動かなければ、その入力に使われていない。

次にやること

- ・実験043の記事に訂正を出す (あの毛はすべて unguided の毛だった)
- ・ガイドを整える guideprocess / guideadvect を触る
- ・毛をレンダリングに乗せる (Karma で毛をどう扱うか)

毛を Karma で出す — 既定の太さは、画の上で 0.1ピクセルしかなかった

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 7 / 接続 9

何を作ったか

hairgen は毛1本ごとに width という太さを書き出す (thickness、既定 0.001) 。その 0.001 が画の上で何ピクセルになるのかを、先に式を立ててから測った。

見かけの太さ(px) = thickness × (1世界単位あたりのピクセル数)。ただし width が「直径」なら答えはこれ、「半径」なら2倍になる。どちらかは書いていないので測るしかない。

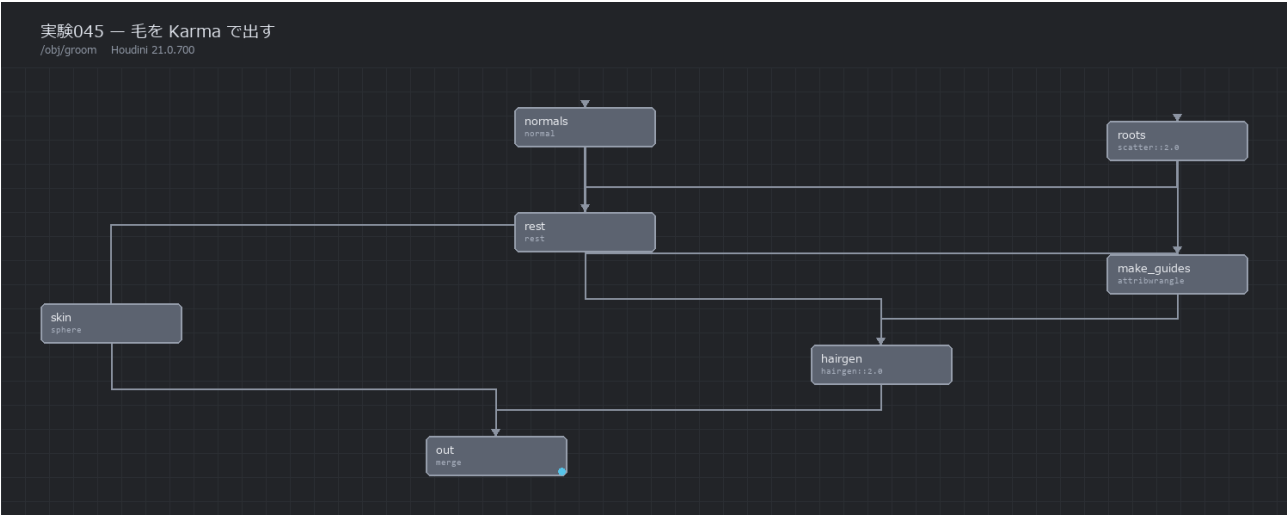
毛玉のままでは毛が重なって面積から太さを割り出せないので、毛を1本だけ置いて真横から撮った。PNG の透明度をそのまま足すと、輪郭のぼけも 0.01ピクセルまで拾える。4通りの太さで、実測÷予測は 0.9969 / 0.9969 / 0.9969 / 1.0019。width は直径だった。

ただし最初の測定は 3~4% ずれていた。Apprentice のレンダには右下に Houdini のロゴが焼き込まれ、そこにも透明度がある。列ごとに透明度を足すやり方だと、ロゴの列だけ値が水増しされていた。ロゴの箱 (行521~566、列343~566、合計 4163.3) を外したところ、4通りとも 0.3% 以内に収まった。

そのうえで既定値に戻すと、球1個を画面いっぱい撮った構図で毛の太さは 0.103ピクセル。1ピクセルの1割しかない。だから既定のままレンダするとザラついた点の集まりにしか見えない。0.008 まで上げると毛の流れが見える絵になる。

レンダ時間は本数に比例しない。毛を16倍 (3,104本 → 49,977本) にしても時間は 4.2倍 (3.1秒 → 13.1秒) 。OpenGL は本数によらず 0.74~0.76秒で一定だった。

ノードの接続



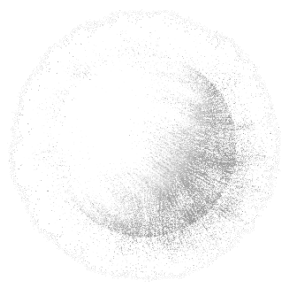
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.008
out	merge	(変更なし)

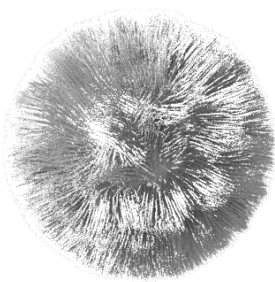
既定の太さと、上げた太さ

毛 12,516本。球も毛も同じで、thickness だけが違う。カメラは2枚とも同じ。Karma、9サンプル、600×600。面積はロゴの箱を除いて数えている。



Houdini

thickness 0.001（既定）。画の上で 0.103px。1ピクセルより細く、ザラついた点にしか見えない。



Houdini

thickness 0.008。画の上で 0.822px。毛の流れが読める。

thickness	画の上の太さ	触れたピクセル	平均の濃さ	時間
0.001（既定）	0.103 px	70,696	0.746	5.0秒
0.002	0.205 px	72,828	0.839	7.0秒
0.004	0.411 px	73,999	0.912	10.3秒
0.008	0.822 px	74,862	0.950	10.7秒

太さは指定どおりに出るか（毛を1本だけ、真横から）

毛の長さは 1.0。カメラの焦点距離と絞りから「1世界単位が何ピクセルか」を出し、512.95 px を正とした。ロゴの箱は測定から外してある。

thickness	予測(px)	実測(px)	実測 ÷ 予測	画に出た長さ（世界）
0.005	2.565	2.557	0.9969	1.00790
0.010	5.129	5.114	0.9969	1.01374
0.020	10.259	10.227	0.9969	1.02544
0.040	20.518	20.557	1.0019	1.04494

本数とレンダ時間

density だけを変えた。枠は毛の伸びる先まで含めて固定してあるので、どの density でもカメラは同じ。面積はロゴの箱を除く。

density	毛	時間	毛1本あたり	覆った面積
250	3,104	3.1秒	0.9841 ms	40,136
500	6,233	3.5秒	0.5629 ms	45,363
1000	12,516	4.7秒	0.3792 ms	52,736
2000	24,916	8.6秒	0.3470 ms	60,813
4000	49,977	13.1秒	0.2630 ms	67,276

OpenGL と Karma

同じ形・同じカメラ・同じ解像度で撮り比べた。

density	毛	OpenGL	Karma	倍率
250	3,104	0.74秒	3.1秒	4.1倍
1000	12,516	0.76秒	4.7秒	6.3倍
4000	49,977	0.75秒	13.1秒	17.4倍

わかったこと

- ・ width は直径だった。4通りで実測÷予測が 0.9969 / 0.9969 / 0.9969 / 1.0019。半径なら 2.0 になるはずで、そうはならない。3通りが 0.9969 でびたりとそろっている。
- ・ 最初の測定は 3〜4% ずれていた。原因はライセンスのロゴ。Apprentice のレンダには右下に Houdini のロゴが焼き込まれ、そこにも透明度がある。列ごとに透明度を足すやり方だと、ロゴのかかる列だけ値が水増しされる。ロゴの箱（行521〜566、列343〜566、透明度の合計 4163.3）を外したら 0.3% 以内に収まった。画を測るときは、画に写り込んでいるものを全部数え上げる。
- ・ 画に出る長さは、指定より少し長い。1.00790 / 1.01374 / 1.02544 / 1.04494。増えた分は太さとほぼ同じ（端が丸いので両側に半径ぶん出る）。太さを引いた残りは 0.0029〜0.0054 で、1.5〜2.8ピクセル分。輪郭のぼけにあたる。
- ・ 既定の thickness 0.001 は、この構図で 0.103ピクセルしかない。1ピクセルの1割。毛1本がピクセルの一部しか覆えないので、点が散らばったようにしか見えない。平均の濃さも 0.746 と薄い。
- ・ 太くすると絵が落ち着く。0.008（0.822px）で平均の濃さが 0.950 になり、毛の流れが読める。ただし時間は 5.0秒 → 10.7秒と倍になる。覆うピクセルが増えるぶん計算も増える。

- ・太さは世界の大きさに対する値である。0.001

が細すぎるのは、土台の球が半径1しかないから。数字そのものではなく「土台の何分の1か」で考える。

- ・レンダ時間は本数に比例しない。毛を16倍にしても時間は 4.2倍。毛1本あたりの時間は 0.9841ms → 0.2630ms と 3.7分の1になる。毛が増えるほど1本あたりは安くなる。

- ・覆う面積も比例しない。毛を16倍にしても、画の上で覆う面積は 40,136 → 67,276 と 1.68倍にしかない。手前の毛が奥の毛を隠すので、増やしても画は濃くなるだけで広がらない。

- ・OpenGL は本数によらず 0.74~0.76秒で一定だった。3,104本でも 49,977本でも変わらない。確認用に何度も撮るなら OpenGL、見せる絵は Karma、という使い分けでよい。

- ・Karma との差は本数とともに開く。4.1倍 → 6.3倍 → 17.4倍。毛のように数が増えるものでは、この差は効いてくる。

- ・毛1本を真横から撮るという測り方が鍵だった。毛玉のままでは毛が重なり、覆った面積から1本の太さを割り出せない。測りたいものだけを残す形に作り替える。

次にやること

- ・毛に色を付ける（根元と毛先で変える・clumpid ごとに変える）
- ・毛を動かす（風・重力）
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）

毛に色を付ける ― 光を消して測ると、色は位置ぴったりに乗った

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 9 / 接続 11

何を作ったか

実験045で毛を Karma に出せた。次は色を付ける。ところが毛が持っているアトリビュートは P と width だけだった。「この点は根元から何割の位置か」を示す数値は付いてこない。プリミティブごとに頂点の並び順から 0~1 を自分で振る。

色が位置どおりに乗っているかを確かめるため、毛を1本だけ置いて根元を赤・毛先を青にし、真横から撮った。画の列ごとに青の割合を測ると、光を当てた状態では $0.6877 \times \text{位置} + 0.1748$

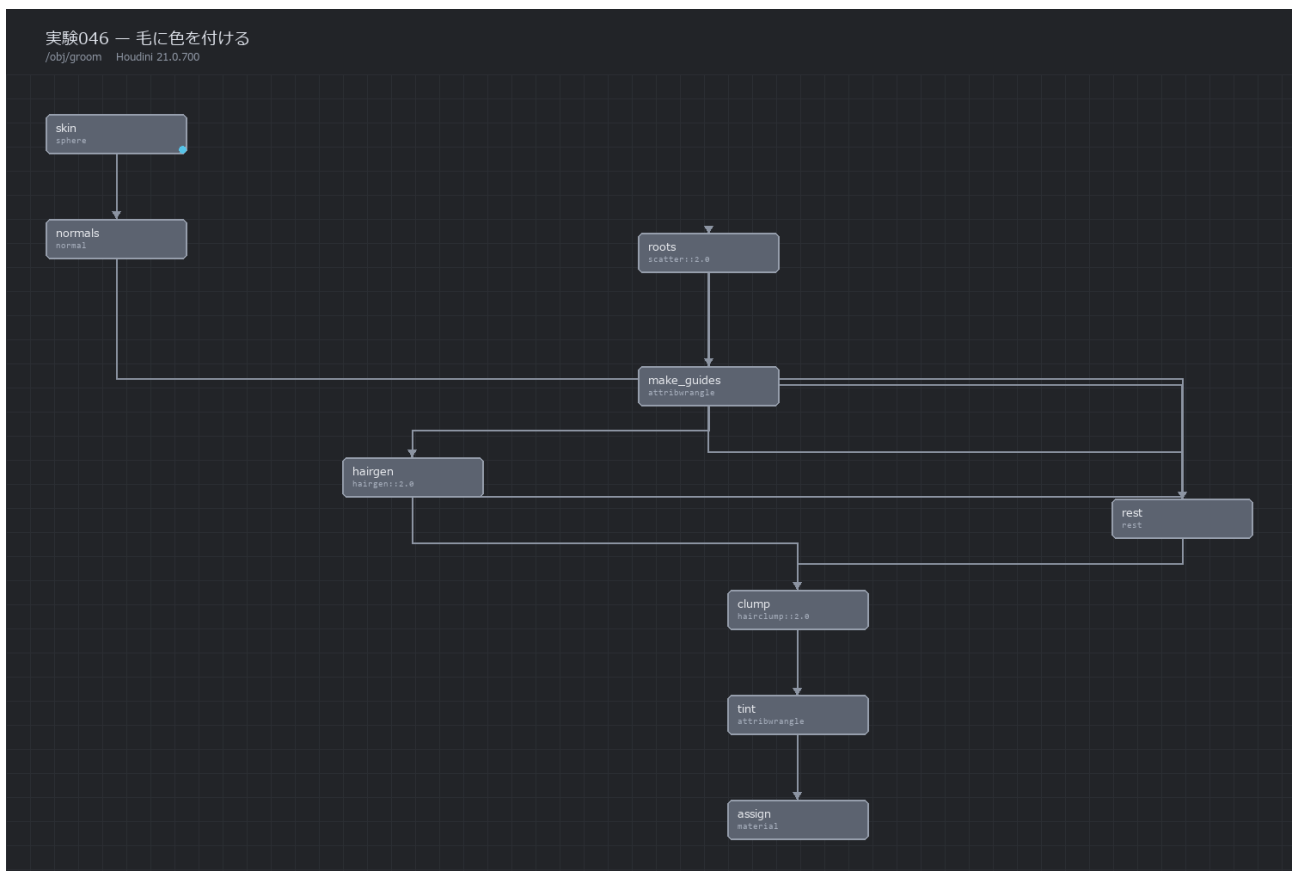
という直線になる。比例はしているが、1:1

ではない。周囲の光が赤にも青にも同じだけ乗るためで、この見立てが正しければ「傾き + 2×切片 = 1」になるはず。実測は 1.0373 だった。

光の影響を外すため、色を発光として出すと $1.0123 \times \text{位置} - 0.0064$ 、 $R^2 = 0.99991$ 、直線からの最大のずれ 0.0037。青の割合は位置そのものになった。

ついでに width も測った。根元から 1/4 までは 0.008 のままで、そこから細くなり、毛先でちょうど 0 になる。パラメータにある hairprofile というカーブのとおりに先細りしている。ばらつきは 12,516本すべてで 0.00000000 だった。

ノードの接続

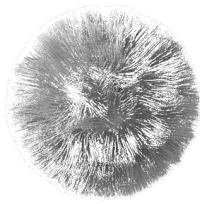


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.008
clump	hairclump::2.0	blendremap ramp2pos = 1.0 blendremap ramp2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件
tint	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); int n = len(pts); for (int i = 0; i < n; i++) { float u = (n > 1) ? float(i...
assign	material	shop_materialpath1 = /mat/hair_shader

色を付けない／根元から毛先へ／束ごと

毛 12,516本。形もカメラも同じで、色の付け方だけが違う。Karma、9サンプル、600×600。



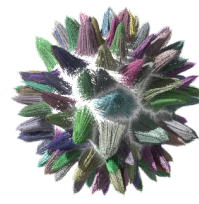
Houdini

色を付けない。彩度のあるピクセルは 0.0%。



Houdini

根元を赤、毛先を青にした。彩度のあ
るピクセルは 90.1%。



Houdini

束ごとに色を変えた。144個の束が目
で数えられる。

条件	毛	時間	色のあるピクセル	彩度の平均
色なし	12,516	8.0秒	0.0%	0.0000
根元→毛先	12,516	8.0秒	90.1%	0.5745
束ごと	12,516	8.1秒	56.4%	0.1808

色は位置どおりに乗るか（毛を1本だけ、真横から）

根元を赤(1,0,0)、毛先を青(0,0,1)にして、その間を位置で混ぜた。画の列ごとに、透明度で重みを付けて赤と青の平均を出し、sRGB

を線形に戻してから青の割合を求めている。17か所を測って直線をあてはめた（表は4か所おき）。

位置	赤(sRGB)	青(sRGB)	青の割合	直線との差	光
0.10	217.8	139.7	0.2718	+0.0282	当てる
0.30	203.1	160.5	0.3718	-0.0093	当てる
0.50	178.7	179.0	0.5009	-0.0177	当てる
0.70	149.7	199.0	0.6530	-0.0032	当てる
0.90	108.3	214.5	0.8176	+0.0239	当てる
0.10	246.1	86.5	0.0926	-0.0023	発光
0.30	220.9	150.3	0.2976	+0.0003	発光
0.50	189.4	190.3	0.5028	+0.0031	発光
0.70	151.3	220.6	0.6987	-0.0035	発光
0.90	85.7	245.3	0.9084	+0.0037	発光

あてはめた直線

周囲の光は赤にも青にも同じだけ乗るので、青の割合は (位置×k + a) ÷ (k + 2a)

という位置の1次式になるはず。この形なら「傾き + 2×切片 = 1」が成り立つ。

光	傾き	切片	R ²	直線からの最大の ずれ	傾き + 2×切片
当てる	0.6877	0.1748	0.99142	0.0282	1.0373
発光	1.0123	-0.0064	0.99991	0.0037	0.9996

width は根元から先まで一定か

thickness 0.008 で 2,500本ぶんを平均した。位置は毛の中の点の並び（9点＝8区間）。

位置	width の平均	根元との比	ばらつき
0.000	0.008000	1.0000	0.00000000
0.125	0.008000	1.0000	0.00000000
0.250	0.008000	1.0000	0.00000000
0.375	0.007987	0.9984	0.00000000
0.500	0.007865	0.9831	0.00000000
0.625	0.007536	0.9419	0.00000000
0.750	0.006802	0.8502	0.00000000
0.875	0.004981	0.6227	0.00000000
1.000	0.000000	0.0000	0.00000000

わかったこと

- ・毛が持っているのは P と width だけだった。「根元から何割の位置か」を示す数値は付いてこない。プリミティブごとに primpoints で点を並べ、順番から 0～1 を自分で振る。頂点にもディテールにも何も入っていない。
- ・光を当てたまま測ると、色は比例するが 1:1 にはならない。青の割合 = $0.6877 \times \text{位置} + 0.1748$ 。周囲の光が赤にも青にも同じだけ乗るので、割合が 0.5 側へ寄る。
- ・その見立ては検算できる。周囲の光を a、色そのものの強さを k とすると、青の割合は $(\text{位置} \times k + a) \div (k + 2a)$ 。この形なら 傾き + $2 \times \text{切片} = 1$ になる。実測は 1.0373（3.7% のずれ）。
- ・発光にして光の影響を外すと、位置そのものになった。 $1.0123 \times \text{位置} - 0.0064$ 、 $R^2 = 0.99991$ 、直線からの最大のずれ 0.0037。傾き + $2 \times \text{切片}$ も 0.9996。色を測りたいときは、光を当てずに測る。
- ・width は一定ではなく、先細りしていた。根元から 1/4 までは 0.008 のままで、そこから落ちて毛先でちょうど 0。hairgen のパラメータにある hairprofile というカーブのとおりに変えられている。
- ・先細りは 12,516本すべてで同じだった。どの位置でもばらつきが 0.00000000。毛ごとの個性ではなく、全部に同じカーブが掛かっている。
- ・色を付けてもレンダ時間はほとんど変わらない。8.0秒 / 8.0秒 / 8.1秒。色は形の複雑さを増やさない。
- ・束ごとに色を変えると、束が目で見えなくなる。実験043で測った144個の束が、そのまま房として見える。数字で出した結果を、見た目で確かめられる形にしておく。
- ・測り方について。実験045で見つけた「Apprentice のロゴが測定に混ざる」問題は、この実験でも最初に出た。光を当てた側の R^2 が 0.94 にしかならず、毛の右半分だけ値が乱れていた。ロゴの箱を外したら 0.99142 になった。同じ罫は、画を測るかぎり毎回出る。

次にやること

- ・毛を動かす（風・重力）

- ・ サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・ 毛を土台のアニメーションに付いてこさせる

土台が動いたとき、毛は付いてくるのか — 入力2をつなぐかどうかだけの差

2026-09-14 | Houdini 21.0.700 | /obj/groom | ノード 8 / 接続 11

何を作ったか

hairgen::2.0 の入力は5つある。実験043で読んだラベルは 0:Rest Skin、1:Guides、2:Animated Skin、3:Volumes、4:Guide Interpolation Mesh。つまり「元の形」と「動いている形」を別々に渡す作りになっている。

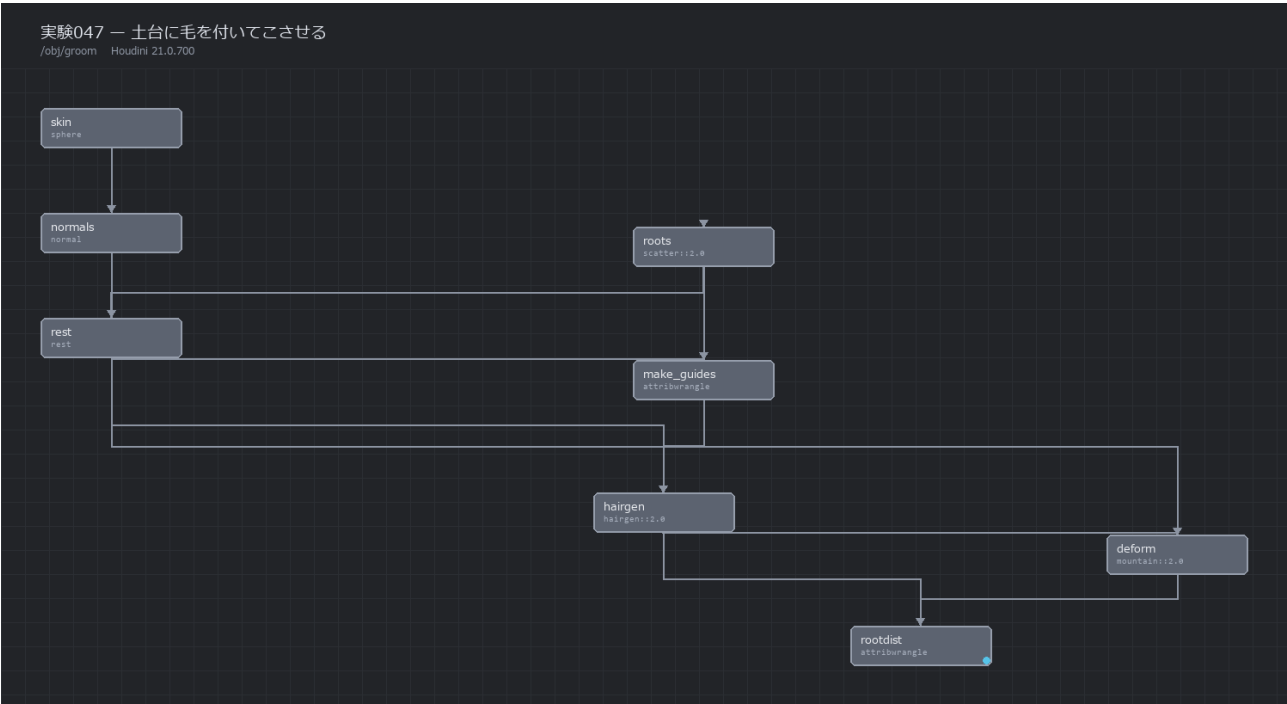
では、動いている形をつながなかったら毛はどうなるのか。土台の球を mountain で少しずつ変形させ、毛の根元から土台の表面までの距離をフレームごとに測った。

つないだ場合、2,500本すべてが全フレームで 0.000000。表面にいる割合は 100.0%。つながらない場合、変形が 0.5 まで進むと平均 0.041807、表面（0.01未満）にいるのは 12.9% だけになる。

そして「離れ」は変形の大きさに比例した。離れ ÷ 変形が 0.08760 / 0.08713 / 0.08629 / 0.08512 / 0.08361。変形の 8~9% しか離れないのは、測っているのが最寄りの表面までの距離だから。近くの別の場所が寄ってくるので、変形そのものの大きさにはならない。

ただし、この差は見た目ではほとんど分からない。0.042 は球の半径の 4% で、毛の長さ 0.5 の 8% にすぎない。目で見つけられないずれは、数字でしか見つからない。

ノードの接続



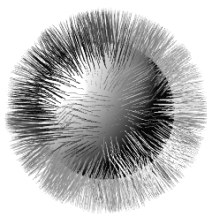
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
rest	rest	(変更なし)
deform	mountain::2.0	elementsiz = 0.9 height = 0.5
roots	scatter::2.0	npts = 200
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 200.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.01
rootdist	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); vector root = point(0, "P", pts[0]); f@rootdist = xyzdist(1, root);

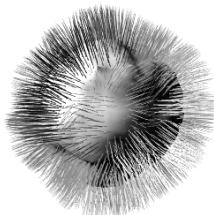
変形の前と後

土台の球を mountain で変形させた。毛は 2,500本、長さ 0.5、太さ 0.010。カメラは2枚とも同じ。OpenGL。



Houdini

フレーム1。変形はまだ 0。



Houdini

フレーム25。変形 0.5。動いている形をつないであるので、根元は表面に乗ったまま。

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合
1	0.00000	0.000000	0.000000	100.0%
5	0.08333	0.000000	0.000000	100.0%
10	0.18750	0.000000	0.000000	100.0%
15	0.29167	0.000000	0.000000	100.0%

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合
20	0.39583	0.000000	0.000000	100.0%
25	0.50000	0.000000	0.000000	100.0%

動いている形をつながないと、どれだけ離れるか

同じシーンで、hairgen の入力2 (Animated

Skin) だけを外した。毛の根元から、動いている土台の表面までの距離を xyzdist で測っている。

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合	離れ ÷ 変形
1	0.00000	0.000000	0.000000	100.0%	—
5	0.08333	0.007300	0.024673	73.6%	0.08760
10	0.18750	0.016337	0.055543	35.4%	0.08713
15	0.29167	0.025169	0.086424	22.8%	0.08629
20	0.39583	0.033692	0.117286	16.3%	0.08512
25	0.50000	0.041807	0.148076	12.9%	0.08361

わかったこと

- ・入力2 (Animated Skin) をつなぐと、根元は完全に表面に乗り続ける。2,500本すべてが全フレームで 0.000000。表面にいる割合は 100.0%。近い、ではなく、ぴったり 0 だった。

- ・つながないと、変形につれて離れていく。変形 0.5 で平均 0.041807、最大 0.148076。表面 (0.01未満) にいるのは 12.9% だけ。

- ・離れは変形の大きさに比例する。離れ ÷ 変形が 0.08760 / 0.08713 / 0.08629 / 0.08512 / 0.08361。5通りで 0.084~0.088

とほぼ一定。わずかに減っていくが、誤差の範囲ではなく単調である。

- ・比が 1 ではなく 0.085 なのは、測っているのが「最寄りの表面までの距離」だから。土台が変形しても、近くの別の場所が寄ってくるので、離れは変形そのものの大きさにはならない。何を測っているかで数字の意味が変わる。

- ・この差は見た目ではほとんど分らない。0.042 は球の半径の 4%、毛の長さ 0.5 の 8% にすぎない。目で見つけられないずれは、数字でしか見つけられない。実験044の「ガイドが使われていなかった」も同じ形の問題だった。

- ・「元の形」と「動いている形」を分ける作りの意味。毛の生える位置は元の形の上で決めておき、描くときだけ動いている形に寄せ替える。こうしないとフレームごとに毛の生える場所が変わり、毛がちらつく。

- ・rest は mountain を通しても壊れない。変形ノードは P を動かすだけで rest はそのまま流れる。だから「元の形」と「動いている形」の両方が、同じ rest を持った状態で hairgen に届く。

次にやること

- ・毛を揺らす (Vellum で毛をシミュレーションする)
- ・サンプル数と毛のザラつきの関係 (実験018と同じ問いを毛で)
- ・毛の生える範囲を絵で指定する (density のテクスチャ指定)

毛を揺らす — 曲がりにくさは「垂れる距離」ではなく「形」に効いていた

2026-09-14 | Houdini 21.0.700 | /obj/hairsim | ノード 9 / 接続 11

何を作ったか

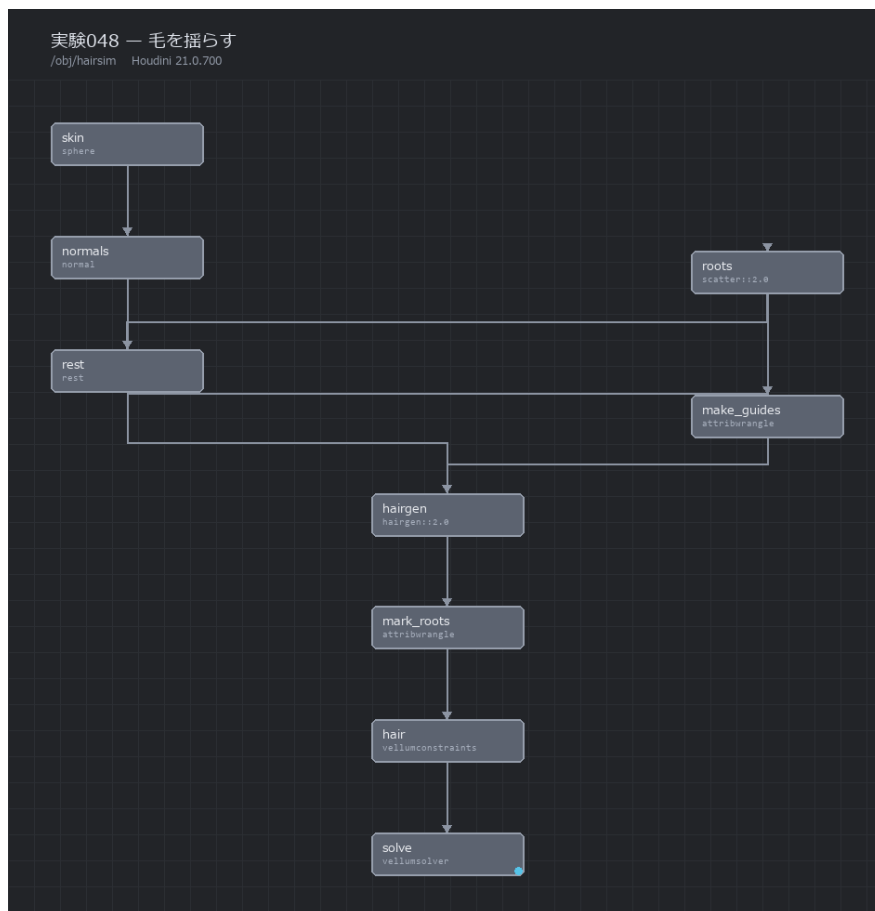
毛はここまで、生やす・束ねる・色を付ける・土台に付いてこさせる、と進めてきた。今回は動かす。vellumconstraints の種類に Hair があり、これでカーブを毛として扱える。

毎回と同じ問いを先に立てる。保たれるはずの量は何か。毛は伸び縮みしないはずなので、毛の長さがそれにあたる。48フレーム流したところ、長さのずれは最大 0.63%。途中のいちばん伸びた瞬間でも 1.49% だった。伸びないのではなく、少し伸びてから戻る。

根元の留め方でつまずいた。vellumconstraints の「Pin to Target」を足しても、拘束は1本も作られず、毛は根元ごと落ちていった。group でも pingroup でも、pintype を3通り変えても同じ。結局、Vellum が見る stopped というアトリビュートを根元の点に直接立てるやり方に変えた。これは効いた。

そして bendstiffness (曲がりにくさ)。0 から 1,000,000 まで100万倍変えても、毛先の高さは -0.5784 から -0.5852 と 1.2% しか変わらない。垂れる距離は決めていない。決めていたのは形のほうだった。毛のまっすぐさ (根元から毛先までの直線距離 ÷ 毛の長さ) は 0.9808 → 0.9994。いちばん曲がった毛で見ると 0.835 → 0.990 とはっきり変わる。100 で頭打ちになる。

ノードの接続

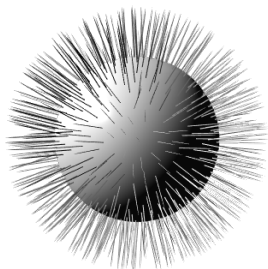


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 30 rows = 30 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 120
make_guides	attribwrangle	class = 0 snippet = int segs = 10; float len = 0.6; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 50.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.25 thickness = 0.012
mark_roots	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); setpointgroup(0, "roots", pts[0], 1); setpointattrib(0, "stopped", pts[0], 1);
hair	vellumconstraints	bendstiffness = 0.05 constrainttype = 4 domass = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

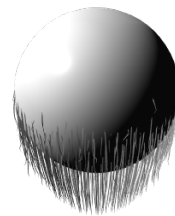
垂れる前と後

毛 620本、長さ 0.596、太さ 0.012。重力だけ。カメラは2枚とも同じ。OpenGL。



Houdini

フレーム1。毛は土台から真っすぐ外へ伸びている。



Houdini

フレーム48。根元は留まったまま、毛先が下へ垂れる。

frame	毛先の高さ 平均	中心からの距離	長さの平均	長さのばらつき	最初との比
1	0.000322	1.590527	0.596209	0.002767	1.000000
4	-0.042383	1.585333	0.597296	0.003049	1.001824
8	-0.241343	1.495563	0.597843	0.003018	1.002741
12	-0.507783	1.244712	0.599988	0.002973	1.006338
18	-0.432239	0.814113	0.598458	0.002978	1.003772
24	-0.446271	0.871080	0.598285	0.002919	1.003481
36	-0.512676	1.288071	0.598500	0.002836	1.003842
48	-0.579763	1.080746	0.599089	0.002810	1.004830

bendstiffness を100万倍まで変える

48フレーム目で測った。まっすぐさは「根元から毛先までの直線距離 ÷ 毛の長さ」。1 なら一直線、小さいほど曲がっている。

bendstiffness	毛先の高さ 平均	中心からの距離	まっすぐさ	いちばん曲がった毛	長さのずれ
0	-0.578420	1.081628	0.980830	0.834974	0.4800%
0.05	-0.579763	1.080746	0.981749	0.811067	0.4830%
1	-0.585164	1.085267	0.990014	0.946751	0.5161%
100	-0.585448	1.098518	0.999359	0.990332	0.5162%
10,000	-0.584058	1.097281	0.999803	0.982212	0.5223%
1,000,000	-0.584029	1.095081	0.999606	0.978315	0.5441%

わかったこと

- ・毛の長さは保たれた。ずれは最大 0.63%。いちばん伸びた瞬間でも 1.49%。伸びないのではなく、少し伸びてから戻る。Vellum の拘束は硬いばねであって、鎖ではない。
- ・根元を留めるのに「Pin to Target」は効かなかった。足しても拘束は1本も作られず、毛は根元ごと落ちていく。group でも pingroup でも、pintype を Permanent / Stopped / Soft と3通り変えても結果は同じだった。Vellum が見る stopped アトリビュートを根元の点に直接立てると効いた。
- ・domass（質量を計算する）は既定でオフ。オフのままだと mass が -1 のまま入る。オンにしてから測っている。
- ・bendstiffness は「どこまで垂れるか」を決めていなかった。0 から 1,000,000 まで100万倍変えても、毛先の高さは -0.5784 → -0.5852 と 1.2% しか動かない。
- ・決めていたのは形のほうだった。まっすぐさは 0.980830 → 0.999359 と上がり、いちばん曲がった毛で見ると 0.834974 → 0.990332。固い毛は「同じところまで垂れるが、まっすぐなまま垂れる」。
- ・100 で頭打ちになる。100 と 10,000 と 1,000,000 でまっすぐさは 0.999359 / 0.999803 / 0.999606。上げ続けても変わらない。これ以上はほぼ一直線なので、変わりようがない。
- ・「平均」だけ見ていたら見落としていた。まっすぐさの平均は 0.9808 → 0.9994 で 1.9% の差にすぎない。いちばん曲がった毛で見ると 0.835 → 0.990 で 18.6%

の差になる。効果が一部にしか出ないときは、平均ではなく端を見る。

・毛先の高さは一本調子に下がらない。-0.508 (12) → -0.432 (18) → -0.446 (24) →
-0.513 (36) →
-0.580 (48)。いったん振り戻している。重力で落ちて、拘束で引き戻されて、揺れている。

次にやること

- ・毛に風を当てる（実験029の popwind と同じ問いを毛で）
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・手順ページをグルームの分だけそろえる

毛に風を当てる — 実験029と同じ「二次の抵抗」だった

2026-09-14 | Houdini 21.0.700 | /obj/hairwind | ノード 9 / 接続 11

何を作ったか

実験029で、粒に当てる popwind は二次の抵抗 ($dv/dt = a(W-v)^2$) だと突き止めた。毛の場合はどうか。

毛は伸びないので、流される量には上限がある。根元から毛先までの距離は、どんなに強い風でも毛の長さを超えられない。つまり比例するはずがなく、どこかで頭打ちになる。問題はその近づき方で、候補を2つ先に立てた。一次の抵抗なら $s = W / (W + k)$ 、二次なら $s = W^2 / (W^2 + k^2)$ 。s

は「流され具合」= (毛先の x - 根元の x) ÷ 毛の長さ。

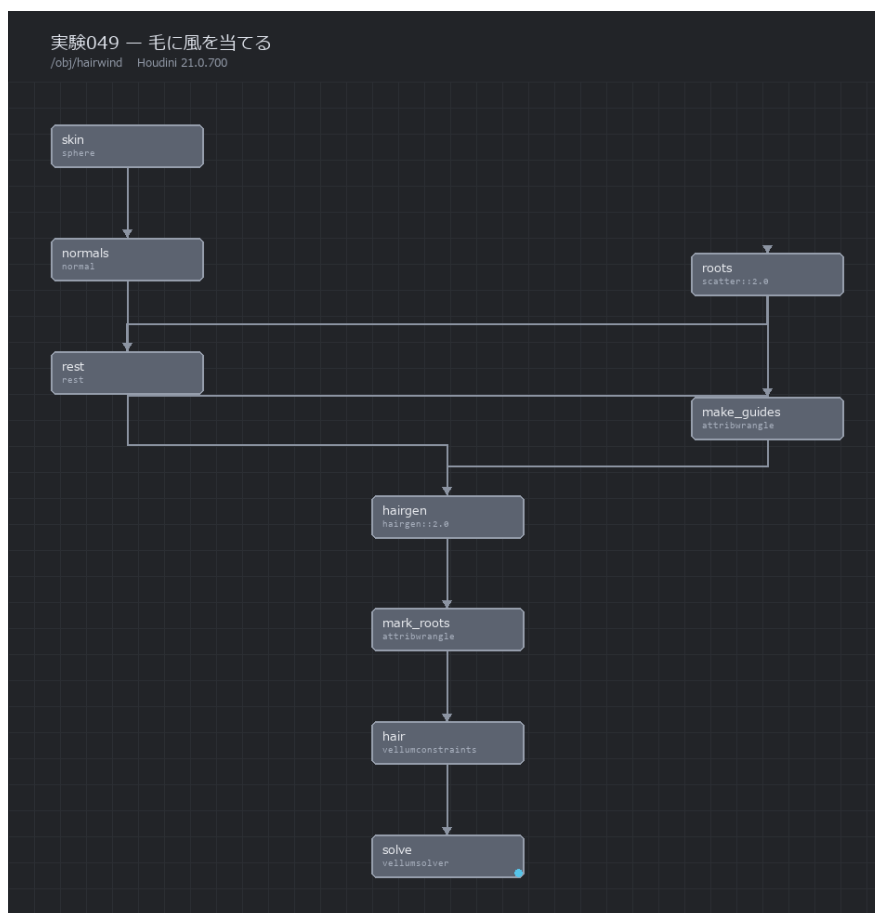
風速を 2 / 5 / 10 / 20 / 40 と変えて測り、k を総当たりで合わせた。残差の二乗和は一次が 0.026267、二次が 0.007404。二次のほうが 3.5倍よく乗る。粒のときと同じだった。

ただしどちらもぴったりにではない。二次でも最大 0.054

ずれる。そして強い風では毛が伸びる。長さは風速 0 で 0.599089 だったものが、40 では 0.631461 と 5.9% 伸びた。重力だけの実験048では 0.63% しか伸びなかったので、これは風の分である。Vellum の拘束は硬いばねであって鎖ではない、という実験048の結論が、強い力をかけると目に見える形で出てくる。

。

ノードの接続

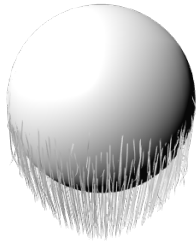


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 30 rows = 30 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 120
make_guides	attribwrangle	class = 0 snippet = int segs = 10; float len = 0.6; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 50.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.25 thickness = 0.012
mark_roots	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); setpointgroup(0, "roots", pts[0], 1); setpointattrib(0, "stopped", pts[0], 1);
hair	vellumconstraints	bendstiffness = 0.05 constrainttype = 4 domass = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618 windspeed = 20.0 windx = 1.0

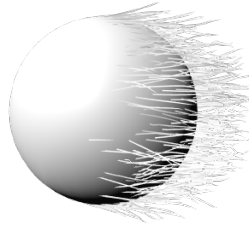
風のあるなし

毛 620本、長さ 0.596、太さ 0.012。風は +x 向き。カメラは2枚とも同じ。48フレーム目。OpenGL。



Houdini

風速 0。重力だけで下に垂れている。



Houdini

風速 40。毛が風下へ寝て、流され具合は 0.950279。

風速	流され具合	ばらつき	毛先の x	毛先の高さ	長さの平均
0	-0.004210	0.117582	0.014733	-0.579763	0.599089
2	0.318497	0.062716	0.207969	-0.552739	0.598813
5	0.736690	0.081876	0.457701	-0.314031	0.597901
10	0.868605	0.073819	0.537896	-0.139250	0.599409
20	0.923036	0.045864	0.578377	-0.042630	0.607896
40	0.950279	0.028984	0.617211	-0.023428	0.631461

2つの式にあてはめる

風速 0 のときの値 (-0.004210) を 0 とみなして正規化し、k を 0.01 刻みで総当たりして残差の二乗和がいちばん小さい値を選んだ。

風速	実測	一次 $W/(W+k)$	二次 $W^2/(W^2+k^2)$
2	0.321354	0.445434	0.303469
5	0.737794	0.667557	0.731403
10	0.869156	0.800641	0.915911
20	0.923358	0.889284	0.977563
40	0.950487	0.941398	0.994295

どちらの式が合うか

残差の二乗和が小さいほどよく乗る。

式	k	残差の二乗和	1点あたりの平均のずれ	最大のずれ
一次 $W/(W+k)$	2.49	0.026267	0.0725	0.1241
二次 $W^2/(W^2+k^2)$	3.03	0.007404	0.0385	0.0542

わかったこと

- ・二次の抵抗のほうがよく乗る。残差の二乗和が一次 0.026267 に対して二次 0.007404 で、3.5倍の差。実験029で粒について突き止めたのと同じ形だった。粒でも毛でも、Houdini の風は速度の差の二乗で効く。
- ・低い風速でいちばんはっきり分かれる。風速2 での実測は 0.321354。一次の式は 0.445434 と大きく外し、二次の式は 0.303469 と近い。頭打ちに入ったあと（10以上）はどちらも似た値になるので、区別は弱い風のところでつく。
- ・それでもぴったりではない。二次でも最大 0.054 ずれる。この式で完全に説明できるとは言えない。毛は粒と違って根元が固定されており、隣の毛ともぶつかる。
- ・強い風では毛が伸びる。長さは風速 0 で 0.599089、40 では 0.631461 と 5.9% 伸びた。重力だけの実験048では 0.63% だったので、これは風の分。Vellum の拘束は硬いばねであって鎖ではない、というのが強い力で見えてくる。
- ・風は毛をそろえる。流され具合のばらつきは 0.117582（風速0）→ 0.028984（風速40）と 4分の1になる。強い風の下では、どの毛もほぼ同じ向きになる。
- ・毛先は上がってくる。高さの平均は -0.579763（風速0）→ -0.023428（風速40）。横向きの風なのに縦の位置が変わるのは、毛が水平に近づくからで、重力に逆らって持ち上がっているわけではない。
- ・風速 0 でも流され具合は 0 ではない（-0.004210）。球の上に散らした毛なので、風下側・風上側が均等に混ざっていて、平均するとほぼ 0 になる。この値を差し引いてから式にあてはめている。
- ・測り方について。流される量そのものではなく「毛の長さで割った値」を見ている。こうすると 0~1 に収まり、上限が 1 だとはっきりする。上限のある量は、上限で割ってから比べる。

次にやること

- ・毛どうしのぶつかり（衝突）を入れると式がどう変わるか
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・実験050の節目 — グループ編のまとめ

APEX に入る —

2,220個の部品と、いちばん小さいグラフ

2026-09-14 | Houdini 21.0.700 | apex::Graph (シーンの中ではなくデータの中にある) | ノード 3 / 接続 2

何を作ったか

APEX は Houdini 20 から入ったリグ (骨組み) の仕組み。特徴は、ノードのネットワークそのものをデータとして持ち歩けること。SOP

のようにシーンの中に置かれるのではなく、ジオメトリの中に入って流れていく。

新しい分野なので、まず大きさを測るところから始めた。apex.Registry().findMatchingNames('*')

で数えると 2,220種。名前空間で分けると geo が 362、sop が 353 で、この2つだけで

715種ある。APEX の中から SOP

の処理をそのまま呼べるということ。名前空間の付いていないもの (算数や比較) が 475種。

次に、いちばん小さいグラフを組んだ。Value を2つ置いて Add につなぐだけ。3 + 4 = 7.0 が出た。

ここで1つつまずいた。Add の2つ目の入力に値を入れても効かない。a=3 / b=4

とパラメータに入れて実行すると、答えは 3.0 になる。エラーは出ない。入力の型を見ると a が Float、b

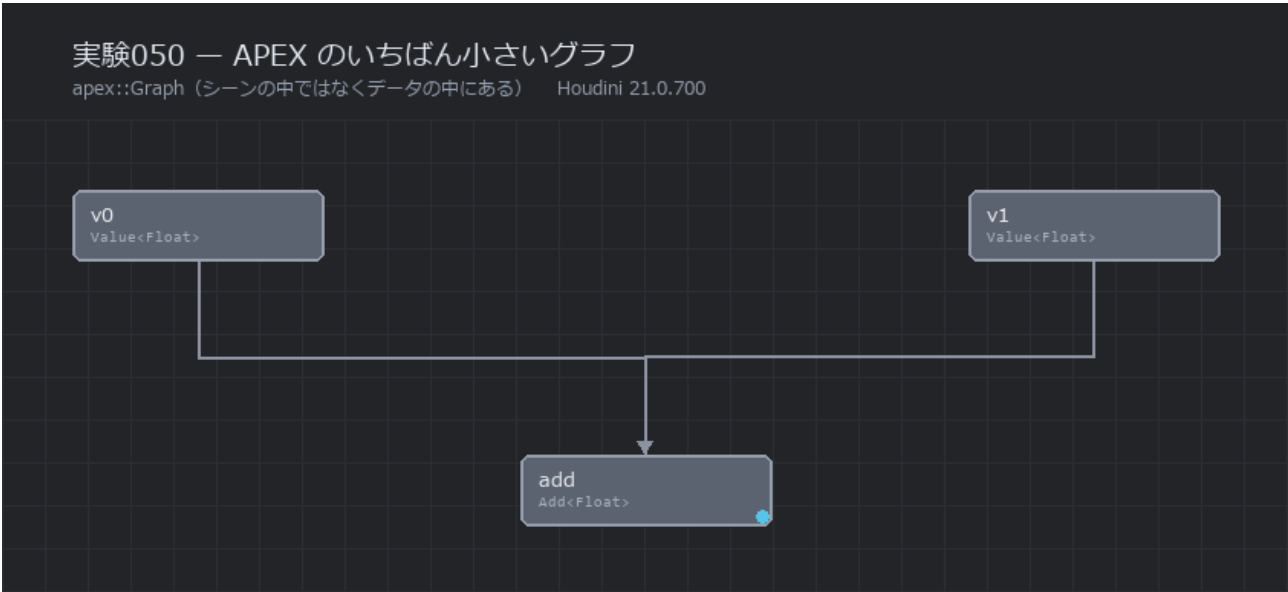
が VariadicArg で、b は「いくつでもつなげる可変長の口」だった。addSubPort

で枝を生やしてからつなぐと動く。可変長なので 1+2+...+10 = 55 もノード1個で通る。

実行は速い。組み立て (compile) が 0.0202ミリ秒、実行が1回あたり

0.499マイクロ秒。組み立ては実行41回分にあたる。

ノードの接続

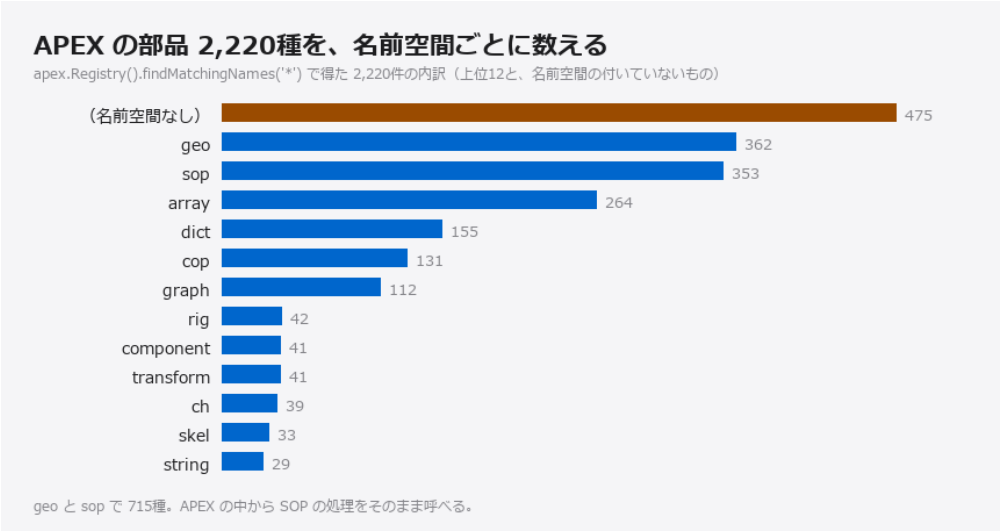


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
v0	Value	parm = 3.0
v1	Value	parm = 4.0
add	Add	a = Float b = VariadicArg

APEX の部品を数える

apex.Registry().findMatchingNames('*') で得た 2,220件を、名前空間で分けた。



上位12と、名前空間の付いていないもの。geo と sop で 715種を占める。

名前空間	数	何のための部品か
(名前空間なし)	475	算数・比較・変換など
geo	362	ジオメトリを触る
sop	353	SOP の処理をそのまま呼ぶ
array	264	配列を扱う
dict	155	辞書を扱う
cop	131	画（Copernicus）を扱う
graph	112	グラフ自身を書き換える
rig	42	リグの部品
component	41	リグの構成要素
transform	41	位置・回転・拡大
ch	39	チャンネル（アニメーション）
skel	33	骨組み
string	29	文字列

いちばん小さいグラフ

Value を2つ置いて Add につなぐ。ノード3個・ポート9個。

調べたこと	結果
ノードの数	3個
ポートの数	9個
1つ目の入力の型	Float
2つ目の入力の型	VariadicArg
3 + 4 の答え	7.0

調べたこと	結果
枝を生やさず parm だけ入れた場合	3.0 (b が無視される)

2つ目の入力に変長

枝（サブポート）を生やした数だけ足せる。ノードは1個のまま。

足す数	式	答え	合っているか
2	$1 + 2$	3.0	はい
3	$1 + 2 + 3$	6.0	はい
5	$1 + 2 + 3 + 4 + 5$	15.0	はい
10	$1 + \dots + 10$	55.0	はい

速さと、SOP 側の入口

実行は10,000回の平均。SOP の種類は `hou.sopNodeTypeCategory()` から数えた。

測ったこと	値
組み立て（compile）	0.0202 ミリ秒
10,000回の実行	0.0050 秒
1回あたりの実行	0.499 マイクロ秒
組み立ては実行何回分か	41回分
apex:: で始まる SOP	38種
kinefx:: で始まる SOP	133種
SOP 全体	1,273種
2つで SOP 全体の	13.4%

わかったこと

- APEX の部品は 2,220種あった。名前空間で分けると geo が 362、sop が 353 で、この2つだけで 715種。APEXの中から SOP の処理をそのまま呼べるということで、リグ専用の小さな言語ではない。

- 名前空間の付いていない部品が 475種。Add / Abs / Clamp / Convert といった算数と変換で、Add<Float> のように型ごとに別の部品になっている（Add だけで16種）。

- いちばん小さいグラフは動いた。Value を2つ置いて Add につなぎ、 $3 + 4 = 7.0$ 。ノード3個・ポート9個。

- 2つ目の入力に値を入れても効かない。 $a=3 / b=4$ をパラメータに入れて実行すると答えは 3.0。パラメータの中身を見ると a も b も入っているのに、b だけ無視される。エラーも警告も出ない。

- 理由は入力の型だった。a が Float、b が VariadicArg<Float>。b は「いくつでもつなげる可変長の口」で、値を直接入れる場所ではない。addSubPort で枝を生やしてからつなぐ。型の合わないつなぎ方をすると Mismatched type: /b:value:Float -> /add:b:VariadicArg<Float> と出るので、そこで気づける。

- 可変長なので、いくつでも足せる。枝を10本生やせば $1+2+\dots+10 = 55.0$ がノード1個で出る。SOP の merge のような考え方が、算数の部品にも入っている。

- ・実行は 0.499マイクロ秒。10,000回で 0.0050秒。組み立て (compile) は 0.0202ミリ秒で、実行41回分。組み立ては1回きり、実行は毎フレーム、という作りに合っている。

- ・ SOP 側の入口は 171種。apex:: が 38種、kinefx:: が 133種で、SOP 全体 1,273種の 13.4%。骨組みまわりだけで SOP の1割を超えている。

- ・ この実験は「新しい分野に入るとき、まず何を測るか」の型でもある。部品の数を知る・いちばん小さいものを動かす・速さを測る。実験042でグループに入ったときも同じ順番だった。

次にやること

- ・ 骨 (skeleton) を作って APEX で動かす
- ・ apex::invokegraph で SOP からグラフを呼ぶ
- ・ リグの部品 (rig 名前空間の42種) を1つずつ見る

骨を作って動かす — rotate と prerotate を取り違えると、関節ごと飛んでいく

2026-09-14 | Houdini 21.0.700 | /obj/rig | ノード 3 / 接続 2

何を作ったか

実験050で APEX の入口を測った。今回はその手前にある骨組み（KineFX）を触る。

KineFX

の骨はただの点と線で持たれている。点1つが関節1つ、線が親子のつながり。名前（name）を付けて `kinefx::rigdoctor` を通すと、足りない属性が埋まる。手で組んだ時点では P と name

の2つしかなく、`rigdoctor` が

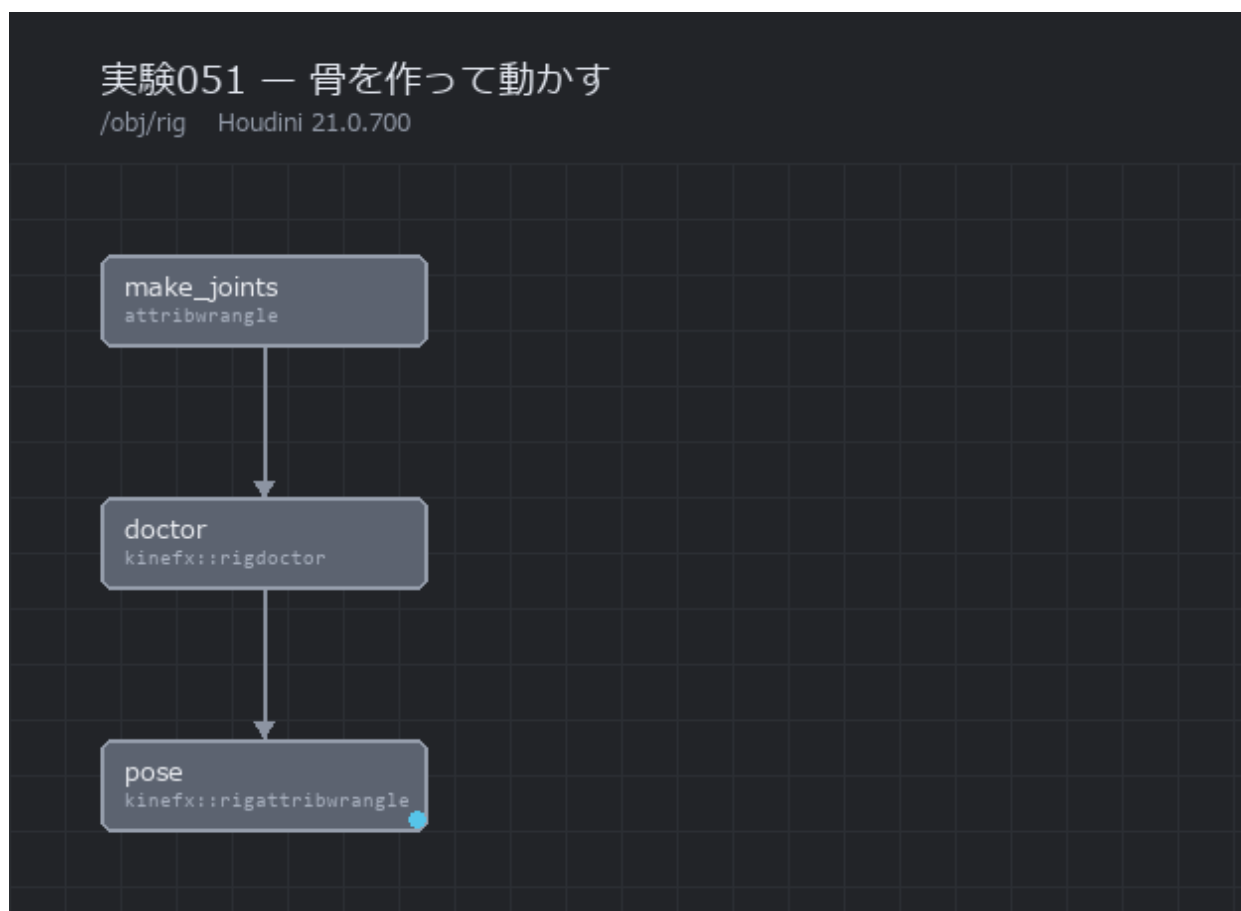
`localtransform` (4×4の行列) ・ `transform` (3×3) ・ `parent_idx` (親が誰か) の3つを足した。

先に式を立てた。根元を原点、関節が (0,1,0) と (0,2,0) にあるなら、真ん中を Z 軸まわりに θ 回したとき先端は $(-\sin \theta, 1 + \cos \theta, 0)$ に来るはず。

6通りの角度で測ったところ、式とのずれは最大 0.0000000060。回した関節そのものの移動は 0.0000000000 だった。

ただし最初は合わなかった。VEX で `rotate()` を使っていて、90度で先端が (-2, 0, 0) に飛んだ。式の値は (-1, 1, 0) なので、ずれは 1.41。回した関節そのものまで (-1, 0, 0) へ動いていた。原因は掛ける順番で、`prerotate()` に変えると一致した。

ノードの接続

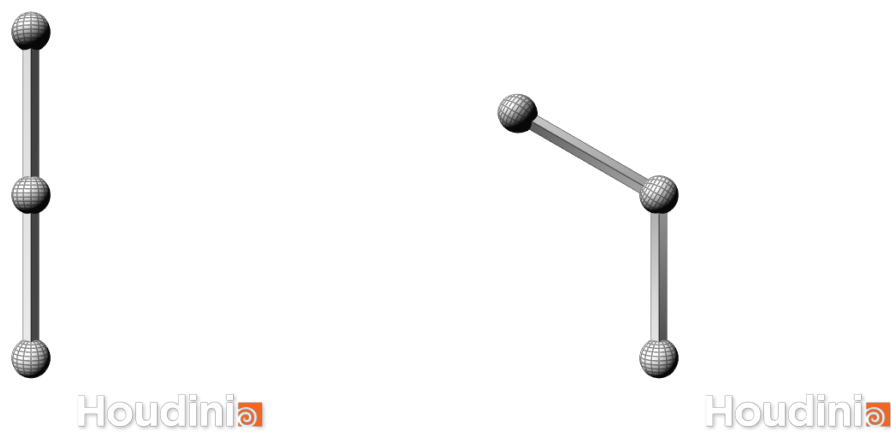


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 45.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; // prerotate は「先に回してから動かす」。rotate だと順番が逆になり、 // 関...

曲げる前と後

関節3つの骨。真ん中を Z 軸まわりに 60度回した。線のままでは細くて写らないので polywire
で筒にしてある。カメラは2枚とも同じ。



回す前。関節は (0,0,0) (0,1,0) (0,2,0) に一直線。真ん中を60度。根元と真ん中は動かず、先だけが折れている。

関節	位置	parent_idx	意味
joint0	(0, 0, 0)	-1	親がない=根元
joint1	(0, 1, 0)	0	親は joint0
joint2	(0, 2, 0)	1	親は joint1

真ん中を回すと、先端はどこへ行くか

式は $(-\sin \theta, 1 + \cos \theta, 0)$ 。ずれは実測と式の距離。

角度	先端 実測 x	先端 実測 y	式 x	式 y	ずれ	真ん中の関節の移動
0°	0.000000	2.000000	-0.000000	2.000000	0.000000000	0.000000000
15°	-0.258819	1.965926	-0.258819	1.965926	0.000000014	0.000000000
30°	-0.500000	1.866025	-0.500000	1.866025	0.000000044	0.000000000
45°	-0.707107	1.707107	-0.707107	1.707107	0.000000049	0.000000000

角度	先端 実測 x	先端 実測 y	式 x	式 y	ずれ	真ん中の関節の移動
60°	-0.866025	1.500000	-0.866025	1.500000	0.000000044	0.000000000
90°	-1.000000	1.000000	-1.000000	1.000000	0.000000060	0.000000000

rotate と prerotate を取り違えるとどうなるか

同じ角度・同じ骨で、VEX の1語だけを変えた。

角度	書き方	真ん中 x	真ん中 y	先端 x	先端 y
30°	prerotate (正)	0.000000	1.000000	-0.500000	1.866025
30°	rotate (誤)	-0.500000	0.866025	-1.000000	1.732051
60°	prerotate (正)	0.000000	1.000000	-0.866025	1.500000
60°	rotate (誤)	-0.866025	0.500000	-1.732051	1.000000
90°	prerotate (正)	0.000000	1.000000	-1.000000	1.000000
90°	rotate (誤)	-1.000000	-0.000000	-2.000000	-0.000000

わかったこと

- ・ KineFX の骨は、ただの点と線だった。点1つが関節1つ、線が親子のつながり。普通の wrangle でそのまま組める。専用のノードで作らなくてよい。

- ・ 手で組んだ時点では P と name しか無い。kinefx::rigdoctor が localtransform (4×4) ・ transform (3×3) ・ parent_idx の3つを足す。親子関係は線のつながりから読み取られる（根元の parent_idx は -1）。

- ・ ただし rigdoctor は既定では何も足さない。transformations / inittransforms / outputparentidx のトグルはすべて既定オフ。オンにして初めて属性が出てくる。

- ・ 式とのずれは最大 0.000000060。6通りの角度すべてで、先端は $(-\sin \theta, 1 + \cos \theta, 0)$ にぴたりと来る。回した関節そのものの移動は 0.000000000 で、桁の限界まで動いていない。

- ・ 最初は合わなかった。原因は掛ける順番。VEX の rotate() を使うと、90度で先端が $(-2, 0, 0)$ へ飛ぶ。式の値 $(-1, 1, 0)$ とのずれは 1.41。回した関節そのものまで $(-1, 0, 0)$ へ動いていた。

- ・ prerotate() に変えると一致した。違いは「回してから動かす」か「動かしてから回す」か。関節の位置は移動の分で決まるので、先に回さないと関節ごと原点のまわりを回ってしまう。

- ・ この間違いは絵で見ても気づきにくい。30度なら「少し曲がった骨」に見えてしまう。気づけたのは、先に式を立てて数字で突き合わせたから。90度まで振ると差が 1.41 まで開くので、大きく振ってみるのが見つけるコツ。

- ・ 回転は子にだけ伝わる。真ん中を回しても根元は動かない。親から子へ行列を掛け合わせていく作りなので、自分より上には影響しない。

次にやること

- ・ 骨に肉を付ける (kinefx::jointdeform で皮を付いてこさせる)
- ・ APEX のグラフから骨を動かす (apex::invokegraph)
- ・ 関節の回る範囲を決める (kinefx::configurejointlimits)

骨に肉を付ける ― 曲げると痩せる。90度で 27% 減った

2026-09-14 | Houdini 21.0.700 | /obj/rigskin | ノード 6 / 接続 7

何を作ったか

実験051で骨を曲げた。今回はその骨に皮（メッシュ）を結び付けて、骨と一緒に動かす。

キャラクターの腕を曲げると肘のあたりが痩せるーというのはこの仕組みでよく言われる話だが、どれくらい痩せるのかは聞いたことがない。測った。

結び付けるノード（`kinefx::jointcaptureproximity`）は、皮に `boneCapture` という属性を1つ足すだけ。これが「この点はどの骨にどれだけ従うか」を持つ。

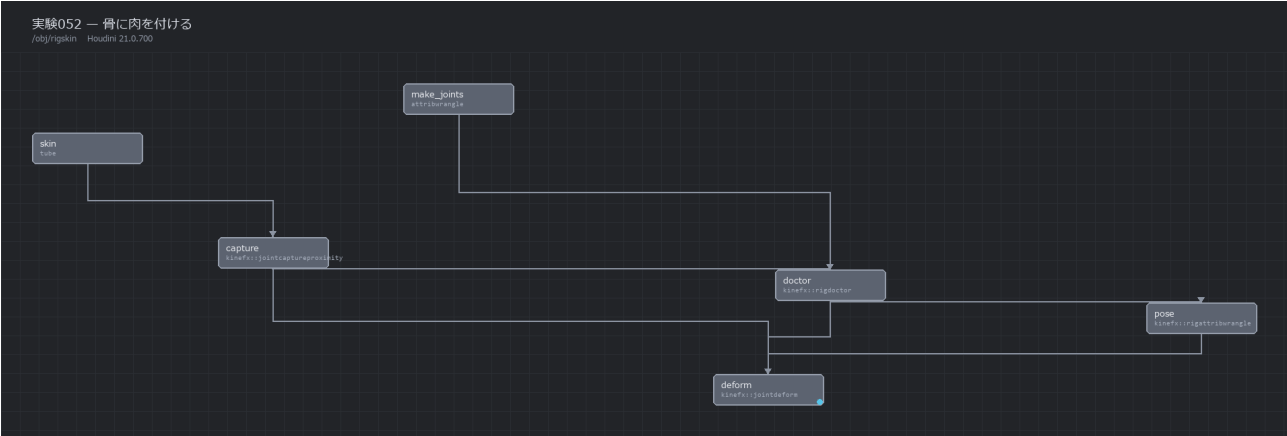
曲げると体積は減った。15度で 0.92%、45度で 7.95%、90度で 27.13%、120度で 40.69%。しかもきれいな式に乗る。減った分を $(1 - \cos \theta)$ で割ると、6通りすべてで 27.1284 になった。つまり 体積の減り = $27.1284\% \times (1 - \cos \theta)$ 。

そして原因が分かった。`maxinfluences`（1つの点が影響を受ける骨の数）が既定で 2 になっている。これを 1 にすると、90度での体積の減りは 27.13% から 0.85% に落ちる。根元の点の移動も 0.443 から 0.0000000000 になる。

ただし代償がある。1本にすると関節が鋭く折れて、内側の角で皮どうしが食い込む。滑らかさと体積は引き換えになっている。

測り方について。体積は面から直接求めた（三角形と原点で四面体を作り、符号付きの体積を足す）。曲げる前の値は 0.636074 で、24角形の筒の理屈の値と小数点以下6桁まで一致した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	<code>class = 0</code> <code>snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...</code>
doctor	kinefx::rigdoctor	<code>inittransforms = 1</code> <code>outputparentidx = 1</code> <code>transformations = 1</code>

ノード	タイプ	デフォルトから変更したパラメータ
pose	kinefx::rigattribwrange	angle = 90.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle"))), set(0, 0, 1)); ...
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	(変更なし)

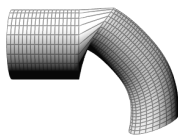
曲げる前と後

皮は 1,440点 / 1,418面 の筒。骨は関節3つ。真ん中を90度回した。カメラは3枚とも同じ。



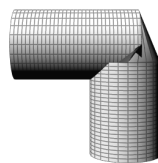
Houdini

曲げる前。体積 0.636074。



Houdini

90度（既定の maxinfluences 2）。なめらかに曲がるが、体積は 27.13% 減る。



Houdini

90度（maxinfluences 1）。体積の減りは 0.85% だが、関節が鋭く折れる。

角度	体積	最初との比	減った分	先端の移動
0°	0.636074	1.000000	0.0000%	0.000000
15°	0.630194	0.990756	0.9244%	0.274093
30°	0.612955	0.963655	3.6345%	0.543495
45°	0.585533	0.920543	7.9457%	0.803599
60°	0.549795	0.864358	13.5642%	1.049952
90°	0.463517	0.728716	27.1284%	1.484857
120°	0.377238	0.593074	40.6926%	1.818571

減り方に式はあるか

減った分を、角度そのもの・その二乗・(1 - cos θ) の3通りで割ってみる。どれかが一定なら、それが式の形になる。

角度	減った分	÷ θ	÷ θ ²	÷ (1 - cos θ)
15°	0.9244%	0.06163	0.0041084	27.1286

角度	減った分	$\div \theta$	$\div \theta^2$	$\div (1 - \cos \theta)$
30°	3.6345%	0.12115	0.0040384	27.1284
45°	7.9457%	0.17657	0.0039238	27.1284
60°	13.5642%	0.22607	0.0037678	27.1284
90°	27.1284%	0.30143	0.0033492	27.1284
120°	40.6926%	0.33911	0.0028259	27.1284

影響を受ける骨の数を変える（90度で比較）

maxinfluences は「1つの点が、いくつの骨に従うか」。既定は 2。

骨の数	体積	減った分	根元の帯の移動 最大	先端の移動 最大
1	0.630683	0.8475%	0.000000000	1.484857
2（既定）	0.463517	27.1284%	0.442943007	1.484857
3	0.401732	36.8419%	0.468320528	1.316201

どの高さの点が、どれだけ動いたか（90度・既定の2本）

元の高さで8つの帯に分けた。骨は (0,0,0) (0,1,0) (0,2,0)。

元の高さ	点の数	移動の平均	移動の最大
0.00～0.25	192	0.426011	0.442943
0.25～0.50	168	0.387273	0.432379
0.50～0.75	192	0.287112	0.370365
0.75～1.00	168	0.176458	0.268907
1.00～1.25	168	0.188160	0.301651
1.25～1.50	192	0.501344	0.829455
1.50～1.75	168	0.942587	1.125673
1.75～2.01	192	1.286820	1.484857

わかったこと

- ・結び付けても、皮に増える属性は1つだけだった。boneCapture。これが「この点はこの骨にどれだけ従うか」を持つ。骨と皮は別々のジオメトリのまま、この属性1つでつながっている。

- ・曲げると体積は減る。15度で 0.92%、45度で 7.95%、90度で 27.13%、120度で 40.69%。「腕を曲げると肘が痩せる」は、この数字のこと。

- ・減り方には、きれいな式があった。減った分を $(1 - \cos \theta)$ で割ると、6通りすべてで 27.1284（15度だけ 27.1286）。角度そのもので割ると 0.062～0.339、二乗で割っても 0.0041～0.0028 とばらけるのに、この形だけがそろそろ。体積の減り = $27.1284\% \times (1 - \cos \theta)$ 。

- ・この式が正しいければ、90度でちょうど 27.1284% になる。 $1 - \cos 90^\circ = 1$ だから。実測もぴたりその値だった。式を立てると、測っていない角度も言える（180度なら 54.26% になるはず）。

- ・原因は maxinfluences が既定で 2 だったこと。1つの点が2本の骨に従うので、関節の近くでは2つの行列が混ざる。混ざった結果は、どちらの行列でもない縮んだ位置になる。

- ・ 1本にすると、体積の減りは 27.13% から 0.85% に落ちる。根元の点の移動も 0.443 から 0.000000000 へ。根元がぴくりとも動かなくなる。
- ・ ただし代償がある。1本にすると関節が鋭く折れて、内側の角で皮どうしが食い込む。なめらかさと体積は引き換え。なお 0.85% という値は、その食い込みの分を含んでいない可能性があるので、そこまでは断定しない。
- ・ 3本にするともっと減る (36.84%)。混ざる行列が増えるほど痩せる。「なめらかにしたいから影響する骨を増やす」は、体積の面では逆効果。
- ・ 既定のままだと、根元の端まで動く。元の高さ 0.00~0.25 の点が平均 0.426 も動いている。真ん中 (0.75~1.00) の 0.176 より大きい。関節から遠いのに大きく動くのは、根元の骨が短く、2本目の骨の重みが残っているため。
- ・ 測り方について。体積は面から直接求めた (三角形と原点で四面体を作り、符号付きの体積を足す)。曲げる前の値 0.636074 は、24角形の筒の理屈の値と小数点以下6桁まで一致した。測り方が正しいことを、先に別の方法で確かめている。

次にやること

- ・ 体積の減りを抑える方法 (dual quaternion skinning) を測る
- ・ 関節の回る範囲を決める (kinex::configurejointlimits)
- ・ APEX のグラフから骨を動かす

痩せない曲げ方はあった — Dual Quaternion で体積の減りが 154分の1 になる

2026-09-14 | Houdini 21.0.700 | /obj/rigskin | ノード 7 / 接続 8

何を作ったか

実験052で、骨を曲げると皮が痩せることを測った。90度で 27.13%、減り方は $27.1284\% \times (1 - \cos \theta)$ というきれいな式に乗っていた。

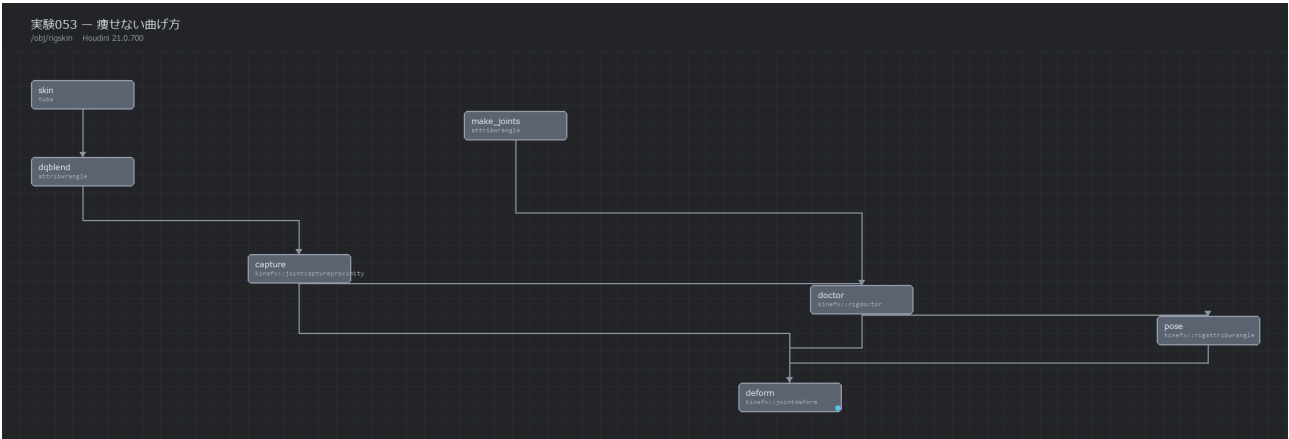
kinefx::jointdeform には混ぜ方が3つある。Linear（既定）、Dual Quaternion、その2つの中間。同じ骨・同じ皮・同じ角度で比べた。

90度での体積の減りは、Linear 27.1284%、Dual Quaternion 0.1764%、中間（半々）14.6904%。Dual Quaternion は Linear の 154分の1 しか痩せない。

そして式のほうも違った。減った分を $(1 - \cos \theta)$ で割ると、Linear は 6通りすべてで 27.1284~27.1286（幅 0.0001）とびたり一定。Dual Quaternion は 0.1513~0.2003（幅 0.0490）でそろわない。中間は 13.6663~15.6573（幅 1.9911）。式に乗るのは Linear だけだった。

「2つの中間」は、そのままでは動かない。点ごとの混ぜ具合を表す属性が要る（Missing dual quaternion blend attribute）。ここでは全点に 0.5 を入れた。すると減りは 14.6904% で、Linear の 27.1284% のほぼ半分。混ぜた割合がそのまま結果に出ている。

ノードの接続



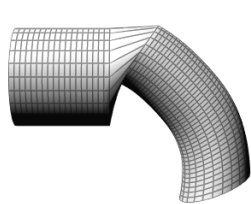
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwrangle	angle = 90.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle")), set(0, 0, 1)); ...

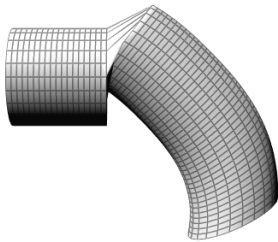
ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
dqblend	attribwrangle	snippet = f@dqblend = 0.5;
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

同じ90度を、2つのやり方で

皮は 1,440点 / 1,418面 の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear（既定）。内側が食い込み、体積は 27.13% 減る。Dual Quaternion。太さが保たれ、体積の減りは 0.18%。

角度	Linear（既定）	Dual Quaternion	2つの中間
0°	0.0000%	0.0000%	0.0000%
15°	0.9244%	0.0052%	0.4657%
30°	3.6345%	0.0205%	1.8417%
45°	7.9457%	0.0459%	4.0666%
60°	13.5642%	0.0808%	7.0413%
90°	27.1284%	0.1764%	14.6904%
120°	40.6926%	0.3005%	23.4860%

それぞれ式に乗るか（減った分 ÷ (1 - cos θ)）

一定なら、その形の式に乗っている。

角度	Linear（既定）	Dual Quaternion	2つの中間
15°	27.1286	0.1513	13.6663

角度	Linear（既定）	Dual Quaternion	2つの中間
30°	27.1284	0.1532	13.7469
45°	27.1284	0.1568	13.8844
60°	27.1284	0.1616	14.0827
90°	27.1284	0.1764	14.6904
120°	27.1284	0.2003	15.6573
ばらつきの幅	0.0001	0.0490	1.9911

中心からの太さ（90度するとき、高さの帯ごと）

その帯の点が、帯の中心からどれだけ離れているかの平均。曲げる前の筒の半径は 0.32。

元の高さ	Linear（既定）	Dual Quaternion	2つの中間
0.00～0.50	0.305559	0.353538	0.329018
0.50～0.75	0.281015	0.329243	0.304530
0.75～1.00	0.279344	0.327047	0.302601
1.00～1.25	0.279592	0.327048	0.302731
1.25～1.50	0.328019	0.341245	0.333388
1.50～2.01	0.350916	0.350916	0.350916

わかったこと

- ・ Dual Quaternion は 154分の1 しか痩せない。90度で Linear が 27.1284% 減るのに対し、0.1764%。 $27.1284 \div 0.1764 = 153.8$ 倍の差。
- ・ 式に乗るのは Linear だけだった。(1 - cos θ) で割った値の幅は、Linear が 0.0001、Dual Quaternion が 0.0490、中間が 1.9911。Linear は式で言い切れるが、Dual Quaternion はこの形では表せない。
- ・ Dual Quaternion も、角度を上げると少しずつ痩せる。0.0052%（15度）→ 0.3005%（120度）。ゼロではない。ただし 120度でも 0.3% なので、実用上は無視できる。
- ・ 「2つの中間」は、そのままでは動かない。「Missing dual quaternion blend attribute」で止まる。点ごとの混ぜ具合を表す属性を、自分で作って名前を指定する。ここでは全点に 0.5 を入れた。
- ・ 半々に混ぜると、減りもほぼ半分になる。14.6904% は Linear の 27.1284% の 0.5415倍。混ぜた割合がそのまま結果に出る。部位ごとに混ぜ方を変えられるのが、この方式の使いどころになる。
- ・ 太さで見ても差ははっきり出る。関節の近く（高さ 0.75～1.00）で、Linear が 0.279344、Dual Quaternion が 0.327047。元の半径 0.32 に対して、Linear は 13% 細く、Dual Quaternion はほぼそのまま。
- ・ 関節から離れた場所では3つとも同じ。高さ 1.50～2.01 の帯はどのやり方でも 0.350916 で完全に一致する。骨1本にしか従っていない場所では、混ぜ方は関係ない。この一致は、測り方が正しいことの確認にもなっている。
- ・ ではなぜ Linear が既定なのか。この実験からは分からない。速さか、ねじれの扱いか、互換性か。確かめていないので断定しない。次に測るとすれば、そこになる。

次にやること

- Linear と Dual Quaternion の速さを比べる
- ねじれ（ひねり）を入れたときの違いを測る
- 関節の回る範囲を決める（`kinefx::configurejointlimits`）

なぜ Linear が既定なのか — 速さではなかった。差は 4%

2026-09-14 | Houdini 21.0.700 | /obj/rigskin | ノード 6 / 接続 7

何を作ったか

実験053で、Dual Quaternion のほうが 154倍も痩せないと分かった。ではなぜ Linear が既定のままなのか。考えられるのは2つ。遅いのか、ねじれで挙動が違うのか。

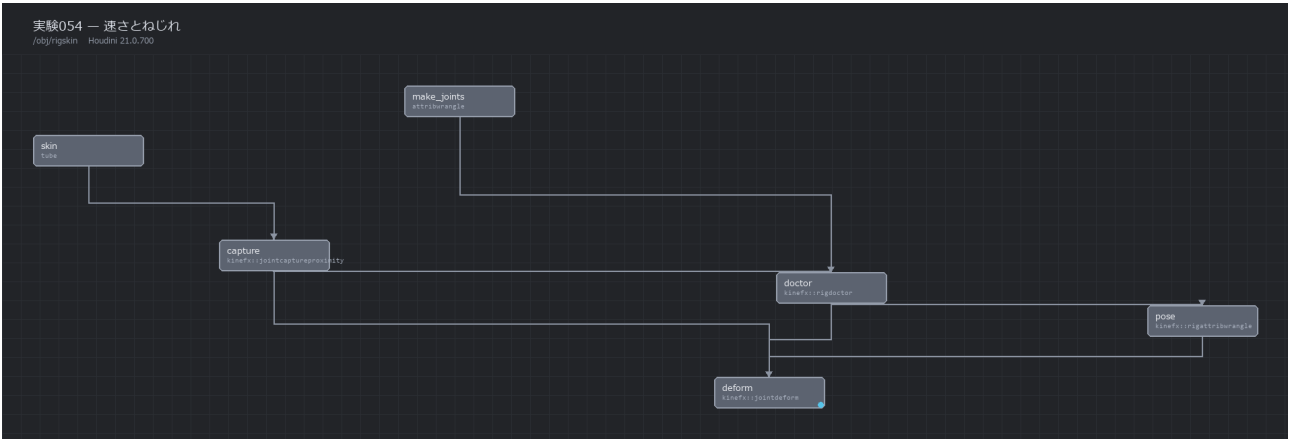
速さから測った。実験009の教訓を使い、毎回新しいノードを作って初回のクックを測る。414,720点の皮で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。点1つあたりの差は約 1.8ナノ秒。5通りの大きさを倍率は 0.98~1.12 に収まる。速さは理由にならない。

ねじれでも Linear は不利だった。骨の軸まわりに 179度ひねると、Linear は体積が 66.81% 減る。有名な「飴の包み紙」の形になり、真ん中がほぼ潰れる。Dual Quaternion は 0.24% しか減らない。

ただし Dual Quaternion にも癖があった。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0~0.5）が Linear では 14.1度しかねじれないのに、Dual Quaternion では 58.9度ねじれる。そのうえ、高さ 1.10~1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。途中に段差ができる。絵にもはっきり出る。

つまり「なぜ Linear が既定なのか」は、この2つの実験では決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が別の癖を持っていることだけである。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 170.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotat(m, radians(ch("angle")), set(0, 1, 0)); ...

ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

骨の軸まわりに 170度ひねる

皮は 1,440点の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear。真ん中がほぼ潰れる。179度で体積は 66.81% 減る。

Dual Quaternion。太さは保たれるが、途中に段差ができる。

ひねり	Linear	減った分	Dual Quaternion	減った分
0°	0.636074	0.0000%	0.636074	0.0000%
45°	0.574068	9.7482%	0.636140	-0.0105%
90°	0.423900	33.3568%	0.635838	0.0371%
135°	0.273535	56.9964%	0.635251	0.1294%
170°	0.214342	66.3024%	0.634689	0.2177%
179°	0.211093	66.8131%	0.634541	0.2409%

速さ（毎回新しいノードを作って、初回のクックを測る）

結び付け（capture）までは先に計算を済ませてあるので、測っているのは皮を動かす処理だけ。1回目は準備の分が乗るので捨てている。

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
2,880	2,858	4.47 ms	4.36 ms	0.98	-36.15 ns

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
11,520	11,474	4.81 ms	5.41 ms	1.12	51.68 ns
46,080	45,986	5.75 ms	5.76 ms	1.00	0.20 ns
184,320	184,130	9.06 ms	9.78 ms	1.08	3.88 ns
414,720	414,434	14.66 ms	15.21 ms	1.04	1.32 ns

高さごとに、実際は何度ねじれたか（170度を指定）

点の x-z 平面での角度が、元の位置から何度動いたかの中央値。軸の近くは角度が不安定なので、外側だけを使っている。

元の高さ	Linear	Dual Quaternion
0.00～0.50	14.1°	58.9°
0.50～0.90	59.7°	80.6°
0.90～1.10	102.6°	87.9°
1.10～1.50	142.2°	99.2°
1.50～2.01	170.0°	170.0°

180度を越えるとうなるか

Dual Quaternion

は角度が半回転を越えると近道を通る、と言われる。体積で見ると限りそうはならなかった。

ひねり	Linear 減った分	Dual Quaternion 減った分
170°	66.3024%	0.2177%
179°	66.8131%	0.2409%
181°	66.8149%	0.5458%
200°	64.8223%	0.4665%
270°	33.4623%	0.1981%
350°	0.5167%	0.0107%

わかったこと

- ・速さは理由ではなかった。414,720点で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。5通りの大きさが 0.98～1.12 に収まる。点1つあたりの差は約 1.8ナノ秒。
- ・ねじれでも Linear は不利だった。179度で体積が 66.81% 減る。真ん中がほぼ潰れて、有名な「飴の包み紙」の形になる。Dual Quaternion は 0.24%。
- ・ただし Dual Quaternion にも癖がある。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0～0.5）は Linear で 14.1度しかねじれないのに、Dual Quaternion では 58.9度。動かしていない側まで巻き込む。
- ・そのうえ途中に段差ができる。高さ 1.10～1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。その境目が絵でも見える。Linear は 14 → 60 → 103 → 142 → 170 と、なめらかにつながっている。

- ・ 180度を越えても、体積では破綻しなかった。Dual Quaternion は半回転を越えると近道を通る、と言われる。181度で 0.55%、270度で 0.20%、350度で 0.01%。少なくとも体積で見るかぎり、そうはならない。向きまでは調べていないので断定はしない。

- ・ Linear は 350度で元に戻る。減りが 0.52% まで下がる。一周に近づけば、ねじれていないのと同じだから。66.8% でいちばん潰れるのは 179度 = 半回転のところ。

- ・ 結局「なぜ Linear が既定なのか」は決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が「ねじれを局所に留めない」という別の癖を持つことだけ。

- ・ 分からなかったことを、分からないまま残す。ここで「互換性のためだろう」と書くのは簡単だが、測っていない。この実験で言えるのは、速さとねじれの体積という2つの筋が消えた、ということ。

- ・ 測り方について。結び付け (capture) までは先に計算を済ませてから測っている。そうしないと、両方に共通の重い処理が時間に混ざる。比べたいものだけを測る形に作り替える。

次にやること

- ・ ねじれの向きまで調べて、180度越えの挙動を確かめる
- ・ 関節の回る範囲を決める (kinexfx::configurejointlimits)
- ・ APEX のグラフから骨を動かす